

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

DSECLZG519 – CS#1

Introduction to DSECLZG519 & Algorithms

Agenda for CS #1



1) Introduction to DSECLZG519

- Course handout
- Books & Evaluation components
- How to make the most out of this course ?

2) Motivation & Synergies between Data Science & DSAD

3) Introduction to Algorithms

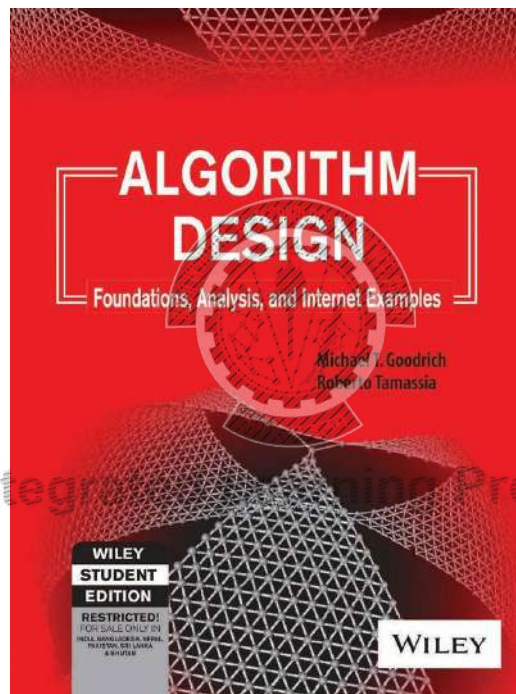
- Notion of an algorithm
- Properties of an algorithm
- Phases of program development
- Why do we have to analyze them ?

4) Analysis of algorithm efficiency

- Analysis Framework
- Pseudocode & Counting primitives
- Order of Growth
- Asymptotic Notations

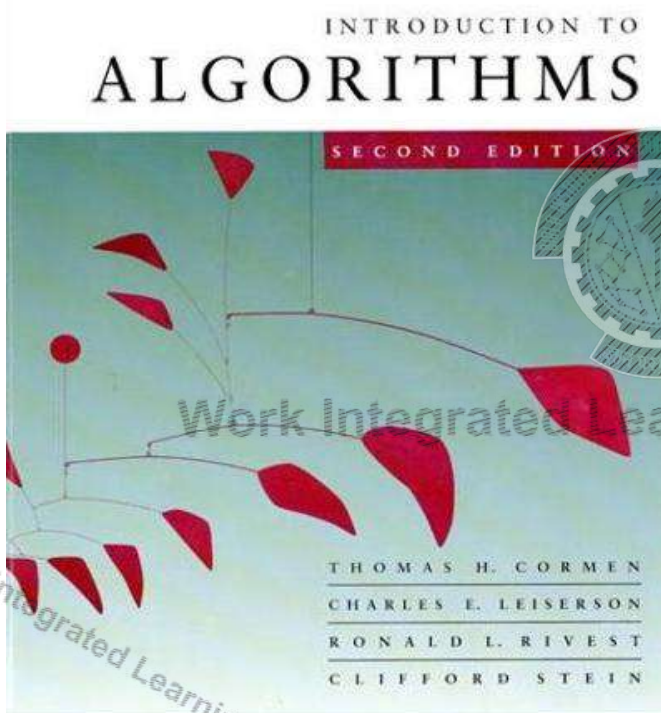
5) Q&A!

Text Book



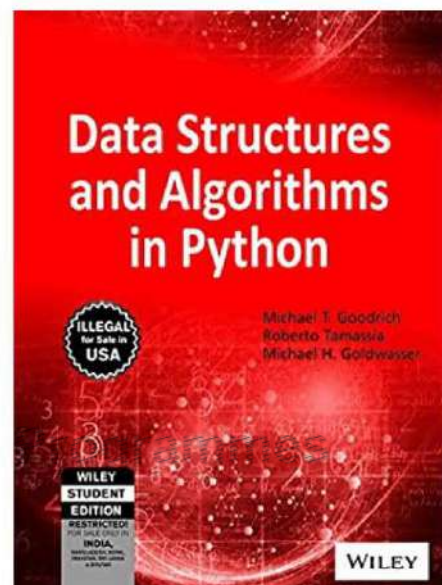
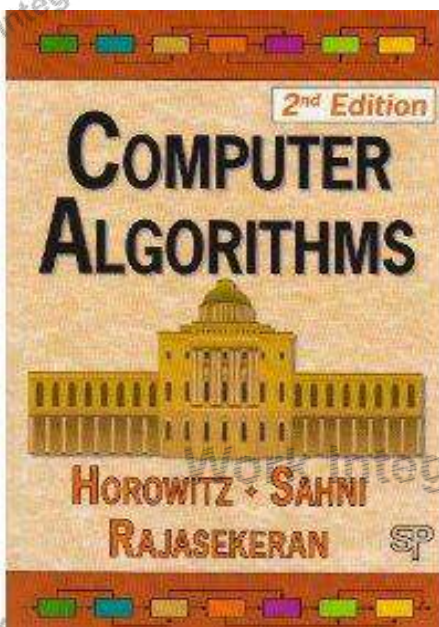
Michael T. Goodrich and Roberto Tamassia: *Algorithm Design: Foundations, Analysis and Internet examples* (John Wiley & Sons, Inc., 2002)

Reference Books



Also known as CLRS book

Reference Books



Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekaran. **Computer Algorithms**

Michael T. Goodrich and Roberto Tamassia and Goldwasser: *Data structures and Algorithms in Python* (John Wiley & Sons)

Ground Rules!



- Be Regular 😊
- Mentally present – Observe!! Listen!! 😊
- Keep your questions for the Q&A section ...
- Use the Discussion Forum in Canvas effectively
- Solve the exercises regularly!
- Go an extra mile 😊

$$1^{365} = 1$$

$$1.01^{365} = 37.8$$

Motivation & DSE – DSAD ?



Aspiring Data Scientist and allied areas ?

- Awesome! Even in such a role, you would be creating solutions that inevitably have code !!
- When you want to code → You need to have a strong understanding of Data structures and algorithms.
- Data Structures and Algorithms Knowledge give us the ability to improve our solution to the problem and the ability to write much better and efficient code.
- But most importantly, it helps to build problem solving mindset ...
- Thus, learning Data Structures and Algorithms can be a major learning curve for any computer science / data science student.

What is an Program ?



Algorithm

An algorithm is a *step-by-step procedure* for solving a problem in a finite amount of time.

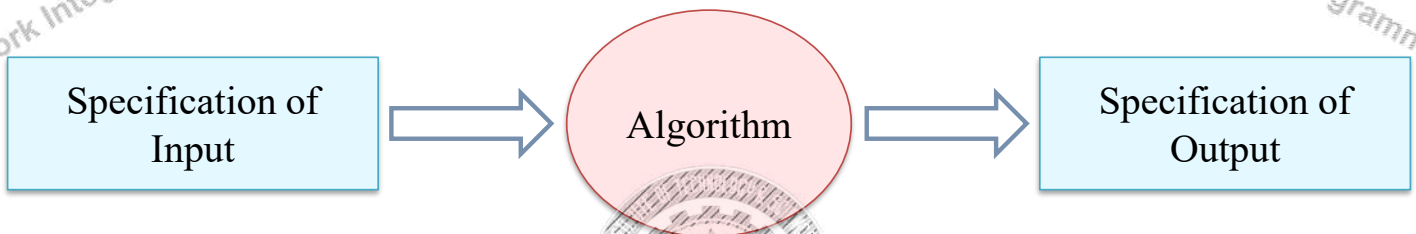
Data Structures

Is a *systematic way of organizing and accessing data, so that data can be used efficiently*

Algorithms + Data Structures = Program

An algorithm is defined as a finite sequence of unambiguous instructions followed to accomplish a given task.

Algorithmic Solution



- Algorithm describes actions on the input instance.
- Infinitely many correct algorithm for the same problem.
- Infinite number of input instances satisfying the specification.



Properties of an Algorithm



- ✓ **Input** : Each algorithm should have zero or more inputs
- ✓ **Output** : The algorithm should produce correct results. Atleast one output has to be produced
- ✓ **Definiteness** : Each instruction should be clear and unambiguous
- ✓ **Effectiveness** : The instructions should be simple and should transform the given input to the desired output.
- ✓ **Finiteness** : The algorithm must terminate after a finite sequence of instructions

Example of an Algorithm

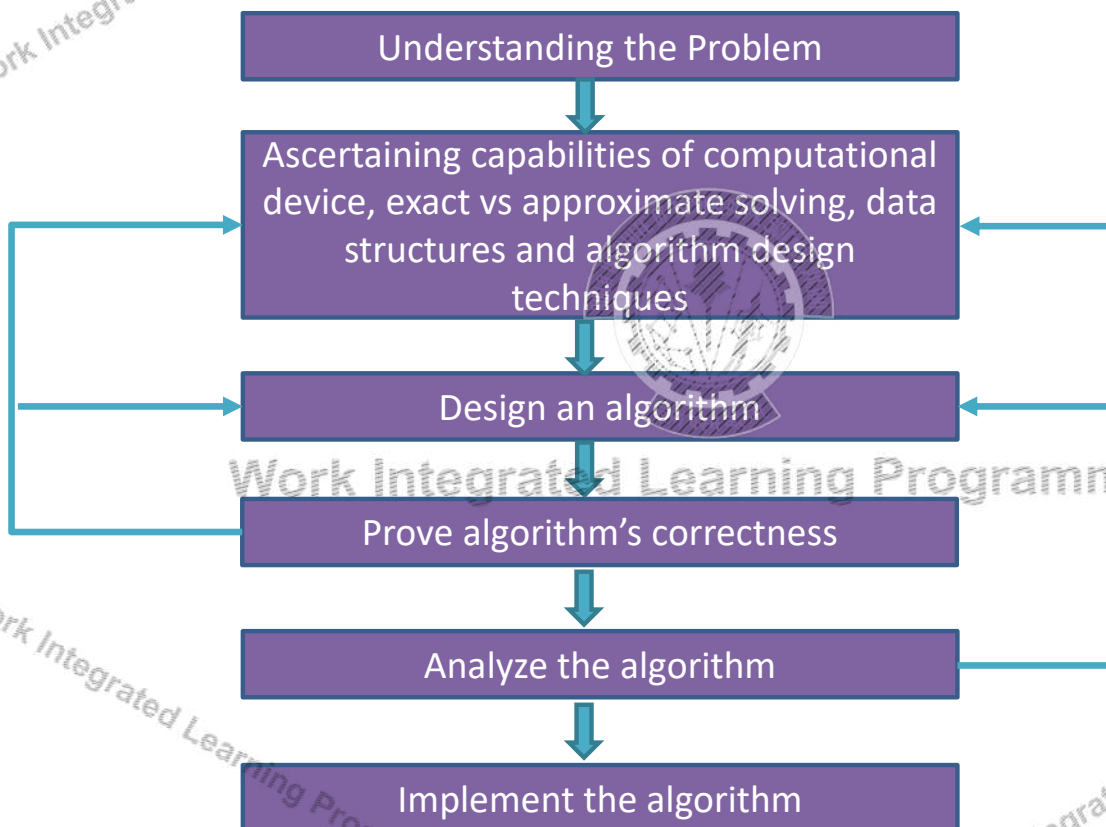


➤ Let us first take an example of a real-life situation for creating algorithm. Here is the algorithm for making tea!

1. Put the teabag in a cup.
2. Fill the kettle with water.
3. Boil the water in the kettle.
4. Pour some of the boiled water into the cup.
5. Add milk to the cup.
6. Add sugar to the cup.
7. Stir the tea.
8. Drink the tea.

➤ Some steps like 5,6 can be interchanged but some like 3,8 cannot be interchanged.

Phases of Program Development



Data Structures Outlook



Name of array (Note that all elements of this array have the same name, **c**)

Array
Linearly
Ordered Set

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Position
number of the
element within
array **c**

*A **data structure** is a particular way of organizing data in a computer so that it can be used effectively.*



Head

Linked List

node



node

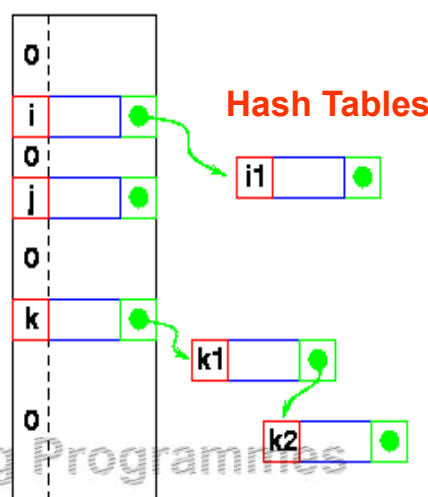
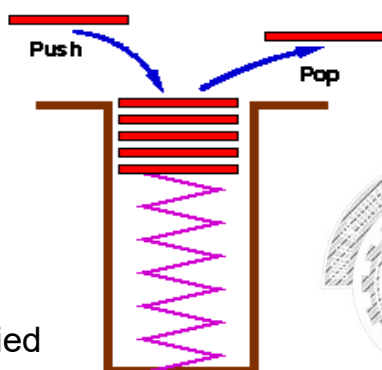


Data Structures Outlook



Stack

Set with delete operation specified (LIFO)



Queue

Remove (Dequeue)

front

(FIFO)

rear

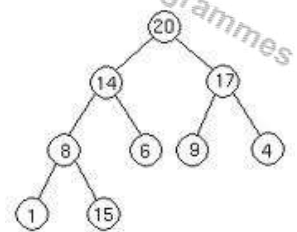
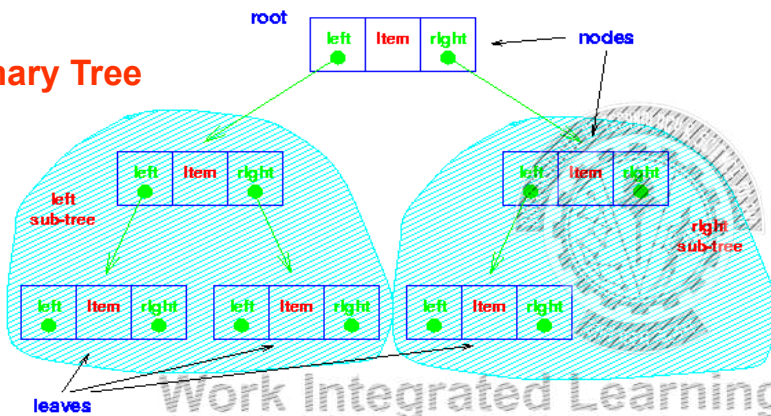
Insert (Enqueue)



Data Structures Outlook



Binary Tree



Heap



AVL Tree and others ...

Algorithm Techniques – Brute Force



Brute Force Algorithms are exactly what they sound like – straightforward methods of solving a problem that rely on sheer computing power and trying every possibility rather than advanced techniques to improve efficiency.



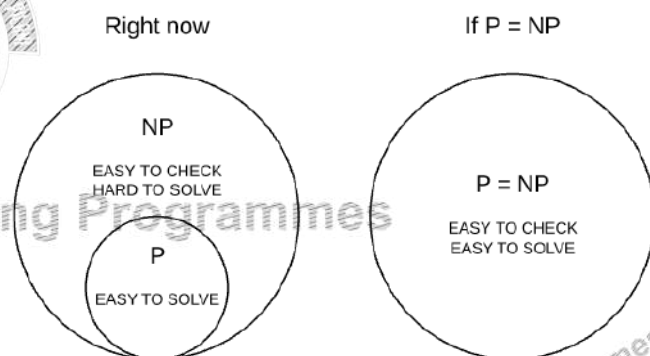
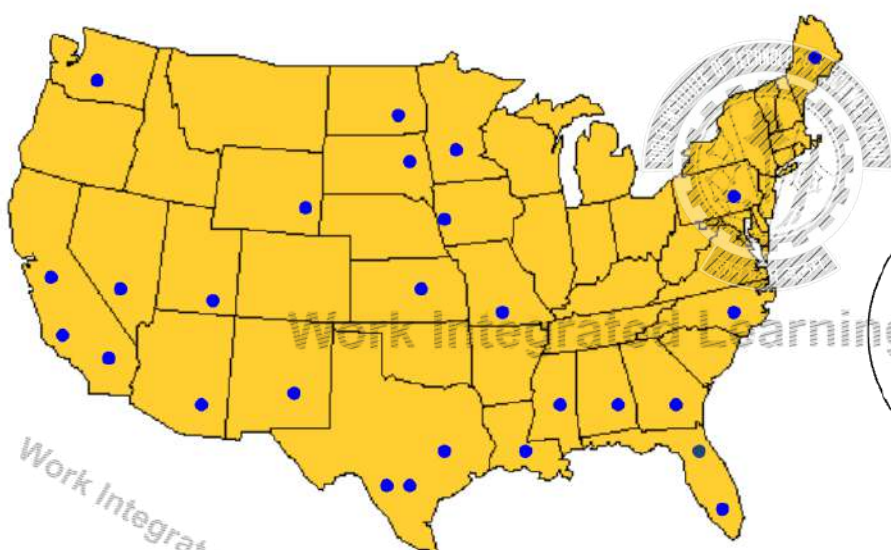
Algorithm Techniques Outlook



- Brute Force
- Greedy Method
 - Knapsack
 - MST
 - Dijkstra's
- Divide & Conquer
 - Merge Sort
 - Quick Sort
 - Integer Multiplication Problem
- Dynamic Programming
 - Matrix Chain Multiplication
 - Floyd – Warshall's
- ...
- ...



P, NP, NP-Complete, NP Hard



Progress Check!



1) Introduction to DSECLZG519

- Course handout
- Books & Evaluation components
- How to make the most out of this course ?

2) Motivation & Synergies between Data Science & DSAD

3) Introduction to Algorithms

- Notion of an algorithm
- Properties of an algorithm
- Phases of program development
- Why do we have to analyze them ?

4) Analysis of algorithm efficiency

- Analysis Framework
- Pseudocode & Counting primitives
- Order of Growth
- Asymptotic Notations

5) Q&A!

Who's the champion?



Which Algorithm is better ?



Who is topper in DSAD?

Algorithm 1:

Sort A into decreasing order

Output A[1].

Algorithm 2:

```
int i;  
int m = A[1];  
for (i = 2; i <= n; i ++)  
    if (A[i] > m)  
        m = A[i];  
return m;
```

Which is better?

What is good algorithm?



- Resources Used
 - Running time (Lesser the running time, better the algorithm)
 - Space used (Lesser the space used, better the algorithm)
- Resource Usage
 - Measured proportional to (input) size



Analysis Framework

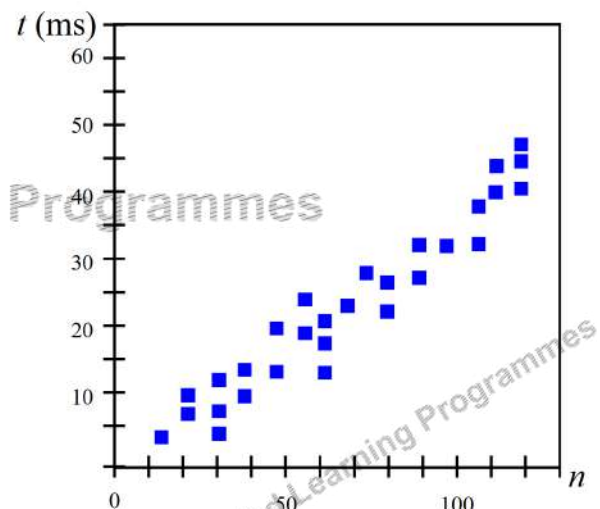


- ✓ “Better” = more efficient
- ✓ Time
- ✓ Space

How should we measure the running time of an algorithm?

Experimental Study

- Write a program that implements the algorithm
- Run the program with data sets of varying size and composition.
- Use a method like `System.currentTimeMillis()` to get an accurate measure of the actual running time.
- The result maybe something similar to this :





Beyond Experimental Studies

Experimental studies have several limitations:

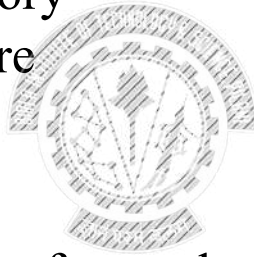
- It is necessary to implement and test the algorithm in order to determine its running time.
- Experiments can be done only on a limited set of inputs, and may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments should be used.
- Analytical Model to analyze algorithm - We will now develop a general methodology for analyzing the running time of algorithms that :
 - Uses a high-level description of the algorithm instead of testing one of its implementations.
 - Takes into account all possible inputs.
 - Allows one to evaluate the efficiency of any algorithm in a way that is independent from the hardware and software environment.

How to Analyze time complexity ?



Running time depends on:

- Single vs multi processor
- Read/write speed to memory
- 32 bit or 64 bit architecture
- Input given to algorithm



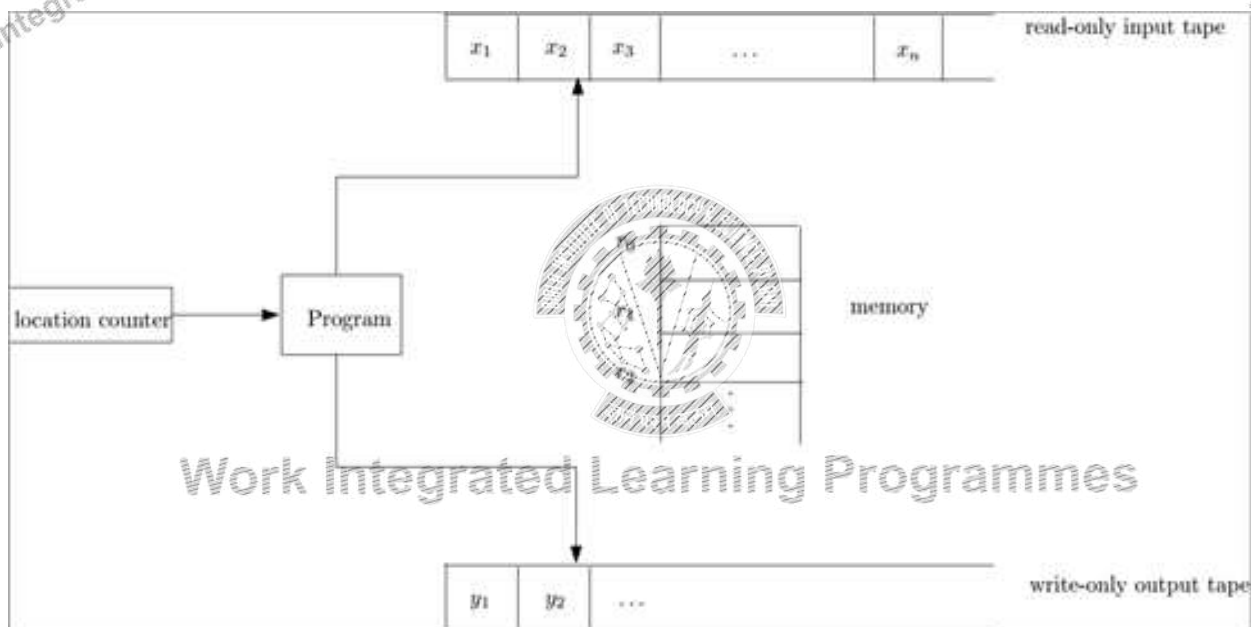
A function which defines rate of growth of time w.r.t input!

Model machine: Random Access Machine model



- Algorithms can be measured in a machine-independent way using the **Random Access Machine (RAM) model**.
- This model assumes a **single processor**.
- In the RAM model, **instructions are executed one after the other, with no concurrent operations**.
- This model of computation is an abstraction that allows us to compare algorithms on the basis of performance.
- The assumptions made in the RAM model to accomplish this are:
 - Each simple operation takes 1 time step.
 - Loops and subroutines are not simple operations.
 - Each memory access takes one time step, and there is no shortage of memory (we assume we have unbounded memory).

Model machine: Random Access Machine model



- ◆ Time complexity (running time) = number of instructions executed
- ◆ Space complexity = the number of memory cells accessed

Pseudo-code



A mixture of natural language and high level programming concepts that describes the main ideas behind a generic implementation of a data structure and algorithms.

- High-level description of an algorithm
- More structured than English prose
- Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design issues

Example: find max element of an array

Algorithm *arrayMax*(*A*, *n*)

Input array *A* of *n* integers

Output maximum element of *A*

currentMax ← *A*[0]

for *i* ← 1 **to** *n* − 1 **do**

if *A*[*i*] > *currentMax* **then**

currentMax ← *A*[*i*]

return *currentMax*

0	1	2	3	4
10	20	30	5	7



Pseudo-code (Some Guidelines)

- Is structured than usual text but less formal than a programming language.
- Control flow
 - **if ... then ... [else ...]**
 - **while ... do ...**
 - **repeat ... until ...**
 - **for ... do ...**
 - Indentation replaces braces
- Method declaration
 - **Algorithm** *method* (*arg* [, *arg*...])
- Expressions
 - Use standard mathematical symbols to describe numeric and Boolean expressions.
 - ← Assignment
(like = in C / Java / Python)
 - = Equality testing
(like == in C / Java / Python)
 - n^2 Superscripts and other mathematical formatting allowed



Primitive Operations



- **Basic** computations performed by an algorithm
- Identifiable in pseudocode
- Largely *independent* from the programming language
- Assumed to take a constant amount of time in the RAM model

Examples:

- Evaluating an expression
- Assigning a value to a variable
- Indexing into an array
- Calling a method
- Returning from a method

Analyzing Pseudo-code (By Counting) / Counting Primitives



1. For each line of pseudocode, count the number of primitive operations in it.
2. Pay attention to the word "*primitive*" here; sorting an array is not a primitive operation.
3. Multiply this count with the number of times this line is executed.
4. Sum up over all lines.



Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Printing each element of an array

```
i=0;  
while(i<n)  
{  
    print a[i];  
    i++;  
}
```




Counting Primitive Operations

Printing each element of an array

```
i=0;  
while (i<n)  
{
```

```
    print a[i];  
    i++;  
}
```



- One Initialization of i
- $n+1$ comparisons
- n array indexing operations
- n invocations of print
- n increments of i

So, we write $T(n)=1+n+1+n+n+n$
 $T(n)=4n+2$

Counting Primitives



We can also use a tabular way for counting primitives as below:

Algorithm ArraySum(A, n)	#Operations	Remarks
(1) Sum = A [0]	2	Indexing , Assignment
(2) i = 1	1	Assignment
(3) while (i<n)	n	Comparison
(a) Sum = Sum + A[i]	3 (n-1)	(n-1) times indexing, addition and assignment
(b) i = i + 1	2 (n-1)	(n-1) times addition and assignment
(4) return Sum	1	1 times returning

$$\begin{aligned} \text{Add the Operations Column} &= 2 + 1 + n + 3n - 3 + 2n - 2 + 1 \\ &= 6n - 1 \end{aligned}$$

Progress Check!



1) Introduction to DSECLZG519

- Course handout
- Books & Evaluation components
- How to make the most out of this course ?

2) Motivation & Synergies between Data Science & DSAD

3) Introduction to Algorithms

- Notion of an algorithm
- Properties of an algorithm
- Phases of program development
- Why do we have to analyze them ?

4) Analysis of algorithm efficiency

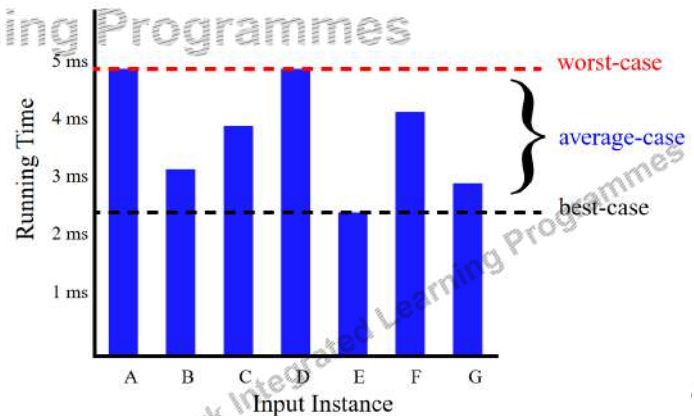
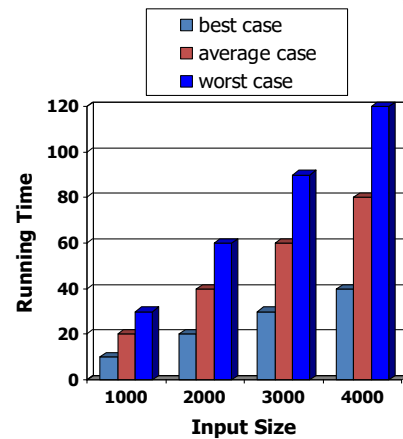
- Analysis Framework
- Pseudocode & Counting primitives
- Order of Growth
- Asymptotic Notations

5) Q&A!

Running Time



- What is best, average, worst case running of a problem?
- An algorithm may run faster on certain data sets than on others.
- Average case time is often difficult to determine – why?
- We focus on the worst case running time.
 - Easier to analyze and best to bet
 - Crucial to applications such as games, finance and robotics
 - Performing well in worst case means it would perform well in normal input too!



Example



To find the sum of two numbers

```
Sum(a,b)
{
Return a+b
}
```

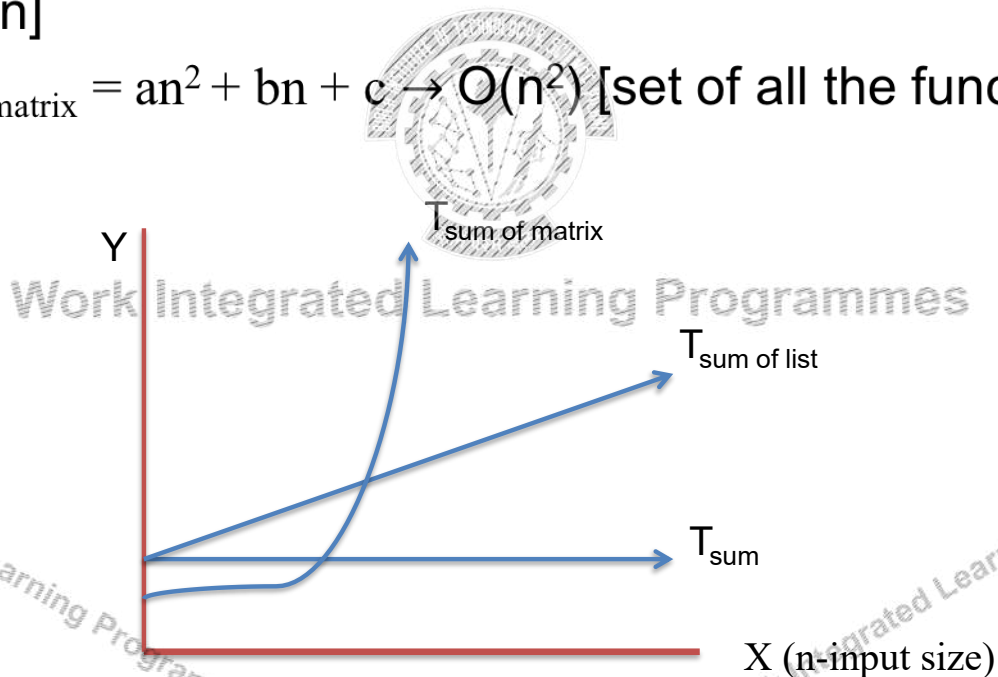


$T_{\text{sum}} = 2$ (it's a constant time algorithm)

Example (Cont.,)



- 1) $T_{\text{sum}} = K \rightarrow$ all the function of form some constant $O(1)$
- 2) $T_{\text{sum of list}} = cn + c' \rightarrow$ all linear function $O(n)$ [linear function]
- 3) $T_{\text{sum of matrix}} = an^2 + bn + c \rightarrow O(n^2)$ [set of all the function]



Time complexity Analysis- Some General Rules.



We analyze time complexity

- a) Very large input-size
- b) Worst case scenario

$$T(n) = n^3 + 3n^2 + 4n + 2$$

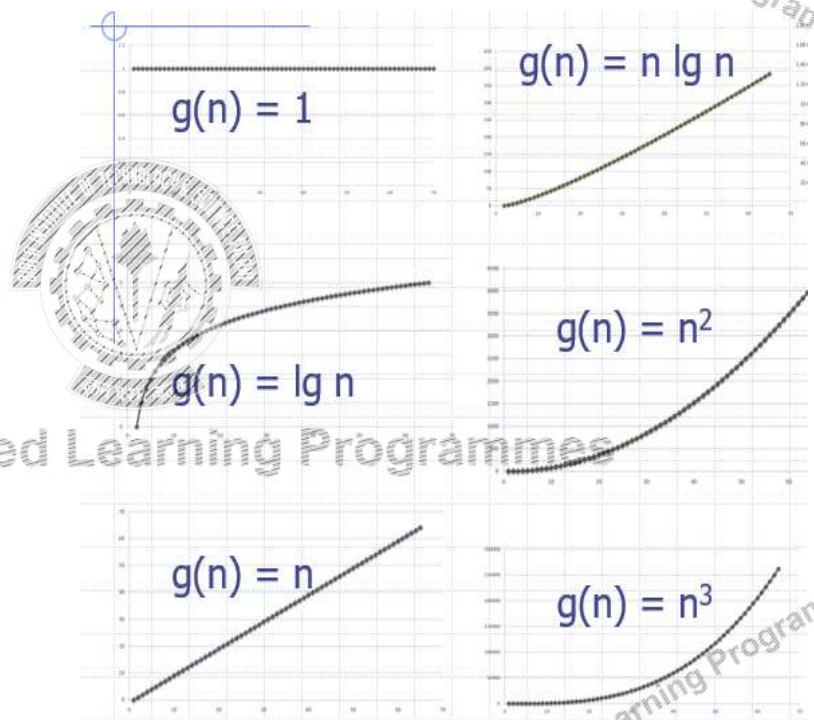
$$\approx n^3 \quad (n \rightarrow \infty)$$

$$c \cdot n^3 \quad O(n^3)$$

- Rule: a) drop lower order terms
b) drop constant multiplier

$$T(n) = 17n^4 + 3n^3 + 4n + 8 = O(n^4)$$

$$T(n) = 16n + \log n = O(n)$$



Time complexity Analysis- Some General Rules.



Eliminate low order terms

$$4n + 5 \Rightarrow 4n$$

$$0.5 n \log n - 2n + 7 \Rightarrow 0.5 n \log n$$

$$2^n + n^3 + 3n \Rightarrow 2^n$$

Eliminate constant coefficients

$$4n \Rightarrow n$$

$$0.5 n \log n \Rightarrow n \log n$$



Order of Growth

- We expect our algorithms to work faster for all values of N. Some algorithms execute faster for smaller values of N. But, as the value of N increases, they tend to be very slow.
- So, the behavior of some algorithms changes with increase in value of N.
- This change in behavior of the algorithm and algorithm's efficiency can be analyzed by considering the highest value of N.
- Order of growth :

As N increases order of growth increases

N	log N	N	N log N	N ²	N ³	2 ^N	N!
1	0	1	0	1	1	2	1
2	1	2	2	4	8	4	2
4	2	4	8	16	64	16	24
8	3	8	24	64	512	256	40320
16	4	16	64	256	4096	65536	high
32	5	32	160	1024	32768	4294967296	very high

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

As n Increases, order of Growth increases ...

42

Asymptotic Notations



- Asymptotic Notations are languages that allow us to analyze an algorithm's running time by identifying its behavior as the input size for the algorithm increases.
- This is also known as an algorithm's growth rate.
- Asymptotic notations are the notations used to express the order of growth of an algorithm and it can be used to compare two algorithms with respect to their efficiency.

Asymptotic Notations

O (Big Oh)

Ω (Big Omega)

Θ (Big Theta)

Asymptotic Notation



- O notation: asymptotic “less than”:

Upper Bound

- $f(n) = O(g(n))$ implies: $f(n) \leq g(n)$

- Ω notation: asymptotic “greater than”:

Lower Bound

- $f(n) = \Omega(g(n))$ implies: $f(n) \geq g(n)$

- Θ notation: asymptotic “equality”:

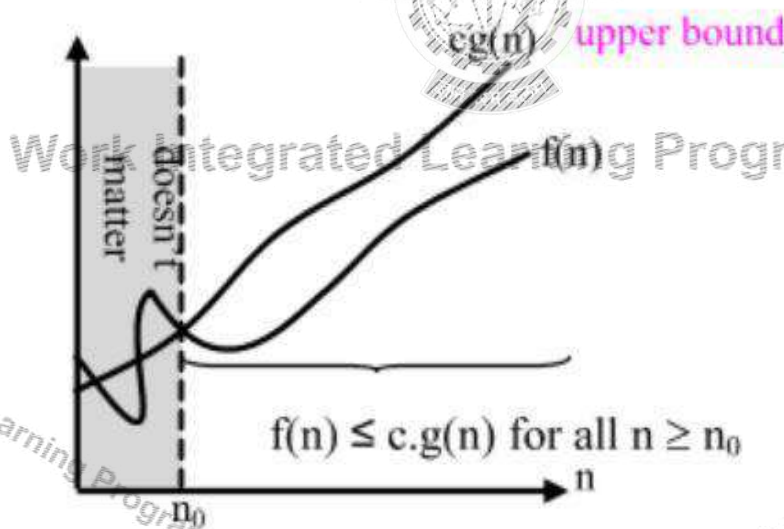
Average Bound

- $f(n) = \Theta(g(n))$ implies: $f(n) = g(n)$

Big - Oh Notation



- Big – Oh is the formal method of expressing the upper bound of an algorithm's running time.
- It is a **measure of the longest amount of time** it could possibly take for the algorithm to complete.

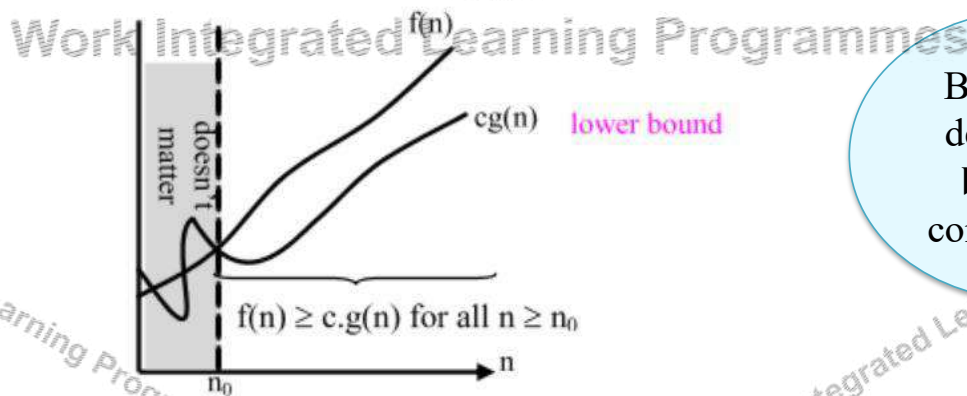


Big Oh denotes the **worst case complexity !!**

Relatives of Big-Oh – Big Omega ($\Omega(n)$)



- Big – Omega is the formal method of expressing the lower bound of an algorithm's running time.
- In general, the lower bound implies that below this time the algorithm cannot perform better.
- It is a **measure of the least amount of time** it could possibly take for the algorithm to complete.

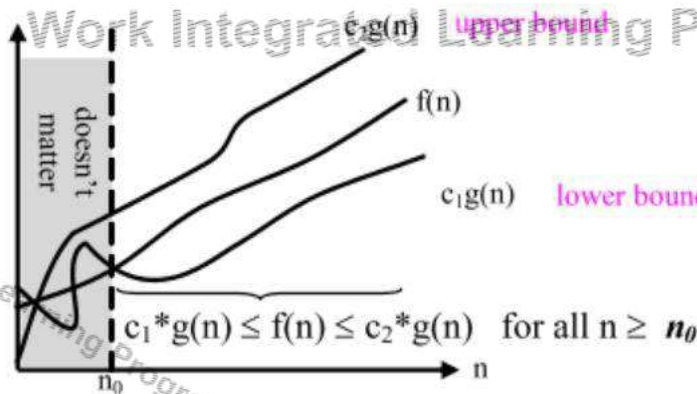


Big Omega denotes the **best case** complexity !!

Relatives of Big-Oh – Big Theta ($\Theta(n)$)



- Big – theta is the formal method of expressing both the lower bound & upper bound of an algorithm's running time.
- The upper bound on $f(n)$ indicates that function $f(n)$ will not consume more than the specified time $c_2 * g(n)$ & the lower bound on $f(n)$ indicates that function $f(n)$ in the best case will consume atleast the specified time $c_1 * g(n)$



Big Theta denotes the **average** case complexity !!

Summary for CS #1



1) Introduction to DSECLZG519

- Courseware
- Books & Evaluation components
- How to make the most out of this course ?

2) Motivation & Synergies between Data Science & DSAD

3) Introduction to Algorithms

- Notion of an algorithm
- Properties of an algorithm
- Phases of program development
- Outline of data structures & Algorithm design strategies
- Why do we have to analyze them ?

4) Analysis of algorithm efficiency

- Analysis Framework
- Pseudocode & Counting primitives
- Asymptotic Notations

5) Exercises, Q&A

Exercises



Write the pseudocode for below and count the primitives:

- Minimum element in an array
- Maximum of 3 numbers
- Write code to count how many elements are even in the given list.
- Linear Search – Finding if an element is present in an array.

State if the below are true/False, Fill in the blanks

◆ Is $T(n) = 9n^4 + 876n = O(n^4)$?

◆ Is $T(n) = 9n^4 + 876n = O(n^3)$?

◆ Is $T(n) = 9n^4 + 876n = O(n^{27})$?

◆ $T(n) = n^2 + 100n = O(?)$

◆ $T(n) = 3n + 32n^3 + 767249999n^2 = O(?)$



Work Integrated Learning Programmes

We will explore more on Algorithm analysis in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in



Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)

51

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #2

DSECLZG519 – Analyzing Algorithms

Work Integrated Learning Programmes

Agenda for CS #2



- 1) Recap of CS#1
- 2) General Rules and Problems
- 3) Mathematical analysis of Non-recursive algorithms
- 4) Mathematical analysis of Recursive algorithms
 - Iteration vs Recursion
 - Recurrence Tree
 - Iterative Substitution
- 5) Masters Theorem with examples
- 6) Exercises, Q&A!

Asymptotic Notations Analogies



- “The delivery will be there *within* your lifetime.”
 - *(big-O, upper-bound)*
- “I can pay you *at least* one dollar.”
 - *(big-omega, lower bound)*
- “The high today will be 25°C *and* the low will be 19°C.”
 - *(big-theta, narrow)*

General rules to determine running time of algorithm



Loops: The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

//executes n-times

```
for(i=1;i<=n;i++)
```

...// constant time

Total number

= a constant $c \times n = cn = O(n)$

$$\begin{aligned} F(n) &= \sum_{i=1}^n 1 \\ &= n - 1 + 1 \\ &= n \end{aligned}$$

General rules to determine running time of algorithm (Cont.,)



Nested loops: Analyze from inside out. Total running time is the *product of the sizes* of all the loops

//outer loop executed n-times

```
for(i=1;i<=n;i++){
```

//inner loop execute n-times

```
for(j=1;j<=n;j++){
```

//constant time

```
k=k+1;
```

```
}
```

Total number = $c \times n \times n = cn^2 = O(n^2)$

General rules to determine running time of algorithm (Cont..)



If-then-else statements: worst-case running time: the test, plus either the then part or the else part (whichever is the larger).

```
//test: constant  $c_0$ 
If(length()==0){
return false; // then part: constant  $c_1$ 
}
else { //else part : (constant  $c_2$  +constant  $c_3$ ) $\times$  length
for(int n =0; n<length();n++){
//another if: constant + constant (no else part)
if(! List[n].equals(otherlist.list[n]))
//constant
return false;
}
}
```

Total time = $c_0 + c_1 + (c_2 + c_3) \times \text{length} = O(\text{length}())$

General rules to determine running time of algorithm (Cont.,)



Logarithmic Complexity: an algorithm is $O(\log n)$ if it takes constant time to cut the problem size by a fraction (usually by $\frac{1}{2}$)

As an example let us consider the following program

```
for(i=1; i<=n;)  
    i=i*2;
```

If we observe carefully, the value of i is doubling every time initially $i=1$, in next step $i=2$, and in subsequent steps $i=4, 8$ and so on.

Let us assume that the loop is executing some k -times.

At k^{th} step $2^k = n$ and we come out of loop

Taking logarithm on both sides, gives

$$\log(2^k) = \log n$$

$$k \log 2 = \log n$$

$$k = \log n \text{ // if we assume base } \sim 2.$$

Total number = $O(\log n)$

Searching in Phone book



Problem-1



What is the complexity of the program given below

```
void function (int n){  
    for (i =1; i<=n; i++)  
        for (j=1;j+n/2<=n;j=j++)  
            //loop execute logn times  
            for (k=1;k<=n;k=k*2)  
                count++;  
}
```

Time complexity of the above function is $O(n^2 \log n)$

More on it in the discussion thread ...

Problem-2



```
function (int n){  
    //constant time  
    if(n==1) return;  
    //outer loop execute n times  
    for (int i =1;i<=n;i++){  
        // inner loop executes only time due to break statement  
        for (int j =1;j<=n;j++){  
            printf("*");  
            break;  
        }  
    }  
}
```

Time complexity of the above function is?

Time Complexity of the above function $O(n)$. *Even though the inner loop is bounded by n , but due to the break statement, it is executing only once.*

Complexity Definition



- **Linear Complexity:** Processing time or storage space is directly proportional to the number of input elements to be processed.
- **Quadratic complexity:** An algorithm is of quadratic complexity if the processing time is proportional to the square of the number of input elements.
- **Cubic complexity:** In the case of cubic complexity, the processing time of an algorithm is proportional to the cube of the input elements.
- **Logarithmic complexity:** An algorithm is of logarithmic complexity if the processing time is proportional to the logarithm of the input elements. The logarithm base is typically 2



Analysis of Algorithms

Techniques employed by algorithms to solve problems

- **Iterative**
- **Recursion**

What is Recursion

- The process in which a function calls **itself** directly or indirectly is called recursion and the corresponding function is called as recursive function
- Define a procedure P that is allowed to make calls to itself, provided those calls to P are for solving sub-problems of smaller size.

Condition for recursive procedure

- A recursive procedure should always define a base case
- Base case
 - Small sized problem which can be solved by the algorithm directly without using recursion

12

Recursive Algorithm



1. if problem is “small enough”
2. solve it directly
3. else
4. break into one or more smaller sub problems
5. solve each sub problem recursively
6. combine results into solution to whole problem

Requirements for Recursive Solution



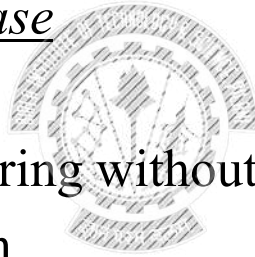
- At least one “small” case that you can solve directly
- A way of breaking a larger problem down into:
 - One or more smaller sub problems
 - Each of the same kind as the original
- A way of combining sub problem results into an overall solution to the larger problem

Recursive Algorithm - Example



Recursive algorithm for finding length of a string:

1. if string is empty (no characters)
2. return 0 $\leftarrow$ base case
3. else $\leftarrow$ recursive case
4. compute length of string without first character
5. return 1 + that length



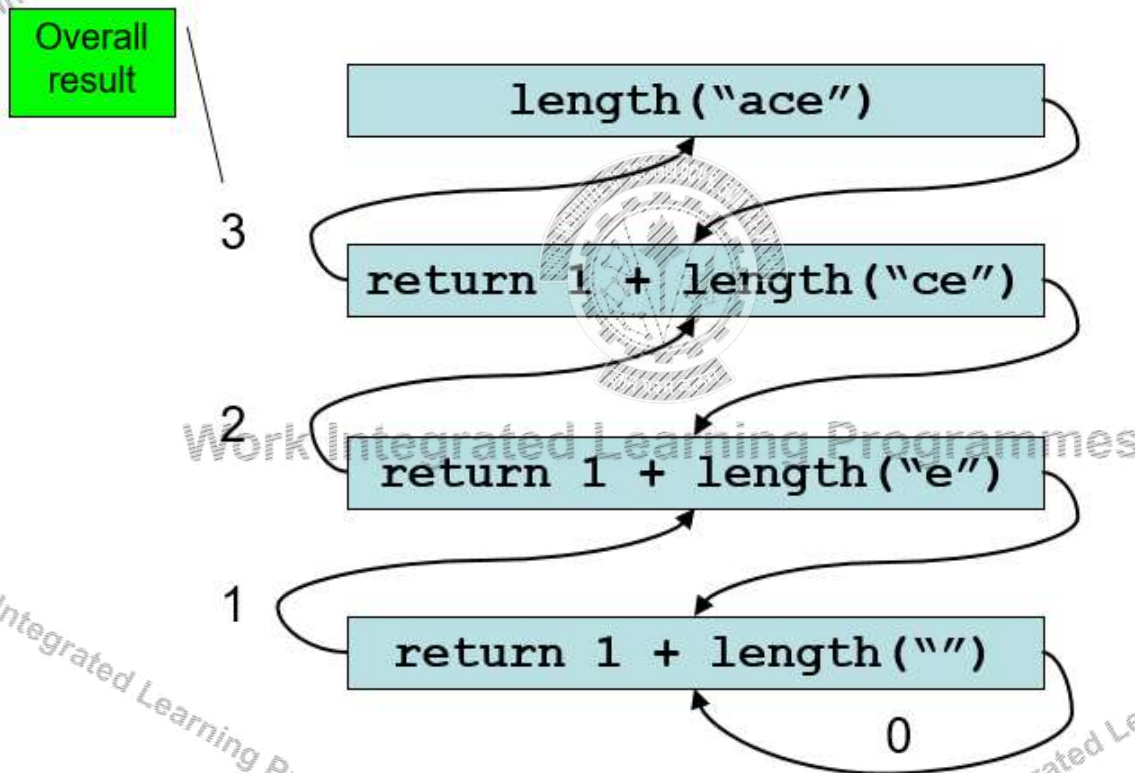
Recursive Design Example: Code



Recursive algorithm for finding length of a string:

```
public static int length (String str) {  
    if (str == null || str.equals(""))  
        return 0  
    else  
        return length(str.substring(1)) + 1  
}
```

Tracing a Recursive Method



Example



- Compute $N! = 1 * 2 * 3 * \dots * N$

```
//An iterative solution
int factorial(int N){
    result = 1;
    for (i = 1; i <= N; i++)
        result = i * result;
    return result;
}
```



- Alternative Solution: factorial calls itself recursively

```
int factorial (int N) {
    if (N == 1) {
        return 1;
    } else {
        return N * factorial (N-1);
    }
}
```

} base case

} recursive case

Analysis of non-recursive Algorithms



- Based on the input size, determine the number of parameters to be considered.
- Identify the basic operation in the algorithm
- Check whether the number of times the basic operation is executed depends only on the size of the input.
- Obtain the total number of times a basic operation is executed.
- Simplify using standard formulas, obtain the order of growth.

Example



Algorithm Linear_Search(key, a[], n)

//Purpose: To search for key in an array of n elements

// Inputs :

n – the number of items present in the table
 a – the items to be searched are present in the table
key – the item to be searched

//Output:

return i whenever key is present in the list
return -1 whenever key is not present in the list

Step 1: [Search for the key in the array/table]

```
for i ← 0 to n-1 do
    if (key = a[i]) return i;    // Key Found at position i
end for
```

Step 2:[Key not found]

```
return -1;
```

- In this problem, the size of input is n and this is the parameter to be considered.
- The statement executed in loop is the basic operation which is “if(key = a[i])”
- The basic operation is inside a single loop and hence the complexity is $O(n)$

Analysis of recursive Algorithms



- Based on the input size, determine the number of parameters to be considered.
- Identify the basic operation in the algorithm
- Obtain the number of times the basic operation is executed on different inputs of the same size.
- Obtain the **recurrence relation** with an appropriate initial condition.
- Solve the recurrence relation and obtain the order of growth and express using asymptotic notations.

Recurrence equation

- When an algorithm contains a recursive call to itself, its running time can often be described by a recurrence. A **recurrence** is an equation or inequality that describes a function in terms of its value on smaller inputs.

21



Solution to recurrence equations

$$T(n) = \begin{cases} 1 & ; n = 0 \text{ // base case} \\ 1 + T(n-1) & ; n > 0 \text{ // recursive case} \end{cases}$$

- Recursion tree method
- Iterative substitution method
- Master's theorem

Recursive Tree method

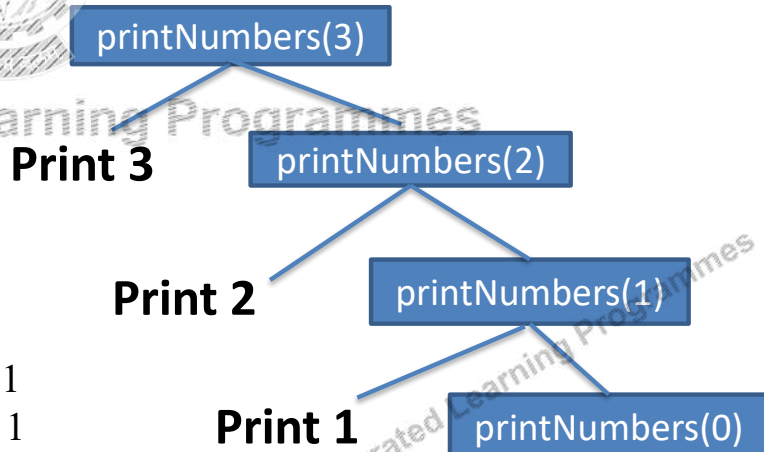


Recursion tree method

- Visual method
- Converts the recurrence into a tree
- Sum all per-level cost to get the total cost of recursion

Lets assume $n = 3$

```
function printNumbers(n)
    if(n > 0)
        print n
        printNumbers(n-1)
    end
```



From the tree, its clear that it takes $T(3) = 3 + 1$
 $T(n) = n + 1$
 $O(n)$

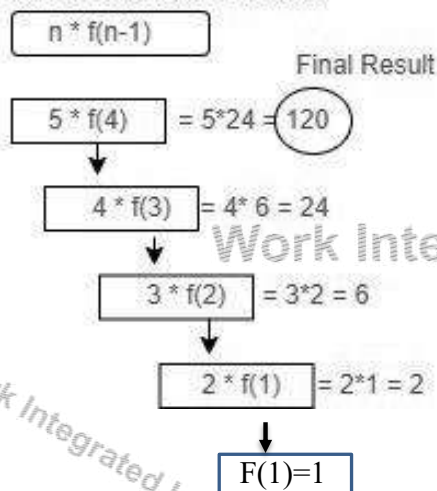
Factorial using Recursion – Recursive Tree Method



Recursive tree method

For user input : 5

Factorial Recursion Function



```
func factorial(n)
    if(n == 1)
        return 1
    return n * factorial(n-1)
end
```

It is clear from the recursive tree that for the recursion unfolds n levels with 1 unit in each level of the tree. Hence, the complexity is $n * 1 = \mathbf{O(n)}$

Iterative substitution method



Iterative substitution method

- Known as the "plug-and-chug" method
- Substitute the general form of the recurrence for each occurrence of the function T on the right-hand side (mathematical induction ☺)
- The hope in applying the iterative substitution method is that, at some point, we will see a pattern that can be converted into a general form equation!
- Then we use the base case to solve the equation

Example

```
function printNumbers(n)
  if(n > 0)
    print n
    printNumbers(n-1)
  end
```

$$T(n) = \begin{cases} 1 & ; n = 0 \text{ // base case} \\ 1 + T(n-1) & ; n > 0 \text{ // recursive case} \end{cases}$$

Let us solve this recurrence relation using iterative substitution method!!

Substitution Method



- A way to solve a linear system algebraically is to use the substitution method. The substitution method functions by substituting the one y value with the other.

➤ Example:

$y = 2x + 4$
 $3x + y = 9$

We can substitute y in the second equation with the first equation since $y =$
 y .

$3x + y = 9$
 $3x + (2x + 4) = 9$
 $5x + 4 = 9$
 $5x = 5$
 $x = 1$

This value of x can then be used to find y by substituting 1 with x e.g. in the first equation

$y = 2x + 4$
 $y = 2 \cdot 1 + 4$
 $y = 6$

The solution of the linear system is $(1, 6)$.

26

Substitution Method



$$T(n) = \begin{cases} 1 & ; n = 0 \text{ // base case} \\ 1 + T(n-1) & ; n > 0 \text{ // recursive case} \end{cases}$$

$$T(n) = T(n-1) + 1$$

Substitute value of $T(n-1)$ in above:

$$T(n) = [T(n-2) + 1] + 1$$

$$T(n) = T(n-2) + 2$$

Substitute value of $T(n-2)$ in above:

$$T(n) = [T(n-3) + 1] + 2$$

$$T(n) = T(n-3) + 3$$

·
·
·

In general, $T(n) = T(n-k) + k \dots (2)$

The recursion will end when the subproblem size reaches the base case, i.e. when $n-k = 0$, which means when $n = k$. Substitute the value of k in (2) and solve:

$$T(n) = T(n-n) + n$$

$$= T(0) + n$$

$$= 1 + n$$

We have now solved the recurrence relation, drop the constant, we get **$O(n)$**

What is $T(n-1)$?

We know that $T(n) = T(n-1) + 1 \dots (1)$

➤ Put $n=n-1$ in (1) :

$$\begin{aligned} T(n-1) &= T(n-1-1) + 1 \\ &= T(n-2) + 1 \end{aligned}$$

➤ Put $n=n-2$ in (1) :

$$\begin{aligned} T(n-2) &= T(n-2-1) + 1 \\ &= T(n-3) + 1 \end{aligned}$$



Work Integrated Learning Programmes

Example 2 – Factorial using Recursion



Iterative substitution method

```
func factorial(n)
    if(n == 1)
        return 1
    return n * factorial(n-1)
end
```

function $T(n)$ is not actually for calculating the value of an factorial but it is telling you about the time complexity of the factorial algorithm.

$$T(n) = \begin{cases} 1 & ; n = 1 \text{ // base case} \\ 1 + T(n-1) & ; n > 1 \text{ // recursive case} \end{cases}$$

Let us solve this recurrence relation using iterative substitution method!!

Find attached CS#2 - PDF for solution. Complexity is $O(n)$

The Master's Method



- Each of the methods described above for solving recurrence equations is ad hoc and requires mathematical sophistication in order to be used effectively.
- The masters method is like a “cook-book” / A utility method for analysis recurrence relations.
- Useful in many cases for divide and conquer algorithms.
- They come handy when the recurrence relation is of the form :

$$T(n) = aT(n/b) + f(n) \text{ where } a \geq 1, b > 1, n \geq d$$

- n = the size of the current problem and $d \geq 1$ is a constant
- a = the number of sub-problems in the recursion.
- n/b = the size of each sub problem
- $f(n)$ = the cost of work that has to be done outside the recursive calls (Cost of dividing + merging).

29



The Master Theorem

The master method for solving such recurrence equations involves simply writing down the answer based on whether one of the three cases applies. Each case is distinguished by comparing $f(n)$ to the special function $n^{\log_b a}$

1. If there is a small constant $\epsilon > 0$, such that $f(n)$ is $O(n^{\log_b a - \epsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$.
2. If there is a constant $k \geq 0$, such that $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$.
3. If there are small constants $\epsilon > 0$ and $\delta < 1$, such that $f(n)$ is $\Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq \delta f(n)$, for $n \geq d$, then $T(n)$ is $\Theta(f(n))$.

Case 1 characterizes the situation where $f(n)$ is polynomially smaller than the special function, $n^{\log_b a}$. Case 2 characterizes the situation when $f(n)$ is asymptotically close to the special function, and Case 3 characterizes the situation when $f(n)$ is polynomially larger than the special function.



Simplified version of Master Theorem

$$T(n) = aT(n/b) + f(n)$$

- ✓ Where $f(n) = \theta(n^k \log^p n)$
- ✓ n = size of the problem
- ✓ a = number of subproblems in the recursion and $a \geq 1$
- ✓ n/b = size of each subproblem
- ✓ $b > 1$, $k \geq 0$ and p is a real number.

Case 1. if $\log_b a > k$, then $T(n) = \theta(n^{\log_b a})$

Case 2. if $\log_b a = k$, then

(a) if $p > -1$, then $T(n) = \theta(n^k \log^{p+1} n)$

(b) if $p = -1$, then $T(n) = \theta(n^k \log \log n)$

(c) if $p < -1$, then $T(n) = \theta(n^k)$

Case 3. if $\log_b a < k$, then

(a) if $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$

(b) if $p < 0$, then $T(n) = \theta(n^k)$

Simplified version Case 1 Example



Case 1: if $\log_b a > k$, then $T(n) = \theta(n^{\log_b a})$

Consider $T(n) = 2T(n/2) + 1$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 2$, $b = 2$, $f(n) = 1$ which can happen when $\theta(n^0 \log^0 n)$. So, $k=0$ and $p=0$

Step 02: $\log_b a = \log_2 2 = 1$ and $k = 0$

Step 03: From step 02, $\log_b a > k$

Step 04: From SMT (Simplified Master theorem) Case 1:

$$T(n) = \theta(n^{\log_b a})$$

Hence, $T(n) = \theta(n^1)$

$$= \theta(n)$$

Simplified version Case 2 (a) Example



Case 2: if $\log_b a = k$, then
(a) if $p > -1$, then $T(n) = \theta(n^k \log^{p+1} n)$

Consider $T(n) = 2 T(n/2) + n$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 2$, $b = 2$, $k=1$ and $p=0$

Step 02: $\log_b a = \log_2 2 = 1$ and $k = 1$

Step 03: From step 02, $\log_b a = k$ so goto case 2

Step 04: $p > -1$ so goto 2(a) of simplified masters theorem.

So, $T(n) = \theta(n^k \log^{p+1} n)$

Hence, $T(n) = \theta(n^1 \log^{0+1} n)$
 $= \theta(n \log n)$

Simplified version Case 2 (b) Example



Case 2 : if $\log_b a = k$, then
(b) if $p = -1$, then $T(n) = \theta(n^k \log \log n)$

Consider $T(n) = 2 T(n/2) + n/\log n$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 2$, $b = 2$,

$n/\log n$ is $n^1 \log^{-1} n$ so $k = 1$ and $p = -1$

Step 02: $\log_b a = \log_2 2 = 1$ and $k = 1$

Step 03: From step 02, $\log_b a = k$ so goto Case 2

Step 04: $p = -1$ So, goto simplified Masters Theorem Case 2b.

So, $T(n) = \theta(n^k \log \log n)$

Hence, $T(n) = \theta(n^1 \log \log n)$
 $= \theta(n \log \log n)$

Simplified version Case 2 (c) Example



Case 2 : if $\log_b a = k$, then
(c) if $p < -1$, then $T(n) = \theta(n^k)$

Consider $T(n) = 2 T(n/2) + n/\log^2 n$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 2$, $b = 2$,

$n/\log^2 n$ is $n^1 \cdot \log^{-2} n$ so $k = 1$ and $p = -2$

Step 02: $\log_b a = \log_2 2 = 1$ and $k = 1$

Step 03: From step 02, $\log_b a = k$ so goto Case 2

Step 04: $p < -1$ So, goto simplified Masters
Theorem Case 2c.

So, $T(n) = \theta(n^k)$

Hence, $T(n) = \theta(n^1)$
 $= \theta(n)$

Simplified version Case 3 (a) Example



Case 3 : if $\log_b a < k$, then
(a) if $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$

Consider $T(n) = 4T(n/2) + n^3 \log n$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 4$, $b = 2$, $k = 3$ and $p = 1$

Step 02: $\log_b a = \log_2 4 = 2$ and $k = 3$

Step 03: From step 02, $\log_b a < k$ so goto Case 3

Step 04: $p > 0$ So, goto simplified Masters Theorem Case 3a.

So, $T(n) = \theta(n^k \log^p n)$

Hence, $T(n) = \theta(n^3 \log^1 n)$
 $= \theta(n^3 \log n)$

Simplified version Case 3 (b) Example



Case 3 : if $\log_b a < k$, then
(b) if $p < 0$, then $T(n) = \theta(n^k)$

Consider $T(n) = 2 T(n/2) + n^2/\log^2 n$

Solution:

$T(n) = aT(n/b) + f(n)$ where $f(n) = \theta(n^k \log^p n)$

Step 01: Here, $a = 2$, $b = 2$, $k = 2$ and $p = -2$ (because $n^2 \cdot \log^{-2} n$)

Step 02: $\log_b a = \log_2 2 = 1$ and $k = 2$

Step 03: From step 02, $\log_b a < k$ so goto Case 3

Step 04: $p < 0$ So, goto simplified Masters Theorem Case 3b.

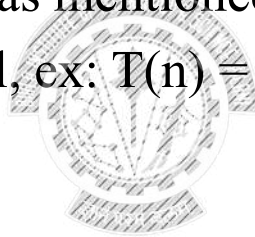
So, $T(n) = \theta(n^k)$

Hence, $T(n) = \theta(n^2)$
 $= \theta(n^2)$

Master Method Capability



- Not all recurrence relations can be solved with the use of the master theorem i.e. if
 - $T(n)$ is not of the form as mentioned earlier, ex: $T(n) = \sin n$
 - $f(n)$ is not a polynomial, ex: $T(n) = 2T(n/2) + 2^n$



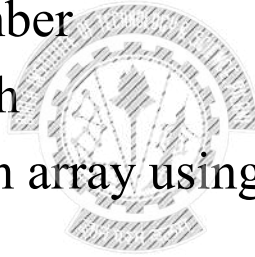
Work Integrated Learning Programmes

Exercises



Find recursive solution for following problems:

1. Return Nth Fibonacci Number
2. Power of a given number
3. Recursive array search
4. Sum of elements of an array using recursive function



Work Integrated Learning Programmes

Exercises



- ❖ Try to run the Practice sheet 1 to get a feel of order of growth,
- ❖ Solve the below recurrence relations using the iterative substitution method:

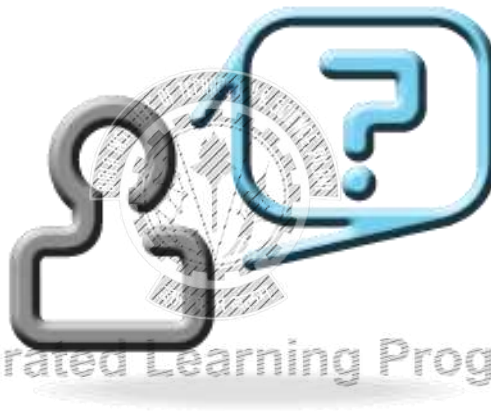
$$1) \quad T(n) = \begin{cases} 1 & ; n = 1 \\ n + T(n-1) & ; n \geq 1 \end{cases}$$

$$2) \quad T(n) = \begin{cases} 1 & ; n = 1 \\ 2T(n-1) & ; n > 1 \end{cases}$$

- ❖ Tree using recurrence tree for: $T(n) = T(n/4) + T(n/2) + n^2$

- ❖ Solve the below recurrence relations masters theorem :

- a) $T(n) = 2T(n/2) + n$
- b) $T(n) = 9T(n/3) + n$
- c) $T(n) = 3T(n/4) + n \log n$
- d) $T(n) = 4T(n/2) + n^2$
- e) $T(n) = 4T(n/2) + n$
- f) $T(n) = 9T(n/3) + 1$



We will look at Linear Data structures in the next class



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in



Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

DSECLZG519 – Contact Session #3

Linear Data Structures

2

Agenda for CS #3

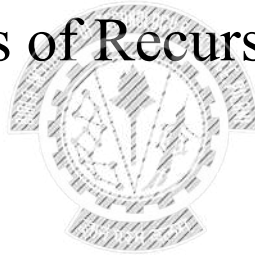


- 1) Recap of CS#2
- 2) Simplified Master's Theorem
- 3) Linear Data structures
- 4) Stacks
 - Stack implementation using array
 - Stack applications
 - Expression Evaluation
 - Infix to postfix conversion [Self-Study] – Available in appendix of this presentation
- 5) Queues
 - Queue implementation using array
 - Circular Queue
 - Applications of Queue
- 6) Linked List
 - Singly Linked list
 - Doubly Linked list – Insertion & Deletion
- 7) Stack & Queue implementation using linked list
- 8) A word on Amortized analysis
- 9) Q&A

Recap of CS#2



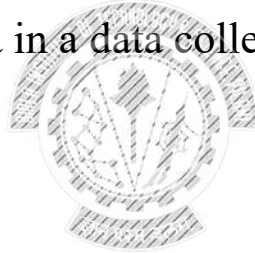
- General Rules and Problems
- Mathematical analysis of Non-recursive algorithms
- Mathematical analysis of Recursive algorithms
 - Iteration vs Recursion
 - Recurrence Tree
 - Iterative Substitution
- Masters Theorem
 - Masters method with an example for each case



Operations on Data



- Typical operations on data:
 - Add data to a data collection (insert)
 - Remove data from a data collection (delete)
 - Ask questions about the data in a data collection (search, exists, update etc.)



Work Integrated Learning Programmes

Linear Data Structures



- Data is organized in a linear fashion.

- Simple ADTs, example :

- *Stack*
- *Queue*
- Vector
- Lists
- Sequences



All these are called *Linear Data Structures*

Non-linear Data Structures



- Data is not organized in a linear fashion.
- Relationship exists among data.

➤ Examples :

- Trees
- Graphs

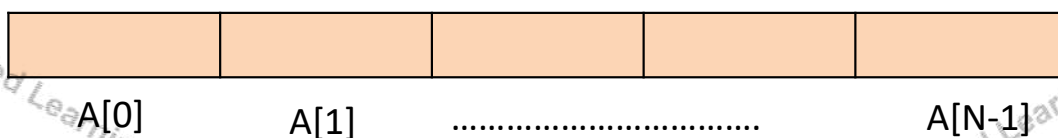


Work Integrated Learning Programmes

Arrays



- Array is a finite set of homogeneous elements stored in adjacent/contiguous memory locations. It is represented by a single name.
- Each location element is identified by a single name with its index. The first element is $A[0]$, the second element is $A[1]$ and so on ..
- If there are N elements in the array, the last element will be $A[N-1]$.

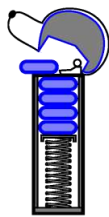


Stacks



- A **stack** is a container of objects that are inserted and removed according to the last-in-first-out (**LIFO**) principle.
- Objects can be inserted at any time, but only the last (the most-recently inserted) object can be removed.
- Inserting an item is known as “**pushing**” onto the stack. “**Popping**” off the stack is synonymous with removing an item.
- Pushing and popping happens only in **one end** called the top.

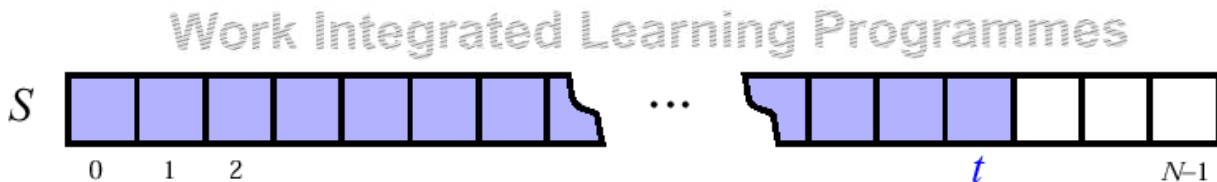
Stacks





Stacks: An Array Implementation

- Create a stack S using an array by specifying a maximum size N for our stack.
- The stack consists of an N -element array S and an integer variable t , the index of the top element in array S .



- Array indices start at 0, so we initialize t to -1



Stacks: An Array Implementation

Pseudo code

```
Algorithm size()
return t+1
```

10	20	33	41		
0	1	2	3	4	5

```
Algorithm isEmpty()
if t == -1
```

```
    return true
```

```
return false
```

t=-1

0	1	2	3	4	5

```
Algorithm top()
if isEmpty() then
    return Error
return S[t]
```

```
Algorithm push(o)
if size() == N then
    return Error/Overflow
t=t+1
S[t]=o
```

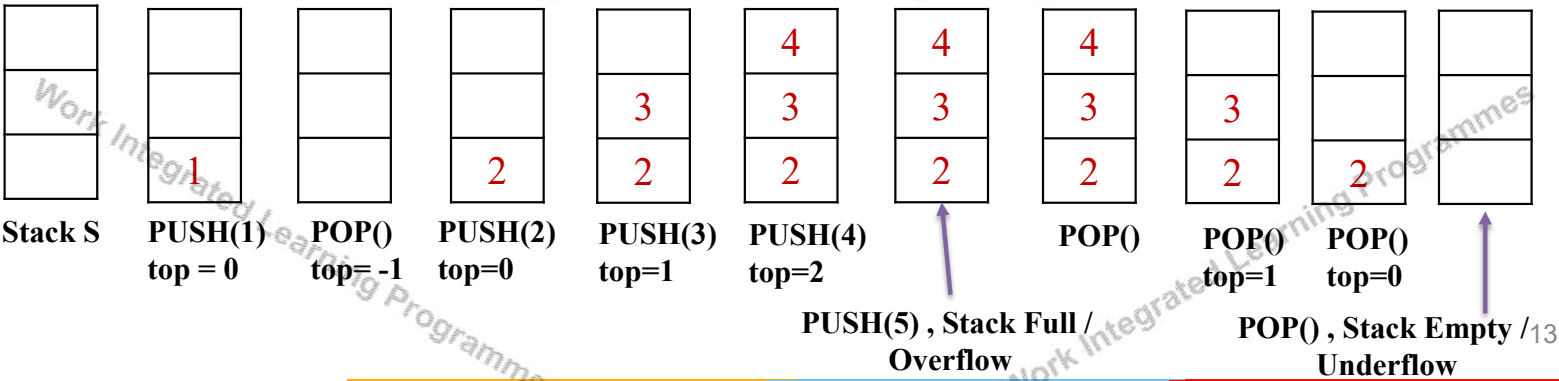
```
Algorithm pop()
if isEmpty() then
    return Error/Underflow
e= S[t]
S[t] = null
t=t-1
return e
```


Operations on Stack Demo



Consider array based implementation of stack S. Assume that the maximum size of the stack is 3 elements. Show the contents of the STACK (trace through) at each step, for the following sequence of operations. Mention exceptions like empty/full if any.

Operations: PUSH (1), POP, PUSH (2), PUSH (3), PUSH (4), PUSH (5), POP, POP, POP, POP





Stacks: An Array Implementation

The array implementation is simple and efficient (methods performed in $O(1)$).

Disadvantage

- There is an upper bound, N , on the size of the stack.
- The arbitrary value N may be too small for a given application
OR a waste of memory.

Applications of Stack



- Stacks can be used for expression evaluation. *[In class]*
- Stacks can be used for Conversion from one form of expression to another. *[In Appendix of this presentation]*
- Stacks can be used to check parenthesis matching in an expression. *[Self-study/Webinar]*
- Stack data structures are used in backtracking problems (recursion). *[Explore!]*

Forms of Expression



Arithmetic expressions can be written in 3 different forms or notations :

- **Infix Notation** : Notation in which the operator symbol is placed in between its operands. Ex : $A + B$, $C - D$, $A * D$ etc.
- **Pre-fix Notation** : It is also called as Polish notation and refers to the notation in which the operator symbol is placed **before** its operands (*when operator precedes the operands*). Ex : $+AB$, $-CD$, $*AD$ etc.
- **Post-fix Notation** : It is also called as reverse polish notation and refers to the notation in which the operator symbol is placed **after** its operands (*when operator follows the operands*). Ex : $AB+$, $CD-$, $AD*$

Algorithm: Evaluation of Postfix expression using Stack



Algorithm:

- 1) Add a right parentheses “)” at the end of the arithmetic expression F
 - 2) Scan F from left to right and repeat step 3 and step 4 for each element of F until the sentinel “)” is encountered.
 - 3) If an operand is encountered, put it onto stack
 - 4) If an operator $\otimes$ is encountered, then :
 - a. Remove the 2 top elements from the stack, where n1 is the top element and n2 is the next-to-top element.
 - b. Evaluate $n2 \otimes n1$
 - c. Place the result of (b) back on stack.
- $\otimes$: is any operator
- Endif
- End of step 2 loop
- 5) Set value equal to the top element of stack.
 - 6) Exit

Example: Evaluation of Postfix expression using stack



Evaluate $AB+C-BA+C^{\wedge}-$ where $A=1, B=2$ and $C=3$

→ $1\ 2\ +\ 3\ -\ 2\ 1\ +\ 3\ ^{\wedge}\ -\)$

Symbol Encountered	N1 (top)	N2(2 nd top)	Value = N2 op N1	STACK
1				1
2				1 2
+	2	1	$1+2=3$	3
3				3 3
-	3	3	$3-3=0$	0
2				0 2
1				0 2 1
+	1	2	$2+1=3$	0 3
3				0 3 3
^	3	3	$3^3=27$	0 27
-	27	0	$0-27=-27$	-27
)	-	-	-27	Empty

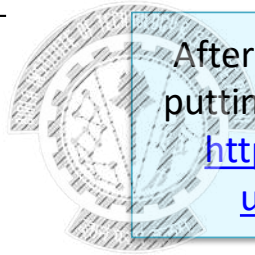
Hence, the evaluation of this expression leads to the answer **-27**

Exercises



Evaluate the following postfix expressions :

- $ABC+*CBA-+*$ where $A=1, B=2$ and $C=3$
- $5\ 7\ 5\ -\ *\ 12\ 4\ *\ 24\ /\ 6\ +\ +$



After you solve it, verify your answer by putting the postfix expression in this tool

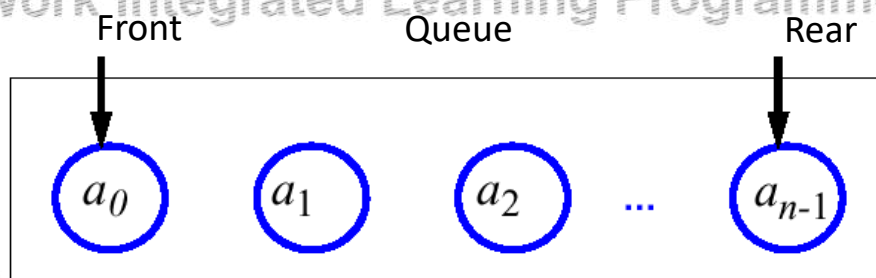
<https://www.free-online-calculator-use.com/postfix-evaluator.html>

Work Integrated Learning Programmes

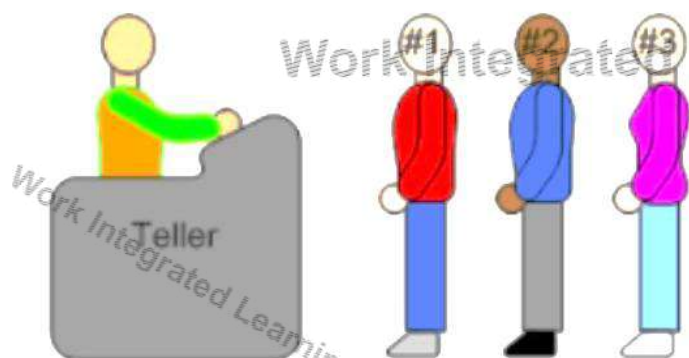
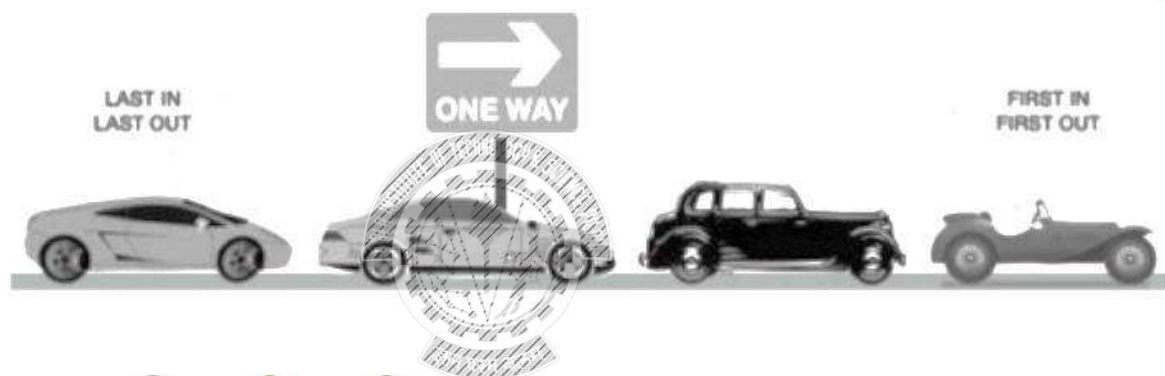
Queues



- A queue differs from a stack in that its insertion and removal routines follow the **first-in-first-out** (FIFO) principle.
- Elements may be inserted at any time, but only the element which has been in the queue the longest may be removed.
- Elements are inserted at the **rear** (enqueued) and removed from the **front** (dequeued)



Queues



Queues



The **queue** supports three fundamental methods:

- **Enqueue($S:ADT, o:element$):** - Inserts object o at the rear of the queue; an error occurs if the queue is full
- **Dequeue($S:ADT$):** - Removes the object from the front of the queue; an error occurs if the queue is empty
- **Front($S:ADT$): $element$** - Returns, but does not remove, the front element; an error occurs if the queue is empty



Queues: An Array Implementation

- Create a queue using an array
- A maximum size N is specified.
- The queue consists of an N -element array Q and two integer variables:
 - f , index of the front element (which is the candidate to be removed by a dequeue operation)
 - r , index of the next available array cell (cell which can be used in enqueue operation)
 - Initially, $f=r=0$ and the queue is empty if $f=r$



Queues



Disadvantage

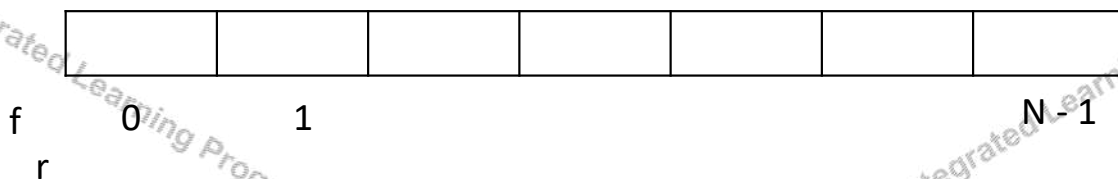
Repeatedly enqueue and dequeue a single element N times.

Finally, $f=r=N$.

- No more elements can be added to the queue, *though there is space in the queue !!*

Solution

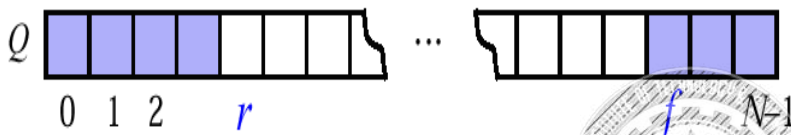
Let f and r wraparound the end of queue (circular queue).



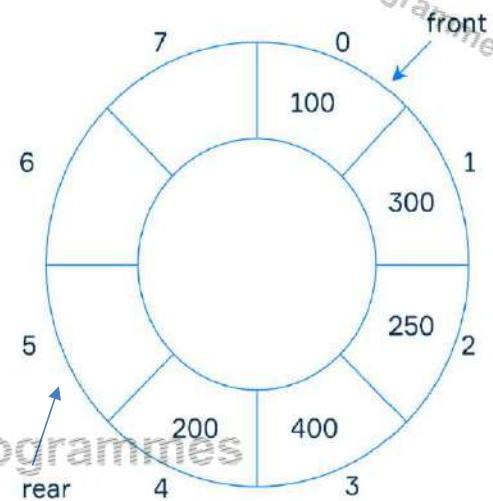
Queues: An Array Implementation



- “wrapped around” configuration



- Each time r or f is incremented, compute this increment as $r = (r+1) \bmod N$ or $f = (f+1) \bmod N$
- When array is full $f = r$ (same as empty condition), then how do we know if it's a queue full or queue empty? There are many ways to handle this (Explore this!!). Simple approach to handle this, insist that Q can never hold more than $N-1$ objects (This is your textbook way but not a good way ☹).



Queues: An Array Implementation



Pseudo code

```
Algorithm size()  
return  $(N - f + r) \bmod N$ 
```

```
Algorithm isEmpty()  
return  $(f = r)$ 
```

```
Algorithm front()  
if isEmpty() then  
    return Error  
return  $Q[f]$ 
```

```
Algorithm dequeue()  
if isEmpty() then  
    return Error
```

```
     $o = Q[f]$   
     $Q[f] = \text{null}$   
     $f = (f + 1) \bmod N$   
    return  $o$ 
```

```
Algorithm enqueue(o)  
if  $\text{size()} = N - 1$  then  
    return Error
```

```
     $Q[r] = o$   
     $r = (r + 1) \bmod N$ 
```


Applications of Queue



- Used in CPU scheduling
- Used in Disk scheduling
- Priority queue is used in heaps [We will look at this in later session]
- ...
- ...



Work Integrated Learning Programmes

Arrays: Pluses and minuses



- + Fast element access.
- Impossible to resize.

- Many applications require resizing!
- Required size not always immediately available.



Work Integrated Learning Programmes

List ADT



- A sequence of items where positional order matter
- $\langle a_1, a_2, \dots, a_{n-1}, a_n \rangle$
- Lists are very general in computing
- e.g. student list, list of events, list of appointments etc



Work Integrated Learning Programmes

List Operations



- Supports the following methods for a list S: refer to relative positions in the list
- first (): Return the position of the first element of S; an error occurs if S is empty.
- last() : Return the position of the last element of S; an error occurs if S is empty.
- isFirst(p) : Return a Boolean value indicating whether the given position is the first one in the list.
- islast (p)
- before(p) : Return the position of the element of S preceding the one at position p; an error occurs if p is the first position.
- after (p) ,size(),isEmpty(),insertAfter(), remove()

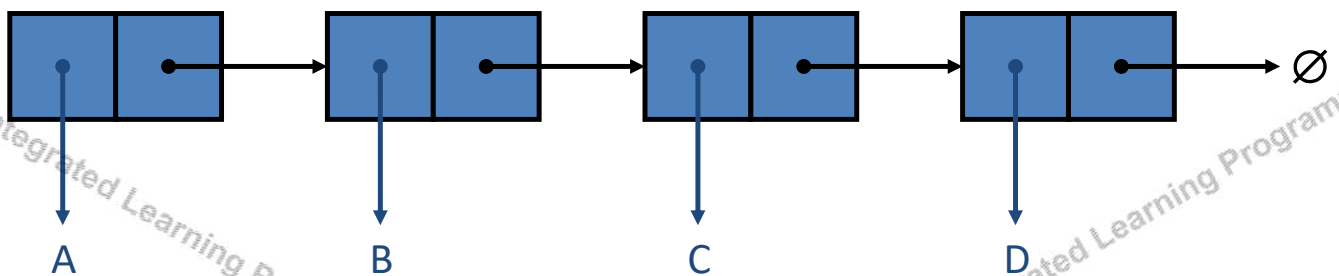
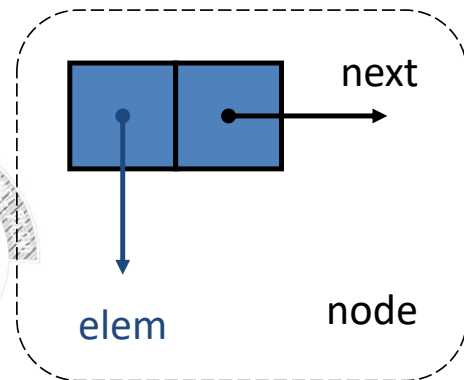
Singly Linked Lists



A singly linked list is a concrete data structure consisting of a sequence of nodes

Each node stores

- element
- link to the next node



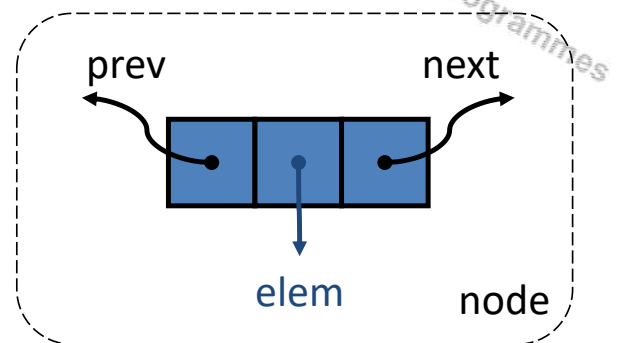
Doubly Linked List



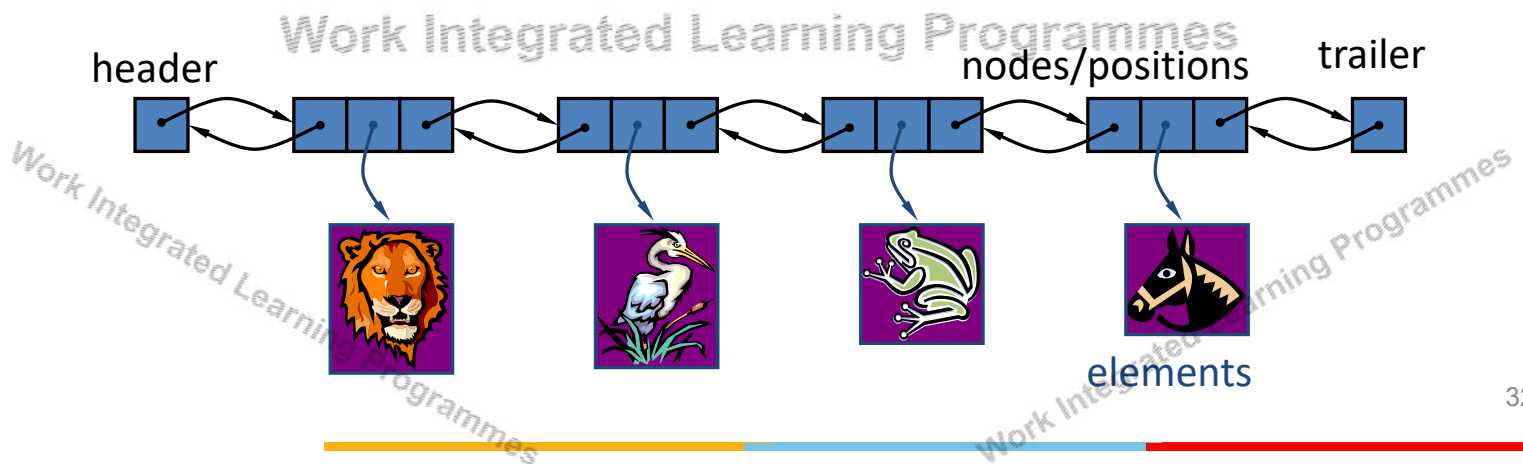
A doubly linked list is often more convenient!

Nodes store:

- element
- link to the previous node
- link to the next node



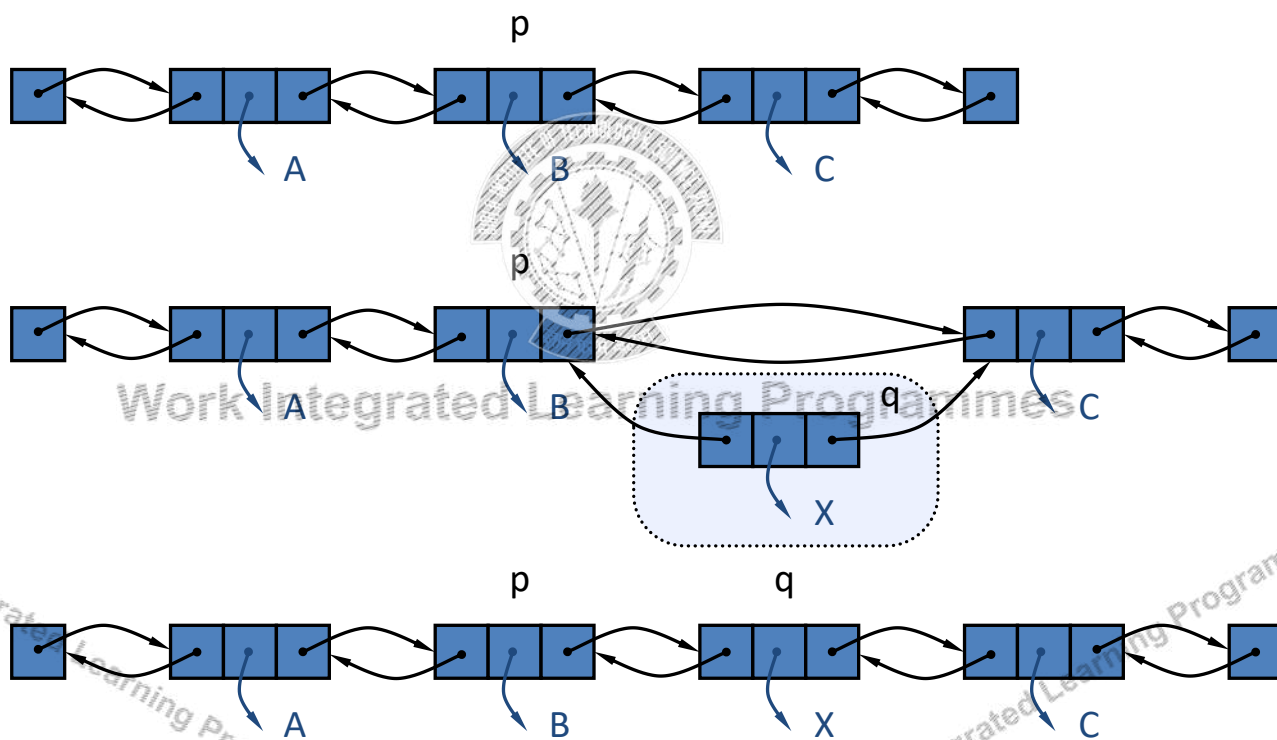
Special trailer and header nodes



Insertion



We visualize operation `insertAfter(p, X)` :



Insertion Algorithm



Algorithm insertAfter(p, e):

Create a new node v

$v.setElement(e)$

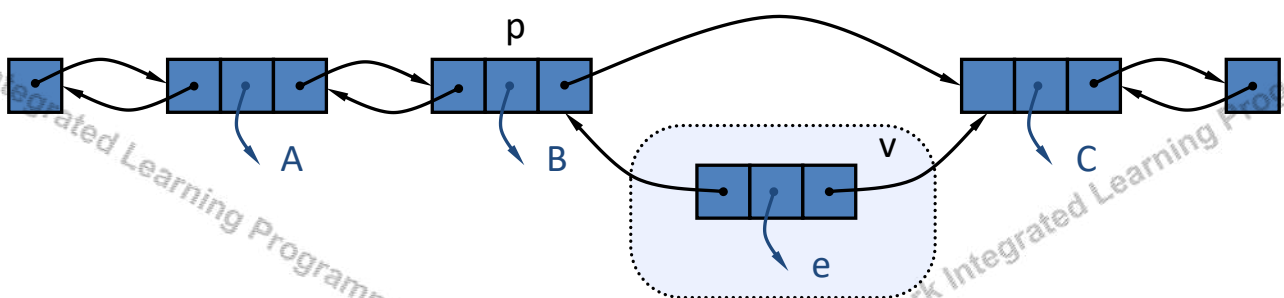
$v.setPrev(p)$ {link v to its predecessor}

$v.setNext(p.getNext())$ {link v to its successor}

$(p.getNext()).setPrev(v)$ {link p 's old successor to v }

$p.setNext(v)$ {link p to its new successor, v }

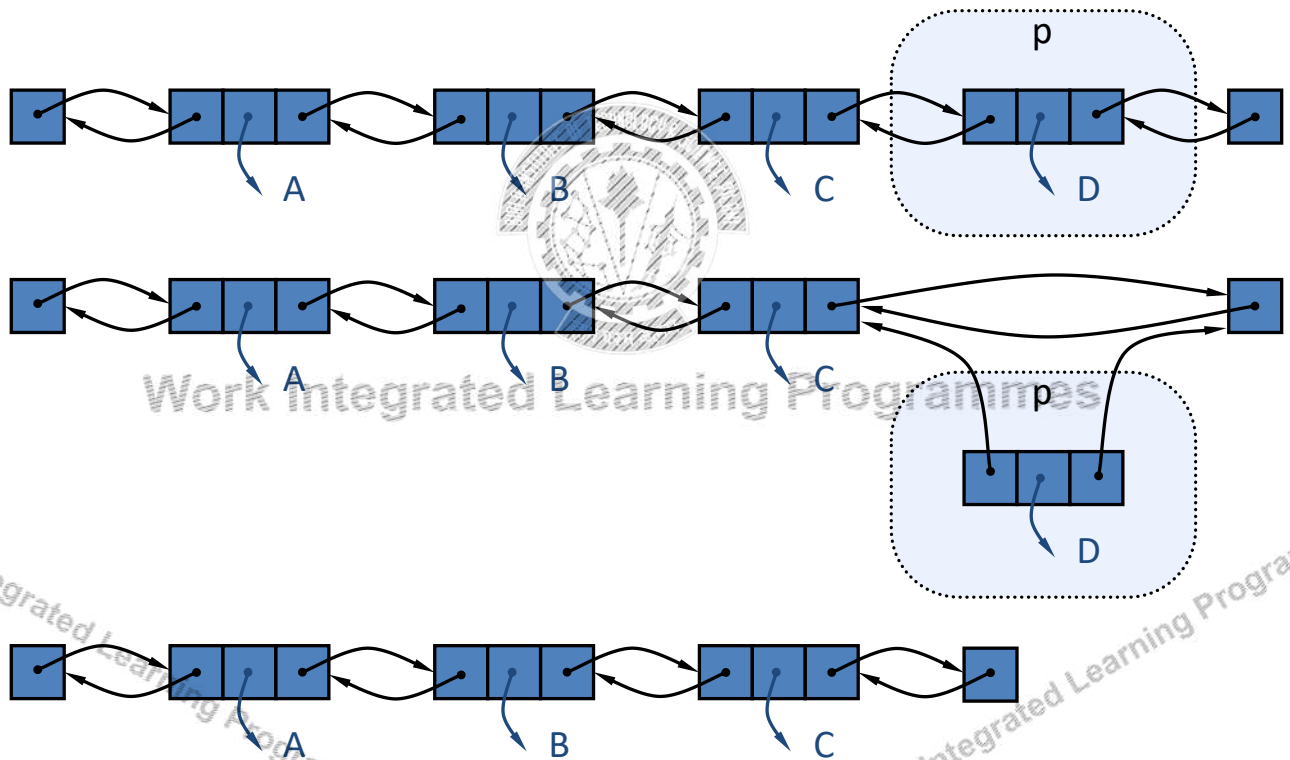
return v {the position for the element e }



Deletion



- We visualize `remove(p)`

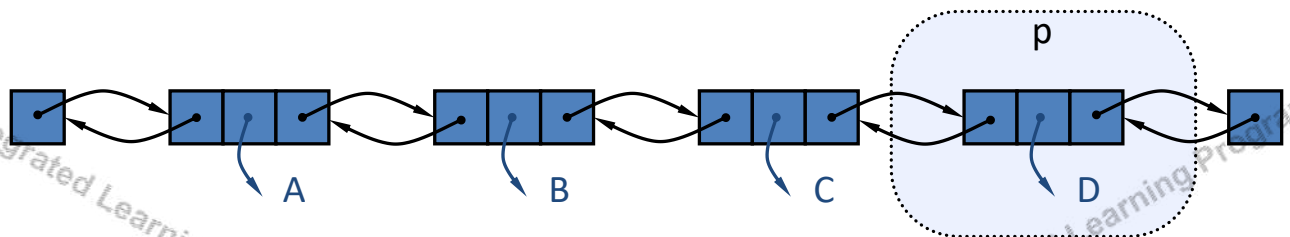


Deletion Algorithm



Algorithm remove(p):

- $t = p.\text{element}$ {a temporary variable to hold the return value}
- $(p.\text{getPrev}()).\text{setNext}(p.\text{getNext}())$ {linking out p }
- $(p.\text{getNext}()).\text{setPrev}(p.\text{getPrev}())$ {linking out p }
- $p.\text{setPrev}(\text{null})$ {invalidating the position p }
- $p.\text{setNext}(\text{null})$
- return** t

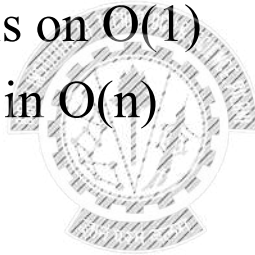


Worst-case running time



In a doubly linked list

- + insertion at head or tail is in $O(1)$
- + deletion at either end is in $O(1)$
- element access is still in $O(n)$

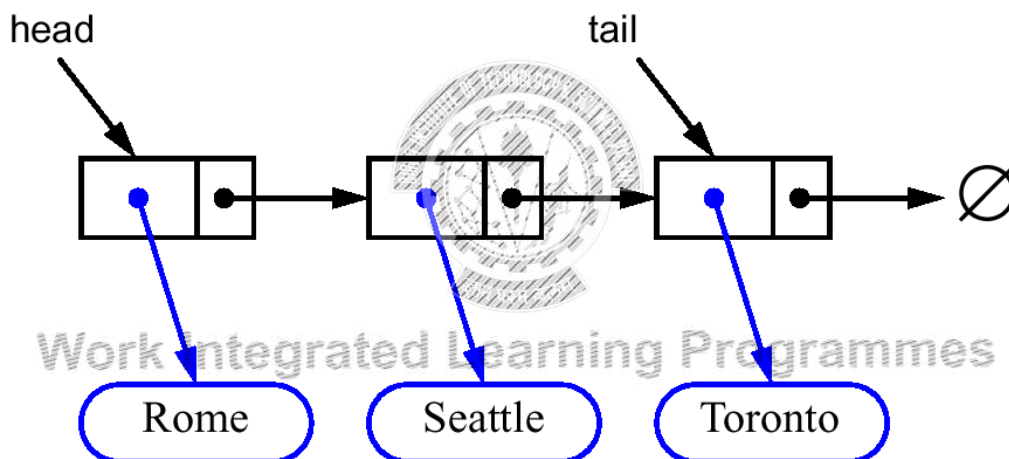


Work Integrated Learning Programmes



Stacks: Singly Linked List implementation

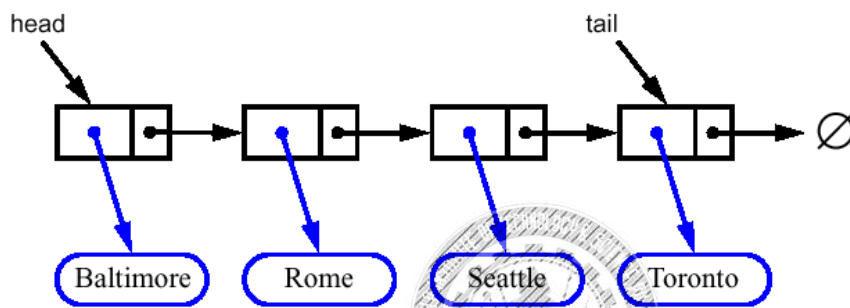
- Nodes (*data, pointer*) connected in a chain by links



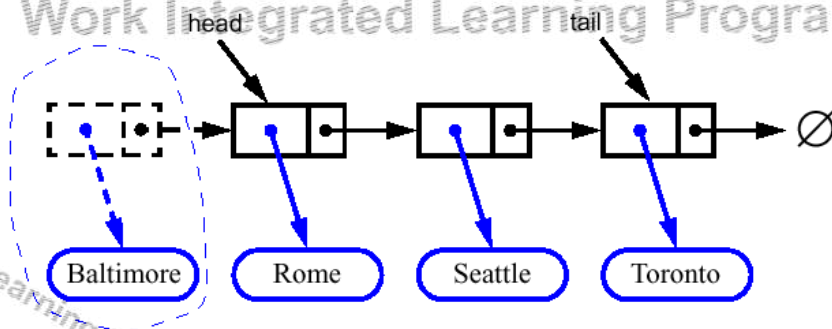
- the head or the tail of the list could serve as the top of the stack



Queues: Linked List Implementation



- Dequeue - advance head reference





Appendix : Forms of Expression



Examples :

#	Infix	Prefix	Postfix
1	$(A+B) * C$	$*+ABC$	$AB+C*$
2	$A + (B * C)$	$+A*BC$	$ABC*+$
3	$A ^ B * C - D$	$- * ^ ABCD$	$AB ^ C * D -$
4	$(A+B)/(C-D)$	$/+AB-CD$	$AB+CD- /$

- The computer usually evaluates an arithmetic expression in infix notation in 2 steps. First, it converts the expression to postfix notation, and then it evaluates the postfix expression. In both these steps, the stack is used !! Let us examine how stack is used ☺



Appendix : Algorithm: Infix to Postfix using Stack

The following algorithm transforms the infix expression Q into its equivalent postfix expression P.

- 1) Push "(" onto STACK, and add ")" to the end of Q
- 2) scan Q from left to right and repeat steps 3 to 6 for each element of Q until the stack is empty
- 3) if an operand is encountered add it to P
- 4) if a left parenthesis is encountered push it onto the stack
- 5) if an operator is encountered, then
 - (a) Repeatedly pop from STACK and add to P each operator (on top of the STACK) which has same precedence as or higher precedence than the operator encountered
 - (b) Add the encountered operator to the stack[end of IF structure]
- 6) if a right parenthesis is encountered, then:
 - (a) repeatedly pop from the stack and add to P each operator (on top of the STACK) until a left parenthesis is encountered
 - (b) remove the left parenthesis [do not add left parenthesis to P][End of IF structure]
[End of step 2 loop]
- 7) Exit

42

Appendix : Example : Converting Infix to Postfix using Stack



Consider the infix expression Q to be $A + B * C \rightarrow A+B*C)$

#	Symbol Encountered	Postfix String P (Output)	STACK Contents
1			(
2	A	A	(
3	+	A	(+
4	B	AB	(+
5	*	AB	(+*
6	C	ABC	(+*
7	)	ABC*	(+
8	-	ABC*+	(
9	-	ABC*+	Empty

Hence, the postfix expression P is **ABC*+**

Appendix : Example : Converting Infix to Postfix using Stack



Consider the infix expression Q to be $(A+B) * C \rightarrow (A+B) * C$

#	Symbol Encountered	Postfix String P (Output)	STACK Contents
1			(
2	(		((
3	A	A	((
4	+	A	((+
5	B	AB	((+
6	)	AB+	(
7	*	AB+	(*
8	C	AB+C	(*
9	)	AB+C*	Empty
10	-	AB+C*	Empty

Hence, the postfix expression P is **AB+C***

Appendix Exercise



1) Convert the following infix expressions into postfix expressions using stack (trace it as we did in class) :

- $A+(B+C)*D$
- $B+C-D/E*F$
- $(A+B)+(C/D)*E$
- $A+B*(C+D-E)*F$

After you solve it, verify your answer by putting the infix expression in this tool :

<https://www.mathblog.dk/tools/infix-postfix-converter/>



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



46

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #4

DSECLZG519 – Introduction to Tree

Work Integrated Learning Programmes

Agenda for CS #4



1) Recap of CS#3 2) Introduction to Trees

- Tree terminologies
- Binary tree and types
- Representation of tree
 - Using array
 - Using linked list
- Binary tree traversals
 - Inorder
 - Preorder
 - Postorder



3) Applications of binary trees 4) Q&A

Linear data structures



Here are some of the data structures we have studied so far:

- Arrays
- Singly-linked lists and doubly-linked lists
- Stacks and queues
- These all have the property that their elements can be adequately displayed in a straight line
- How to obtain data structures for data that have non-linear relationships?

How should one decide which data structure to use ?



- What need to be stored
- Cost of operations
- Memory use
- Ease of implementation 😊



Work Integrated Learning Programmes

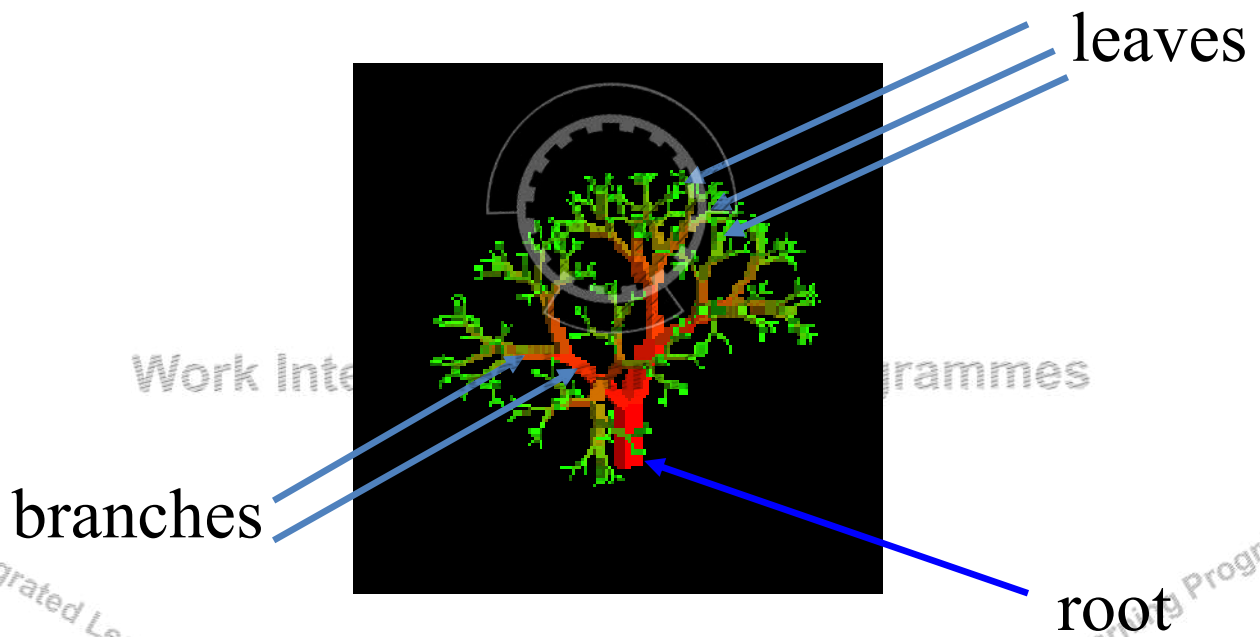
Non-linear Data Structures



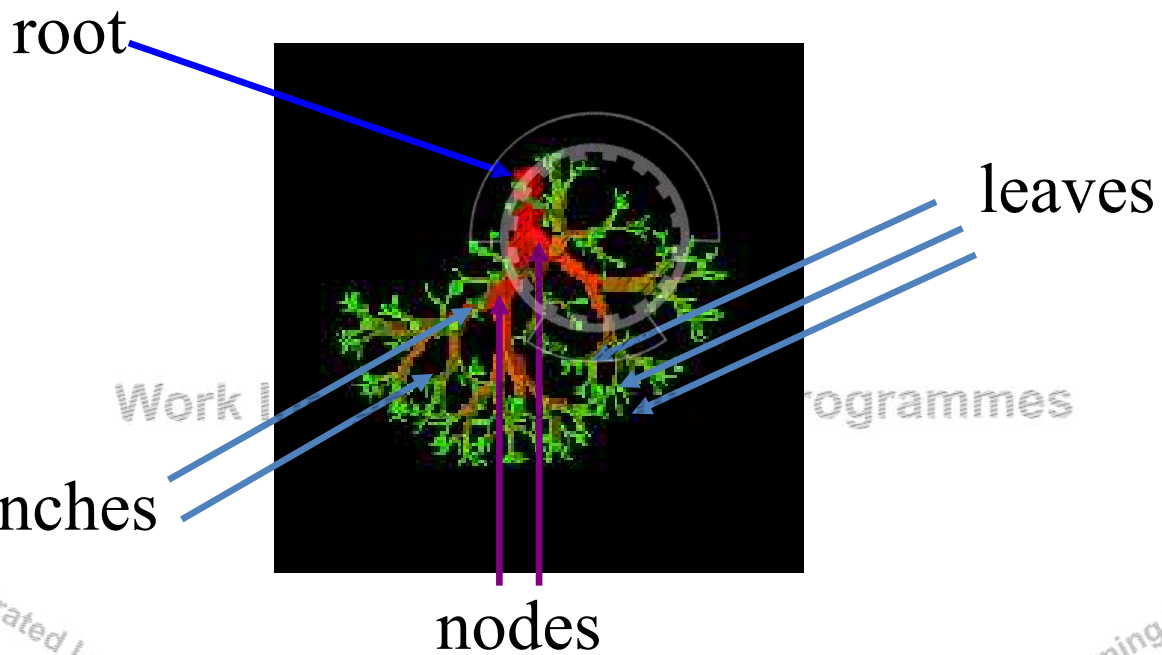
- Linear access time of linked lists is expensive
 - Does there exist any simple data structure for which the running time of most operations (search, insert, delete) is $O(\log N)$?
- Trees
 - Basic concepts
 - Tree traversal
 - Binary tree
 - Binary search tree and its operations
- Stores elements hierarchically
- Top element is called as **root**
- Example
 - Directory structure
 - Family tree
 - Organizational structure , Content of a book etc.



Natural View of a Tree



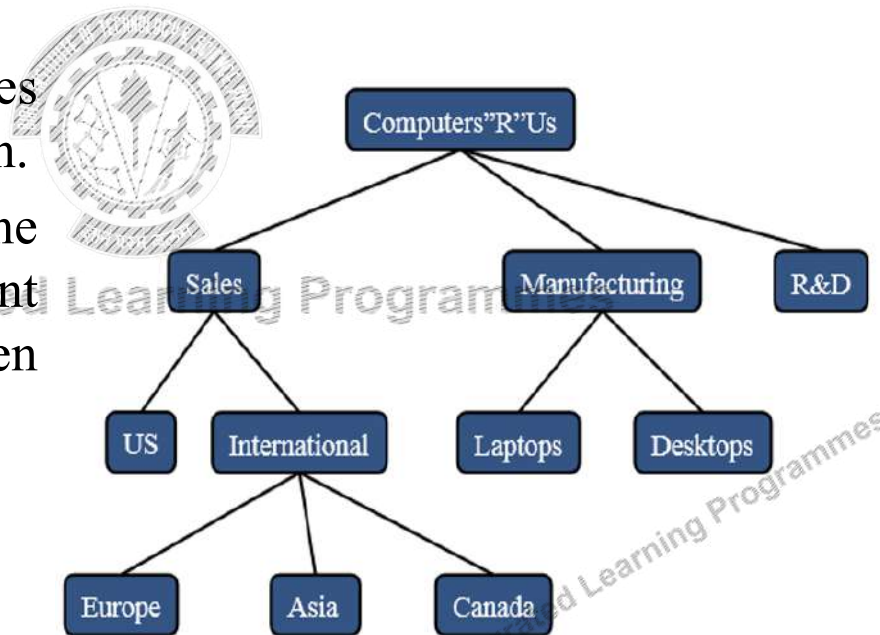
Computer Scientist's View



Tree: A Hierarchical ADT



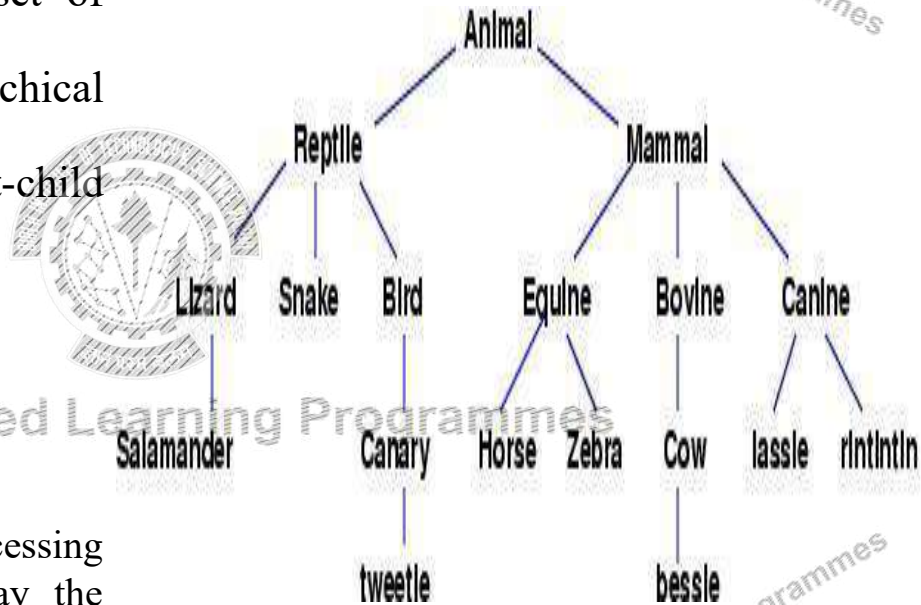
- A tree (upside down) is an abstract model of a hierarchical structure.
- A tree consists of nodes with a parent-child relation.
- Each element (except the top element) has a parent and zero or more children elements.



What is a Tree



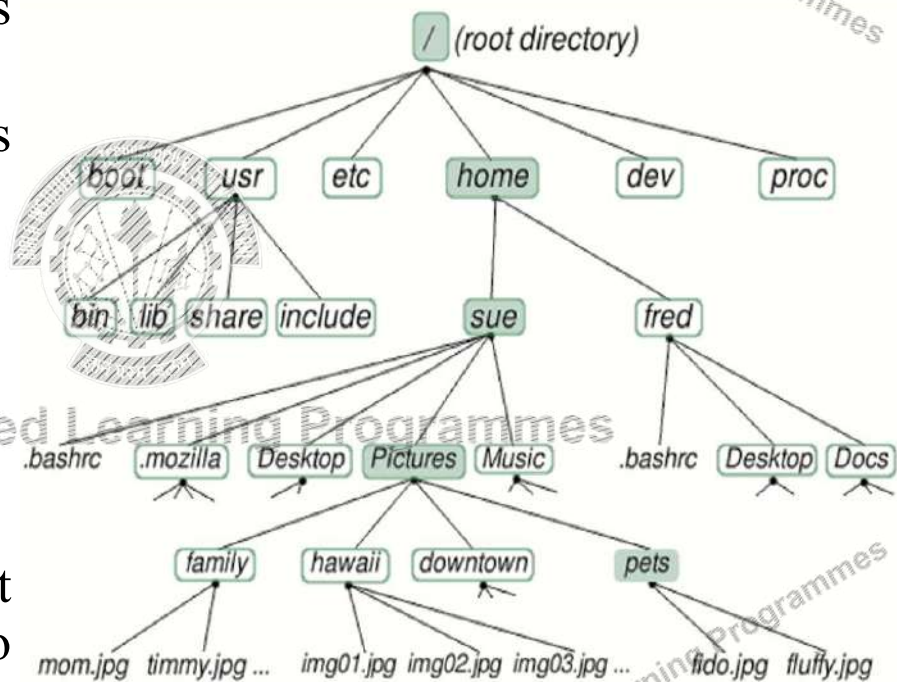
- A tree is a finite non-empty set of elements.
- It is an abstract model of a hierarchical structure.
- Consists of nodes with a parent-child relation.
- Applications:
 - Organizational data
 - File systems /storing naturally
 - Network routing algorithm.
 - Programming environments
 - Compiler Design/Text processing (syntax analysis & to display the structure of a sentence in a language)
 - Searching Algorithms and sorting
 - Evaluating a mathematical expression.



Why do we need trees ?



- Lists, Stacks, and Queues are *linear* relationships
- Information often contains hierarchical relationships:
 - File directories or folders
 - Moves in a game
 - Hierarchies in organizations
- Can build a tree to support these relationships and also support fast searching.

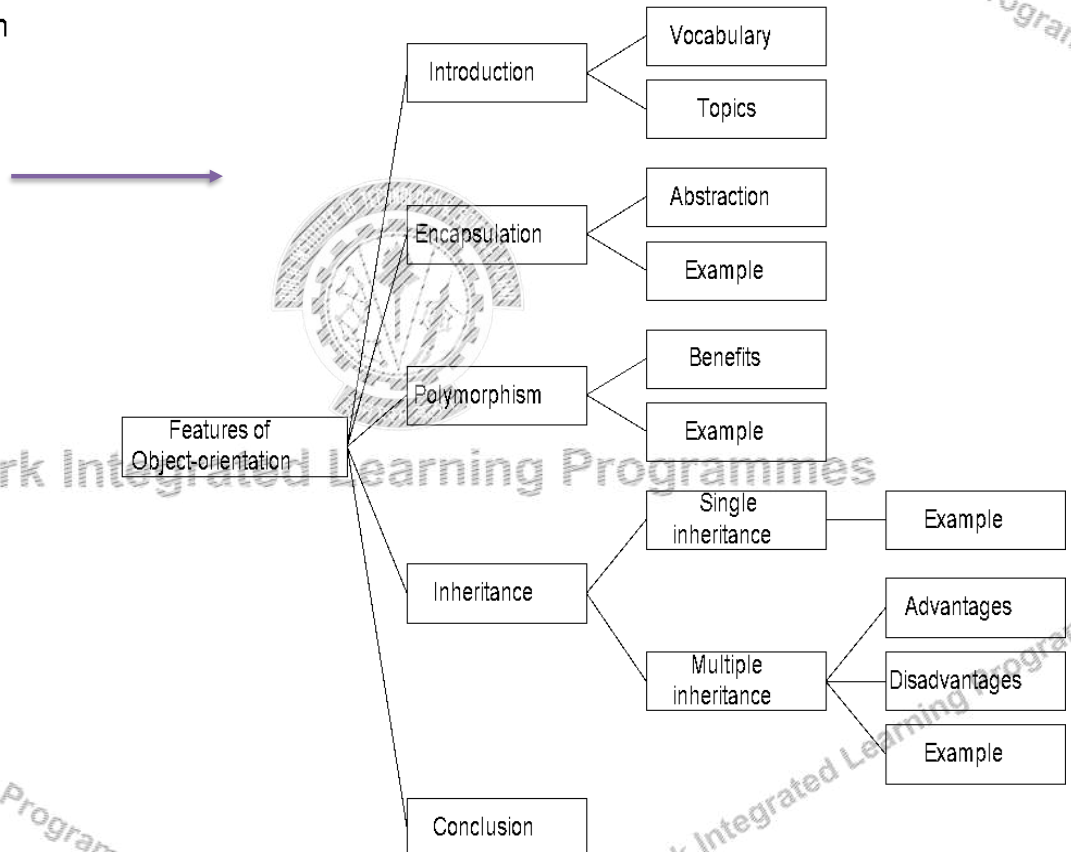


Tree Applications : Outline as Tree



Features of Object-orientation

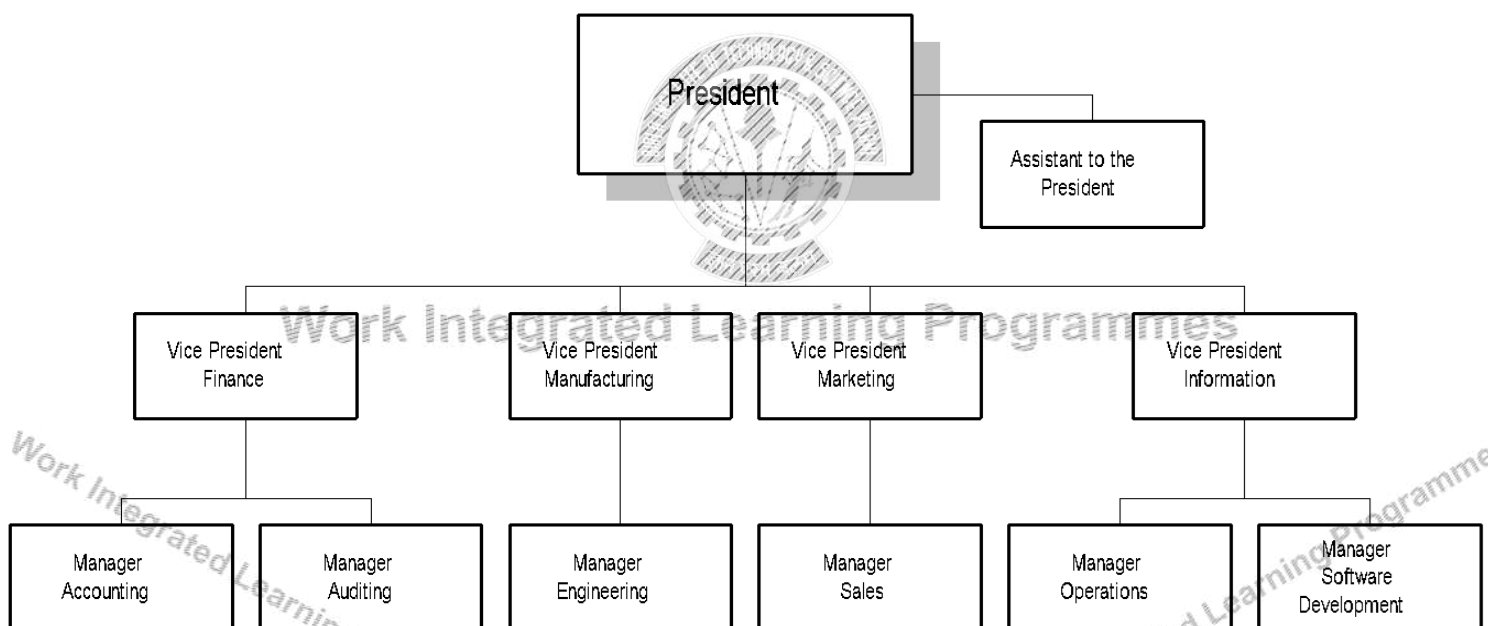
- I. Introduction
 - A. Vocabulary
 - B. Topics
- II. Encapsulation
 - A. Abstraction
 - B. Example
- III. Polymorphism
 - A. Benefits
 - B. Example
- IV. Inheritance
 - A. Single inheritance
 - 1. Example
 - B. Multiple inheritance
 - 1. Advantages
 - 2. Disadvantages
 - 3. Example
- IV. Conclusion



Tree Applications : Organizational Chart



XYZ Corporation

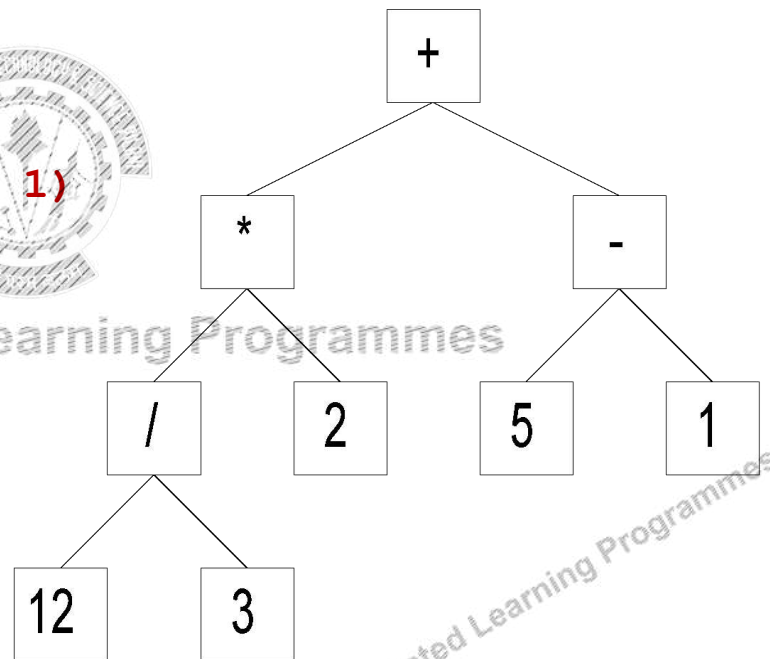


Tree Applications : Expression Tree



Can represent an arithmetic expression as a tree

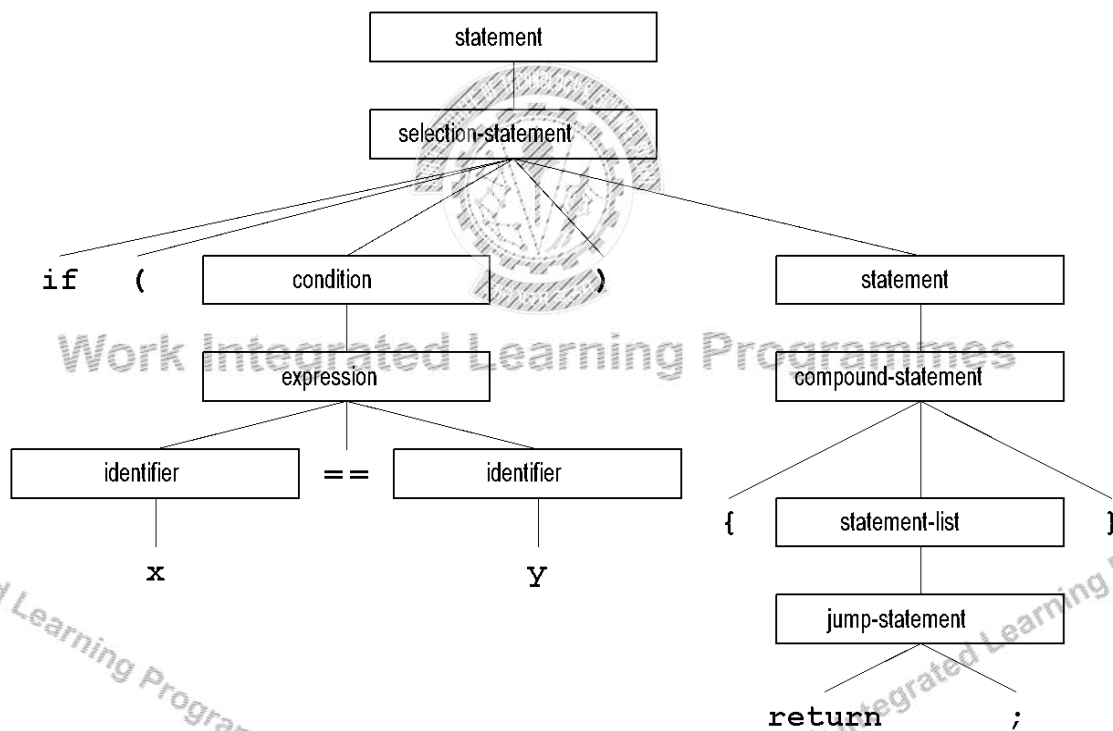
$((12 / 3) * 2) + (5 - 1)$



Tree Applications : Programs as Trees



➤ Many compilers represent programs as trees



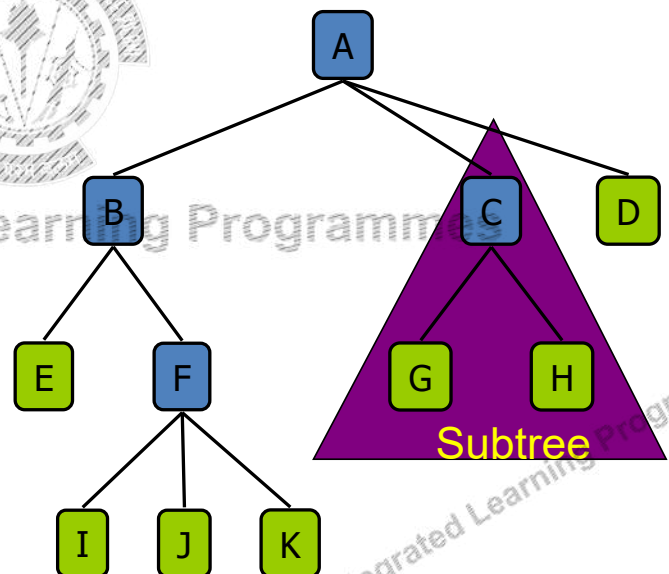
15

Tree Terminology



- ✓ **Root:** node without parent (A)
- ✓ **Siblings:** nodes share the same parent (B,C,D)
- ✓ **Internal node:** node with at least one child (A, B, C, F)
- ✓ **External node (leaf):** node without children (E, I, J, K, G, H, D)
- ✓ **Ancestors** of a node: parent, grandparent, great-grandparent, etc. (For E, B and A are ancestors).
- ✓ **Descendant** of a node: child, grandchild, great-grandchild, etc. (For B, E and F,I,J,K are descendants)
- ✓ **Degree** of a node: the number of its children (For A its 3, F its 3, C its 2).
- ✓ **Depth of x** = no of edges in path from root to x. (depth of F is 2)
- ✓ **Height of x** = no of edges in longest path from x to a leaf. (height of B is 2, height of A is 3).

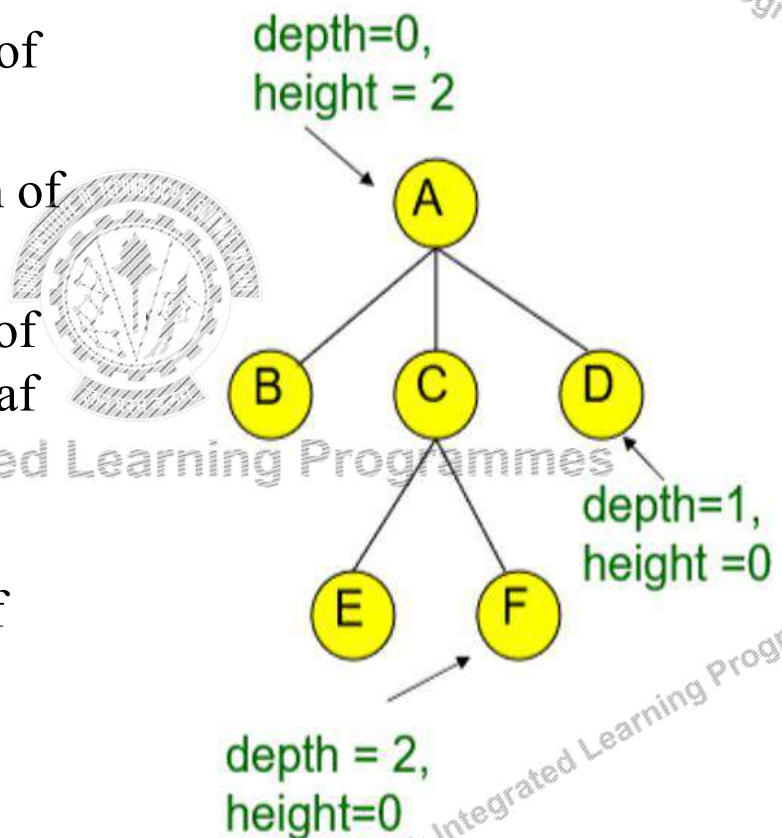
- ✓ **Sub-tree:** tree consisting of a node and its descendants
- ✓ **Level:** set of all nodes at the same depth. Root at 0.
- ✓ **Forest:** set of disjoint trees.



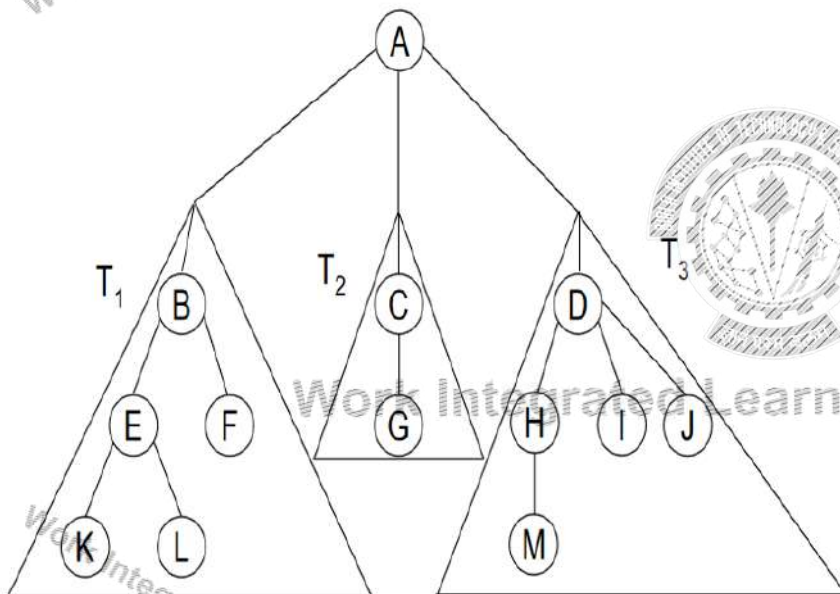
More Tree Jargon



- *Length of a path* = number of edges
- *Depth of a node N* = length of path from root to N
- *Height of node N* = length of longest path from N to a leaf
- *Depth of a tree* = depth of deepest node
- *Height of a tree* = height of root



Tree Terminology Activity



- ✓ Root ?
- ✓ Internal nodes ?
- ✓ Leaf's ?
- ✓ Ancestors of M ?
- ✓ Descendants of E ?
- ✓ Degree of A ?
- ✓ Height of tree ?
- ✓ I is in which level ?
- ✓ Depth of H ?
- ✓ Height of D ?

Tree ADT



➤ We use positions to abstract nodes

➤ **Generic methods:**

integer size(): Return the number of nodes in the tree.

boolean isEmpty() : Return if tree is empty

ObjectIterator elements(): Return an iterator of all the elements stored at nodes of the tree.

➤ **Accessor methods:**

position root(): Return the root of the tree

position parent(p): Return the parent of node p; error if p is root.

positionIterator children(p): Return an iterator of the children of node p.

Tree ADT



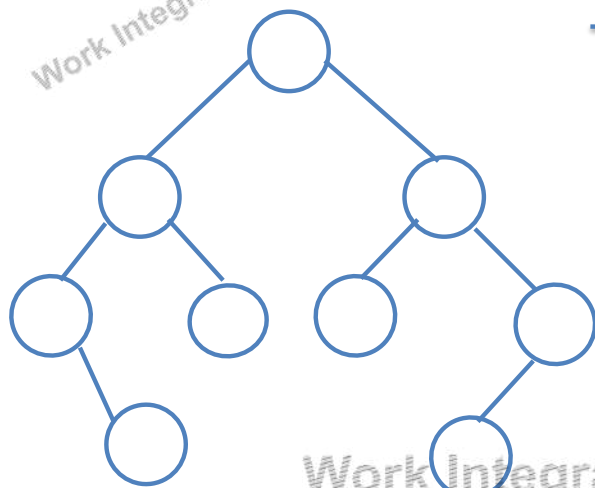
➤ Query methods:

- boolean `isInternal(v)`: Test whether node `v` is internal.
- boolean `isExternal(v)`: Test whether node `v` is external
- boolean `isRoot(v)`: Test whether node `v` is the root.

➤ Update methods:

- `swapElements(v, w)`: Swap the elements stored at the nodes `v` and `w`.
- object `replaceElement(v, e)`: Replace with `e` and return the element stored at node `v`

Binary Tree



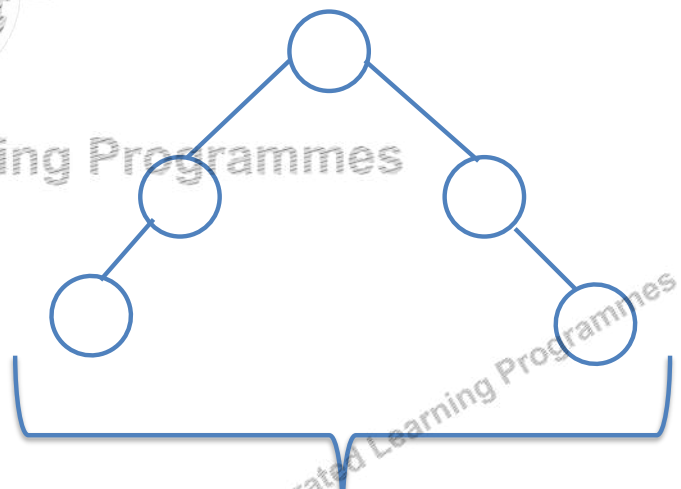
Each node can have at most 2-children

A binary tree is a tree with the following properties:

- Each internal node has **at most two** children.
- The children of a node are called as left child and right child.



This is also binary tree



What about this ?

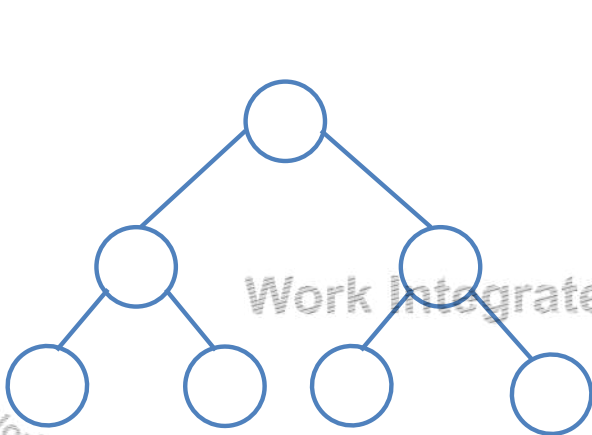
21

Strict / Proper/ Full Binary tree



Proper Binary Tree

- Each internal node has **exactly** two children



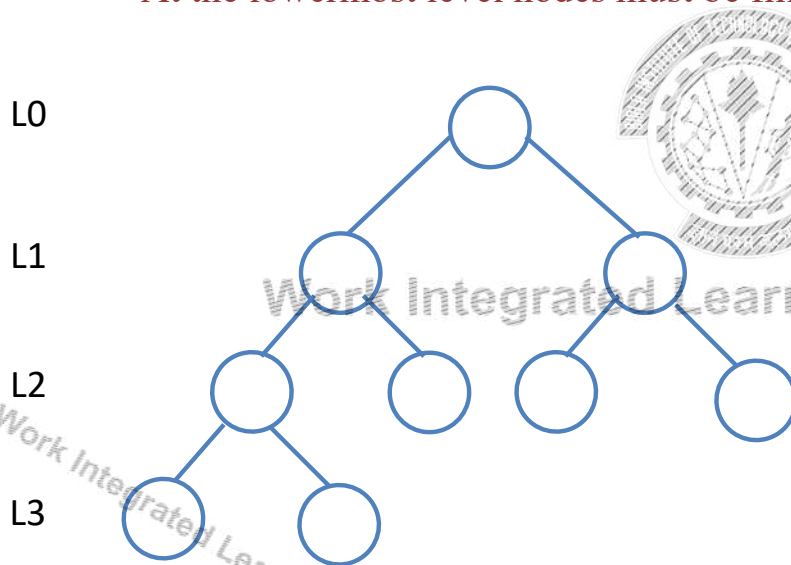
Each node can have either 2 or 0 children.

Complete Binary tree



Complete binary tree

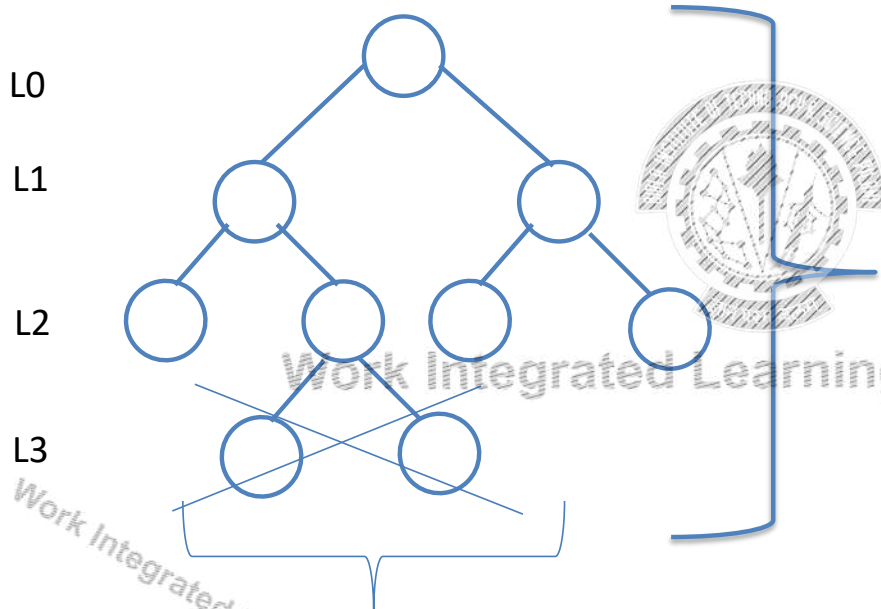
- A binary tree in which every level, except the lowermost, is completely (max) filled
- At the lowermost level nodes must be filled from left to right



All levels except possibly the last are completely filled and all nodes are as left as possible.

Max no of nodes at level $i = 2^i$

Complete Binary tree



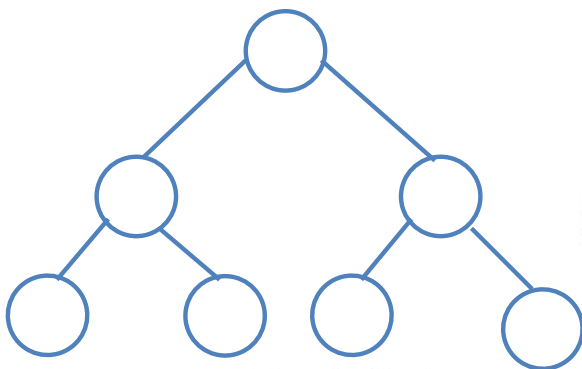
All levels except possibly the last are completely filled and all nodes are not as left as possible. So this is not a complete Binary tree

This is not the left as possible

Perfect binary tree



A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.



In perfect binary all levels will be completely filled.

Proof of complete binary can be found in the introduction to algorithms textbook

Maximum no of nodes in a binary tree with height h

$$\begin{aligned} &= 2^0 + 2^1 + \dots + 2^h \\ &= 2^{h+1} - 1 \\ &= 2^{(\text{no of levels})} - 1 \\ n &= 2^{h+1} - 1 \end{aligned}$$

- ❖ Now you can find the high of a tree, with n
 $n = 2^{h+1} - 1$
 $2^{h+1} = n+1$
 $h = \log_2(n+1) - 1$
- ❖ Height of complete binary tree $h = \text{floor}(\log_2 n)$

25

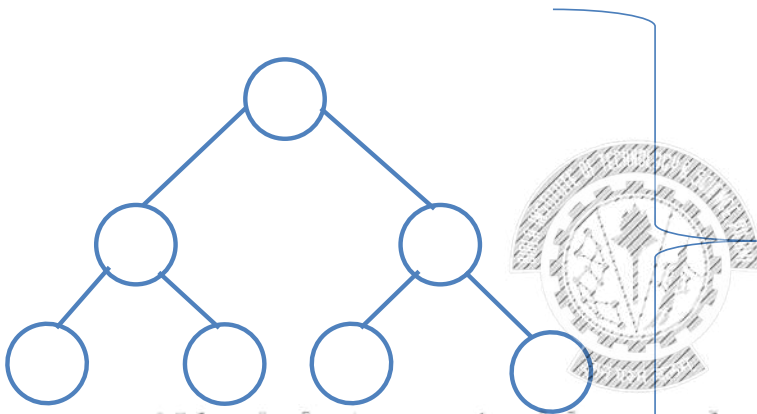
Min and max height of a binary tree



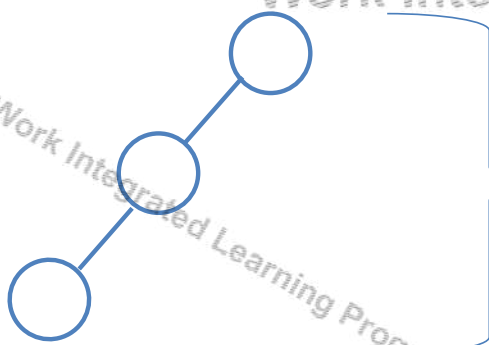
L0

L1

L2

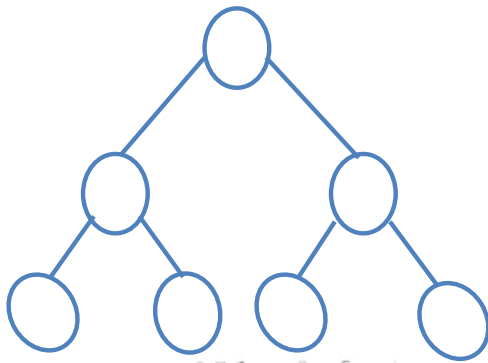


Min-height of n-nodes = floor ($\log_2 n$)



Max-height of n-nodes = n-1

Balanced binary tree



Difference between height of left & right subtree for every node is not more than k (mostly 1).

- ❖ Height $\rightarrow$ no of edges in longest path from root to a leaf.
- ❖ Height of an empty tree $= -1$
- ❖ Height of tree with one node $= 0$

Binary tree implementation



We can implement binary tree using

- Linked List [Dynamically created nodes]
- Arrays [this will be efficient only for complete binary tree]



Work Integrated Learning Programmes

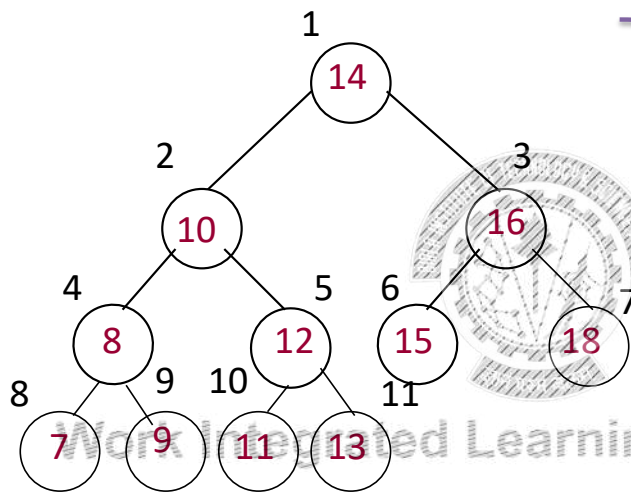
Complete Binary Trees: Array Representation



Complete Binary Trees can be represented in memory with the use of an array A so that all nodes can be accessed in $O(1)$ time:

- Root will be the first position in A
- Label nodes sequentially top-to-bottom and left-to-right
- Left child of $A[i]$ is at position $A[2i]$
- Right child of $A[i]$ is at position $A[2i + 1]$
- Parent of $A[i]$ is at $A[i/2]$

Complete Binary Trees: Array Representation



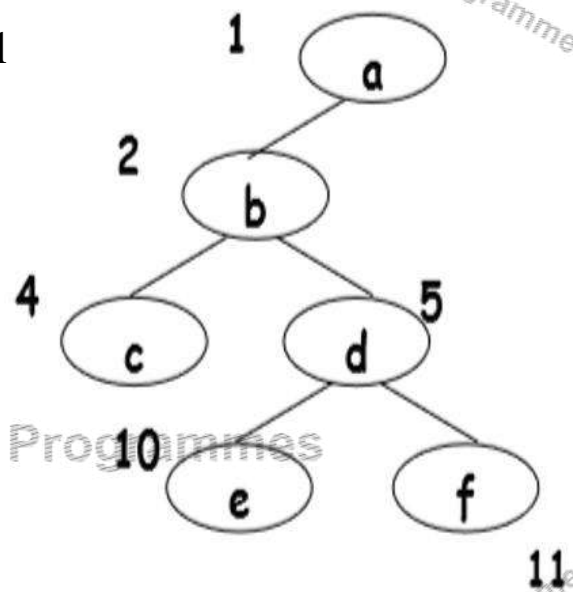
- Left child of $A[i]$ is at position $A[2i]$
- Right child of $A[i]$ is at position $A[2i + 1]$
- Parent of i is at position $\text{floor}(i/2)$

14	10	16	8	12	15	18	7	9	11	13
1	2	3	4	5	6	7	8	9	10	11

Binary Trees: Array Representation

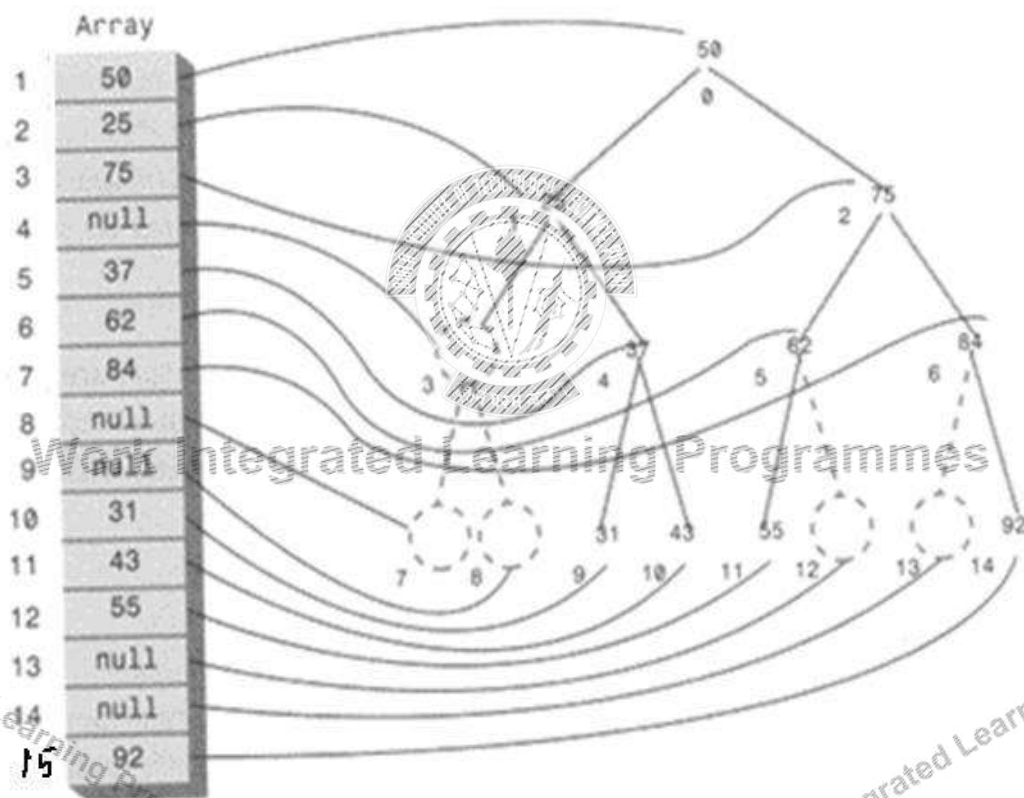


Sequential representation of a tree with **depth d** will require an array with approx. $(2^{d+1} - 1)$ elements



1	2	3	4	5	6	7	8	9	10	11
a	b		c	d					e	f

Binary Trees: Array Representation



32

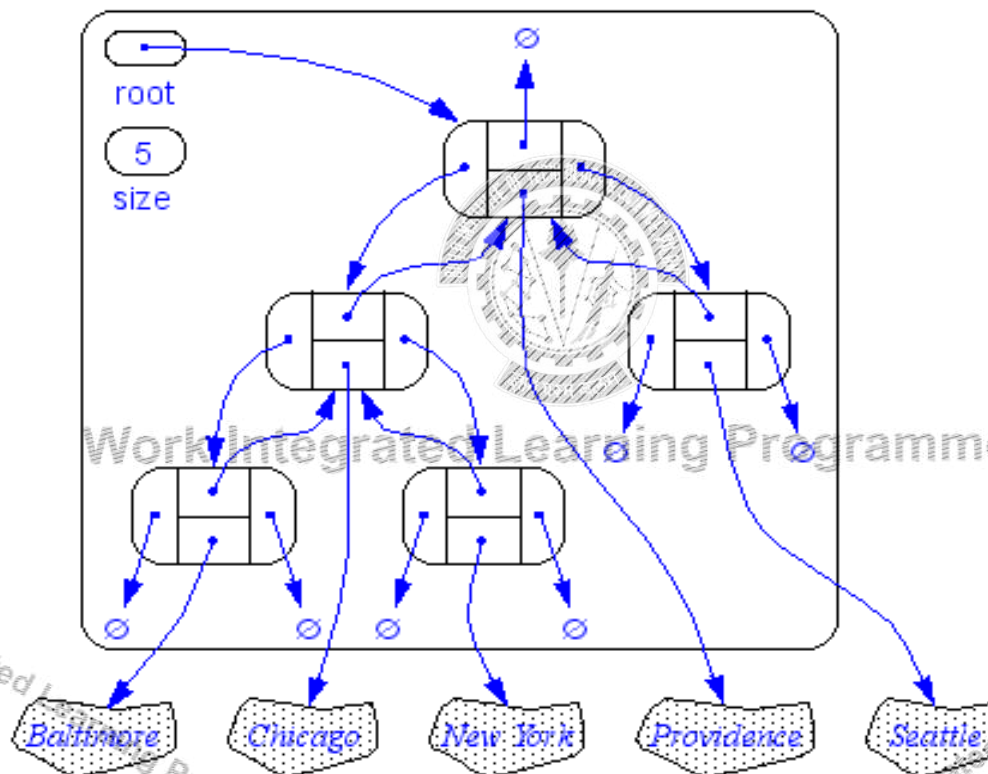
Binary Trees: Linked List Representation



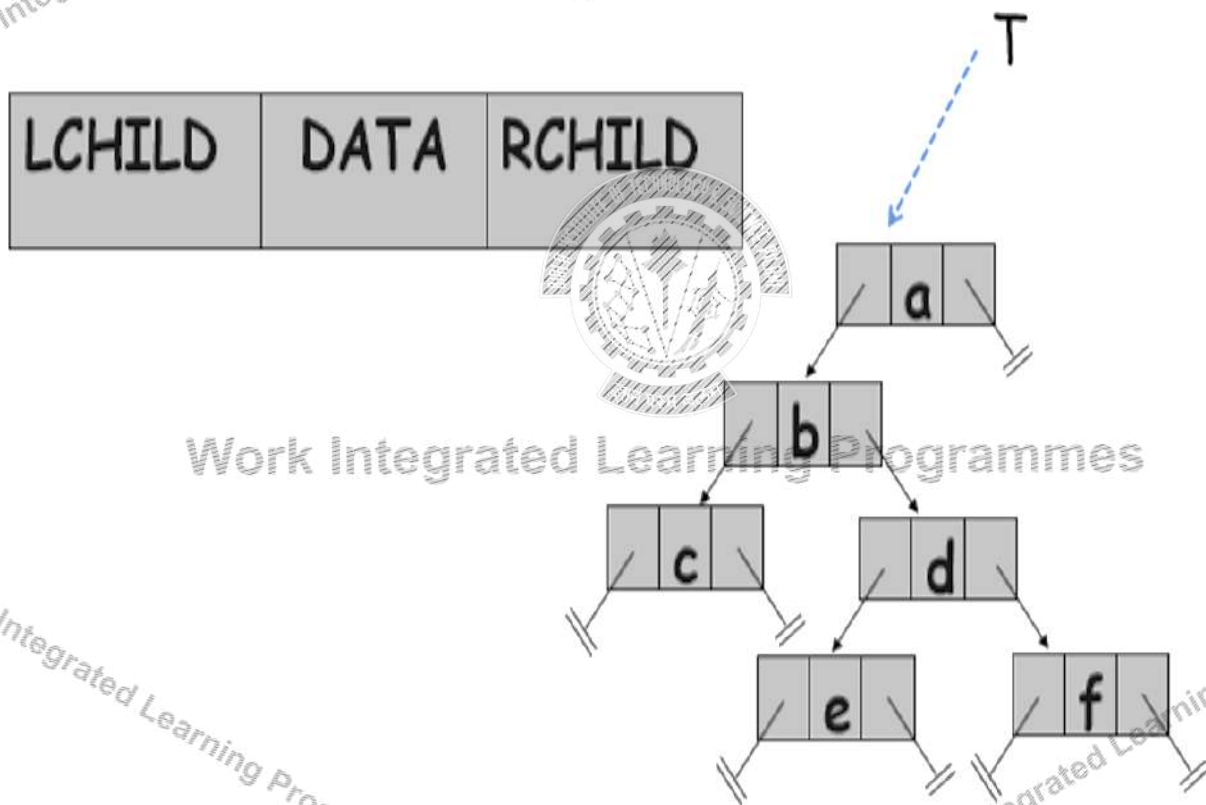
A **Binary tree** is a linked data structure. Each node contains data, and pointers left, right and p.

- **Left** points to the left child of the node.
- **Right** points to the right child of the node.
- **p** points to the parent of the node.
- If a child is missing, the pointer is NIL.
- If a parent is missing, p is NIL.
- The **root** of the tree is the only node for which p is NIL.
- Nodes for which both left and right are NIL are **leaves**.

Binary Trees: Linked List Representation



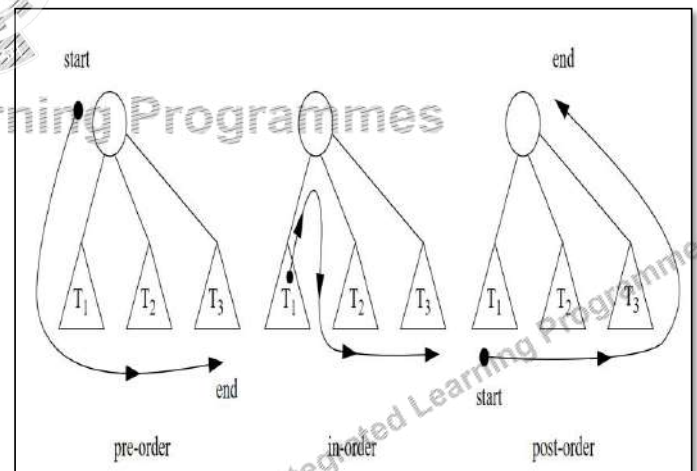
Binary Trees: Linked Representation



Binary Tree Traversals



- To **traverse (or walk)** the binary tree is to visit each node in the binary tree exactly once.
- Tree traversals are naturally recursive.
- Since a binary tree has three parts, the ways to traverse the binary tree are:
 - ✓ root, left, right : **Preorder**
 - ✓ left, root, right: **Inorder**
 - ✓ left, right, root: **Postorder**



36



Binary Tree Traversals

Three ways of traversing the binary tree T with root R:

Preorder

- Process the root R
- Traverse the left sub-tree of R in preorder
- Traverse the right sub-tree of R in preorder

a.k.a node-left-right
(**NLR**)

In-order

- Traverse the left sub-tree of R in in-order
- Process the root R
- Traverse the right sub-tree of R in in-order

a.k.a left-node-right
(**LNR**)

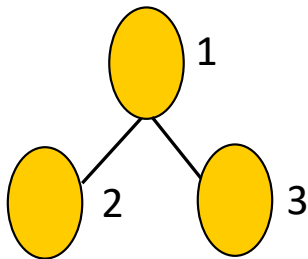
Post-order

- Traverse the left sub-tree of R in post-order
- Traverse the right sub-tree of R in post-order
- Process the root R

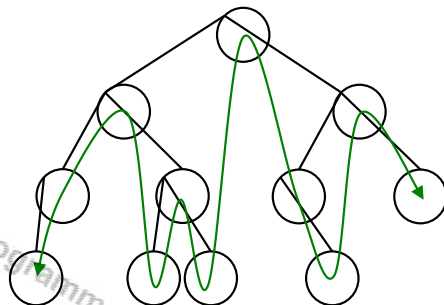
a.k.a left-right-node
(**LRN**)

37

Binary Tree Traversals



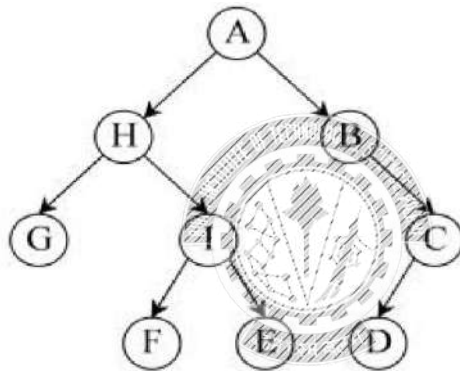
- Preorder(Root(N),left (L),right (R)): 1 2 3
- Inorder(left(L),root(N),right(R)): 2 1 3
- Postorder(left(L),right(R),root(N)): 2 3 1



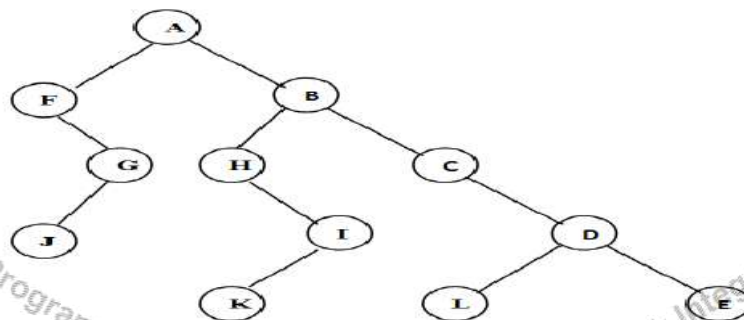
Exercises for CS #4



1) Perform the 3 traversals on the below:



2) Represent below in array and Linked list:



39

Appendix 1: TREE ADT-Depth



- Let v be a node of a tree T .
- The depth of v is the number of ancestors of v , excluding v itself
- If v is the root, then the depth of v is 0.
- Otherwise, the depth of v is one plus the depth of the parent of v .

Algorithm $\text{depth}(T, v)$:

if $T.\text{isRoot}(v)$ then

 return 0

else

 return $1 + \text{depth}(T, T.\text{parent}(v))$



- The running time of algorithm $\text{depth}(T, v)$ is $O(1 + d_v)$, where d_v denotes the depth of the node v in the tree T .
- In the worst case, the depth algorithm runs in $O(n)$ time, where n is the total number of nodes in the tree T

Appendix 2: TREE ADT-Height



- The *height of a tree* T is equal to the maximum depth of an external node of T .
- If v is an external node, then the height of v is 0.
- Otherwise, the height of v is one plus the maximum height of a child of v .

Algorithm *height*(T, v) :

if $T.isExternal(v)$ then

return 0

else

$h = 0$

for each $w \in T.children(v)$ do

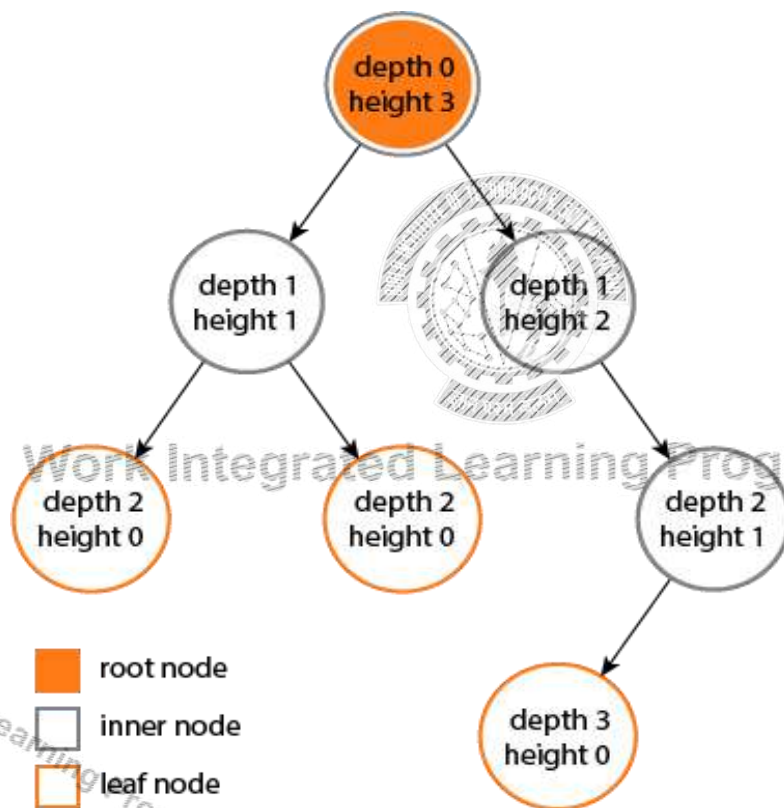
$h = \max(h, height(T, w))$

return $1 + h$

The height of a tree T is obtained by calling $height(T, T.root())$

Complexity of this algorithm is also $O(n)$, Refer Chapter 2, T1. (This involves Amortized analysis).

TREE ADT-Dept h and Height



Summary for CS #4



1) Recap of CS#3

2) Introduction to Trees

- Tree terminologies
- Binary tree and types
- Representation of tree
 - Using array
 - Using linked list



3) Q&A



See you in the next class to explore heaps!



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in



Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design

DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #5

DSECLZG519 – Heaps & Heap Sort

Agenda for CS #5



1) Recap of CS#4

2) Introduction to Heap

- What is Heap ?
- Types of Heap
- Heapification
- Build a Heap
- Insertion into a Heap
- Removal from a Heap
- Exercises



3) Heap Sort

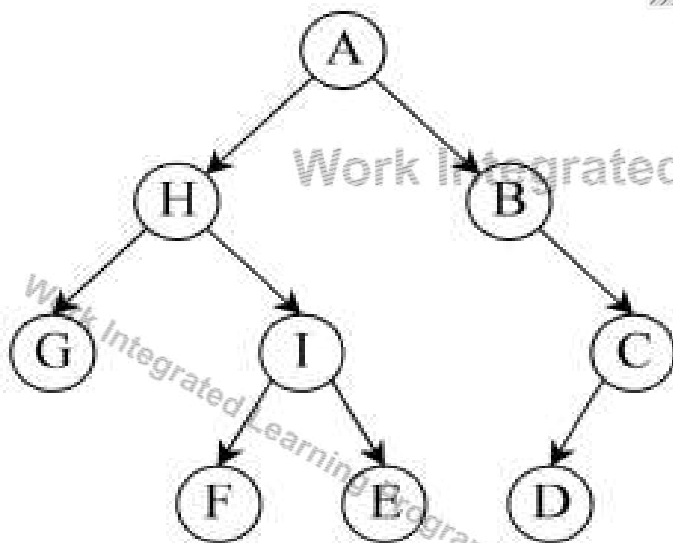
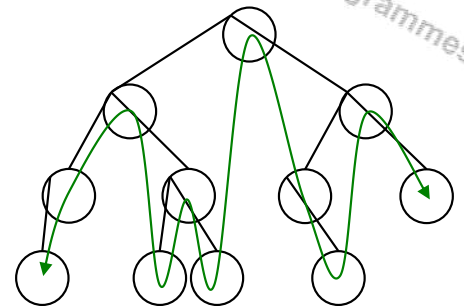
4) Q&A

Tree Traversal: In-order



In-order

- Traverse the left sub-tree of R in in-order
 - Process the root R
 - Traverse the right sub-tree of R in in-order
- a.k.a left-node-right (**LNR**)



In-order (LNR) traversal yields:

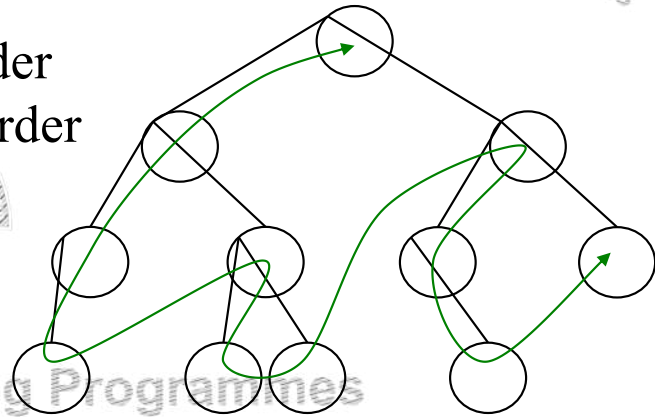
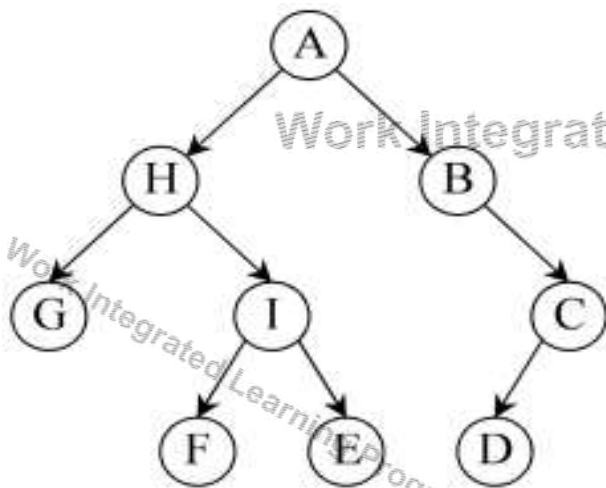
G, H, F, I, E, A, B, D, C

Tree Traversal: Pre-order



Preorder

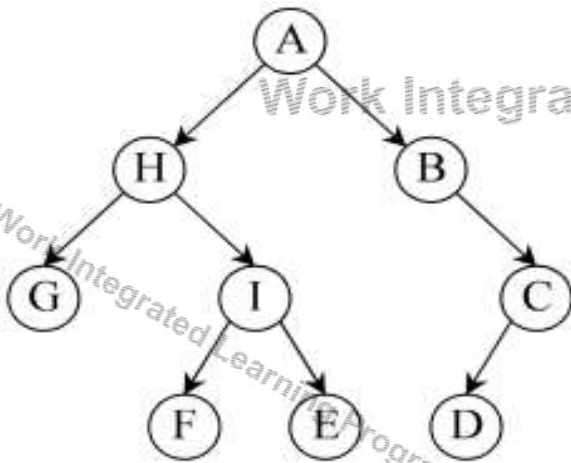
- Process the root R
 - Traverse the left sub-tree of R in preorder
 - Traverse the right sub-tree of R in preorder
- a.k.a node-left-right (**NLR**)



Preorder (NLR) traversal
yields: **A, H, G, I, F, E, B, C, D**

innovate achieve lead

-



Postorder (LRN) traversal
yields: **G, F, E, I, H, D, C, B, A**

Application of Binary Trees



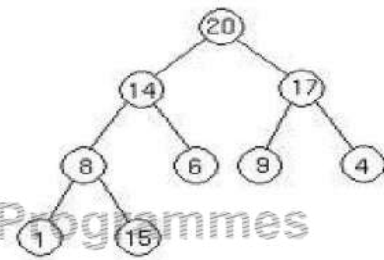
- Data Base indexing
- In video games
- Path finding algorithms in AI applications
- Huffman Coding
- Heaps
- Syntax tree.
-
- ...



Heap

- Heap is a special tree-based data structure, that satisfies the following special heap properties:

- *Shape Property*
- *Heap Property*



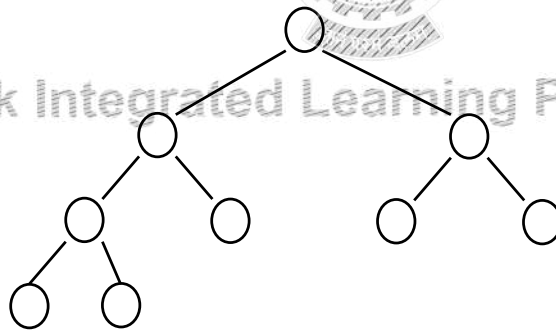
Heap



Shape Property



Heap data structure is always a **complete binary tree**, which means all levels of the tree are fully filled till $h-1$ and at level h (last level), the nodes are filled from left to right.



Complete Binary tree

Heap Property



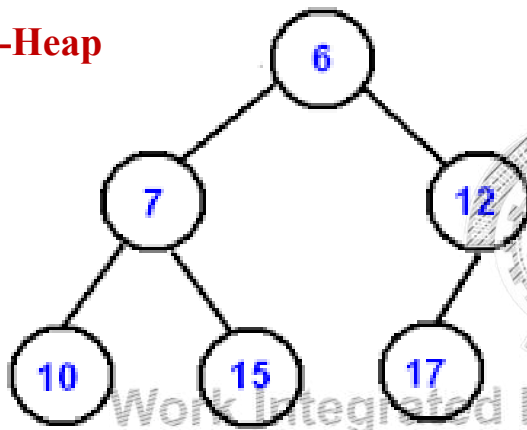
- All nodes are either [greater than or equal to] or [less than or equal to] each of its children.
- If the parent nodes are greater than their children, then such a heap is called: **Max-Heap**. So, The root of any sub-tree holds the **greatest** value in the sub-tree
- If the parent nodes are smaller than their children, then such a heap is called: **Min-Heap**. So, the root of any sub-tree holds the **least** value in that sub-tree.

- **Max-Heap**
for every node v other than the root
 $\text{element}(\text{parent}(v)) \geq \text{element}(v)$
- **Min-Heap**
for every node v other than the root
 $\text{element}(\text{parent}(v)) \leq \text{element}(v)$

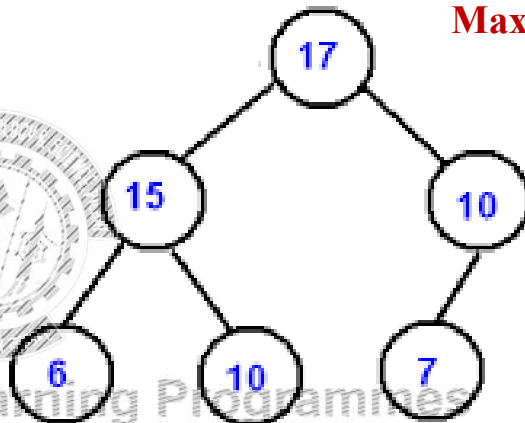


Examples of min-heap and max-heap

Min-Heap

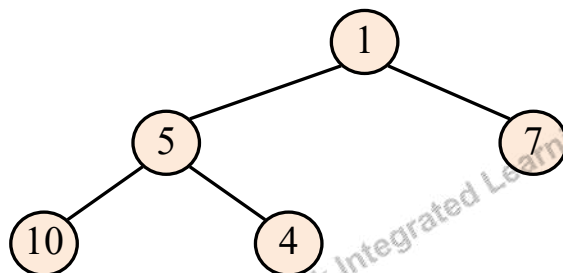
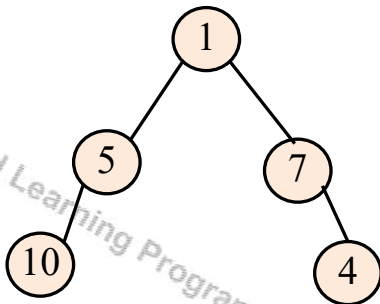


Max-Heap



Check :

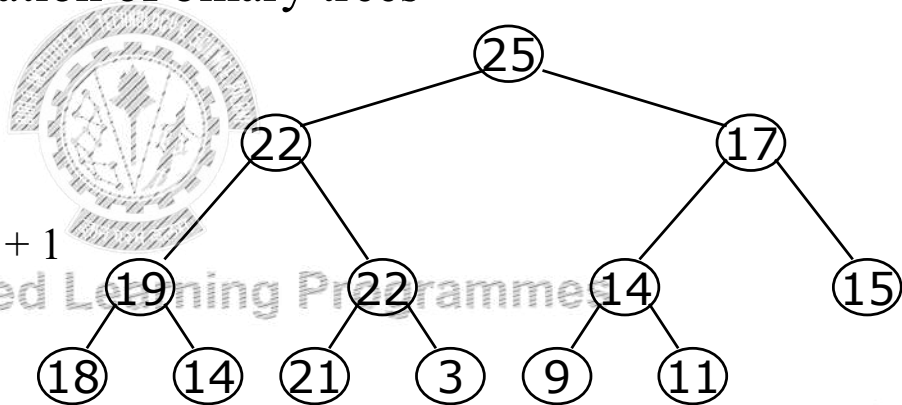
*These
are not
heaps!!*



Heap Representation



- Since, a heap is always a complete binary tree, can we represent heap as array easily and effectively ? **YES**
- Similar to array implementation of binary trees
- Root is at index 1
- For any node at index i
 - The left child is at index $2i$
 - The right child is at index $2i + 1$
 - Parent is at $\text{floor}(i/2)$

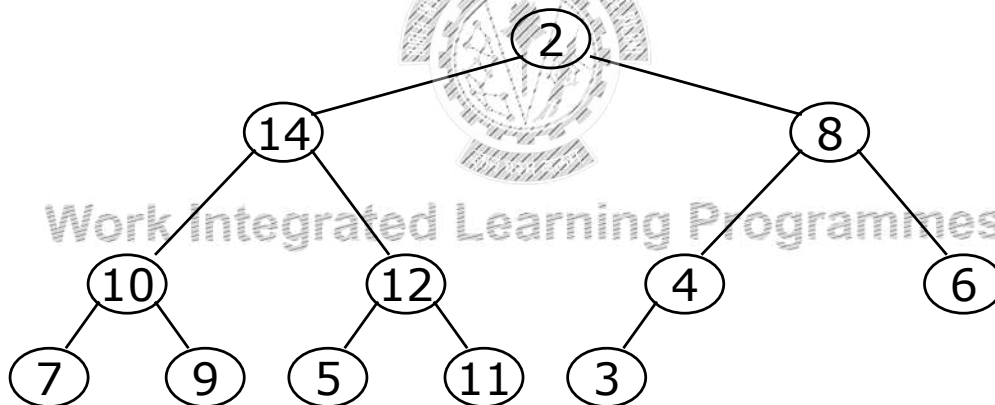


1	2	3	4	5	6	7	8	9	10	11	12	13
25	22	17	19	22	14	15	18	14	21	3	9	11

Heapification (Max-Heapify)



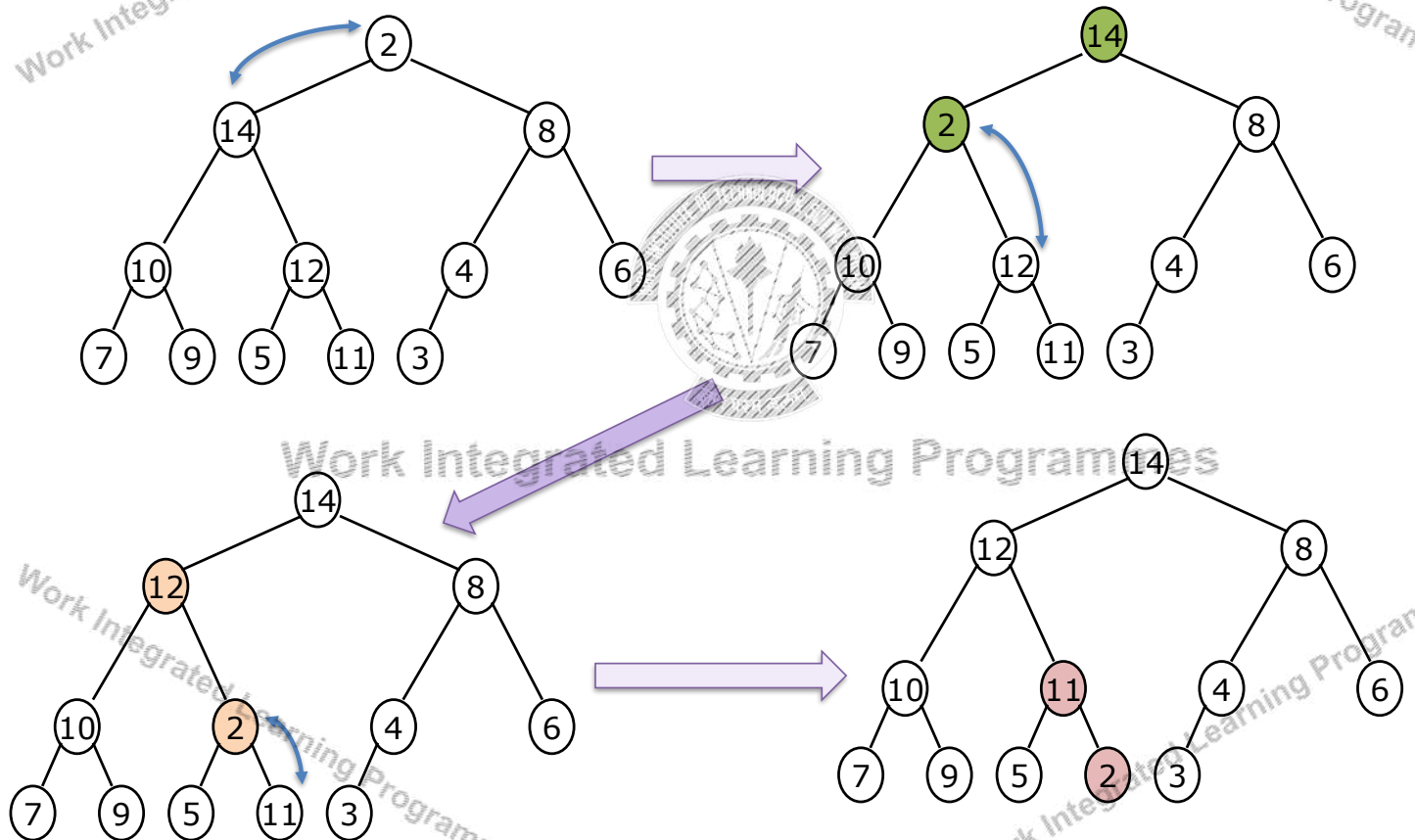
- Before discussing the method for building heap of an arbitrary complete binary tree, we discuss a simpler problem.
- Let us consider a binary tree in which left and right subtrees of the root satisfy the heap property but **not the root**. See the following fig:



- Now the Question is how to transform the above tree into a Heap ?
- Heapification !! Commonly referred as Max-Heapify() if max-heap is required and Min-Heapify() if min-heap is required.

13

Sequence Depicting the Heapification process



Algorithm: Max-Heapify(B, s)



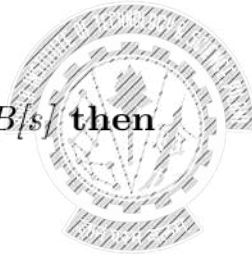
Algorithm 1: Max-Heapify Pseudocode

Data: B : input array; s : an index of the node

Result: Heap tree that obeys max-heap property

Procedure Max-Heapify(B, s)

```
    left = 2s;
    right = 2s + 1;
    if left ≤ B.length and B[left] > B[s] then
        | largest = left;
    else
        | largest = s;
    end
    if right ≤ B.length and B[right] > B[largest] then
        | largest = right;
    end
    if largest ≠ s then
        | swap(B[s], B[largest]);
        | Max-Heapify(B, largest);
    end
end
```



The time complexity of max-Heapify is $O(\log n)$

**Since the complete binary tree is perfectly balanced, shifting up a single node takes $O(\log n)$ time.*

Build Heap



- Heap building can be done efficiently with **bottom up fashion**.
- Given an arbitrary complete binary tree, we can assume each leaf is a heap
- Start building the heap from the *parents of these leaves* i.e., Max-Heapify subtrees rooted at the parents.
- The Heapify process continues till we reach the root of the tree.

Algorithm 2: Building a Max-Heap Pseudocode

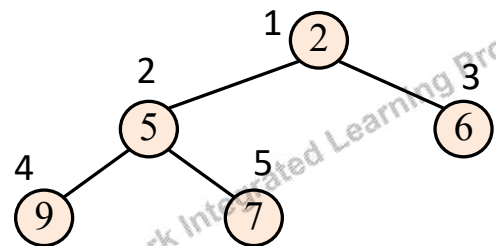
Data: B : input array

Result: Heap tree

Procedure Max-Heap-Building(B)

```
 $B.\text{heapsize} = B.\text{length};$   
for  $k = B.\text{length}/2$  down to 1 do  
|  $\text{Max-Heapify}(B, k);$   
end  
end
```

All leaf nodes are from $\lfloor n/2 \rfloor + 1$ to n
All non-leaf nodes are from 1 to $\lfloor n/2 \rfloor$

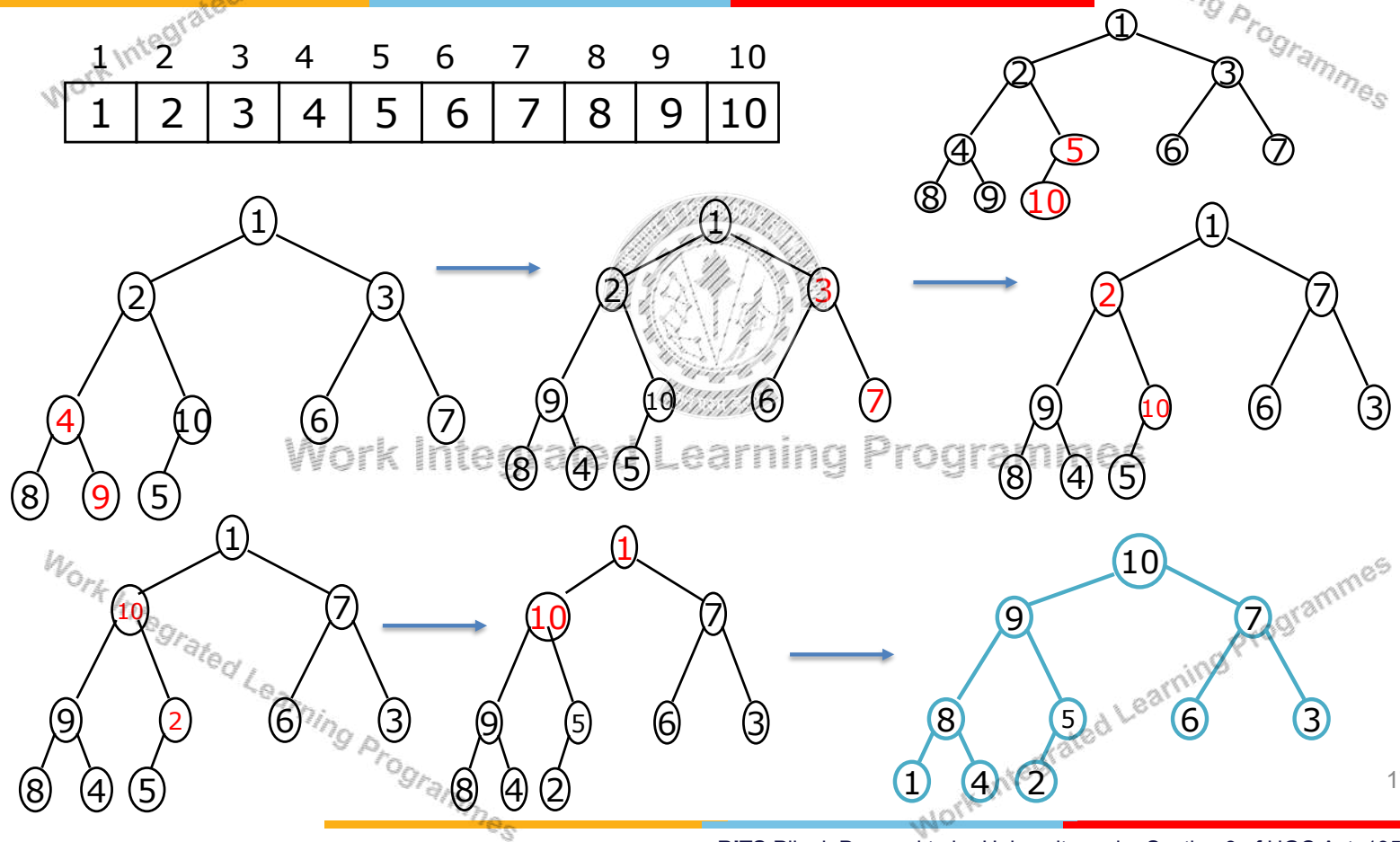


16

Build Heap

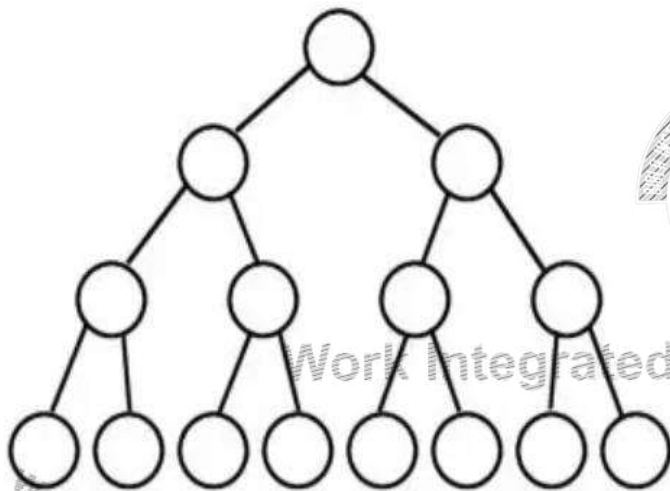


1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



17

Build Heap Analysis



We are calling $\text{MAX_HEAPIFY}()$ on $n/2$ nodes (as leaf's are already a heap).

Since we call MAX_HEAPIFY $O(n)$ times and MAX_HEAPIFY takes $O(\log n)$, the overall complexity is $O(n \log n)$.

*But this is not tight! We can also prove that building a heap is not $O(n \log n)$ but just **$O(n)$** .*

There is a beautiful proof about this in CLRS using progression. Please refer the same and start a discussion if required!

Insertion into a Heap



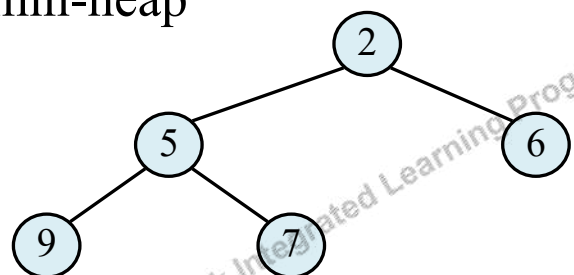
Inserting an element e in the heap has

– Three steps

- Find the insertion point z
 - So that we maintain complete binary tree property
- Store e at insertion point z
- Check if the heap follows heap-order property
 - Restore the heap-order property by *Up-heap bubbling*

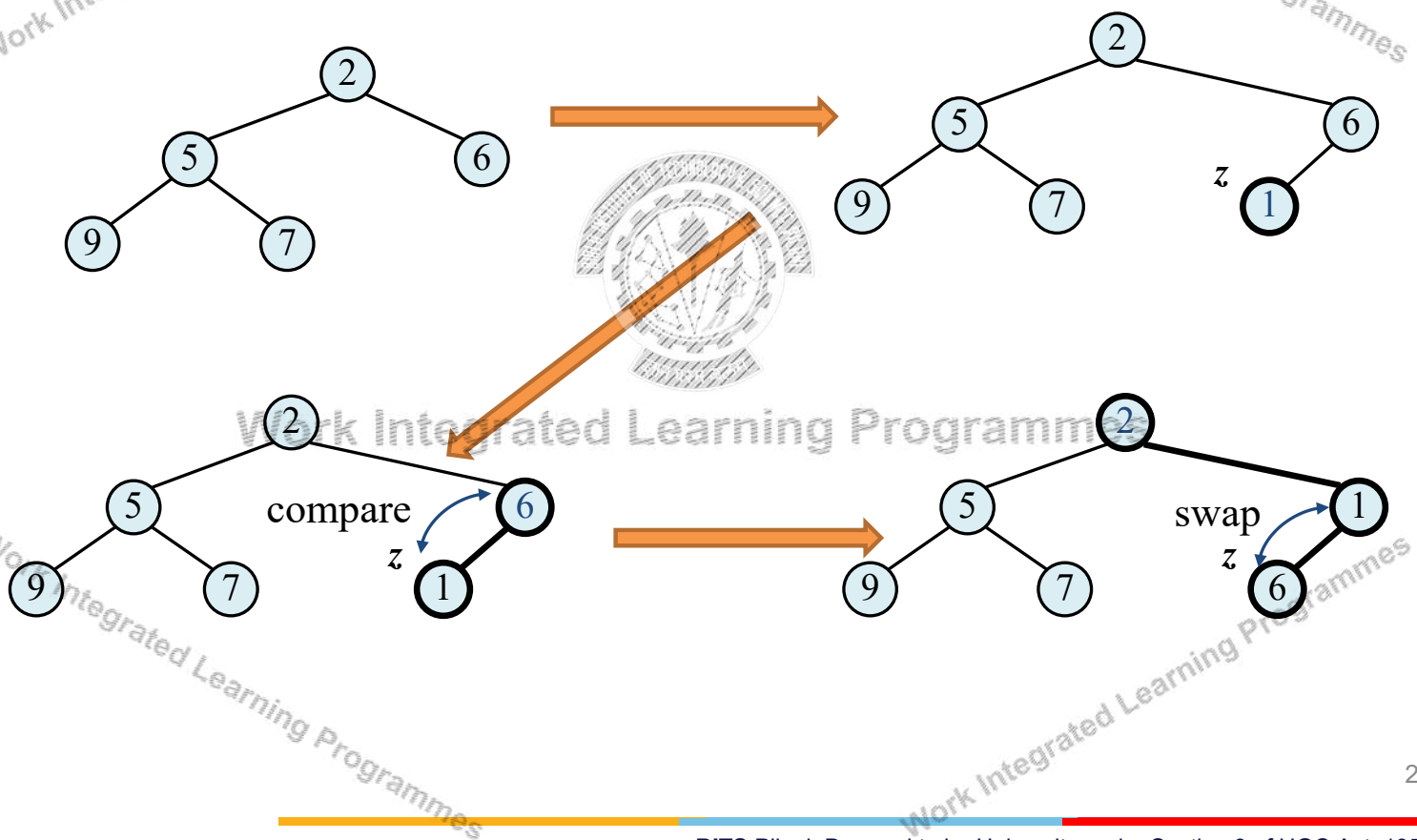
Example:

- Insert the element 1 into the min-heap

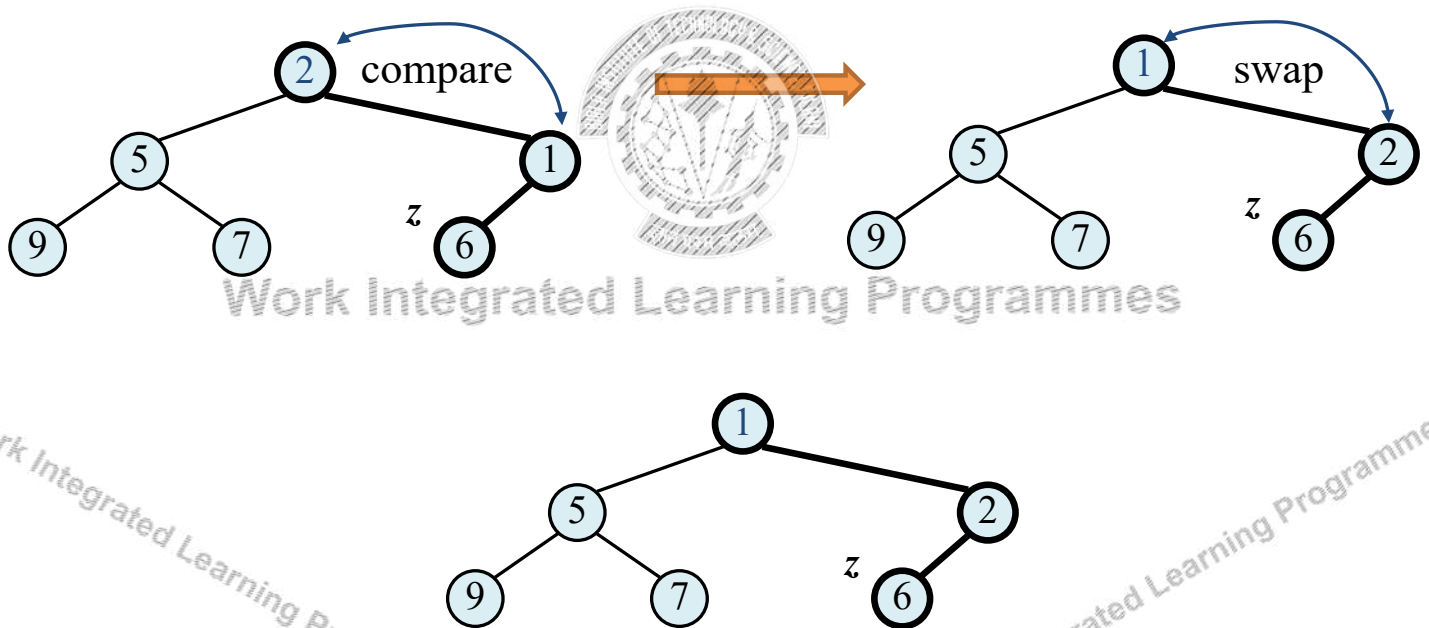


19

Insertion into a Heap



Insertion into a Heap



Insertion into a Heap



- After the insertion of a new element e , the heap-order property may be violated.
- **Up-heap bubbling** restores the heap-order property
 - Compare and swap e along an upward path from the insertion point
 - Up-heap bubbling terminates when the element e reaches
 - the root
 - a node where the heap order property is satisfied
- Since the heap has a complete binary tree structure, its height = $\log n$ (where n is no of elements). In the worst case (element inserted at the bottom has to be swapped at every level from bottom to top up to the root node to maintain the heap property), 1 swap is needed on every level. Therefore, the maximum no of times this swap is performed is $\log n$. Hence, Insertion in a heap takes **$O(\log n)$ time**.

22

Removal from a Heap



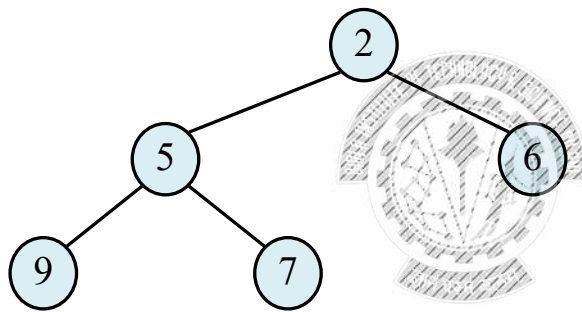
Three steps

- Remove the element **at the root node** from the heap
- Fill the root node with the element from **the last node**
 - maintain Complete binary tree property
- Check if the heap follows the heap-order property
 - Restore the heap-order property by *down-heap bubbling*

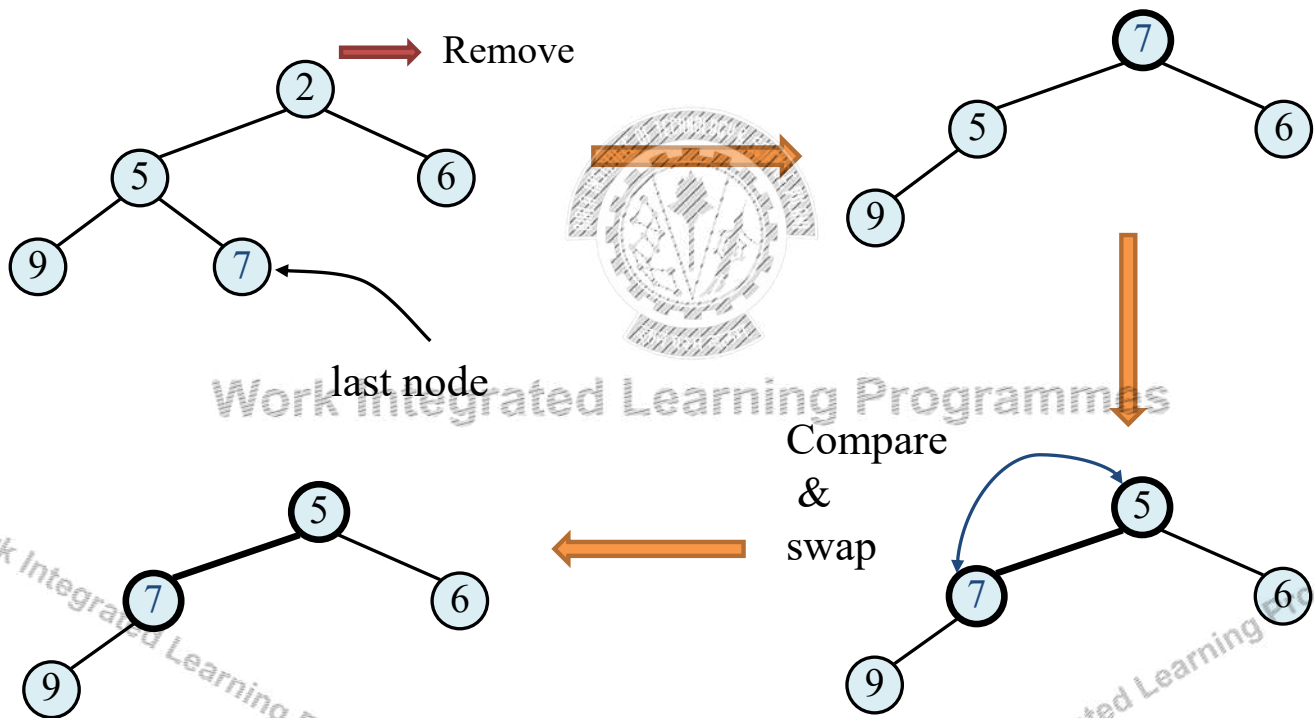
Removal from a Heap



Example: Perform a delete operation on the given min-heap



Removal from a Heap



Removal from a Heap



- After replacing the root with the element from the last node e , the heap-order property may be violated
- **Down-heap bubbling** restores the heap-order property
 - Compare and swap e along a downward path from the root node
 - Choose the eligible (min/max) child of e and swap it with e
 - Down-heap terminates when the element e reaches
 - A leaf
 - A node where the heap order property is satisfied
- Here again, in worst case, we may have to perform down-heap bubbling till the node reaches the leaf. Complexity is **$O(\log n)$**

Exercise 1



Min-Heap

- Illustrate the result of inserting the elements 35, 33, 42, 10, 14, 19 and 27 one at a time, into an initially empty binary min-heap in that order. Draw the resulting min-heap after each insertion.
- Perform 2 delete operation for the min-heap constructed in the earlier example.

Max Heap

- Illustrate the result of inserting the elements 35, 33, 42, 10, 14, 19 and 27 one at a time, into an initially empty binary max-heap in that order. Draw the resulting max-heap after each insertion.
- Perform 2 delete operation for the max-heap constructed in the earlier example.

Exercise 2



Consider a binary max-heap implemented using an array. Which one of the following array represents a binary max-heap?

25, 14, 16, 13, 10, 8, 12

25, 12, 16, 13, 10, 8, 14

25, 14, 12, 13, 10, 8, 16

25, 14, 13, 16, 10, 8, 12

Draw the heap structure and find out the right answer/s

Heap-Sort Algorithm



In-Place: A sorting algorithm is said to be “in-place” if it moves the items within the array itself and, thus, requires only a small $O(1)$ amount of extra storage.

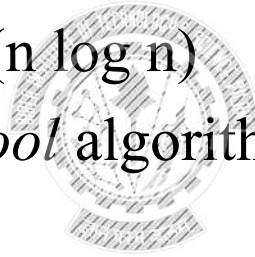
- Heap sort is one of the best sorting methods being **in-place** and with no quadratic worst-case scenarios.
- Heap sort is divided into two basic parts :
 - Creating a heap of the unsorted list
 - Then a sorted array is created by repeatedly removing the largest/smallest element from the heap and inserting it into the array
 - *Heap is reconstructed after each removal*

29

Why study Heapsort?



- It is a well-known, traditional sorting algorithm you will be expected to know!!
- Heapsort is *always* $O(n \log n)$
- Heapsort is a *really cool* algorithm!



Work Integrated Learning Programmes

Heap Sort

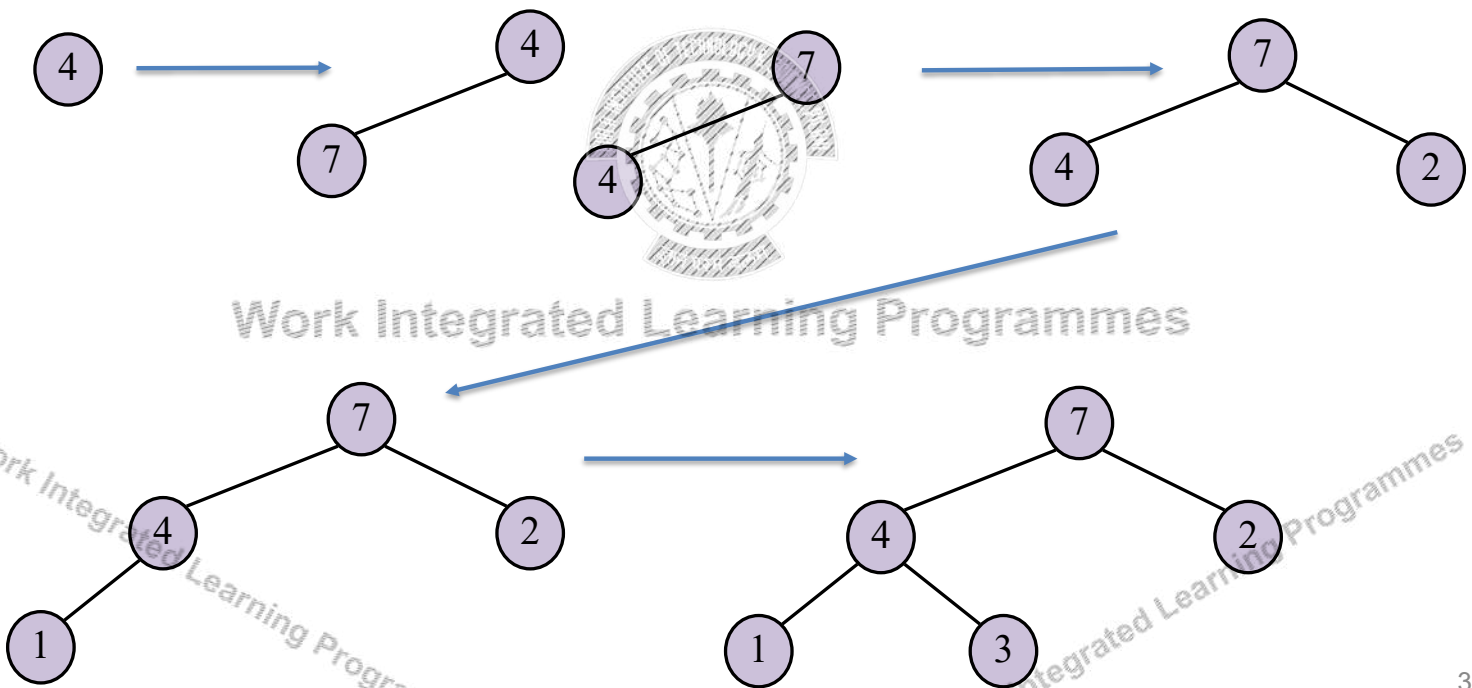


- Given an array of n element, first we build the heap ..
- The largest element is at the root, but its position in sorted array should be at last. So swap the root with the last.
- We have placed the highest element in its correct position we left with an array of $n-1$ elements. Repeat the same of these remaining $n-1$ element to place the next largest elements in its correct position.
- Repeat the above step till an elements are placed in their correct positions.
- For increasing (ascending) order → Create a Max-Heap
- For a decreasing (descending) order → Create a Min-Heap

Heap Sort Example



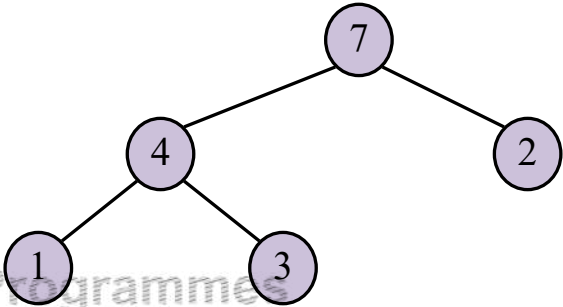
Illustrate heap sort for the given array $S = [4, 7, 2, 1, 3]$. Sort S in increasing order.
First phase: Build max-heap



Heap Sort Example



	S[1]	S[2]	S[3]	S[4]	S[5]
Initial	4	7	2	1	3
i=1	4	7	2	1	3
i=2	7	4	2	1	3
i=3	7	4	2	1	3
i=4	7	4	2	1	3
i=5	7	4	2	1	3
	Represents heap				
	Represents array elements not in heap				



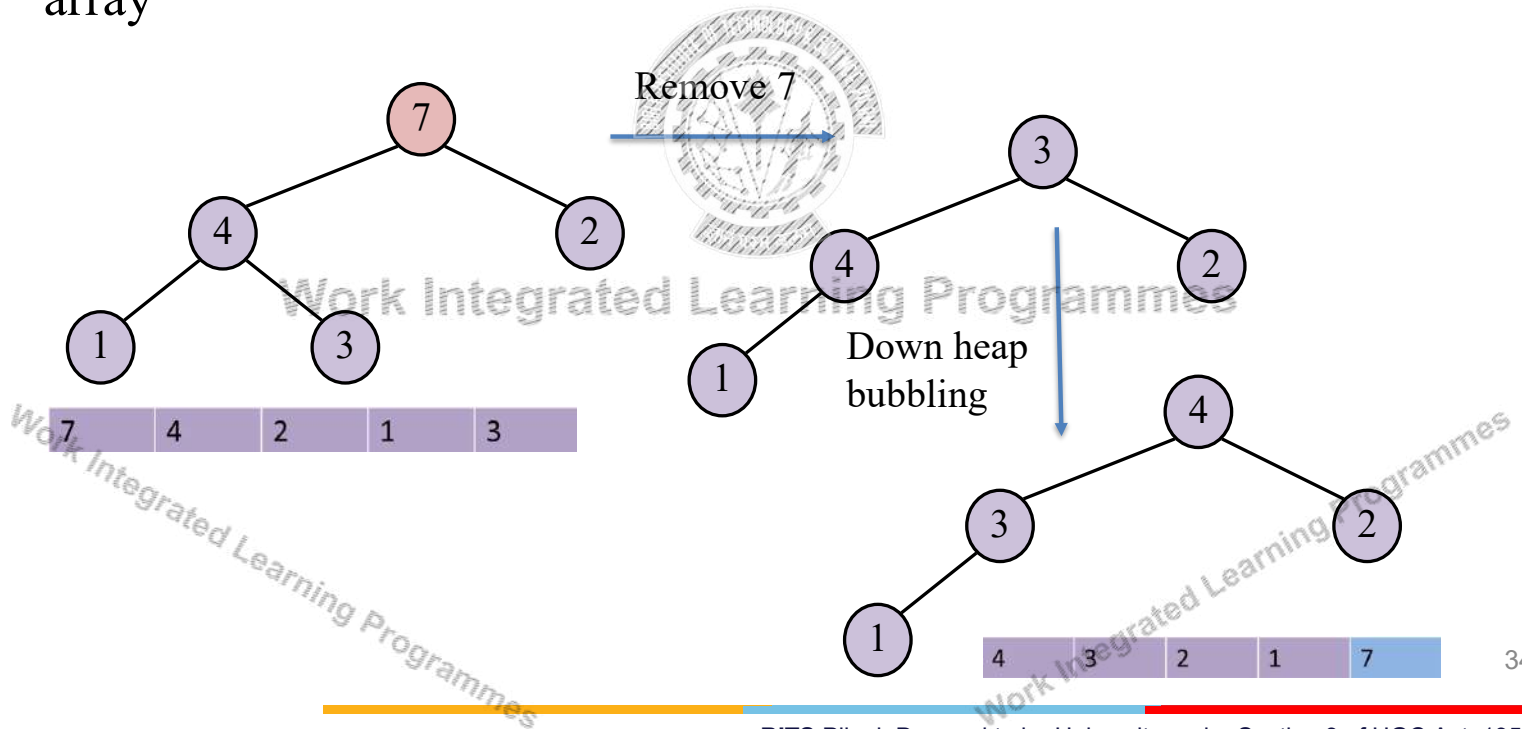
Phase 1: Heap Creation is completed!

Heap Sort Example

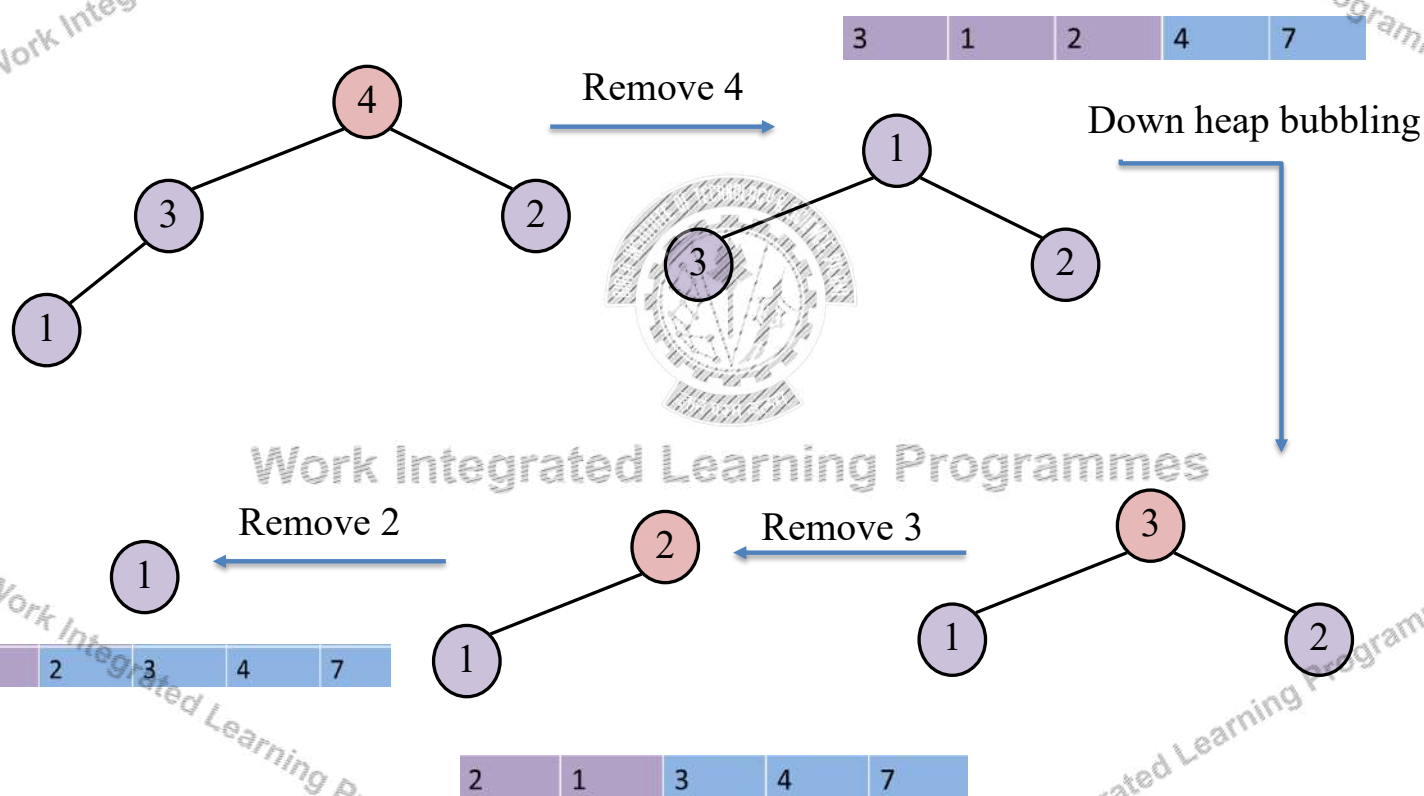


Heap $S = [7, 4, 2, 1, 3]$

Second phase: Remove elements from heap and add them to the sorted array



Heap Sort Example



Heap Sort Example



	S[1]	S[2]	S[3]	S[4]	S[5]
Initial	7	4	2	1	3
i=1	4	3	2	1	7
i =2	3	1	2	4	7
i =3	2	1	3	4	7
i =4	1	2	3	4	7
i =5	1	2	3	4	7
	Represents heap				
	Sorted array				

Phase 2: Heap Deletion is completed! And result is sorted elements!!

Pseudocode for Heap Sort



Heapsort(A)

1. Build-Max-Heap(A)
2. for $i \leftarrow \text{length}[A]$ **downto** 2
3. **do** exchange $A[1] \leftrightarrow A[i]$
4. $\text{heap-size}[A] \leftarrow \text{heap-size}[A]-1$
5. Max-Heapify(A, 1)

Work Integrated Learning Programmes

The time complexity of the heap sort algorithm is in ?

Phase 1 – Building the heap takes $O(n)$

Phase 2 – Remove the roots till we are left with only 1 element ($\log n$)

Overall Complexity : $O(n \log n)$

37

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Exercise 3



- Given set of elements: 16,14,10,8,7,9,3,2,4,1

Exercise 1:

- Implement Heap Sort by showing each step and the resultant must be in increasing order. Hint: Create max-heap!

Exercise 2:

- Implement Heap Sort by showing each step and the resultant must be in decreasing order. Hint: Create min-heap!

Exercise 4



Find K'th smallest element in an array using Heap.

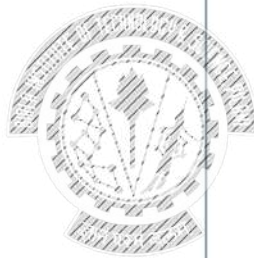
Input:

```
arr = [7, 4, 6, 3, 9, 1]
```

```
k = 3
```

Output:

k'th smallest element in the array is 4



Procedure

1. Construct a min-heap of size 'N'
2. Pop first K-1 elements from it
3. Now K'th smallest element will reside at the root of the min-heap.



See you in the next class to explore Graphs!



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



41

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design

DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Goa | Hyderabad



Work Integrated Learning Programmes

Contact Session #6

DSECLZG519 – Introduction to Graphs

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Agenda for CS #6



- 1) Recap of CS#5
- 2) Priority Queues & Heaps
- 3) Introduction to Graphs

- What is a Graph ?
- Types of Graph
- Terminologies
- Applications



- 4) Graph Implementation
 - Adjacency matrix
 - Adjacency List
 - Edge List

- 5) Exercises

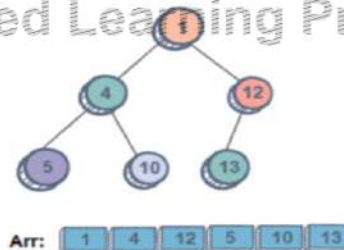
Recap of #5



- Heaps & Types
- Heapification
 - Max-Heapify()
 - Min-Heapify()
- Building Heap
 - Using *bottom up* using appropriate heapification
 - Using *top down* using repetitive insertion
- Insertion
 - Up Heap bubbling
- Deletion
 - Down Heap bubbling
- Application 1: Heap Sort

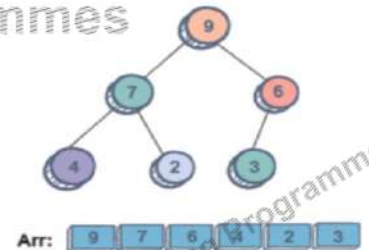
Min-Heap

- The root node has the **minimum** value.
- The value of each node is equal to or greater than the value of its parent node.
- A complete binary tree.



Max-Heap

- The root node has the **maximum** value.
- The value of each node is equal to or less than the value of its parent node.
- A complete binary tree.



Priority Queues

(An Application of Heap)



- A priority queue is a data structure for maintaining a set S of elements, each with an associated value called a key/priority.
- There's no “real” FIFO rule anymore.
- Two kinds : max-priority queues and min-priority queues, according to max-heaps and min-heaps.
- The key denotes the **priority**
- Max-heap is used to implement a max-Priority Queue
 - Always deletes & returns (extracts) an element with maximum priority
- Min-heap is used to implement a min-Priority Queue
 - Always deletes & returns (extracts) an element with minimum priority

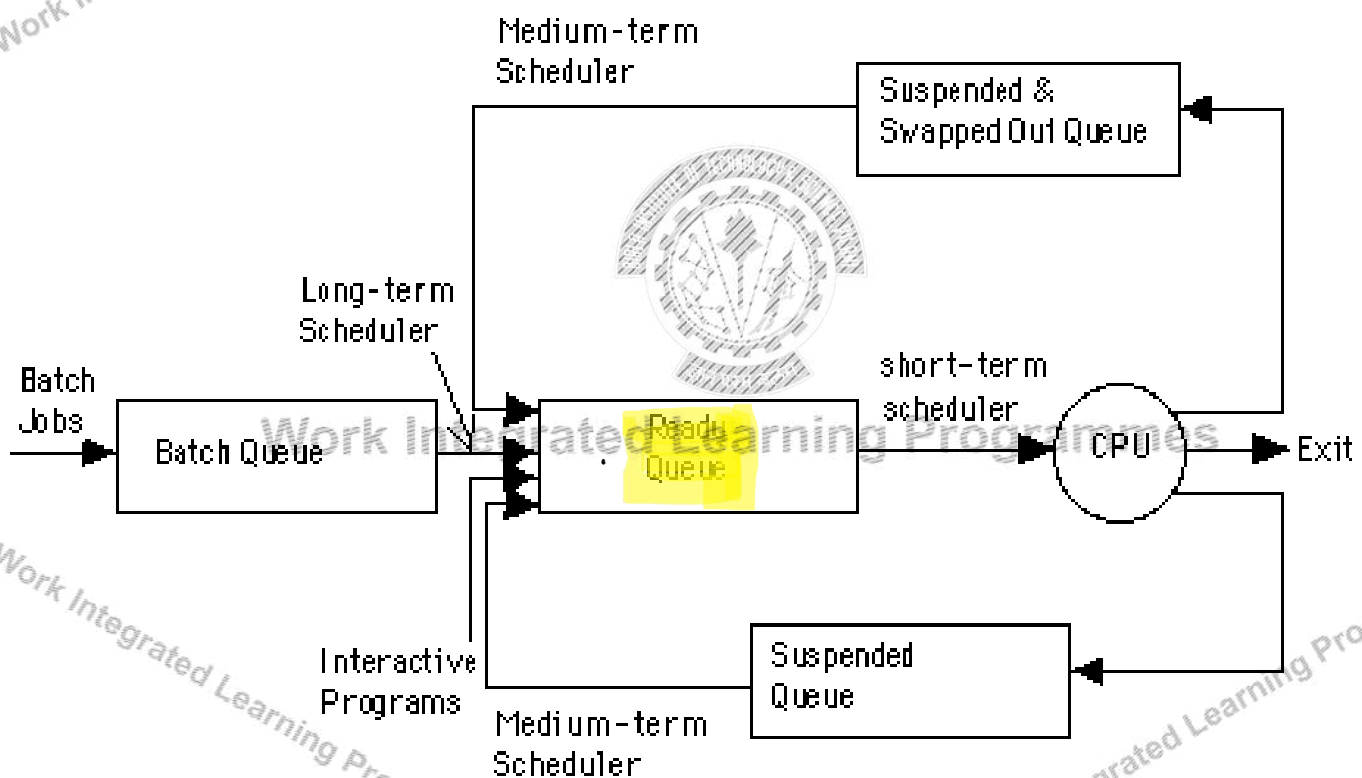
Max – Priority Queue Applications



Example: **job scheduling** on shared computer

- jobs have priorities, are stored in a max-priority queue
- each time a new job is to be scheduled, it's got to be one of highest priority (**Extract-Max** operation)
- new jobs can be inserted using **Insert** operation
- in order to avoid “starvation”, priorities can be increased (**Increase-Key** operation)

Max – Priority Queue Applications





Max- Priority Queue Operations

- **Insert**(S, x) inserts element x into set S
- **Maximum**(S) returns element of S with largest key
- **Extract-Max**(S) removes and returns element of S with largest key
- **Increase-Key**(S, x, k) increases x 's key to new value k , assuming k is at least as large as x 's old key

Min-priority queues offer Insert, Minimum, Extract-Min, and Decrease-Key.

Max – Priority Queue



Heaps are very convenient here:

- using max-heaps, we know that the largest element is in $A[1]$: we have $O(1)$ access to largest element
- removing/inserting elements and increasing keys means that we (basically) can call **Max-Heapify** at the right place (relatively efficient operation)

Max-Priority Queue Implementation



Heap-Maximum(A)

←----- **O(1)**

1. **return** A[1]

Heap-Extract-Max(A)

←----- **O(log n)**

1. if $\text{heap-size}[A] < 1$

2. **then error** "heap underflow"

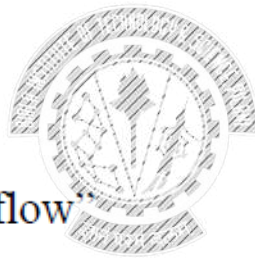
3. $\text{max} \leftarrow A[1]$

4. $A[1] \leftarrow A[\text{heap-size}[A]]$

5. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$

6. Max-Heapify(A,1)

7. **return** max



Technically, this part is removal from heap!

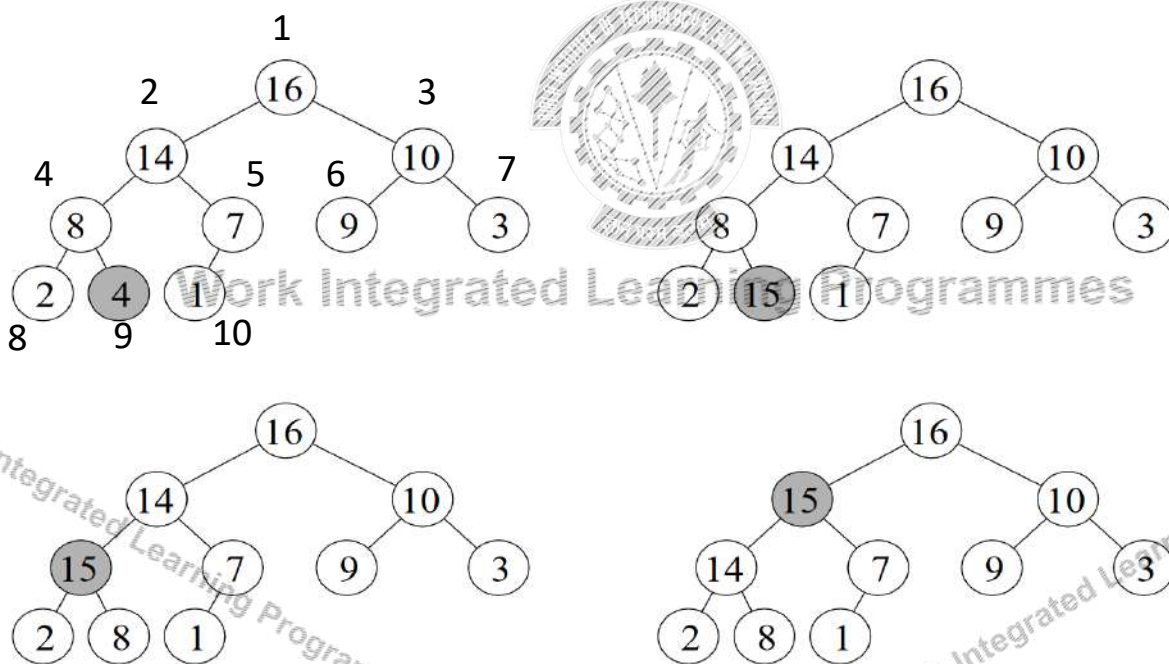
Example of Heap-Increase-Key



Example:

Heap-Increase-Key(A , 9, 15)

Updates node 9 from 4 to 15



Max-Priority Queue Implementation



Heap-Increase-Key(A,i,key)

1. **if** $\text{key} < A[i]$
2. **then error** “new key is smaller than current key”
3. $A[i] \leftarrow \text{key}$
4. **while** $i > 1$ and $A[\text{Parent}(i)] < A[i]$
5. **do** exchange $A[i] \leftrightarrow A[\text{Parent}(i)]$
6. $i \leftarrow \text{Parent}(i)$

<----- $O(\log n)$
Height of the tree

Technically, this part is insertion into heap!

Max-Heap-Insert(A,key)

<----- $O(\log n)$

1. $\text{heap-size}[A] \leftarrow \text{heap-size}[A] + 1$
2. $A[\text{heap-size}[A]] \leftarrow -\infty$
3. Heap-Increase-Key(A, $\text{heap-size}[A]$, key)

Applications of Heap



- Heaps are used in heapsort!
- *Priority Queues*: Priority queues can be efficiently implemented using Heaps
- *Order statistics*: The Heap data structure can be used to efficiently find the kth smallest (or largest) element in an array.
- Heap Implemented priority queues are used in Graph algorithms like Prim's Algorithm and Dijkstra's algorithm.

Exercise 1



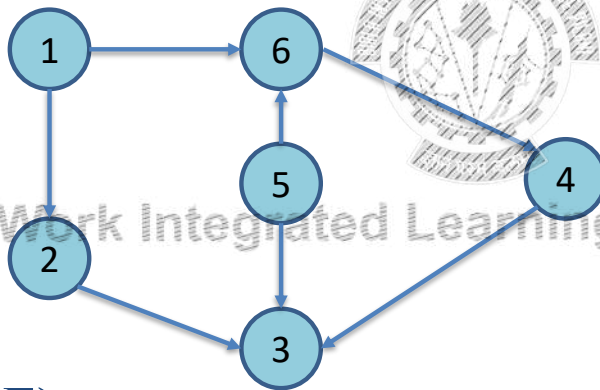
- Write an algorithm to compute product of three minimum numbers [least 3 numbers] of a given array using min-heap.
- Example:
 - Original array elements:
[12, 74, 9, 50, 61, 41]
 - Product of three minimum numbers of the given array:
4428 [9 X 12 X 41]
- Write both steps and the algorithm
- Analyze the complexity of your solution

Graph



A graph G is defined as a pair of two sets V and E , where

- V is a set of nodes, called **vertices**
- E is a collection of pairs of vertices, called **edges**



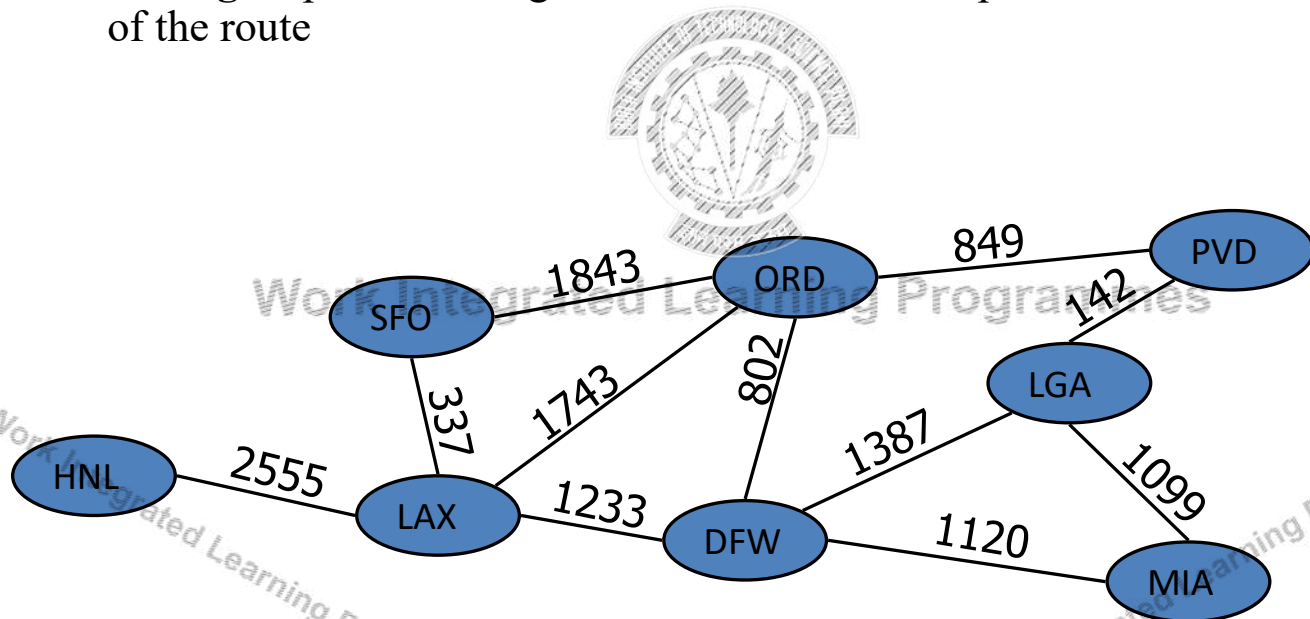
- $G = (V, E)$
- $V = \{1, 2, 3, 4, 5, 6\}$ and $|V| = 6$
- $E = \{(1, 6), (1, 2), (2, 3), (4, 3), (5, 3), (5, 6), (6, 4)\}$ and $|E| = 7$

Example



Example:

- A **vertex** represents an **airport** and stores the **three-letter airport code**
- An **edge** represents a flight route between two airports and stores the **mileage** of the route



Other Examples



Can you think of other examples of graphs modelling real – world situations ?

- Google maps !
- Facebook and friends ☺
- World Wide Web and Hyperlinks ...
- Operating Systems for resource allocation
- ...



Electronic circuits

Printed circuit board
Integrated circuit

Transportation networks

Highway network
Flight network

Computer networks

Local area network
Internet
Web

Databases

Entity-relationship diagram 17

Tree vs Graph



BASIS FOR COMPARISON	TREE	GRAPH
Path	Only one between two vertices.	More than one path is allowed.
Root node	It has exactly one root node.	Graph doesn't have a root node.
Loops	No loops are permitted.	Graph can have loops.
Complexity	Less complex	More complex comparatively
Traversal techniques	Pre-order, In-order and Post-order.	Breadth-first search and depth-first search.
Number of edges	$n-1$ (where n is the number of nodes)	Not defined
Model type	Hierarchical	Network

18

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Edge Types



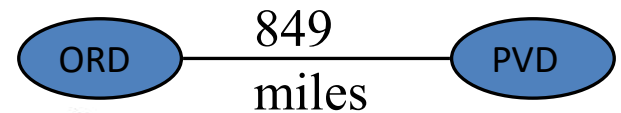
Directed edge (with arrow)

- ordered pair of vertices (u, v)
- first vertex u is the origin
- second vertex v is the destination
- e.g., a flight



Undirected edge (no arrow)

- unordered pair of vertices (u, v)
- e.g., a flight route



Directed graph

- all the edges are directed
- e.g., flight network

Undirected graph

- all the edges are undirected
- e.g., route network

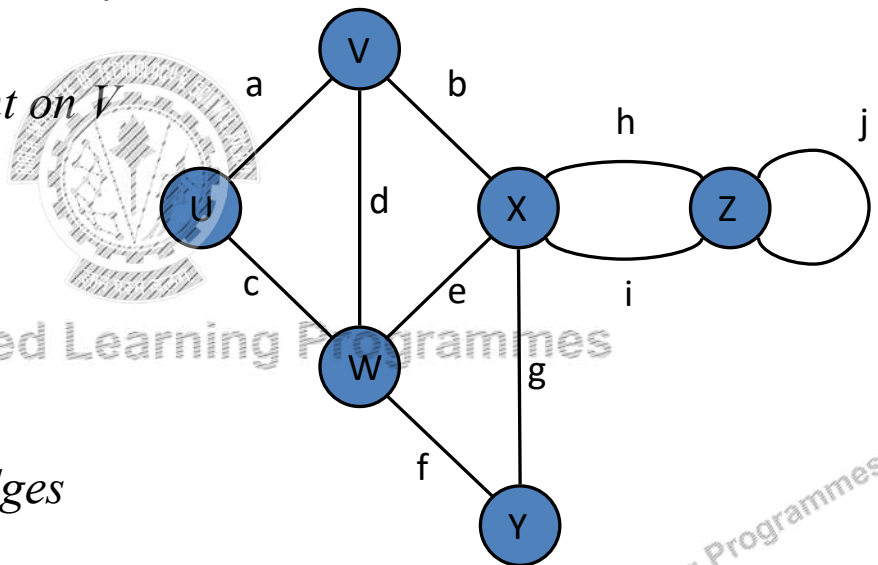
Analogy:

- A road between two points could be **one way** (directed) Or **two way** (undirected)
- Could have more than one edge (e.g. toll road and public road)



Graph Terminology

- **End vertices** (or endpoints) of an edge
 - Ex: U and V are the endpoints of a
- **Edges incident** on a vertex
 - Ex: a , d , and b are incident on V
- **Adjacent vertices**
 - Ex: U and V are adjacent
- **Degree of a vertex**
 - Ex: X has degree 5
- **Parallel edges**
 - Ex: h and i are parallel edges
- **Self-loop**
 - Ex: j is a self-loop



Graph Terminology



Path

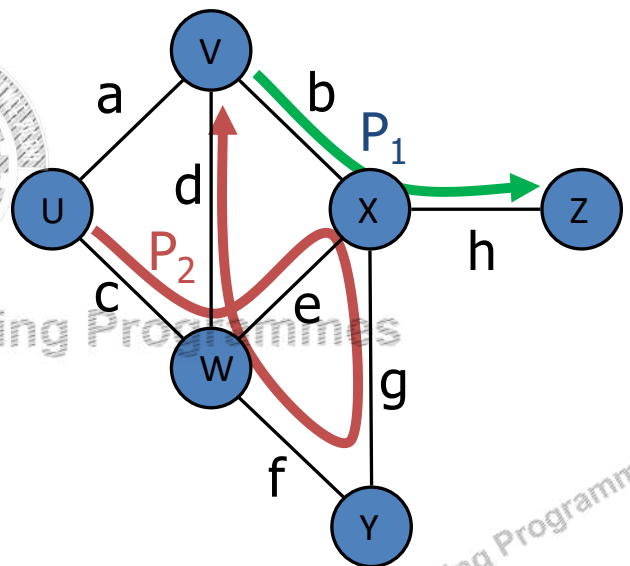
- Sequence of alternating vertices and edges
- Begins with a vertex
- Ends with a vertex
- Each edge is preceded and followed by its endpoints

Simple path

- Path such that all its vertices and edges are *distinct*

Examples

- $P_1 = (V, b, X, h, Z)$ is a **simple** path
- $P_2 = (U, c, W, e, X, g, Y, f, W, d, V)$ is a path that is **not simple**



Graph Terminology



Cycle

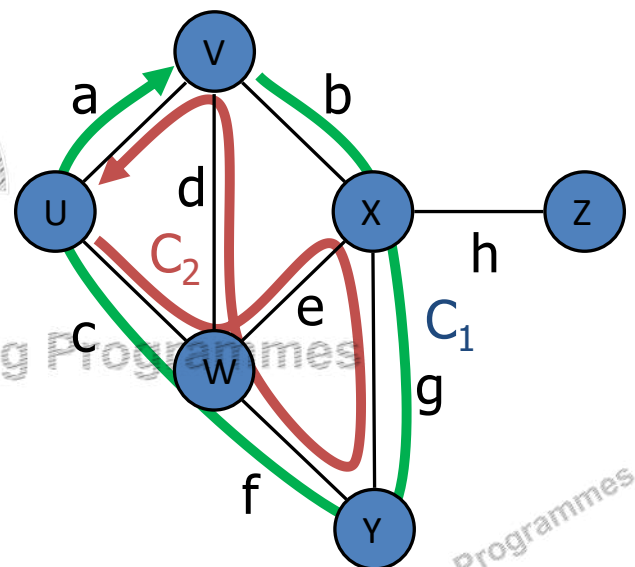
- **Circular** sequence of alternating vertices and edges
- each edge is preceded and followed by its endpoints

Simple Cycle

- Cycle such that all its vertices and edges are **distinct**

Examples

- $C_1 = (V, b, X, g, Y, f, W, c, U, a, \downarrow)$ is a **simple cycle**
- $C_2 = (U, c, W, e, X, g, Y, f, W, d, V, a, \downarrow)$ is a cycle that is **not simple**



Graph Terminology

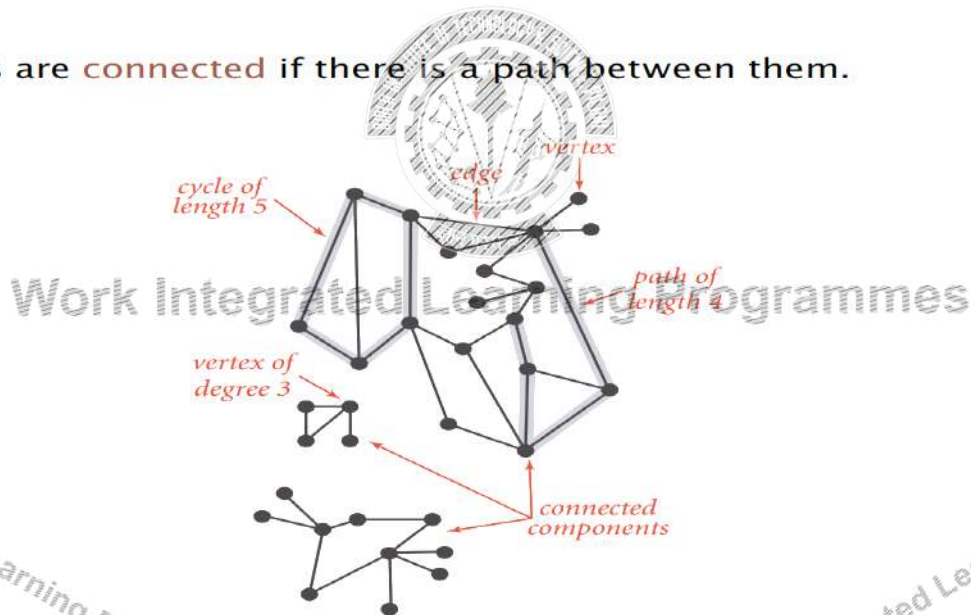


Graph terminology

Path. Sequence of vertices connected by edges.

Cycle. Path whose first and last vertices are the same.

Two vertices are **connected** if there is a path between them.



Graph Types



Null Graph : A graph having no edges.



Trivial Graph : A graph with only one vertex.



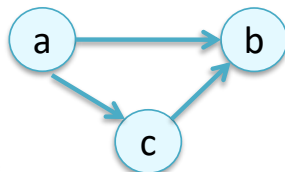
Undirected Graph : A graph that contains edges but the edges are not directed ones.



Directed Graph : A graph that contains edges which have direction.



Simple Graph : A graph that contains no loops and no parallel edges.

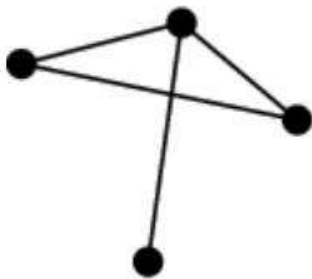


Many more

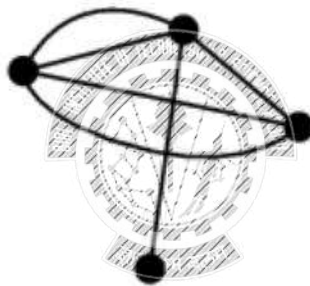
Quick Check



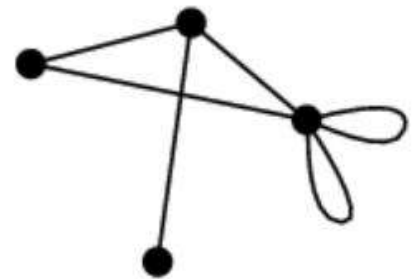
Which of the below is a Simple Graph ?



Simple Graph



Non-Simple Graph
with multiple (parallel)
edges

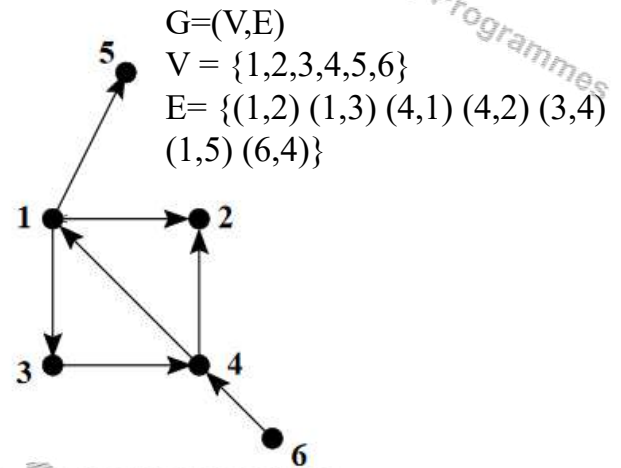


Non-Simple Graph
with loops

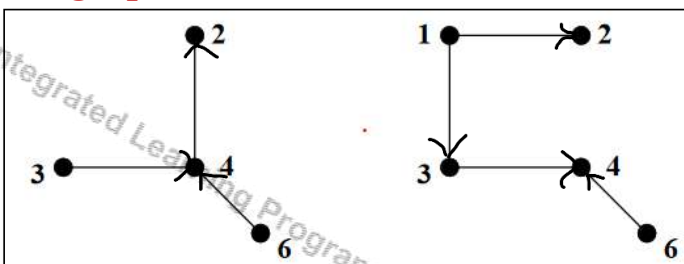
Subgraphs



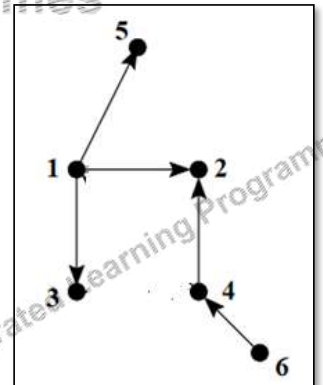
- A **subgraph** S of a graph G is a graph such that :
 - The **vertices** of S are a **subset** of the vertices of G
 - The **edges** of S are a **subset** of the edges of G
- A **spanning subgraph** of G is a subgraph that contains all the **vertices** of G .



Subgraphs :



Spanning subgraph:

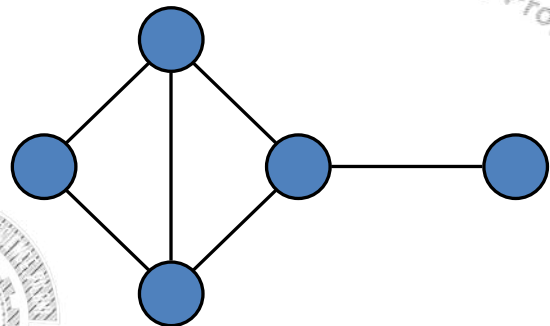


26

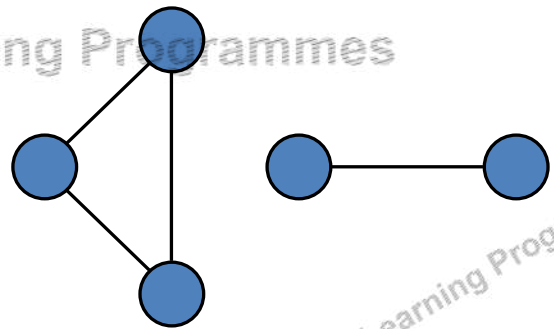
Connectivity



- A graph is **connected** if there is a path between every pair of vertices.
- From every vertex to any other vertex, there should be some path to traverse.
- That is called the connectivity of a graph. A graph with multiple disconnected vertices and edges is said to be disconnected.



Connected graph



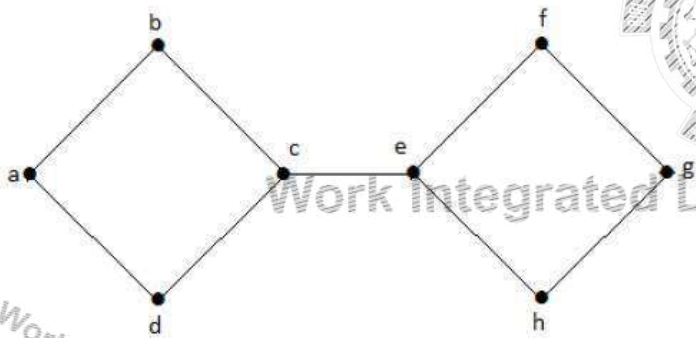
Non connected graph with two connected components

Connectivity – Cut Vertex

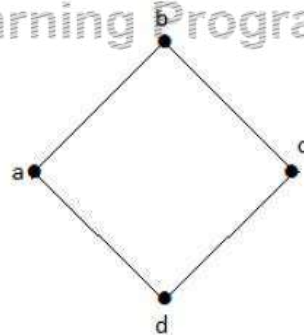


- Let 'G' be a connected graph. A vertex $V \in G$ is called a cut vertex of 'G', if 'G-V' (Delete 'V' from 'G') results in a disconnected graph. Removing a cut vertex from a graph breaks it into two or more graphs.

Note – Removing a cut vertex will render a graph disconnected.



By removing 'e' or 'c', the graph will become a disconnected graph.



Hence it is a disconnected graph with cut vertex as 'e'. Similarly, 'c' is also a cut vertex for the above graph.

Connectivity – Cut Edge

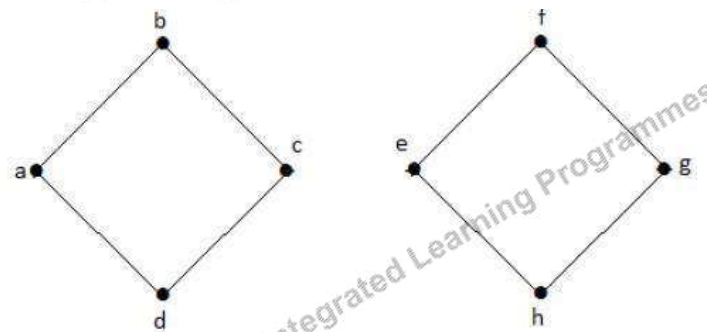


- Let 'G' be a connected graph. An edge 'e' $\in G$ is called a cut edge if 'G-e' results in a disconnected graph.
- If removing an edge in a graph results in two or more graphs, then that edge is called a Cut Edge.

Note – Removing a cut edge will render a graph disconnected.



By removing the edge (c, e) from the graph, it becomes a disconnected graph.



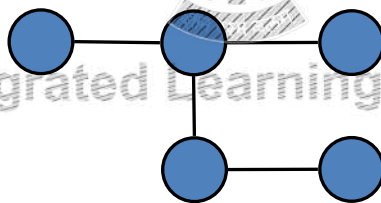
In the above graph, removing the edge (c, e) breaks the graph into two which is nothing but a disconnected graph. Hence, the edge (c, e) is a cut edge of the graph.

Trees and Forests



- A (free) **tree** is an undirected graph T such that
- T is **connected**
 - T has **no cycles**

[This definition of tree is different from the one of a rooted tree]

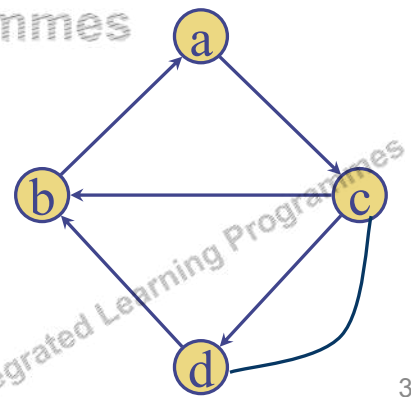
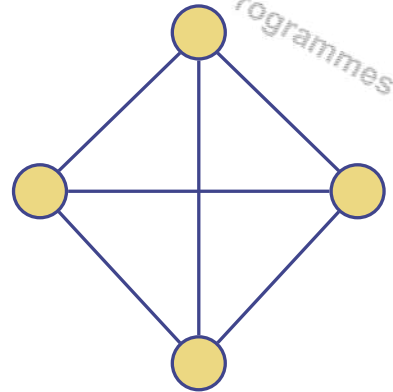


Tree

Operations on a Graph



- Return the number, n , of vertices in G .
- Return the number, m , of edges in G .
- Return a set or list containing all n vertices in G .
- Return a set or list containing all m edges in G .
- Return some vertex, v , in G .
- Return the degree, $\deg(v)$, of a given vertex, v , in G .
- Return a set or list containing all the edges incident upon a given vertex, v , in G .
- Return a set or list containing all the vertices adjacent to a given vertex, v , in G .
- Return the two end vertices of an edge, e , in G ; if e is directed, indicate which vertex is the origin of e and which is the destination of e .
- Return whether two given vertices, v and w , are adjacent in G .

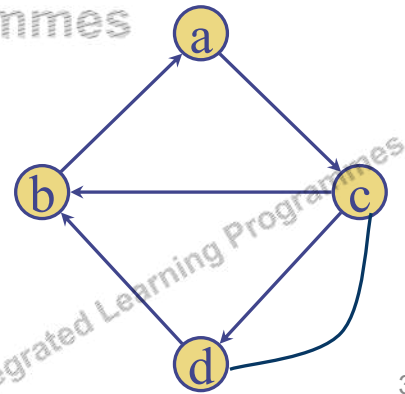
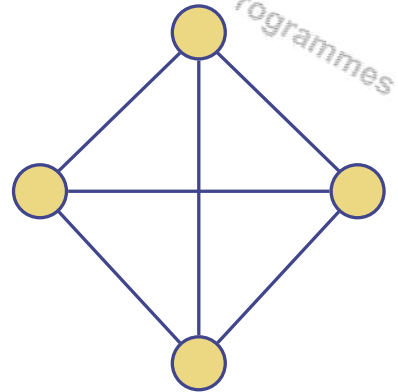


31

Operations on a Graph



- Indicate whether a given edge, e , is directed in G .
- Return the in-degree of v , $\text{inDegree}(v)$.
- Return a set or list containing all the incoming (or outgoing) edges incident upon a given vertex, v , in G .
- Return a set or list containing all the vertices adjacent to a given vertex, v , along incoming (or outgoing) edges in G .
- Insert a new directed (or undirected) edge, e , between two given vertices, v and w , in G .
- Insert a new (isolated) vertex, v , in G .
- Remove a given edge, e , from G .
- Remove a given vertex, v , and all its incident edges from G .



32

Graph Representation Strategies



- Edge List
- Adjacency Matrix
- Adjacency List



Work Integrated Learning Programmes

Edge List



➤ Vertex object

- element
- reference to position in vertex sequence

➤ Edge object

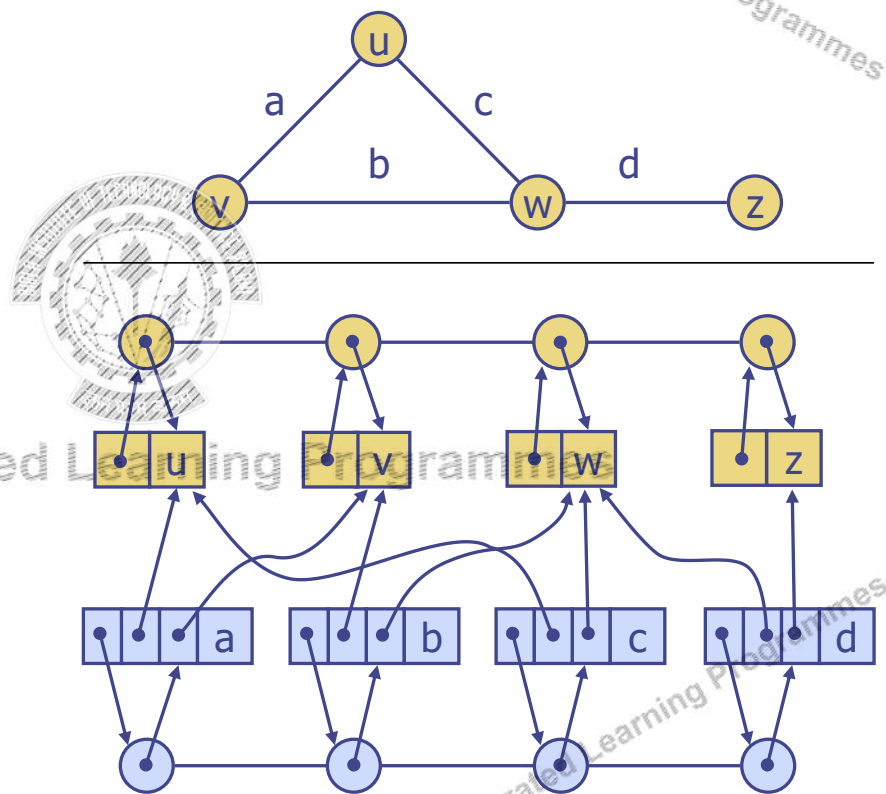
- element
- origin vertex object
- destination vertex object
- reference to position in edge sequence

➤ Vertex sequence

- sequence of vertex objects

➤ Edge sequence

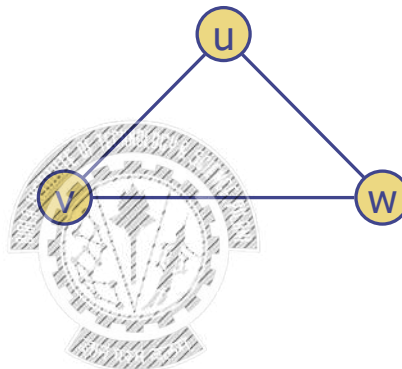
- sequence of edge objects



Edge List Simplified



```
[  
  (u, v) ,  
  (v, w) ,  
  (u, w)  
]
```



Example 2:

edge_list = [(0, 1), (1, 2), (2, 3), (0, 2), (3, 2), (4, 5), (5, 4)]

Adjacency Matrix

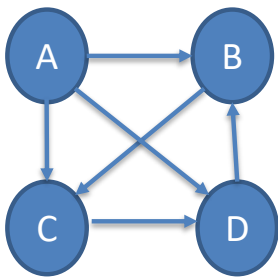


The adjacency matrix A of a graph G is formally defined as :

$$A[i][j] = \begin{cases} 1 & \text{if there is an edge from vertex } i \text{ to vertex } j \\ 0 & \text{if there is no edge from vertex } i \text{ to vertex } j \end{cases}$$

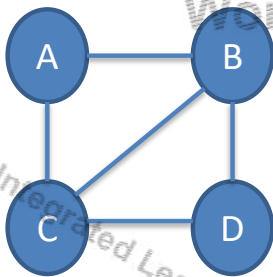
- It is clear from the definition that an adjacency matrix of a graph with n vertices is a boolean square matrix with n rows and n columns with entries 1's and 0's (bit-matrix)
- Note that the above definition holds true for both directed and undirected graph. *If it's a adjacency matrix of a undirected graph, then the matrix will look symmetric and diagonal being 0's.*
- Note: If there are multiple edges from vertex u to v , then the number of edges can also be used in the matrix.

Adjacency Matrix Examples



Adjacency Matrix

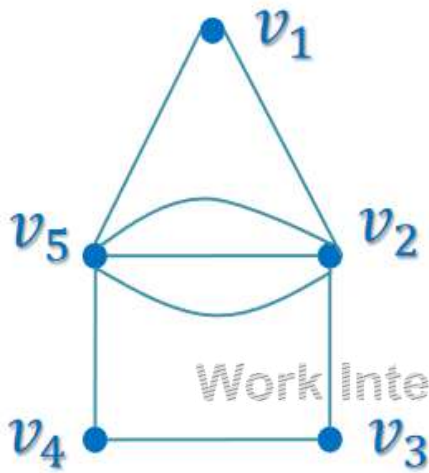
	A	B	C	D
A	0	1	1	1
B	0	0	1	0
C	0	0	0	1
D	0	1	0	0



Adjacency Matrix

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Adjacency Matrix Examples



$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 3 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 3 & 0 & 1 & 0 \end{pmatrix}$$

Adjacency Matrix: Pros and Cons



Advantages

- Fast to tell whether edge exists between any two vertices i and j (and to get its weight)

Disadvantages

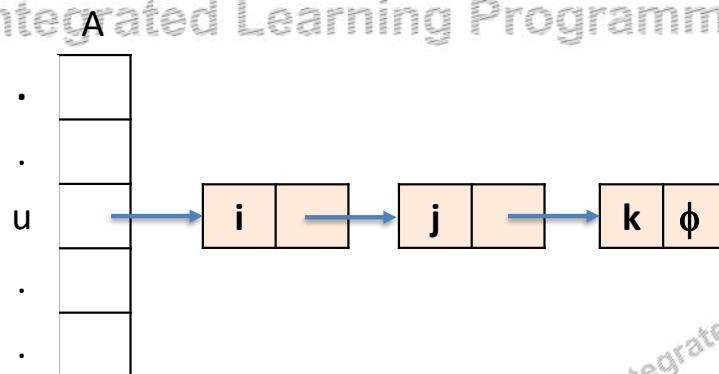
- Consumes a lot of memory on **sparse** graphs (ones with few edges)
- Redundant information for undirected graphs



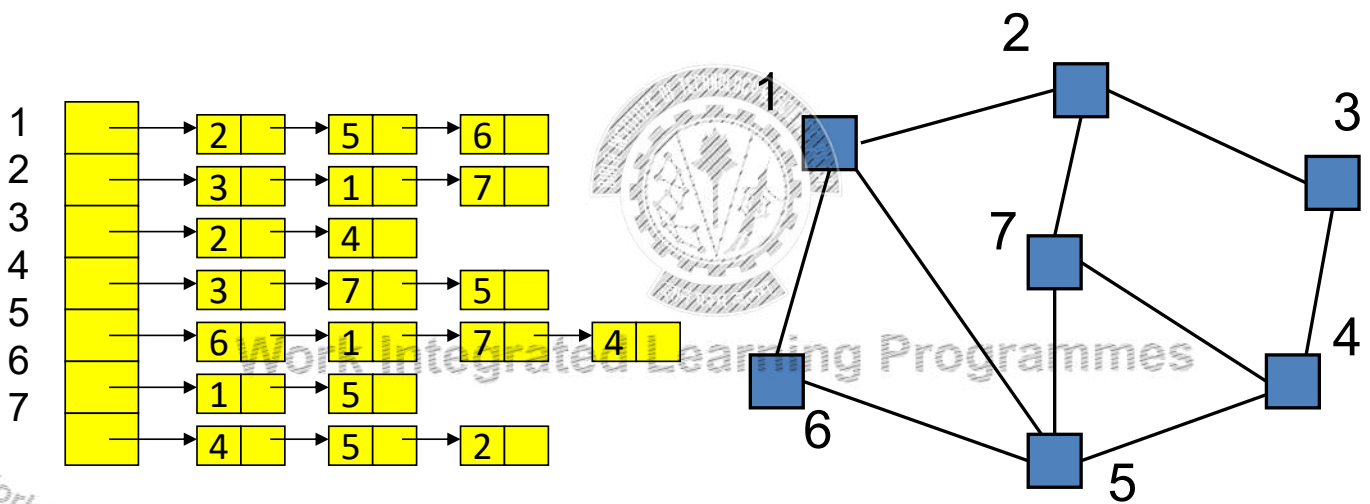
Adjacency Lists



- An adjacency linked list is an array of n linked lists where n is the number of vertices in graph G . Each location of the array represents a vertex of the graph. For each vertex $u \in V$, a linked list consisting of all the vertices adjacent to u is created and stored in $A[u]$. The resulting array A is an adjacency list.
- Note: It is clear from the def. that if i, j and k are the vertices adjacent to the vertex u , then i, j and k are stored in a linked list and starting address of linked list is stored in $A[u]$ as shown below:



Adjacency List Example



Adjacency List: Pros and Cons

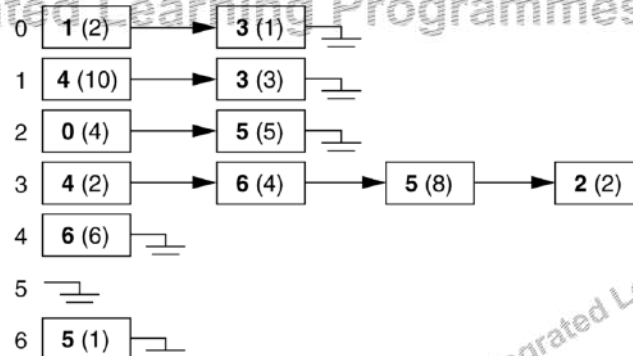


Advantages:

- new nodes can be added easily
- new nodes can be connected with existing nodes easily
- "who are my neighbors" easily answered

Disadvantages:

- determining whether an edge exists between two nodes:
 $O(\text{average degree})$



Asymptotic Performance



▶ n vertices, m edges ▶ no parallel edges ▶ no self-loops ▶ Bounds are “big-Oh”	Edge List	Adjacency List	Adjacency Matrix
Space	$n + m$	$n + m$	n^2
incidentEdges (v)	m	$\text{deg}(v)$	n
areAdjacent (v, w)	m	$\min(\text{deg}(v), \text{deg}(w))$	1
insertVertex (o)	1	1	n^2
insertEdge (v, w, o)	1	1	1
removeVertex (v)	m	$\text{deg}(v)$	n^2
removeEdge (e)	1	1	1

Exercise 2



Consider the following two adjacency matrices:

(1)

	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	1	0	0	1
3	0	1	0	0	0	1
4	0	0	0	0	1	0
5	1	0	0	1	0	1
6	0	1	1	0	1	0

(2)

	1	2	3	4	5	6
1	0	1	0	0	0	0
2	1	0	0	0	0	0
3	0	1	0	0	0	1
4	1	0	0	1	1	0
5	1	0	0	1	0	1
6	0	1	1	0	0	0

For each, answer the following questions:

- Can this matrix be the adjacency matrix of an undirected graph? Could it represent a directed graph? Why or why not?
- For both adjacency matrices draw the corresponding undirected graph if possible, otherwise draw the corresponding directed graph.
- Give the adjacency list representation for both graphs.

Exercise 3



Create graph from given adjacency matrix:

a)

	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	0
3	1	0	0	0



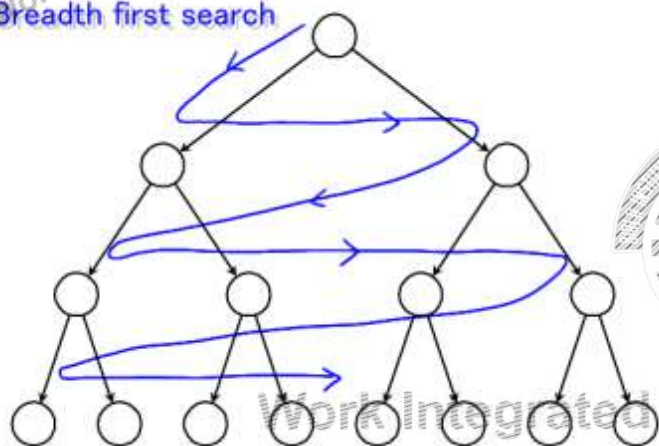
b)

$$A = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 3 \\ 0 & 0 & 0 & 0 & 3 & 0 \end{bmatrix}$$

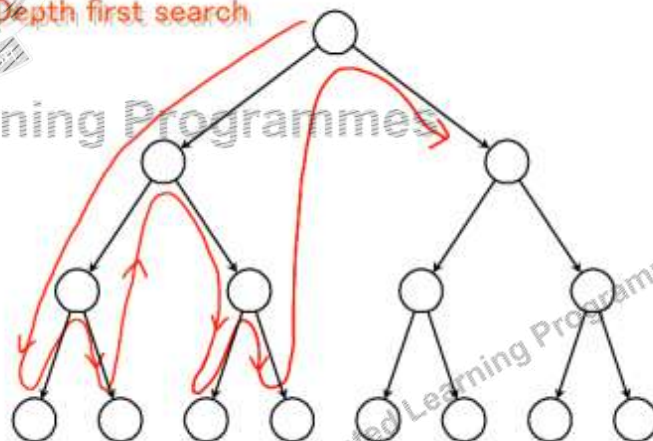
Graph Traversals



Breadth first search



Depth first search





See you in the next class to explore more on Graphs!



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



48

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #7

DSECLZG519 – Graph Traversals & BST

Agenda for CS #7



1) Recap and continuation of CS#6

2) Graph Traversals:

- BFS
- DFS

3) BST

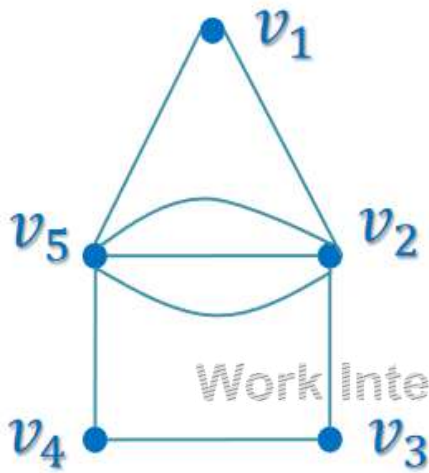
- Motivation
- Insertion
- Deletion

4) Exercises

5) Q&A



Adjacency Matrix Examples



$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 3 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 3 & 0 & 1 & 0 \end{pmatrix}$$

Adjacency Matrix: Pros and Cons



Advantages

- Fast to tell whether edge exists between any two vertices i and j (and to get its weight)

Disadvantages

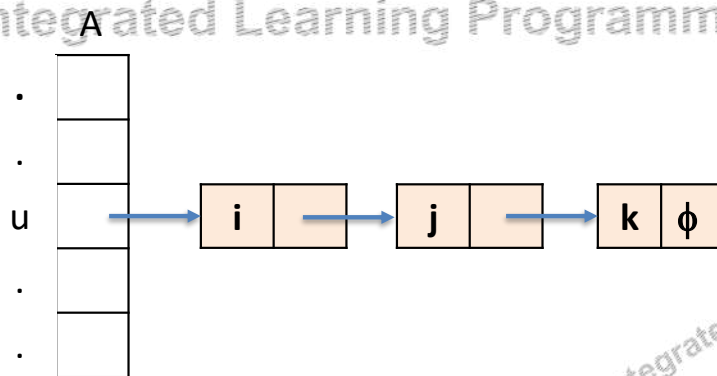
- Consumes a lot of memory on **sparse** graphs (ones with few edges)
- Redundant information for undirected graphs



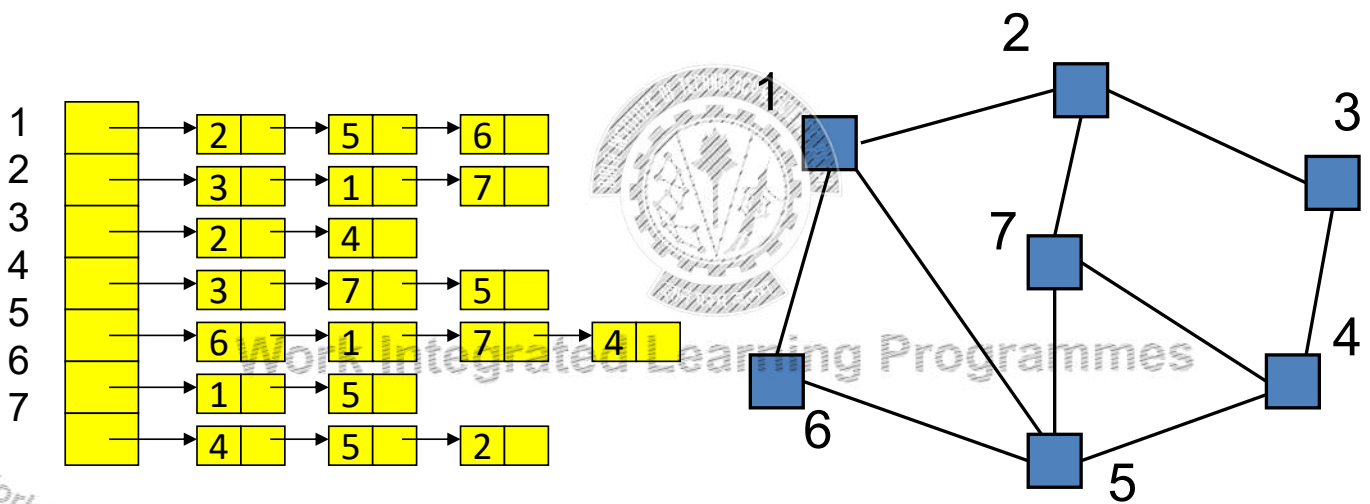
Adjacency Lists



- An adjacency linked list is an array of n linked lists where n is the number of vertices in graph G . Each location of the array represents a vertex of the graph. For each vertex $u \in V$, a linked list consisting of all the vertices adjacent to u is created and stored in $A[u]$. The resulting array A is an adjacency list.
- Note: It is clear from the def. that if i, j and k are the vertices adjacent to the vertex u , then i, j and k are stored in a linked list and starting address of linked list is stored in $A[u]$ as shown below:



Adjacency List Example



Adjacency List: Pros and Cons

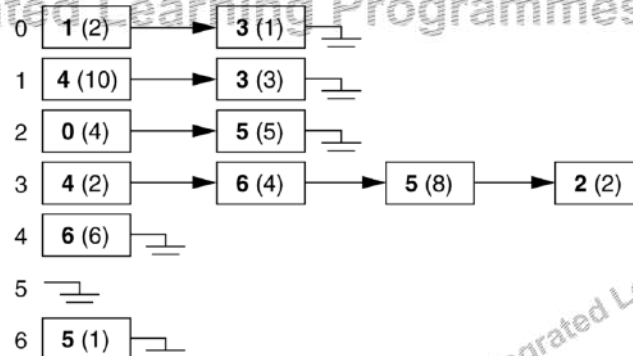


Advantages:

- new nodes can be added easily
- new nodes can be connected with existing nodes easily
- "who are my neighbors" easily answered

Disadvantages:

- determining whether an edge exists between two nodes:
 $O(\text{average degree})$



Breadth First Search (BFS)



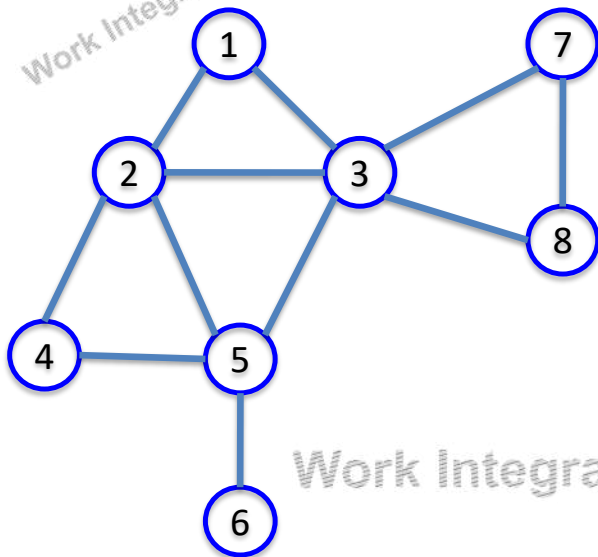
- Breadth-first search (BFS) is a general technique for traversing a graph.
- A BFS traversal of a graph G :
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- BFS on a graph with n vertices and m edges takes $O(n + m)$ time
- BFS can be further extended to solve other graph problems
 - Find and report a path with the minimum number of edges between two given vertices
 - Find a simple cycle, if there is one

Breadth First Search (BFS)

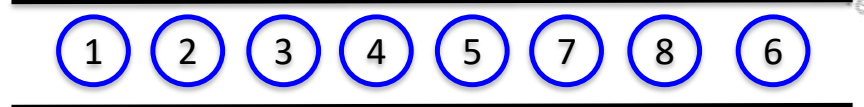


- It is very common to use a *queue* to keep track of:
 - nodes to be visited next, or
 - nodes that we have already visited.
- Typically, use of a queue leads to a *breadth-first* visit order.
- Breadth-first visit order is “cautious” in the sense that it examines every path of length i before going on to paths of length $i+1$.
- Dotted lines depict *Cross edges* and solid lines depict the *discovery edges*.

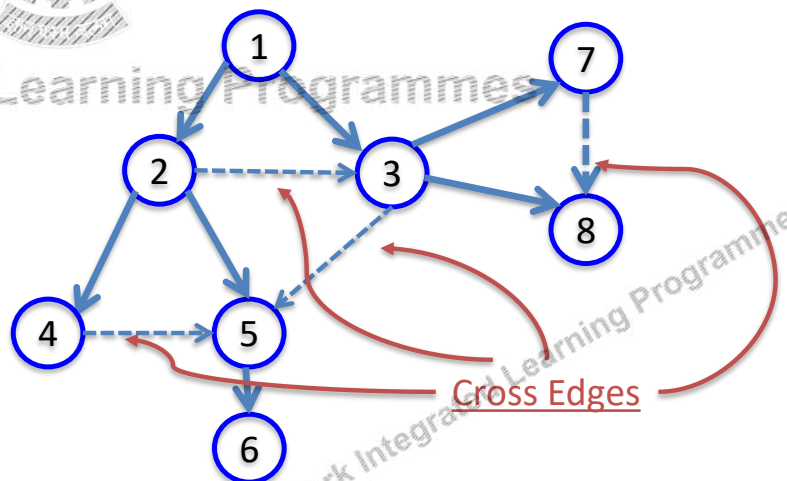
BFS - Demo



Queue - FIFO

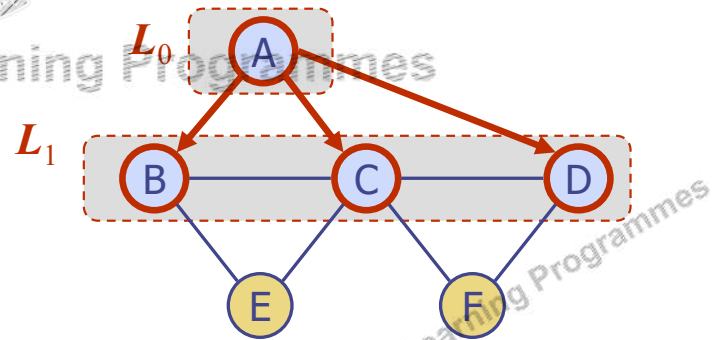
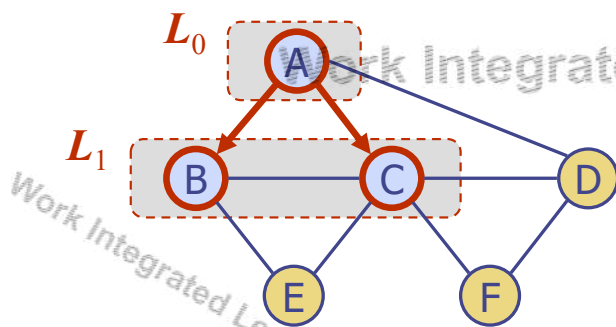
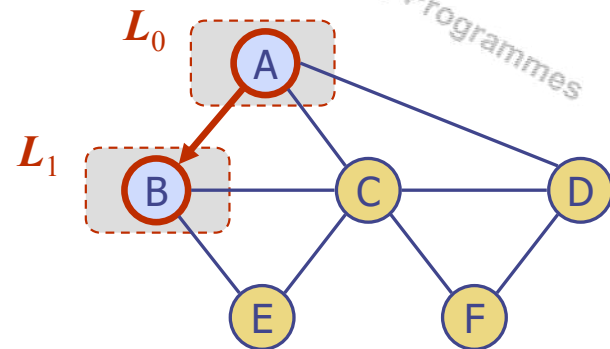
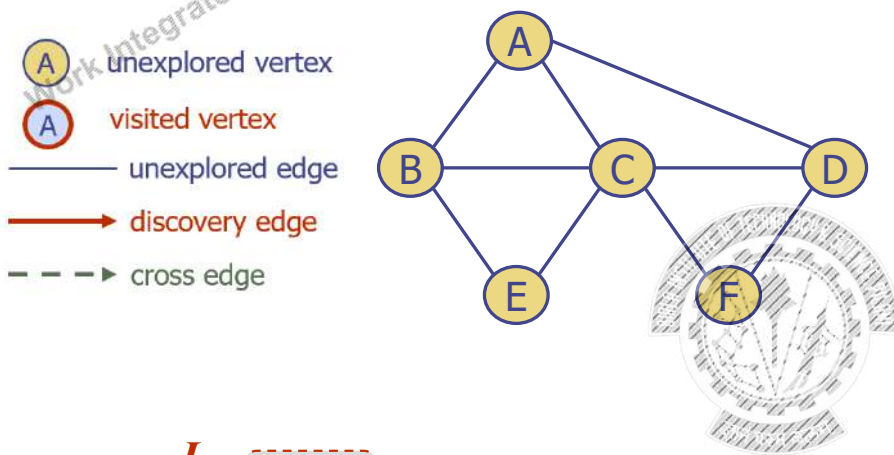


BFS Spanning Tree

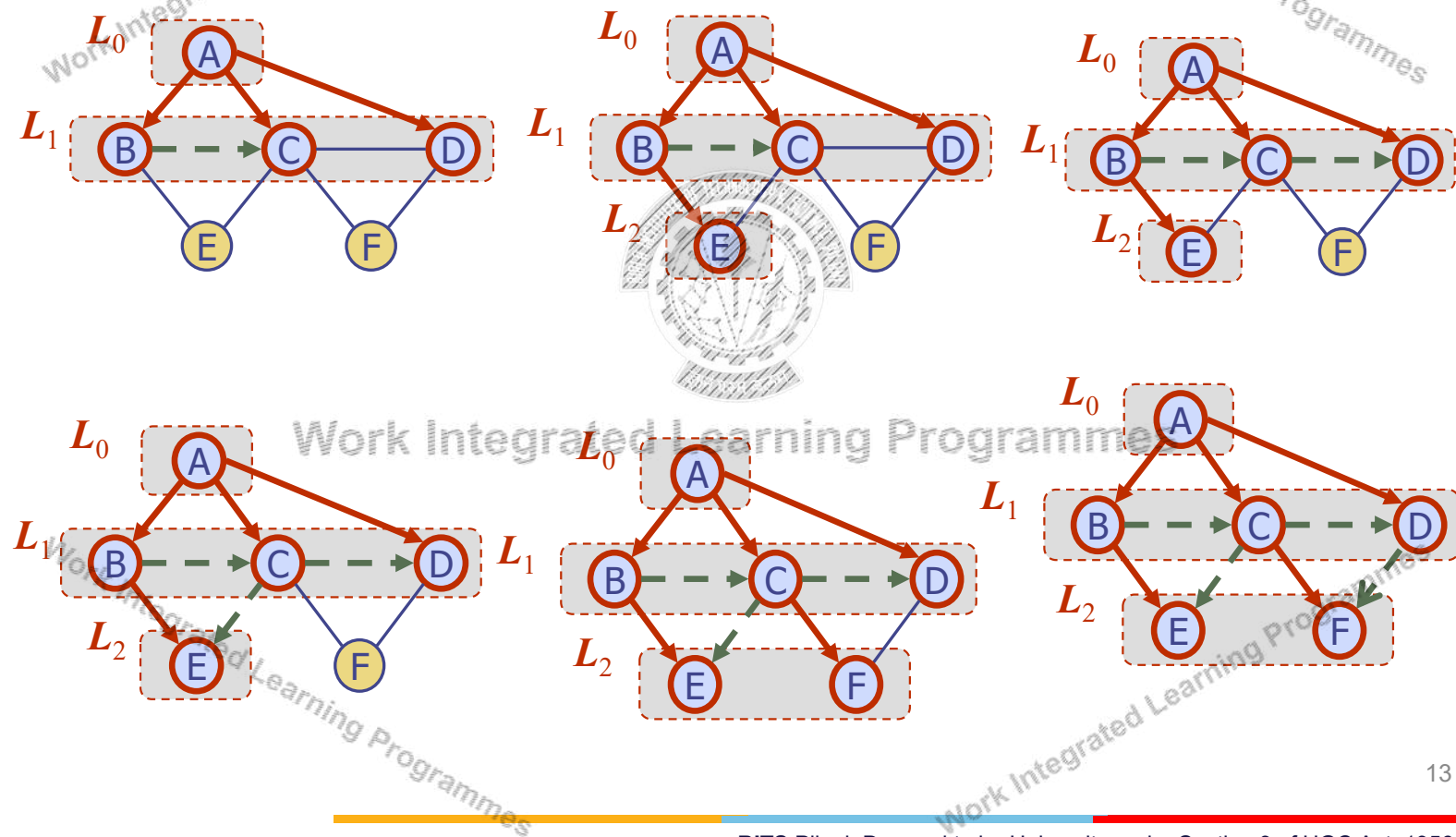


BFS: 1,2,3,4,5,7,8,6

BFS – Example 2



BFS – Example 2



BFS- Pseudocode



```
BFS (G, s)                                //Where G is the graph and s is the source node
    let Q be queue.
    Q.enqueue( s ) //Inserting s in queue until all its neighbour vertices are marked.

    mark s as visited.
    while ( Q is not empty)
        //Removing that vertex from queue whose neighbour will be visited now
        v = Q.dequeue( )

        //processing all the neighbours of v
        for all neighbours w of v in Graph G
            if w is not visited
                Q.enqueue( w )                //Stores w in Q to further visit its neighbour
                mark w as visited.
```

The time complexity of BFS is $O(V+E)$ where V is the number of nodes and E is the number of edges.

BFS Applications



- We can specialize the BFS traversal of a graph G to solve the following problems in $O(n + m)$ time
 - Compute the connected components of G
 - Compute a spanning forest of G
 - Find a simple cycle in G , or report that G is a forest
 - Given two vertices of G , find a path in G between them with the minimum number of edges, or report that no such path exists

Good Design exercise! Try the above problems using BFS traversal

Depth First Search (DFS)



- Depth-first search (DFS) is a general technique for traversing a graph
- A DFS traversal of a graph G
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- DFS on a graph with n vertices and m edges takes $O(n + m)$ time
- DFS can be further extended to solve other graph problems
 - Find and report a path between two given vertices
 - Find a cycle in the graph

DFS

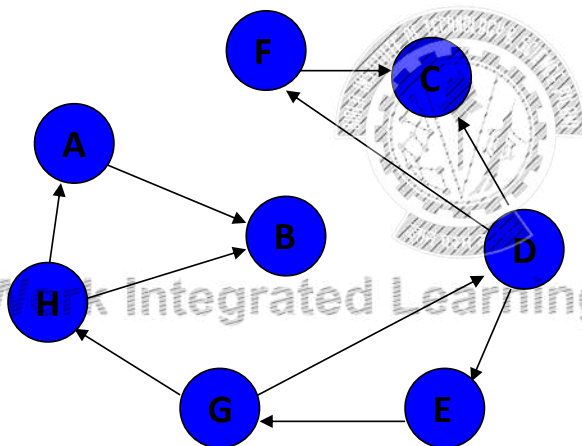


- It is very common to use a *stack* to keep track of:
 - nodes to be visited next, or
 - nodes that we have already visited.
- Typically, use of a stack leads to a *depth-first* visit order.
- Depth-first visit order is “aggressive” in the sense that it examines complete paths.
- Solid lines depict the *discovery edges* and dotted lines depict the *Back edges*.

DFS-Demo



Consider this graph below and perform DFS with D as start node.

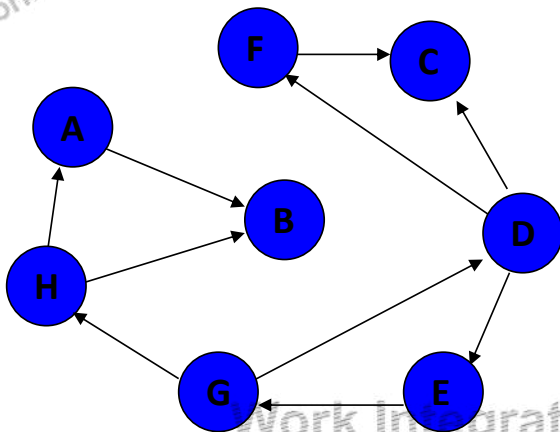
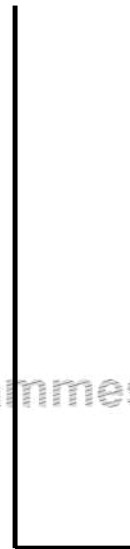


Walk-Through



Visited Array

A	
B	
C	
D	
E	
F	
G	
H	



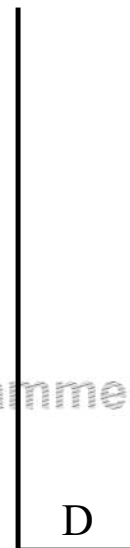
Task: Conduct a depth-first search of the graph starting with node D

Walk-Through



Visited Array

A	
B	
C	
D	✓
E	
F	
G	
H	

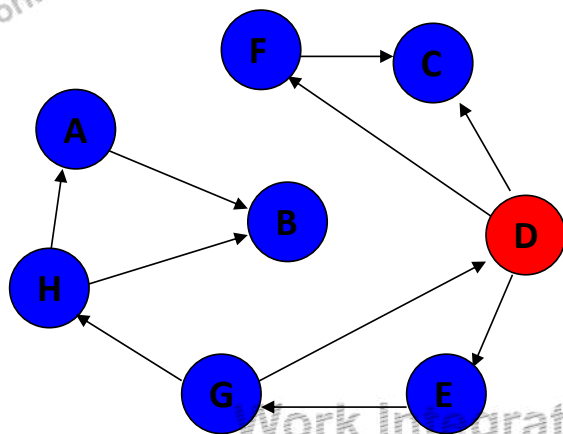


The order nodes are visited:

D

Visit D

Walk Through



Visited Array

A	
B	
C	
D	✓
E	
F	
G	
H	

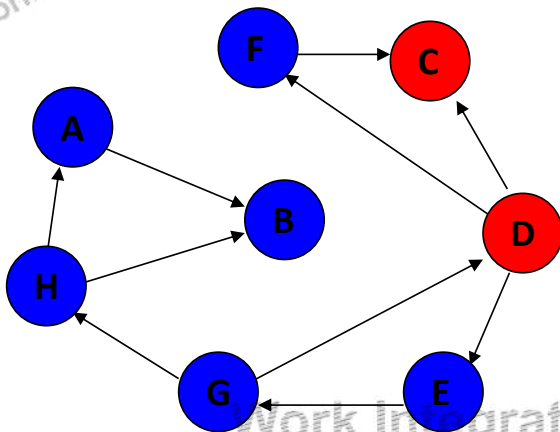
D

The order nodes are visited:

D

Consider nodes adjacent to D, decide to visit C first (Rule: visit adjacent nodes in alphabetical order)

Walk Through



A	
B	
C	✓
D	✓
E	
F	
G	
H	

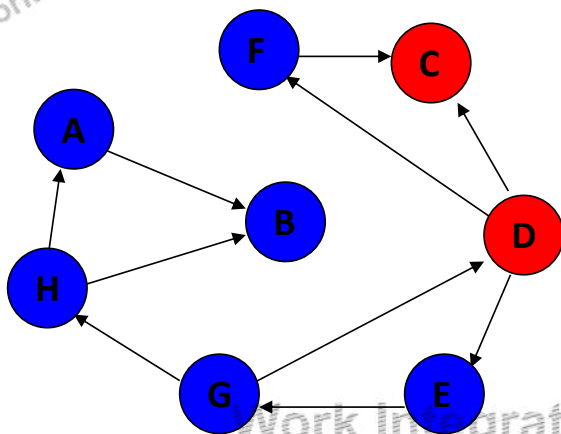
C
D

The order nodes are visited:

D, C

Visit C

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	
F	
G	
H	

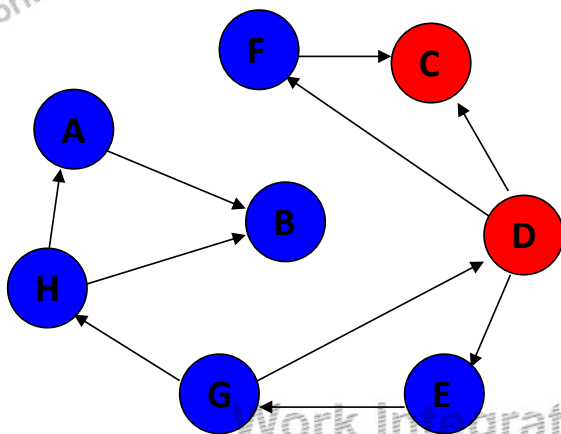
C
D

The order nodes are visited:
D, C

No nodes adjacent to C; cannot continue → *backtrack*, i.e., pop stack and restore previous state

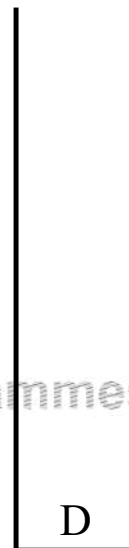
23

Walk-Through



Visited Array

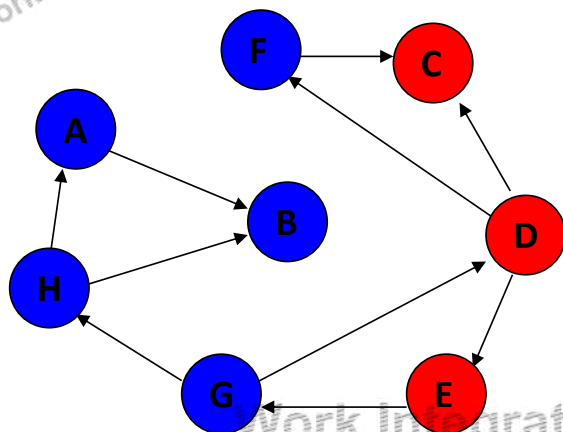
A	
B	
C	✓
D	✓
E	
F	
G	
H	



The order nodes are visited:
D, C

**Back to D – C has been visited,
decide to visit E next**

Walk Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	
H	

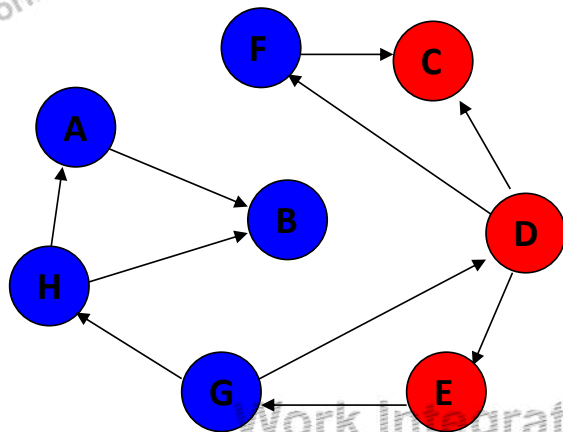
E
D

The order nodes are visited:

D, C, E

**Back to D – C has been visited,
decide to visit E next**

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	
H	

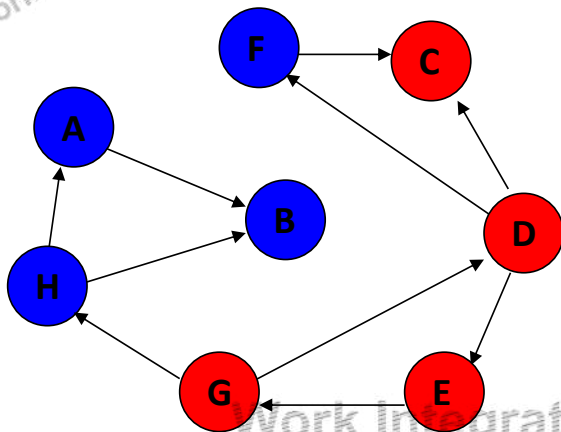
E
D

The order nodes are visited:

D, C, E

Only G is adjacent to E

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	✓
H	

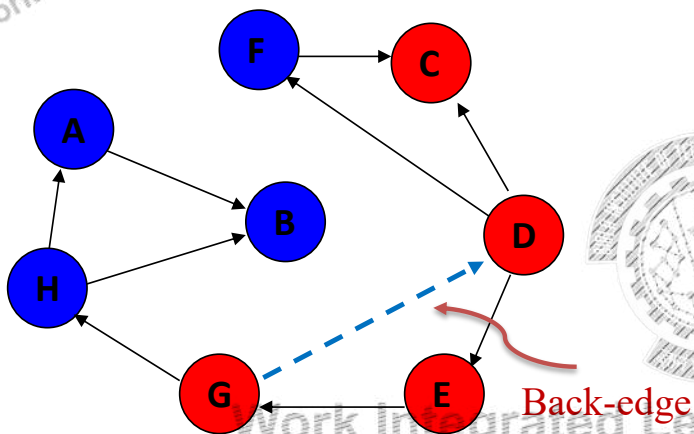
G
E
D

The order nodes are visited:

D, C, E, G

Visit G

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	✓
H	

G
E
D

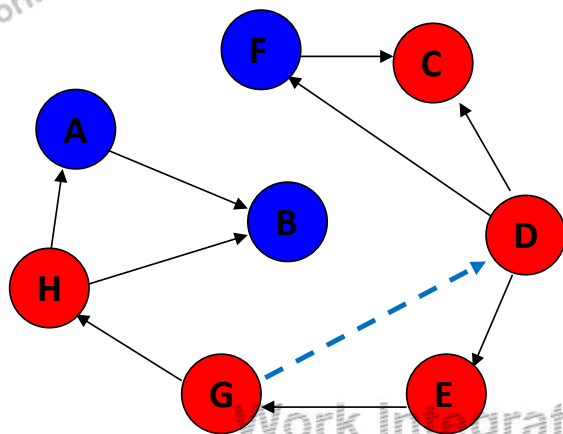
The order nodes are visited:

D, C, E, G

Nodes D and H are adjacent to G. D has already been visited. Decide to visit H.

28

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	✓
H	✓

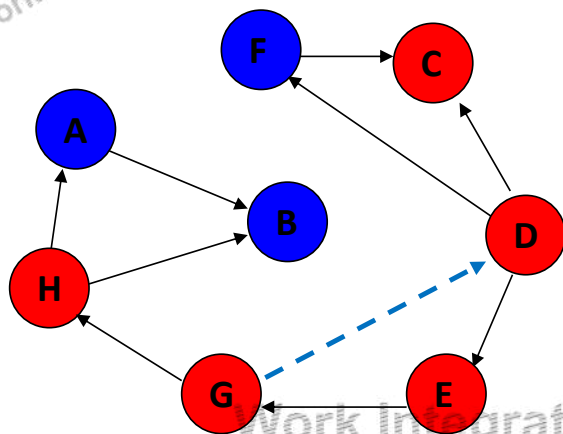
H
G
E
D

The order nodes are visited:

D, C, E, G, H

Visit H

Walk-Through



Visited Array

A	
B	
C	✓
D	✓
E	✓
F	
G	✓
H	✓

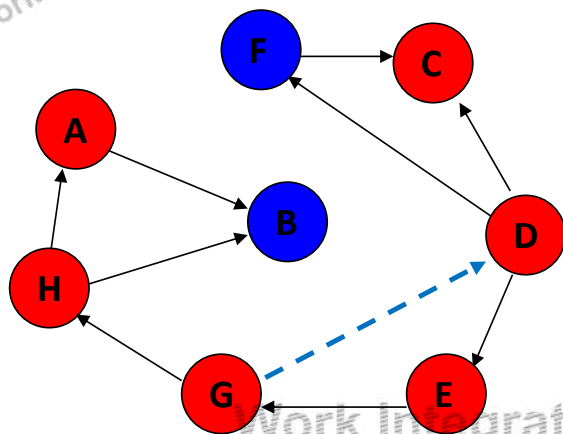
H
G
E
D

The order nodes are visited:

D, C, E, G, H

**Nodes A and B are adjacent to F.
Decide to visit A next.**

Walk-Through



Visited Array

A	✓
B	
C	✓
D	✓
E	✓
F	
G	✓
H	✓

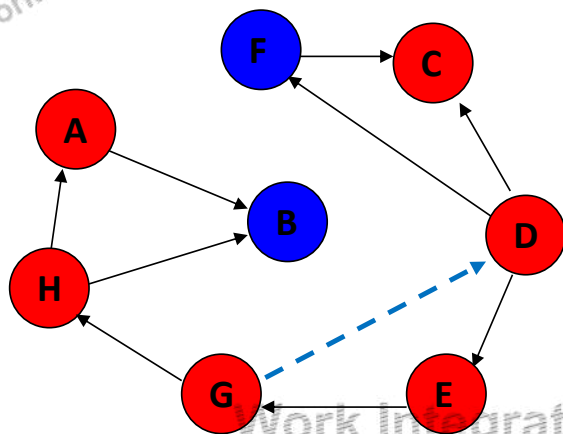
A
H
G
E
D

The order nodes are visited:

D, C, E, G, H, A

Visit A

Walk-Through



Visited Array

A	✓
B	
C	✓
D	✓
E	✓
F	
G	✓
H	✓

A
H
G
E
D

The order nodes are visited:

D, C, E, G, H, A

**Only Node B is adjacent to A.
Decide to visit B next.**

Walk Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

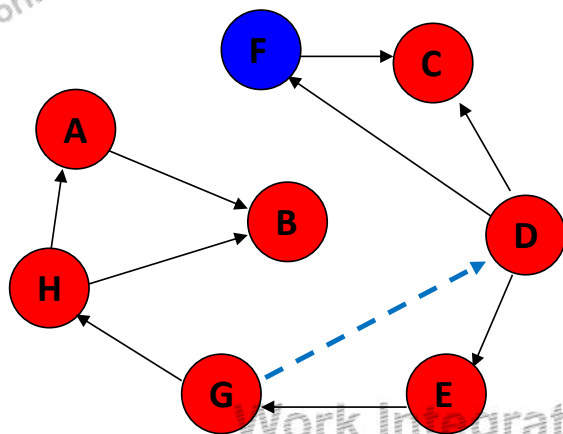
B
A
H
G
E
D

Visit B

The order nodes are visited:

D, C, E, G, H, A, B

Walk Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

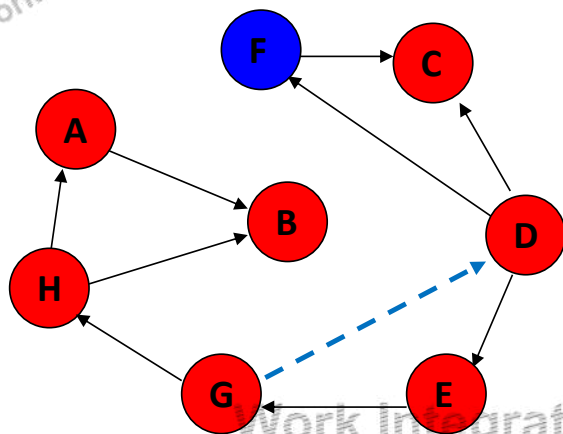
A
H
G
E
D

The order nodes are visited:

D, C, E, G, H, A, B

No unvisited nodes adjacent to B. Backtrack (pop the stack).

Walk Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

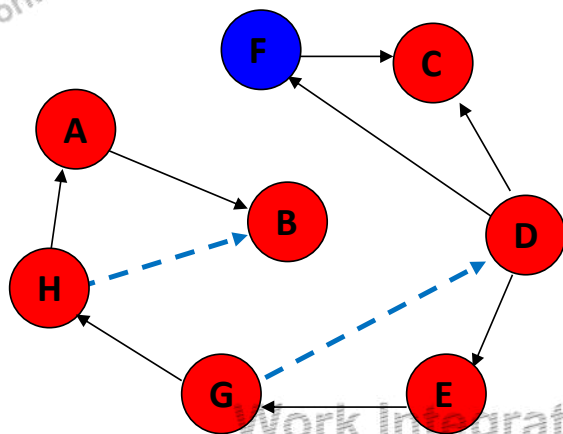
H
G
E
D

The order nodes are visited:

D, C, E, G, H, A, B

No unvisited nodes adjacent to A. Backtrack (pop the stack).

Walk Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

G
E
D

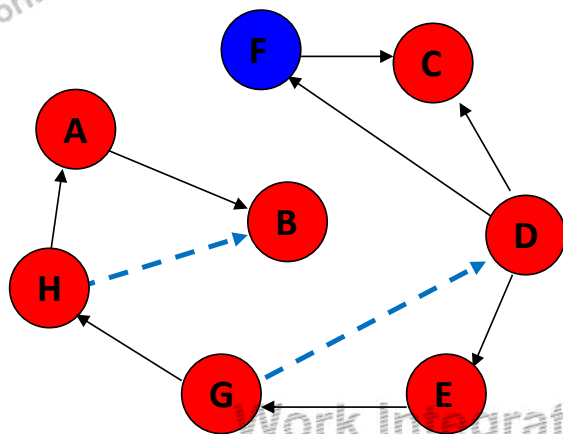
The order nodes are visited:

D, C, E, G, H, A, B

No unvisited nodes adjacent to H. Backtrack (pop the stack).

36

Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

E
D

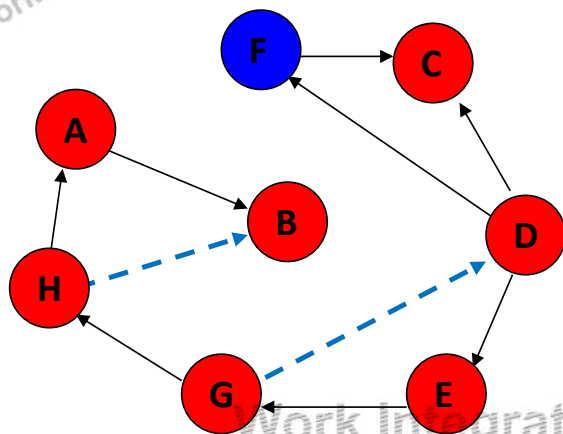
The order nodes are visited:

D, C, E, G, H, A, B

No unvisited nodes adjacent to G. Backtrack (pop the stack).

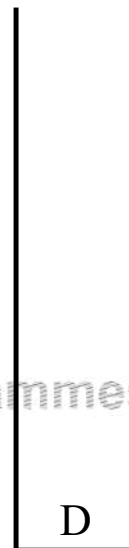
37

Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

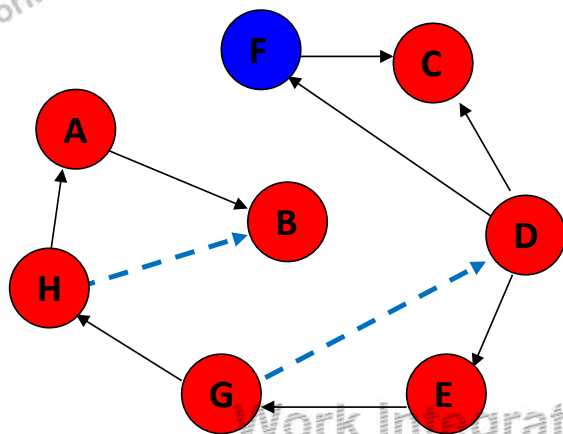


The order nodes are visited:

D, C, E, G, H, A, B

No unvisited nodes adjacent to E. Backtrack (pop the stack).

Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	
G	✓
H	✓

D

The order nodes are visited:

D, C, E, G, H, A, B

F is unvisited and is adjacent to D. Decide to visit F next.

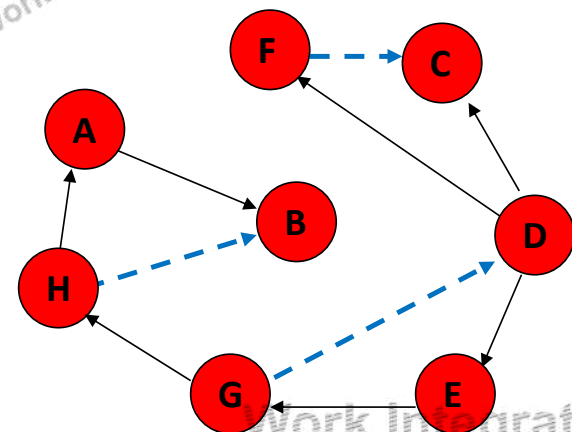
Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	✓
G	✓
H	✓

F
D

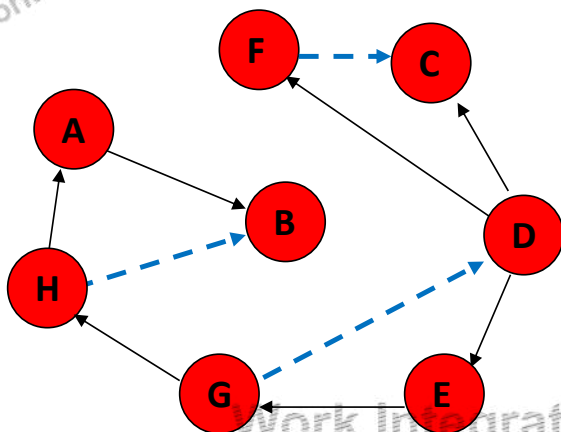


The order nodes are visited:

D, C, E, G, H, A, B, F

Visit F

Walk-Through



A	✓
B	✓
C	✓
D	✓
E	✓
F	✓
G	✓
H	✓

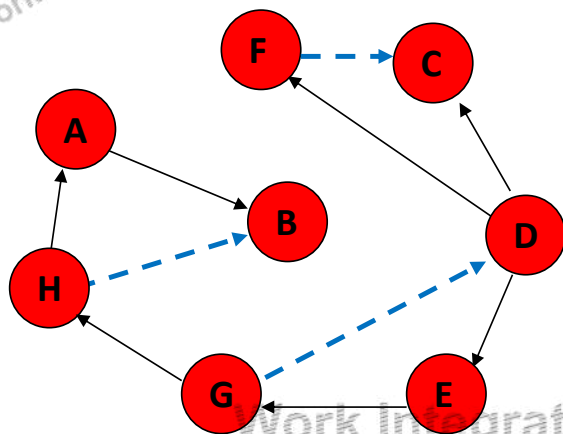
D

The order nodes are visited:

D, C, E, G, H, A, B, F

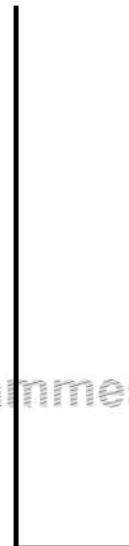
No unvisited nodes adjacent to F. Backtrack.

Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	✓
G	✓
H	✓

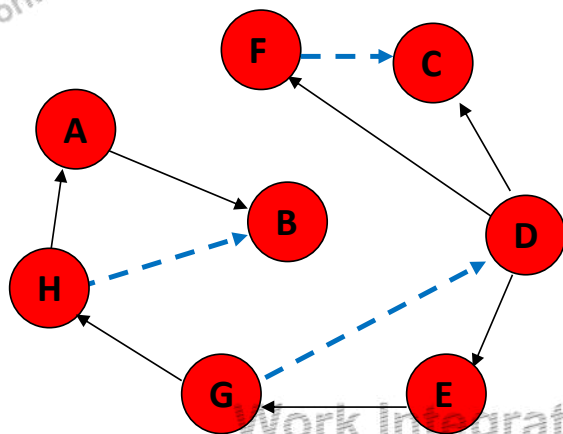


The order nodes are visited:

D, C, E, G, H, A, B, F

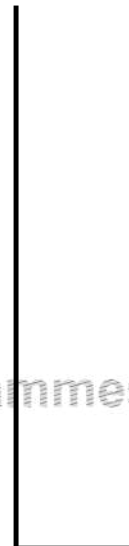
No unvisited nodes adjacent to D. Backtrack.

Walk-Through



Visited Array

A	✓
B	✓
C	✓
D	✓
E	✓
F	✓
G	✓
H	✓

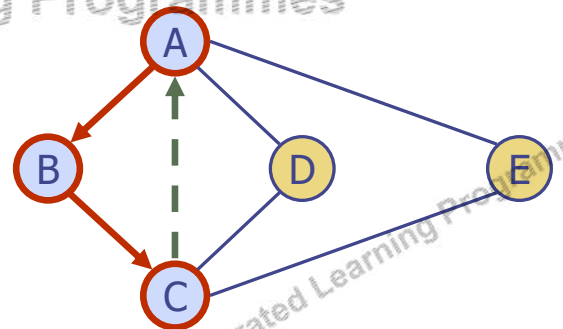
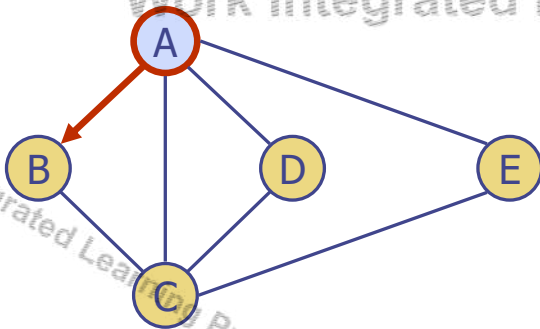
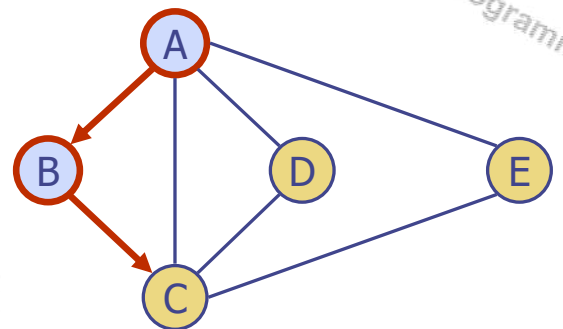
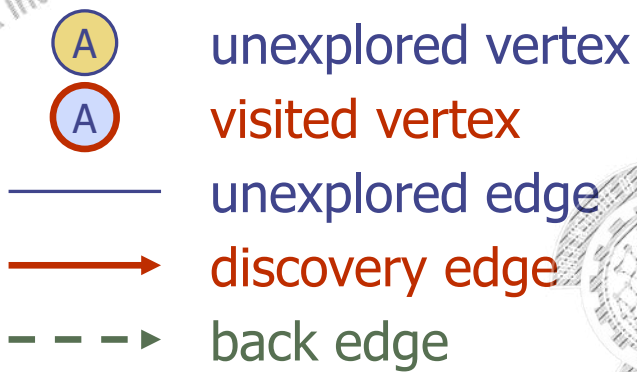


The order nodes are visited:

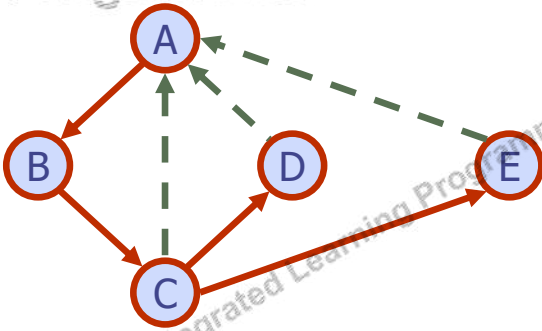
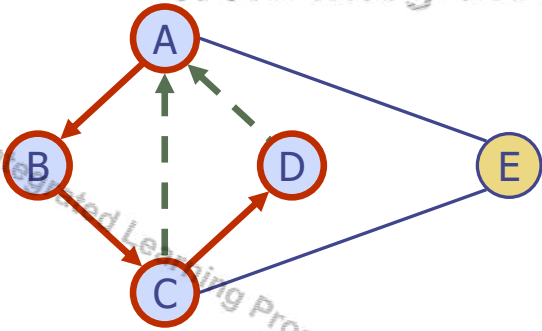
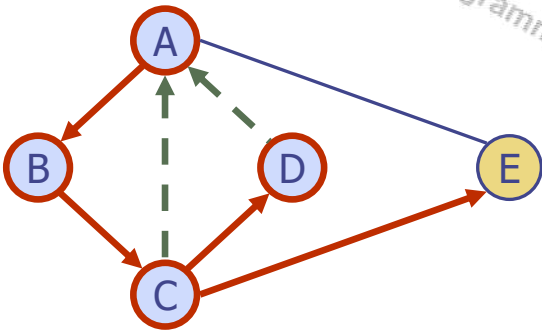
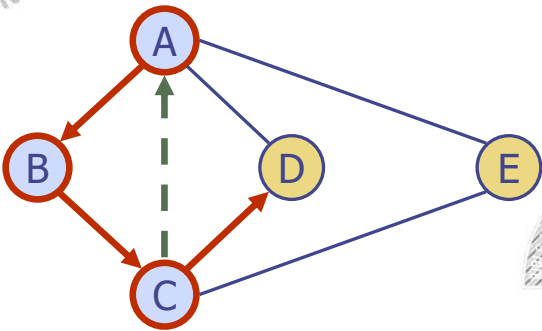
D, C, E, G, H, A, B, F

Stack is empty. Depth-first traversal is done.

DFS Example 2



DFS Example 2



DFS Pseudocode



```
DFS-iterative (G, s):                                     //Where G is graph and s is source vertex
let S be stack
S.push( s )                                              //Inserting s in stack
mark s as visited.
while ( S is not empty):
    //Pop a vertex from stack to visit next
    v = S.top( )
    S.pop( )
    //Push all the neighbours of v in stack that are not visited
    for all neighbours w of v in Graph G:
        if w is not visited :
            S.push( w )
            mark w as visited

DFS-recursive(G, s):
mark s as visited
for all neighbours w of s in Graph G:
    if w is not visited:
        DFS-recursive(G, w)
```

The time complexity of DFS is **$O(V+E)$** where **V** is the number of nodes and **E** is the number of edges.

46

DFS Applications



- Let G be a graph with n vertices and m edges represented with the adjacency list structure. A DFS traversal of G can be performed in $O(n + m)$ time. Also, there exist $O(n + m)$ -time algorithms based on DFS for the following problems:
 - Testing whether G is connected
 - Computing a spanning forest of G
 - Computing the connected components of G
 - Computing a path between two vertices of G , or reporting that no such path exists [*Ex: Good algorithm design exercise*]
 - Computing a cycle in G , or reporting that G has no cycles. [*Ex: Good algorithm design exercise.*]

Binary Search Tree (BST)



Why Binary Search Tree?

- We have already discussed, different tree representations and in all of them we *did not* impose any restriction on the nodes data !!
- As a result, to search for an element we need to check both in left subtree and in right subtree.
- Due to this, the worst case complexity of search operation is $O(n)$.

Binary Search Trees (BSTs)



- As the name suggests, the main use of this representation is for searching. In this representation we impose restriction on the kind of data a node can contain.
- As a result, it reduces the worst case average search operation to $\log n$.

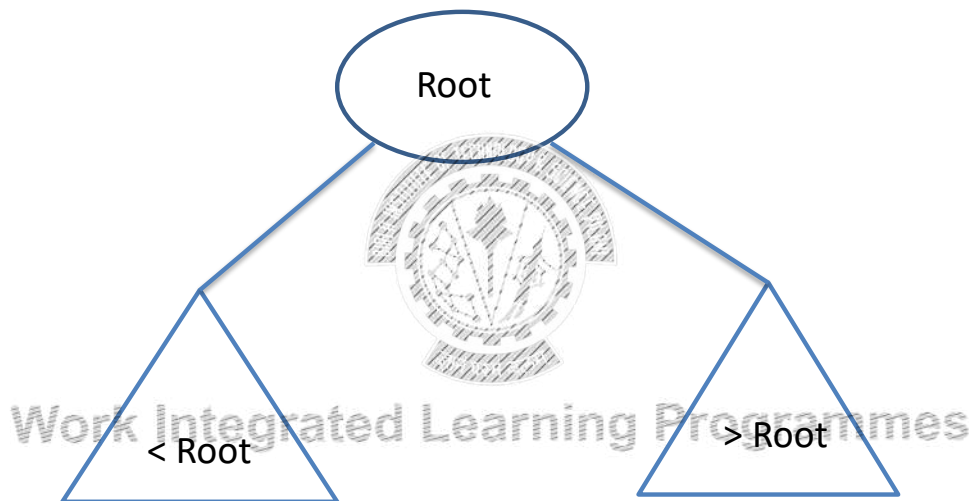
Binary Search Tree - Property



➤ In binary search tree, *all the left subtree elements should be less than root data and all the right subtree elements should be greater than root data.*

- This is called binary search tree property, note that, this property should be satisfied at **every node** in the tree.
 - The left subtree of a node, contains only nodes with keys **less** than the nodes key.
 - The right subtree of a node, contains only nodes with keys **greater** than the nodes key.
 - Both the left and right subtrees must also be binary search trees.

BST Property Diagrammatic Representation



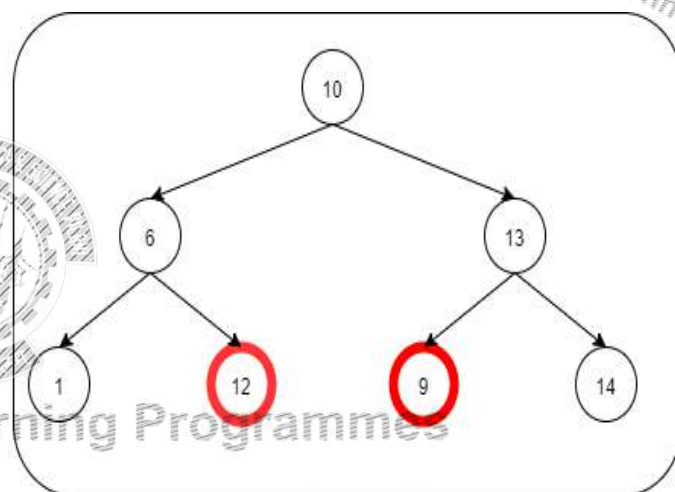
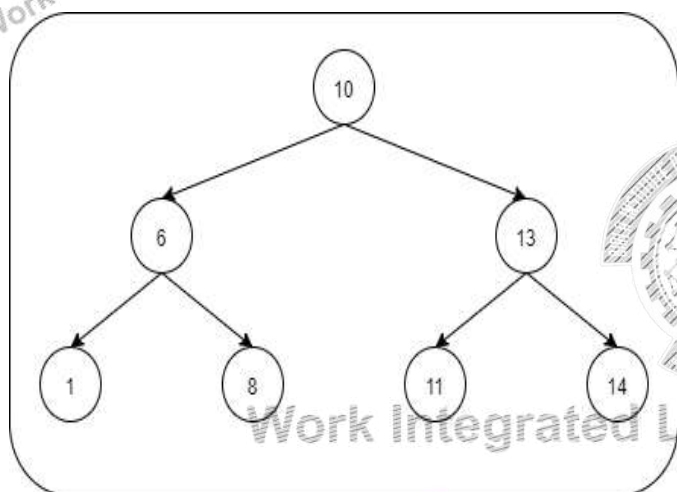
Note: Many textbooks would depict that a BST cannot have duplicates and the = is not discussed. Nevertheless, refer to Introduction to Algorithm by CLRS – Chapter 12.

Example



Example: The left tree is a binary search tree and right tree is not binary search tree [at node 6 it's not satisfying the binary search tree property]

Binary Search Trees

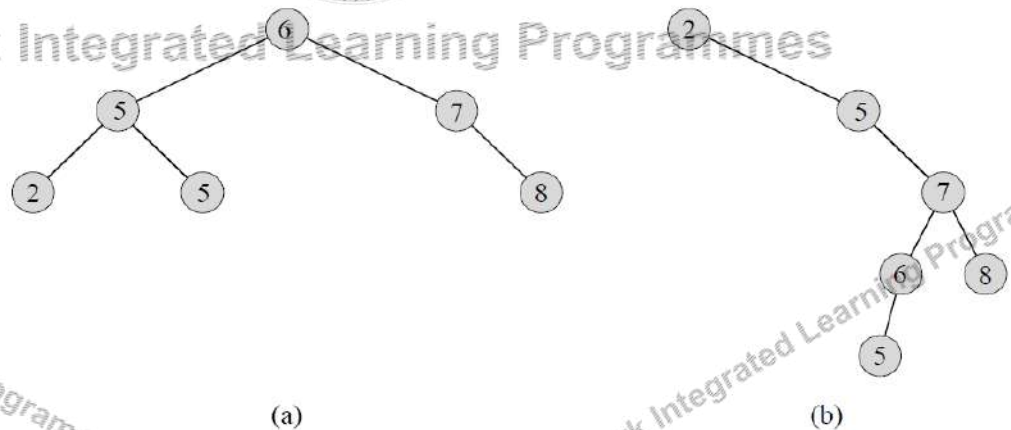




BST with Duplicate Keys

The keys in a binary search tree are always stored in such a way as to satisfy the binary search tree property :

Let x be a node in a binary search tree. If y is a node in the left subtree of x , then $y.key \leq x.key$. If y is a node in the right subtree of x , then $y.key \geq x.key$.

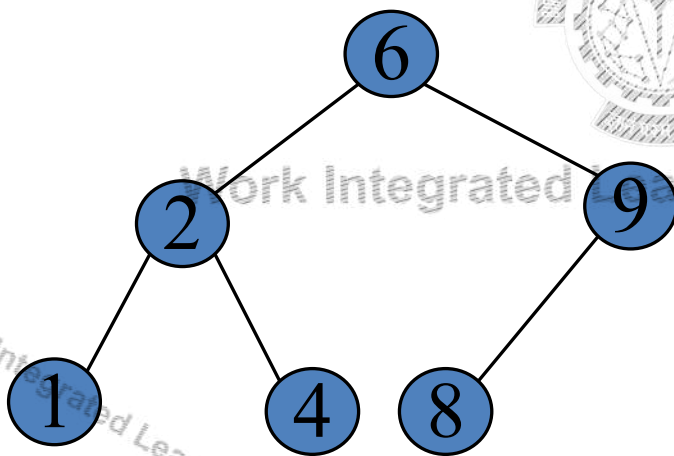


54

BST – Inorder gives Sorted!



- An Inorder traversal of a binary search trees visits the keys in increasing order.
- Outputs keys in sorted order.



Inorder Traversal (Left, root, right)

1,2,4,6,8,9 → Sorted !

Operations on BST



Main Operations: Following are the main operations that are supported by binary search trees:

- **Find**/find minimum/find maximum element in binary search trees.
- **Inserting** an element in binary search trees.
- **Deleting** an element from binary search trees.

Work Integrated Learning Programmes

Important Notes on Binary Search Trees

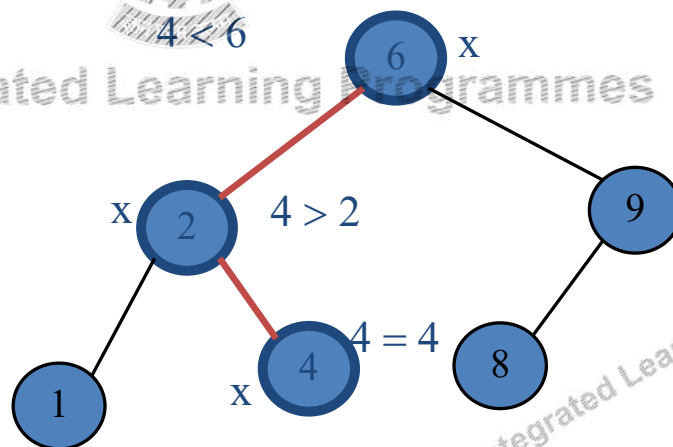


- Since root data is always between left subtree data and right subtree data, performing Inorder traversal on binary search tree produces a sorted list.
- While solving problems on binary search trees, first we process left subtree, then root data and finally we process right subtree. This means, depending on the problem only the intermediate step (processing root data change and we do not touch the first and third steps)
- If we are searching for an element x , then at each internal node we compare the value of the current node to our search element x . If the answer is “smaller” then the search continues in the left subtree. If the answer is “greater” then the search continues in the right subtree. If the answer is “equal” then the search terminates successfully. Finally, if we reach an external empty node, then search terminates unsuccessfully.
- In other words, the binary search trees consider either left or right subtrees for searching an element but not both.

Finding an element in Binary Search Trees



- Find operation is straight forward in a BST. Start with the root and keep moving left or right using the BST property.
- If the data we are searching is same as nodes then we return current nodes. if the data is not present, we end up in a Null Link.
- Find/Search element 4

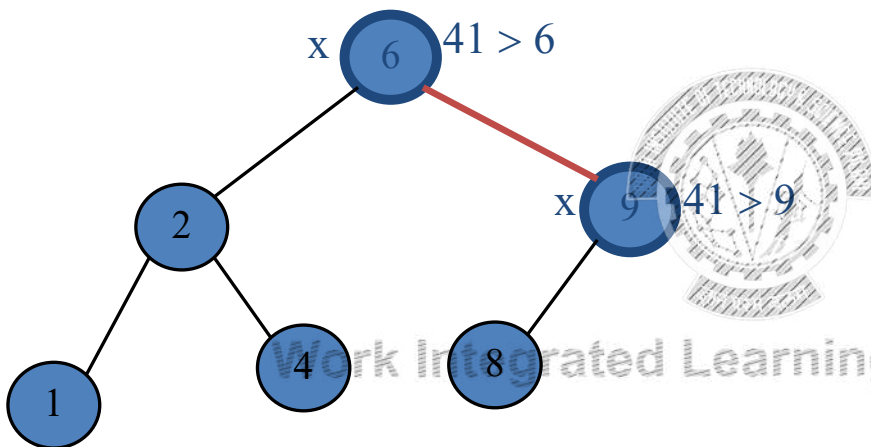


58

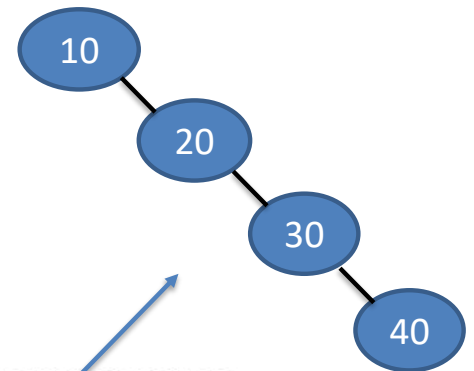
Find in BST



Search element 41



Search element 41



Skewed Tree

What is the time complexity of finding the element in binary search tree ?

$O(n)$ Why?!

Finding element in BST



```
Struct Binary Search TreeNode*find(struct  
Binary search tree node* root,int data){  
If(root == Null)  
    return Null;  
If(data < root → data)  
    return find (root → left,data);  
Else if (data > root → data)  
    return find (root → right, data);  
return root ;  
}
```

Time Complexity : $O(n)$, in worst case (when BST is a skew tree)
Space complexity : $O(n)$, for recursive stack.

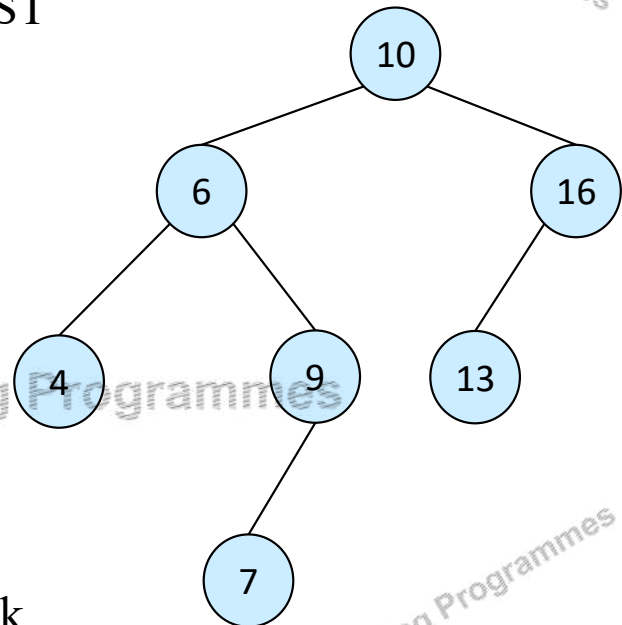
Finding minimum element in Binary search trees



In BSTs, the minimum element, is the left-most node, which does not have left child. In the BST below, the minimum element is 4.

Tree-Minimum(x)

1. **while** $left[x] \neq NIL$
2. **do** $x \leftarrow left[x]$
3. **return** x



- Time complexity $O(n)$ in the worst case
- Space complexity $O(n)$ for recursive Stack

Finding max element in Binary search trees

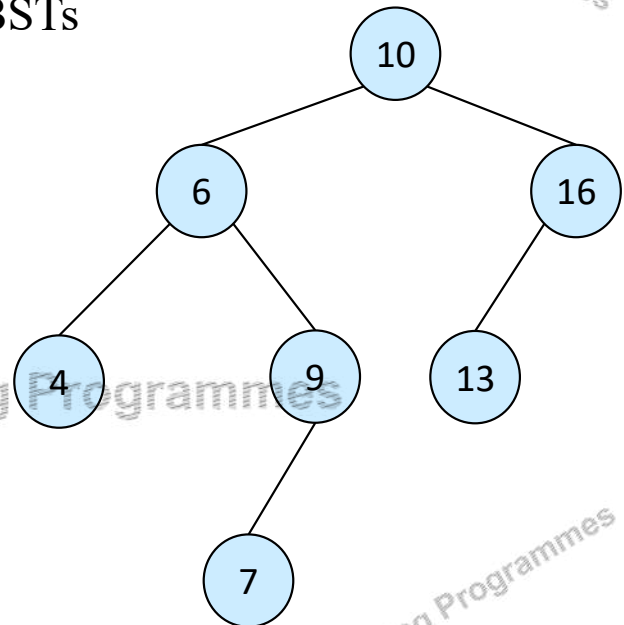


In BSTs the Maximum element is the right –most node which does not have right child. In the BSTs above the maximum element is 16.

Tree-Maximum(x)

1. **while** $right[x] \neq NIL$
2. **do** $x \leftarrow right[x]$
3. **return** x

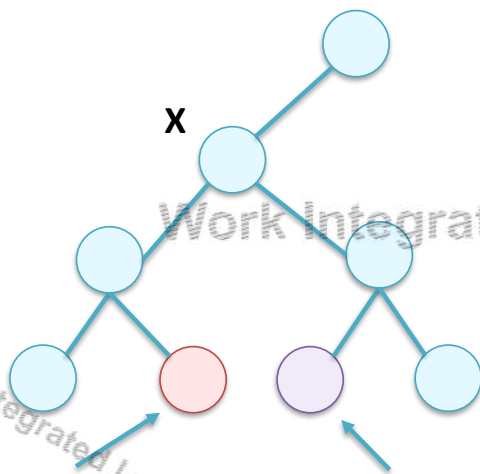
- Time complexity $O(n)$ in the worst case
- Space complexity $O(n)$ for recursive Stack



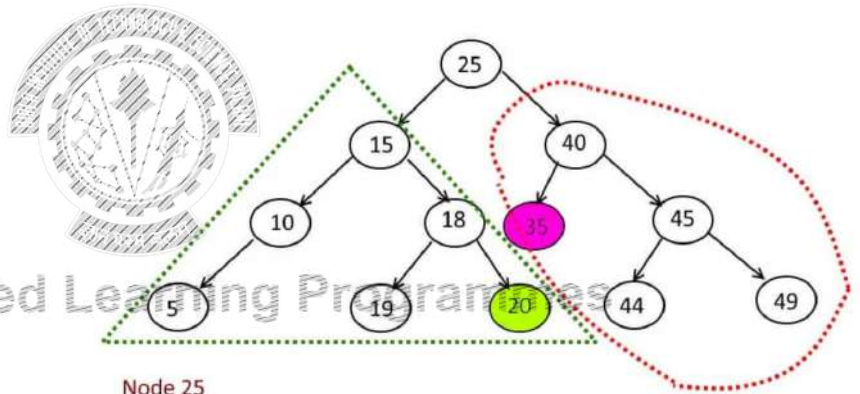
Where is In-order Predecessor & Successor ?



If X has two children then its in-order predecessor is the maximum value in its left subtree and its in-order successor is the minimum value in its right subtree.



Predecessor of X **Successor of X**



Node 25

Predecessor of node 25 will be the right most element in the left subtree.
which is 20

Successor of node 25 will be the left most element in the right subtree
which is 35

Note : When the node X does not have children, how to find the predecessor ? Refer CLRS

63

Insertion in BST



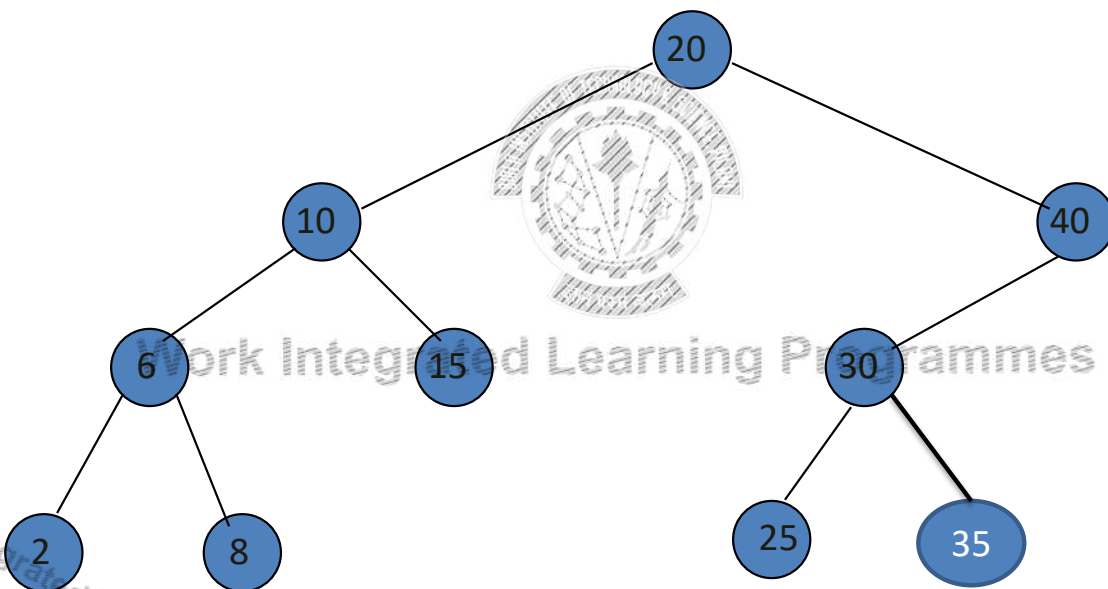
Insert k in a BST:

- Begin at the root
- Compare the element k with the element at current node
- If value we want to insert is less than key of current node, go to the left subtree
- Otherwise go to the right subtree
- If we reach a NULL, create a node with the value we are inserting and place it here

Insertion in BST Example



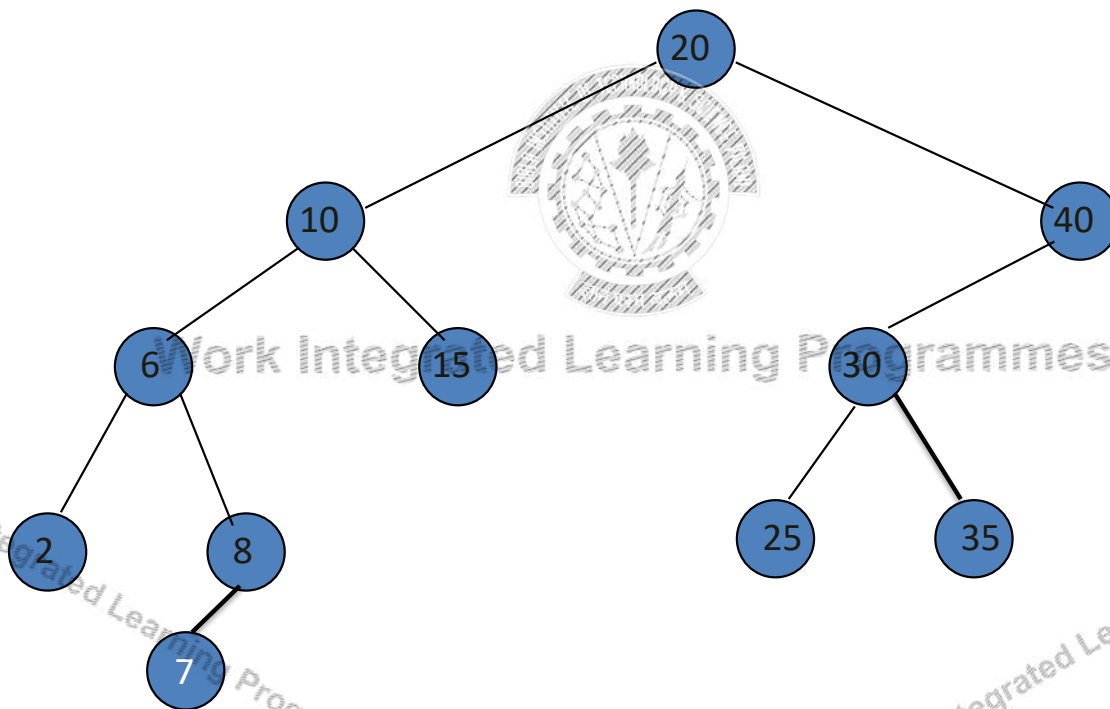
Insert element 35 in the given BST and draw the resulting BST



Insertion in BST Example



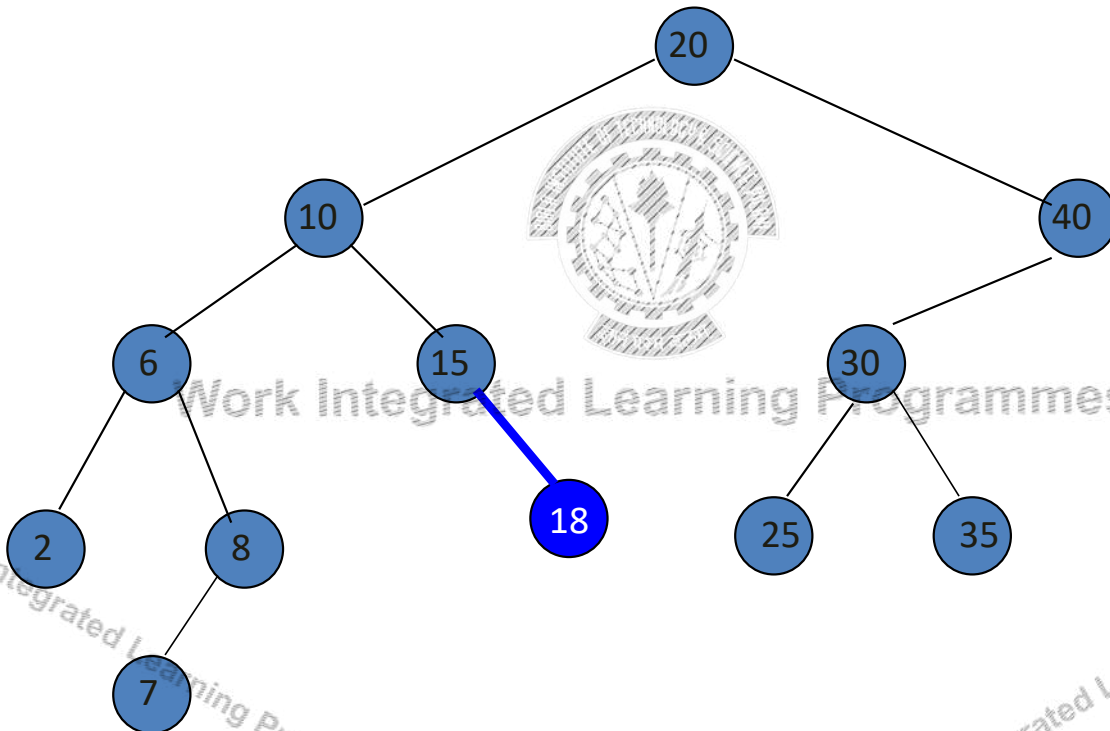
Insert element 7 in the given BST and draw the resulting BST



Insertion in BST Example



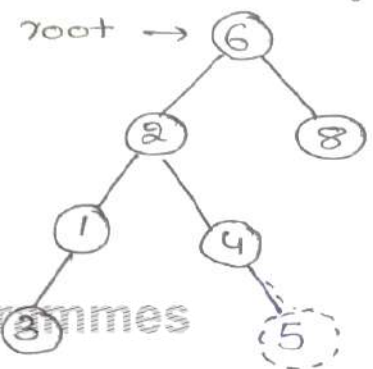
Insert element 18 in the given BST and draw the resulting BST



Analysis of Insertion in BST



- To insert data into binary search tree, first we need to find the location for that element. We can find the location of insertion by following the same mechanism of that of find operation.
- While finding the location if the data is already there then we can simply neglect and come out.
- Otherwise, insert data at the last location on the path traversed.
- As an example, let us consider the tree:
- The dotted node indicates the element (5) to be inserted, to insert 5, traverse the tree as using the find function. At node with key 4, we need to go right but there is no subtree, so 5 is not in the tree, and this is the correct location for insertion.
- Time complexity : $O(n)$, in worst case



Deletion in BST



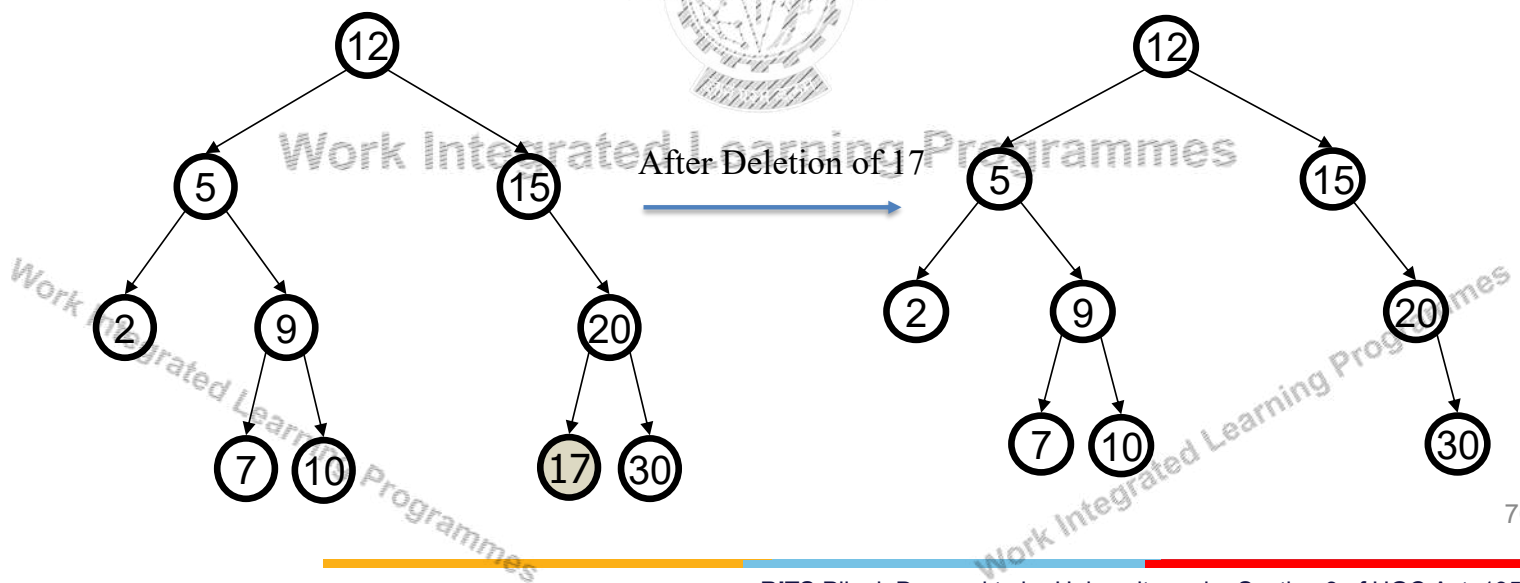
Delete k

- Search k
- If k is present
 - Deletion involves three cases
 - Node has no children (leaf)
 - Node has one child
 - Node has two children
- If k not present
 - Return a message as element not present



Deletion in BST : Case 1

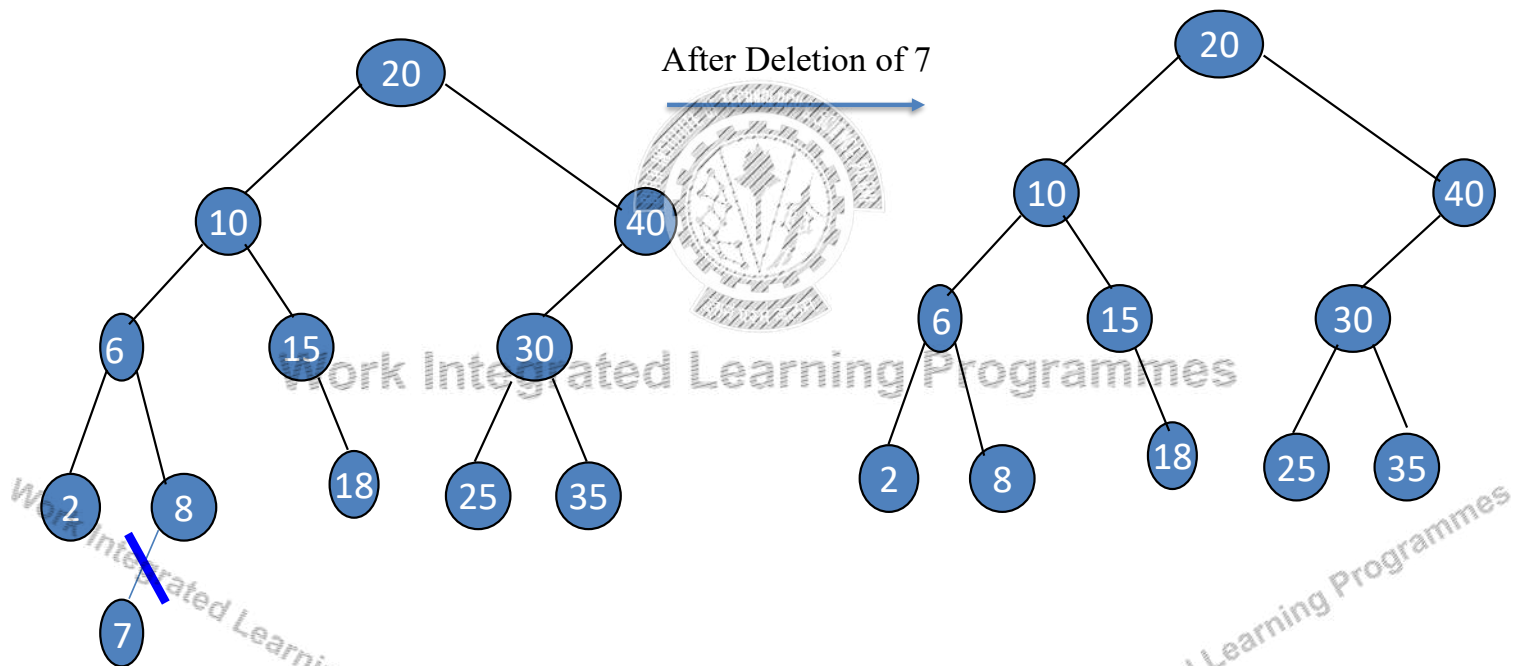
- **Case 1:** Node (z) to be deleted has no children (leaf)
 - ✓ Remove the node (z)
 - ✓ Modify z's parent to replace the node with NULL
- Delete element 17 from the BST and draw the resulting BST



Deletion in BST



- Delete element 7 from the BST and draw the resulting BST

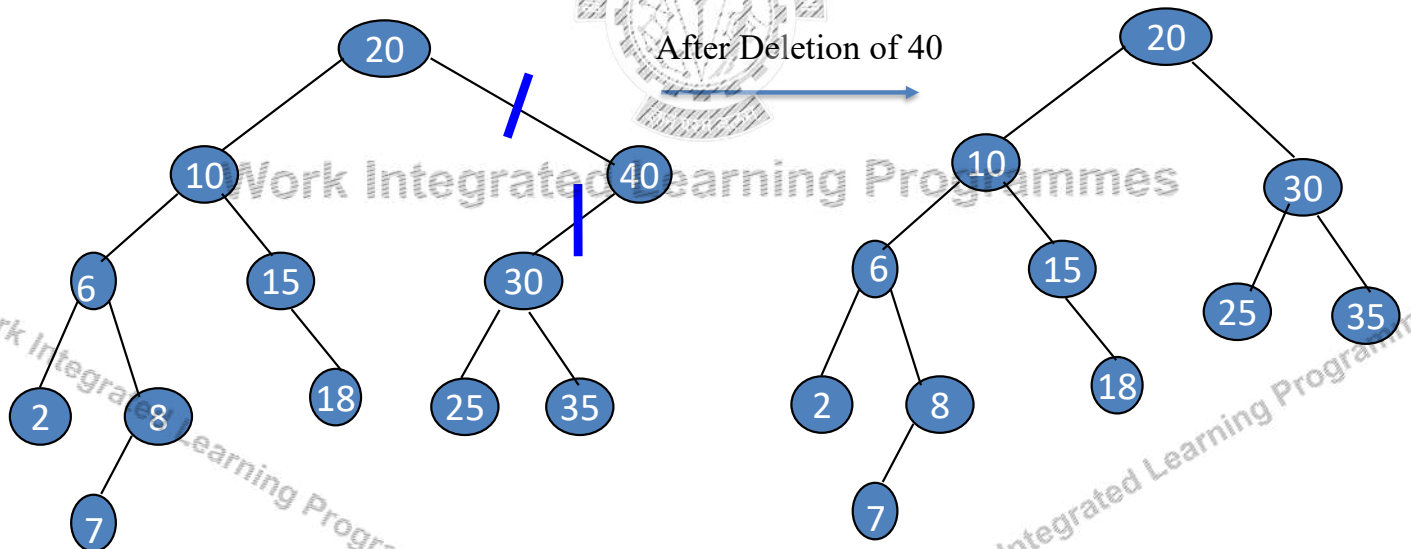


- Exercise : Delete 35 from the above tree.

Deletion in BST : Case 2



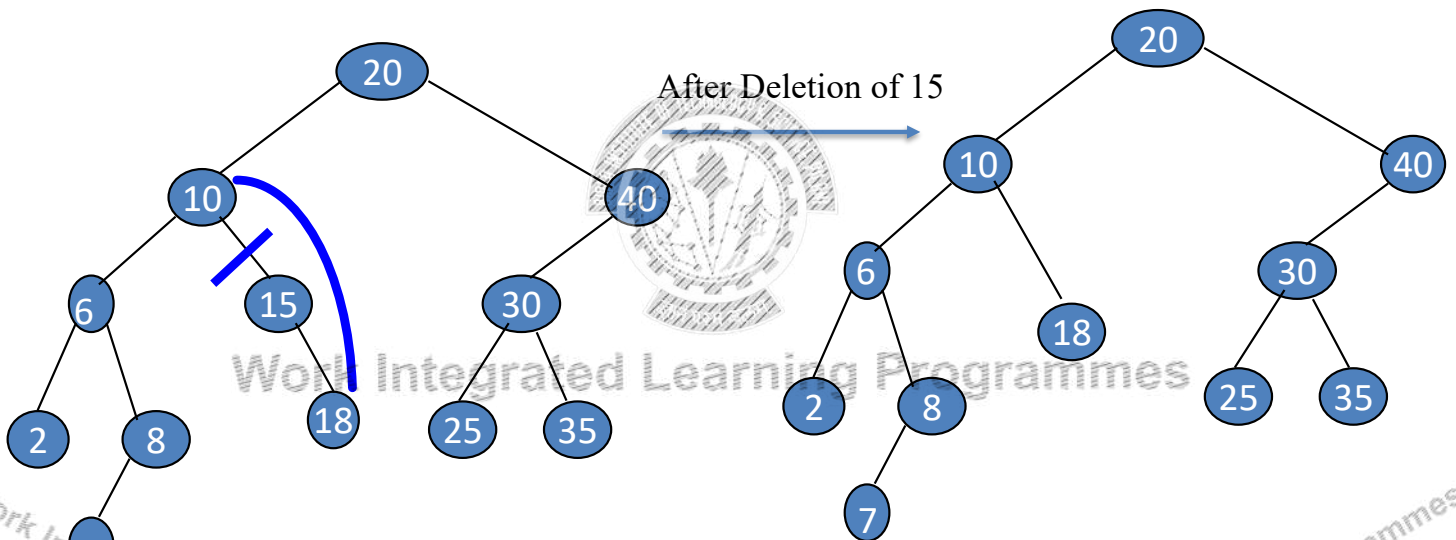
- **Case2:** Node (z) to be deleted has one child
 - ✓ Remove the node (z)
 - ✓ Modify z's parent to point to z's child
- Delete element 40



Deletion in BST



➤ Delete node 15

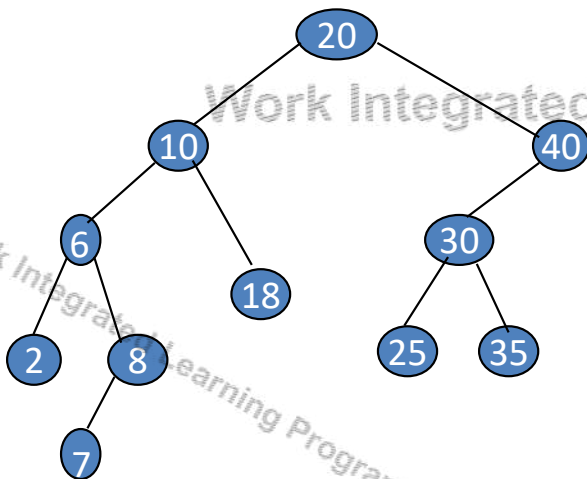


Deletion in BST : Case 3



➤ **Case3:** Node (z) to be deleted has two children

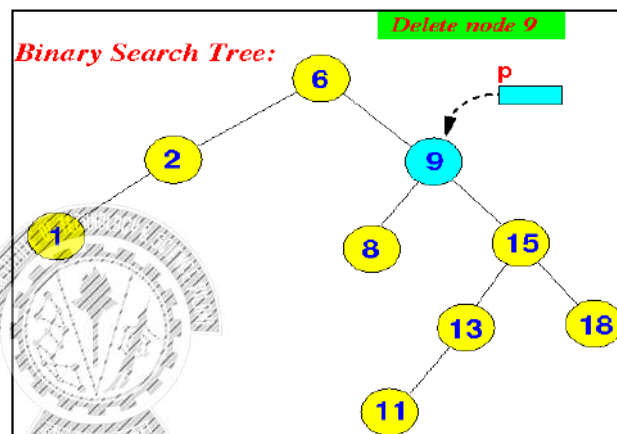
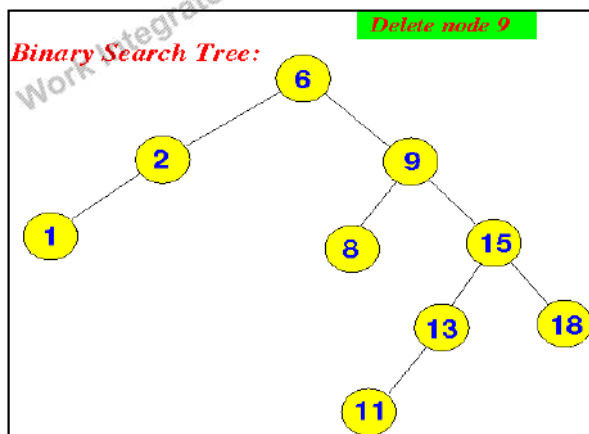
1. Find successor (or predecessor) of the node z
2. Replace the element at the node to be deleted with the element of the successor node. Now, right subtree contains a duplicate!
3. Apply remove to the right subtree to remove a duplicate



Find the successor of nodes 20, 10

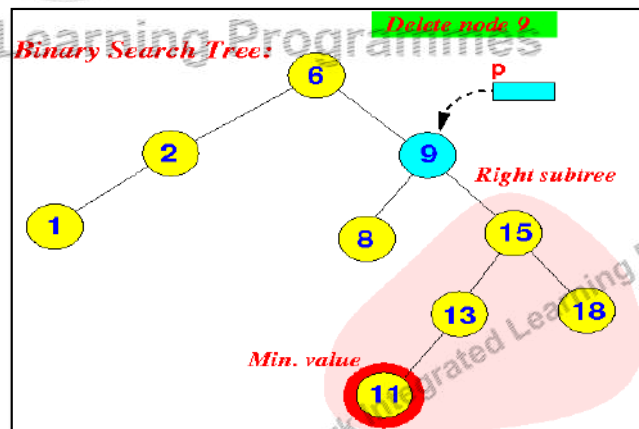
*Successor of node 20 is 25 and
Successor of node 10 is 18*

Deletion in BST



We use the variable *p* to point to the deletion node

Next, we find the *successor* node of the node 9. The *successor* node is the node with the *minimum* value in the *right* subtree



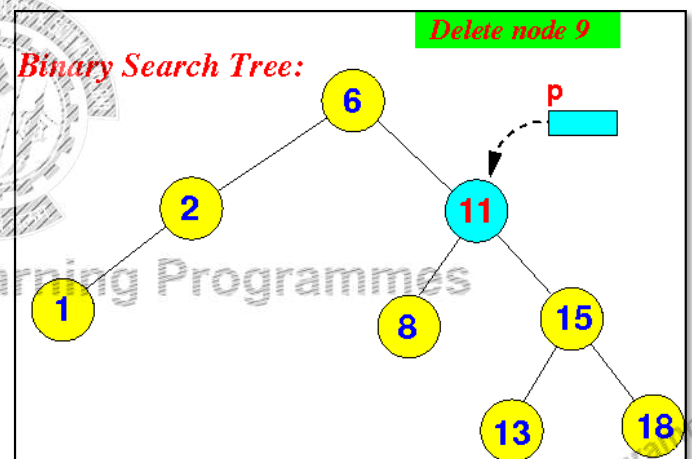
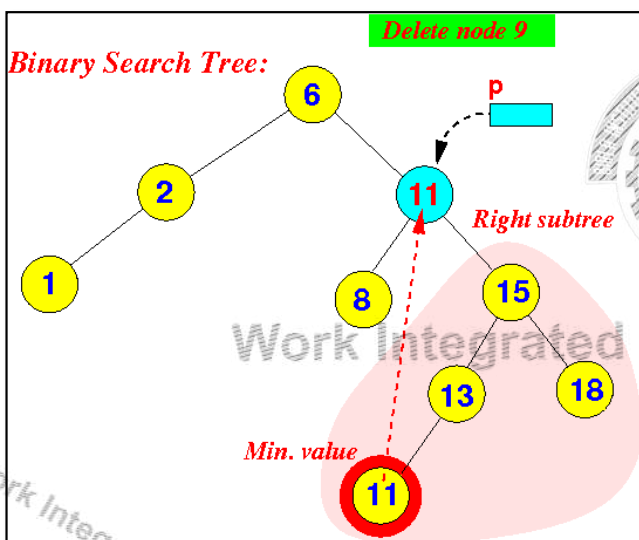
75

Deletion in BST



We **copy** the **content** of the *successor* node of the node into the *deletion* node

The **last step** is **deleting** the *successor* node from the **BST**

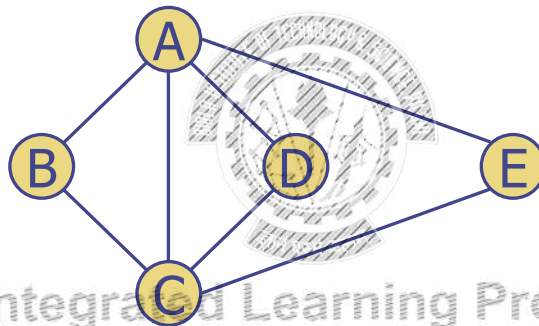


- all nodes in *left subtree* has *smaller values* and
- all nodes in the *right subtree* has *larger values*.

Exercise 1



- Perform BFS and DFS on the below and assume A as the start node. Also show the final result and the cross-edges.

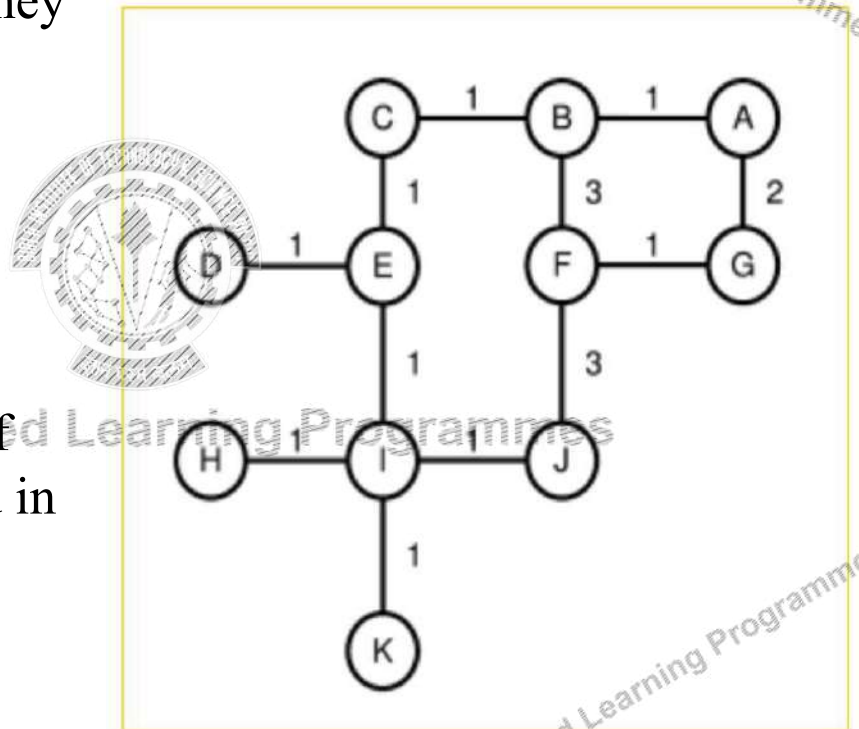


Exercise 2



List the labels in the order they would be visited when performing:

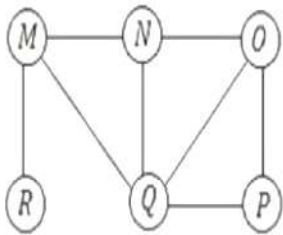
- Breadth First Search
- Depth First Search
- Source vertex: A
- Assume that neighbors of the same node are visited in alphabetical order



Exercise 3



The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one of the following is a possible order of visiting the nodes in the graph below?

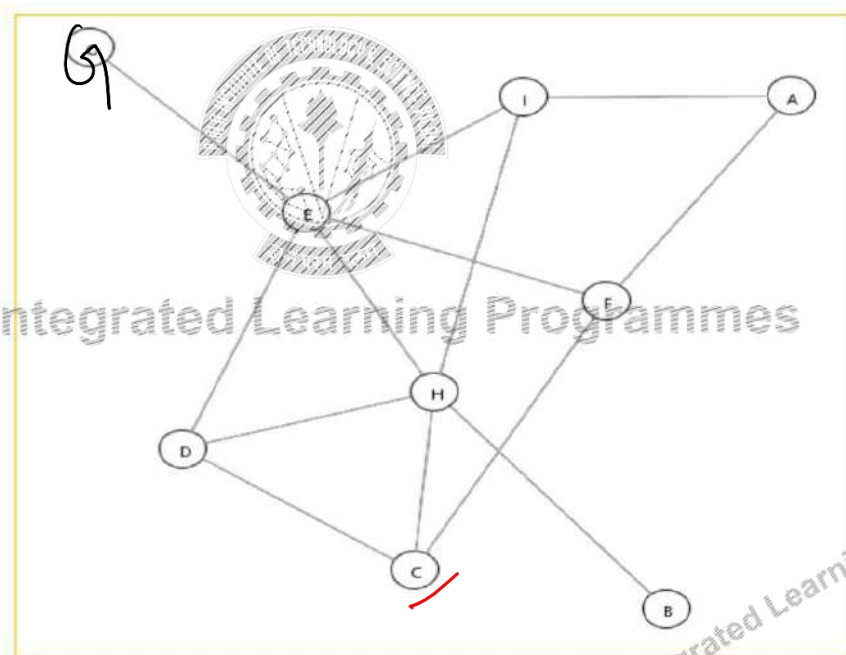


- A** MNOPQR
- B** NQMPOR
- C** QMNROP
- D** POQNMR

Exercise 4



- Implement both DFS and BFS traversal. Assume C to be the source vertex.





See you in the next class to explore more on Hashing !



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



82

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Contact Session #8

DSECLZG519 – Hashing & Collisions!

Agenda for CS #8

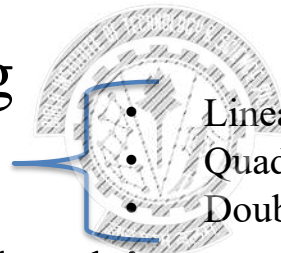


1) Recap and BST Deletion

1) Introduction

2) Separate Chaining

2) Open addressing



• Linear Probing

• Quadratic Probing (Later)

• Double Hashing (Later)

3) Load Factor & Rehashing

4) Journey so far & Mid-Sem discussion

5) Exercises

6) Q&A

Hash Table



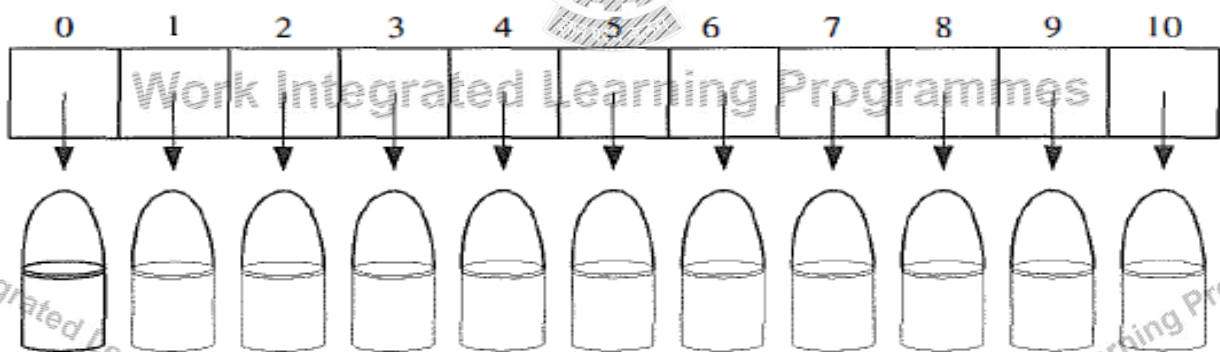
- A hash table is a data structure used to store key-element (k, e) pairs
- The key and element together is called an item
- Keys are used to insert, search and remove elements
- Applications :
 - Address book
 - Mapping host names (e.g., cs16.net) to internet addresses (e.g., 128.148.34.101)
- A hash table consists of two major components
 - Hash function
 - (Bucket) Array (called table)

Bucket Array



➤ An (bucket) array A of size N

- Each cell is thought of as a bucket
- N defines the capacity of the array
- If the keys are integers well distributed in the range $[0, N - 1]$
 - An element e with key k is inserted into the bucket $A[k]$



The bucket for items with
key = 6

Array vs Hash Table



Array

Value
New York
Boston
Mexico
Kansas
Detroit
California

Hash Table

Key	Value
1	New York
2	Boston
3	Mexico
4	Kansas
5	Detroit
6	California

Hash Function



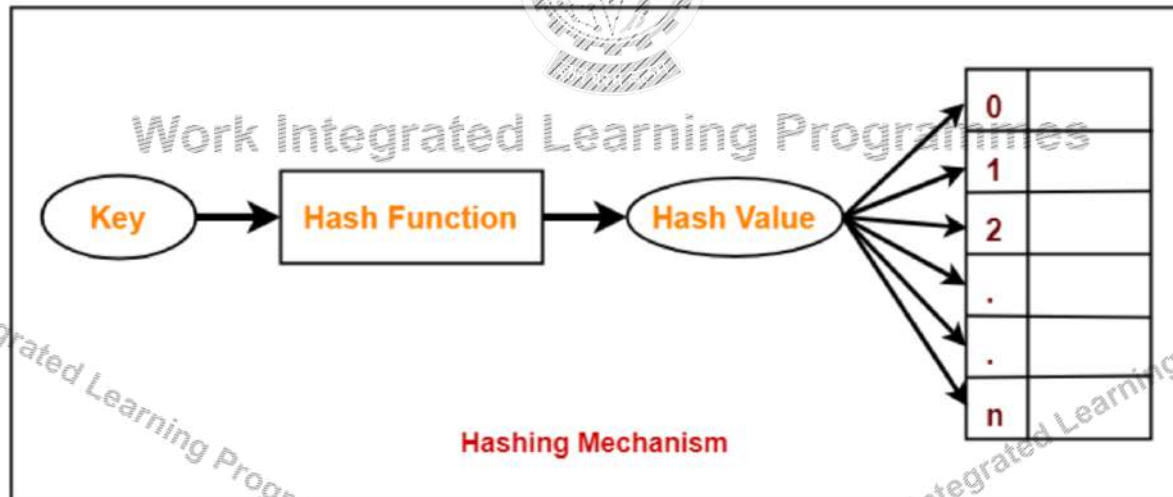
- The keys are not always in the range $[0, N - 1]$.
- Hash function helps to map each key k to an integer in the range $[0, N - 1]$.
- **Example:** $h(x) = x \bmod N$ is a hash function and the integer $h(x)$ is called the hash value of key x .
- Hash function, maps each key k in our dictionary to an integer in the range $[0, N - 1]$, where N is the capacity of the bucket array for this table.
- Use the hash function value, $h(k)$, as an index to bucket array, A , instead of the key k (which is most likely inappropriate for use as a bucket array index). That is, store the item(k , e) in the bucket $A[h(k)]$.
- Requirements of good hash function
 - Minimize collisions as much as possible
 - Fast and easy to compute

Hash Function



Division:

- $h(k) = |k| \bmod N$
- N is the size of the hash table
- N is usually chosen to be a prime that helps to “spread out” the hashed values



Hash Function



➤ Consider the following keys: 200, 205, 210, 300, 305, 310, 400, 405, 410

Apply the hash function $h(k) = |k| \bmod N$ *when* $N=100$ and $N=101$ and compute the hashed values

– $N=100$

- hashed values are 0, 5, 10, 0, 5, 10, 0, 5, 10

– $N=101$

- hashed values are 99, 3, 8, 98, 2, 7, 97, 1, 6

Collision

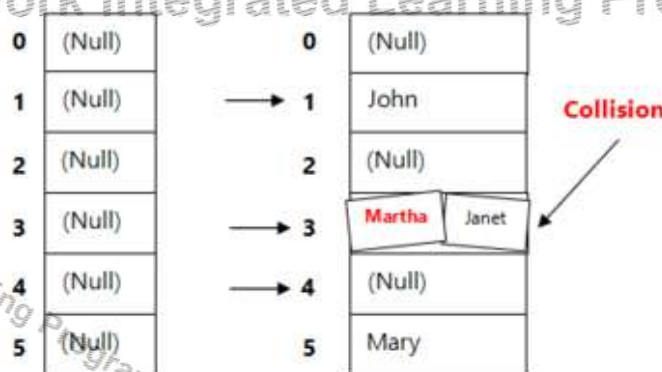


➤ A bucket can store only one item 😊

➤ Collision occurs when

- If two different elements have same keys
- When two distinct keys, k_1 and k_2 , are mapped to the same bucket in the hash table such that $h(k_1)$ is equal to $h(k_2)$

Figure 1: Hash Table: Empty / Collision Occurrence

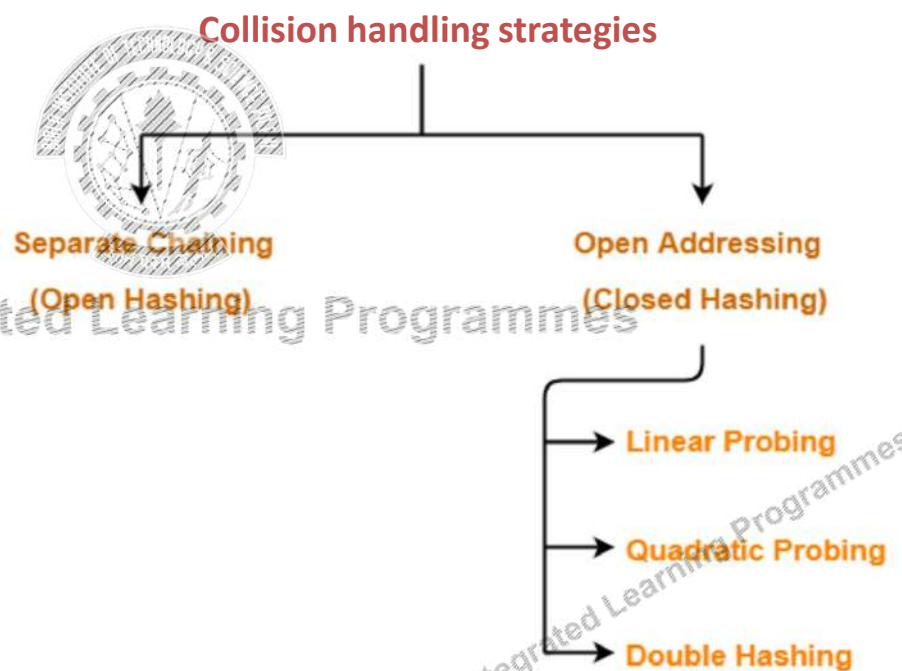


10

Collision handling strategies



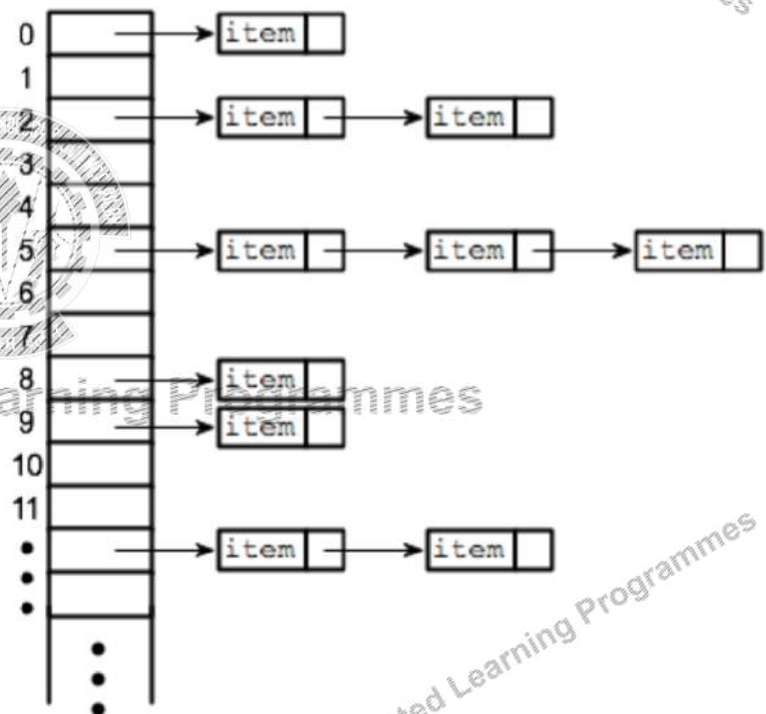
- Separate chaining
- Open addressing
 - Linear probing
 - Quadratic probing
 - Double hashing



Separate chaining



- Colliding item is placed in a *linked list*
- Each cell in the array
 - Stores a pointer that points to a linked list of elements that are mapped to that cell
 - Otherwise stores NULL
- Requires additional memory apart from the table.



Separate chaining

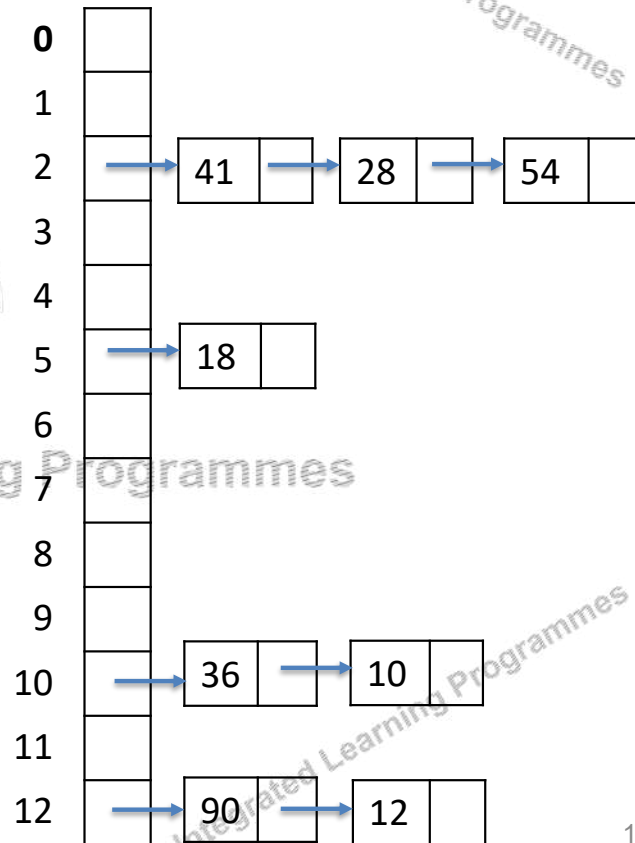
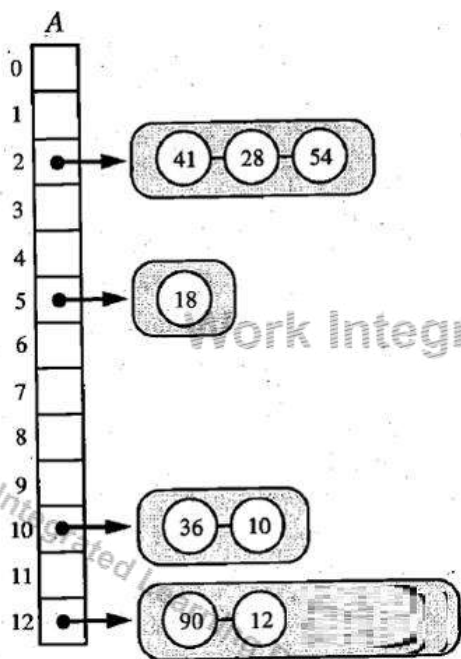


- Insert the elements using separate chaining in this order into an initially empty hash table using the hash function $h(k) = k \bmod 13$. Show your work step by step.
- Keys: 90, 41, 36, 18, 28, 12, 54, 10
- Hash values = 12, 2, 10, 5, 2, 12, 2, 10

Separate chaining



- Keys: 90, 41, 36, 18, 28, 12, 54, 10
- Hash values = 12, 2, 10, 5, 2, 12, 2, 10



Separate Chaining Insert Algorithm



1. Declare an array of a linked list with the hash table size.
2. Initialize an array of a linked list to NULL.
3. Find hash key.
4. If $\text{chain}[\text{key}] == \text{NULL}$



Make $\text{chain}[\text{key}]$ points to the key node.

5. Otherwise (collision),
Insert the key node at the end of the $\text{chain}[\text{key}]$.

Separate chaining



○ Search (k)

- Compute $i = h(k)$
- Go to cell $A[i]$
- if $A[i]$ is not null traverse the list and search for the key
 - If the key exists return index (i), KEY PRESENT
- Else return NO SUCH KEY

Separate chaining



- Search(10)

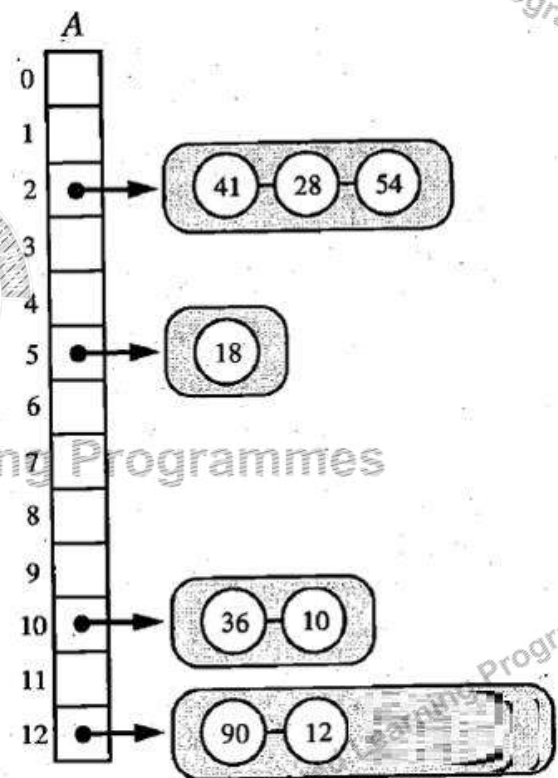
- ✓ $i = h(10) = 10 \bmod 13 = 10$
- ✓ Goto $A[i] = A[10]$ & check if 10 exists.

- Search(20)

- ✓ $i = h(20) = 20 \bmod 13 = 7$
- ✓ Goto $A[i] = A[7]$ & check if 20 exists.

- Search(15)

- ✓ $i = h(15) = 15 \bmod 13 = 2$
- ✓ Goto $A[i] = A[2]$ & check if 15 exists



Open addressing



- Colliding item is placed in a different cell of the array

Separate Chaining	Open Addressing
Keys are stored inside the hash table as well as outside the hash table.	All the keys are stored only inside the hash table. No key is present outside the hash table.
The number of keys to be stored in the hash table can even exceed the size of the hash table.	The number of keys to be stored in the hash table can never exceed the size of the hash table.
Deletion is easier.	Deletion is difficult.
Extra space is required for the pointers to store the keys outside the hash table.	No extra space is required.
Cache performance is poor. This is because of linked lists which store the keys outside the hash table.	Cache performance is better. This is because here no linked lists are used.
Some buckets of the hash table are never used which leads to wastage of space.	Buckets may be used even if no key maps to those particular buckets.

18

Linear Probing



- Linear Probing handles collisions by placing the colliding item in the next (circularly) available table cell.
- Each table cell inspected is referred to as “probe”.
- In this method, if we try to insert an element (k,v) into a bucket $A[i]$ that is already occupied, where $i=h(k)$, then we try next at $A[(i+1) \bmod N]$.
- This process will continue until we find an empty bucket that can accept the new entry.

Linear probing



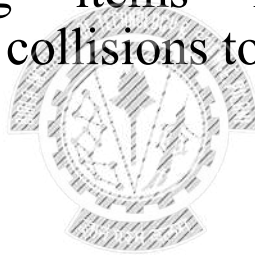
➤ Insert an item (k, e)

- Compute $i = h(k)$
- If $A[i]$ is not occupied then place element e in $A[i]$
- If $A[i]$ is already occupied (collision) then try the buckets $A[(i+1) \bmod N]$
- If $A[(i+1) \bmod N]$ is occupied, then we try $A[(i+2) \bmod N]$, and so on, until we find an empty bucket in A that can accept the new item or we have inspected the entire array $A[(i+N-1) \bmod N]$

Linear Probing



- Probe: Each table cell inspected to place the item is referred to as a “probe”
- *Disadvantage:* Colliding items lump together (primary clustering), causing future collisions to cause a longer sequence of probes during insertion



Work Integrated Learning Programmes

Linear probing

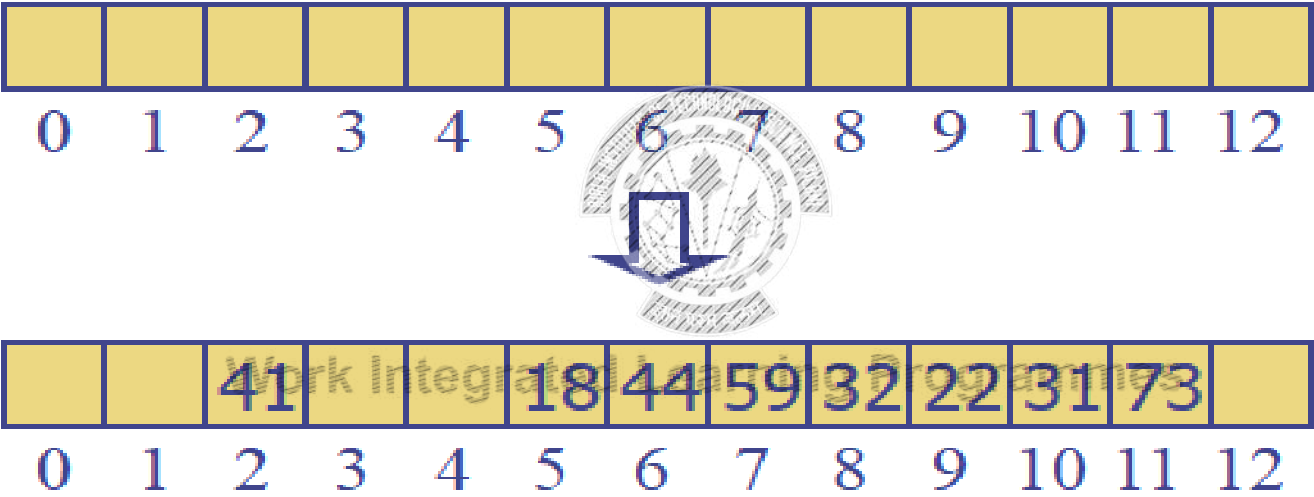


- Insert the keys using linear probing in this order into a hash table using the hash function $h(k) = k \bmod 13$. Show your work step by step. Keys: 18, 41, 22, 44, 59, 32, 31, 73
 - $h(k) = (5, 2, 9, 5, 7, 6, 5, 8)$

Linear Probing
in action!

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Linear probing



Linear probing



Search (k)

- Compute $i = h(k)$
- Start at array cell $A[i]$
- Probe consecutive locations until one of the following occurs
 - An item with key k is found return **KEY PRESENT**
 - An empty cell is found return **NO SUCH KEY**
 - N cells have been unsuccessfully probed return **NO SUCH KEY**

Linear probing



- Consider the following hash table and the hash function $h(k) = k \bmod 13$. Search/Find 18, 31 and 57

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

- Search(18) = $18 \bmod 13 = 5$, look at A[5] ; $18 == 18$, “Success”
- Search(31) = $31 \bmod 13 = 5$, look at A[5] ; $18 \neq 31$, Look at A[6] and so on .. A[10] = 31, “Success”
- Search(57) = $57 \bmod 13 = 5$, look at A[5]; $18 \neq 57$, Look at A[6] and so on ... until its found or an empty location is found. Here, A[12] = null and hence “57 not found”

Linear probing



Delete (k) is (1) Search(k) (2) Once found, delete it!

Delete (k)

- Consider the following hash table and the hash function $h(k) = k \bmod 13$

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

- **Delete(18)** = $18 \bmod 13 = 5$, goto A[5] and $18 \neq 18$, “Delete 18”

		41				44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

- **Search(31)** = $31 \bmod 13 = 5$, goto A[5] and it's empty!
- Oops!! 31 not found

Linear probing



- Cannot just delete the key we want
- Doing so might make it impossible to retrieve any key k during whose insertion we had probed this slot & found it occupied
- Use a special value **AVAILABLE** when we delete a key from a slot
 - **Search** should treat **AVAILABLE** as though the slot holds a key that does not match the one being searched for
 - **Insert** should treat **AVAILABLE** as though the slot were empty, so that it can be reused

Linear probing



- Example

- Consider the following hash table and the hash function $h(k) = k \bmod 13$

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Delete(18) = $18 \bmod 13 = 5$, goto A[5] and delete element and mark as A

		41			A	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Search(31)

treat **AVAILABLE** as though the slot holds a key that does not match the one being searched for and continue .. A[10] = 31

Linear probing



Example

- Consider the following hash table and the hash function $h(k) = k \bmod 13$

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

- **Delete(18)**

		41			A	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

- **Insert(57)**

		41			57	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Load factor



- Suppose that
 - the bucket array is of capacity N
 - there are n entries in the hash table
- The load factor is defined as $\lambda = n/N$
- Load factor is kept below a small constant, preferably below 1 (i.e.) $\lambda < 1$
- Usually λ is kept as 0.75
 - But this requires additional work

Rehashing



- Whenever we add elements, it is important to keep the load factor below a specified constant
 - Increase the size of bucket array
 - double the size of the original array, choosing the size of the new array to be a prime number
 - Change hash function to match this new size
 - Insert all the existing hash table elements into the new bucket array using the new hash function
- Such a size increase and hash table rebuild is called **rehashing**

Rehashing



- What is the load factor of the hash table? Elements of the hash table are rehashed into another hash table of size 17 using linear probing. Let the new hash function be $h(x) = x \bmod 17$.

0	6
1	15
2	23
3	24
4	
5	
6	13

Rehashing



0	6
1	15
2	23
3	24
4	
5	
6	13

$$\lambda = n/N$$
$$= 5/7 = 0.7$$



0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

$$\lambda = n/N$$
$$= 5/17 = 0.3$$

Hash Table



- The worst case running time of operations like insert, delete and search is $O(n)$ where n is the number of items in the hash table.

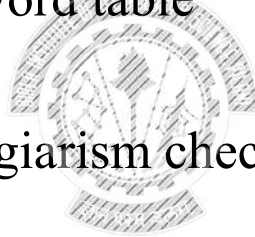


Method	Worst case performance
Separate Chaining	$O(N)$
Linear Probing	$O(N)$
Quadratic Probing	$O(N)$
Double Hashing	$O(N)$

Applications



- DNS resolution
- Password Verification (local and server-side)
- Compilers: Symbol/keyword table
- Online spell check tools
- Pattern matching and plagiarism check



Work Integrated Learning Programmes

Journey so far ...



CS#1
Introduction
& Analysis
framework

CS#2
Analysis and
Master's
Theorem

CS#3
Linear Data
structures

CS#4
Introduction
to Trees &
traversals

CS#5
Heaps ,
Heap Sort!

CS#6
Priority
Queues,
Introduction
to Graphs

CS#7
Exploring
graph
traversals,
BST

CS#8
Hashing &
Collision.

Webinar 1: Stacks & Amortized. Webinar 2: Trees

Mid Semester Examination – All the Best!

Mid-Semester Examination



- Portions for mid-semester examination will be till CS#8 (today's session). Portions will be clearly put as an announcement as well in eLearn by the IC. Do have an eye on it !
- Sample/past paper will be available in canvas - today.
- Weightage : 30%
- Nature: Subjective
- Mode : Online (At an institute assigned center).
- Type : **CLOSED** book
- Duration : 2Hours

Exercise 1



1) Given the input {4371,1323,6173,4199,4344,9679,1989}, and a fixed table size of 10, and a hash-function $H(X) = X \bmod 10$, show the resulting

1.Linear probing hash-table

2.Separate chaining hash-table

2) Given the input {42,39,57,3,18,5,67,13,70,26}, and a fixed table size of 13, and a hash-function $H(X) = X \bmod 13$, show the resulting

1.Linear probing hash-table (Circle the elements which causes collision)

2.Separate chaining hash-table

Exercise 2



1. Implement Linear probing for the below given list:

- Table Size 15
- Insert 16, 7, 28, 31, 67, 28, 29, 73, 99, 43, 218
- Use below given hash functions.
 - $k \bmod 15$

- Visualizations:
<https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>
- Credits: USFCA
- Purpose: Many concepts are illustrated visually here. Have a look if this interests you 😊



Work Integrated Learning Programmes

All the Best !!



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



41

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #9

DSECLZG519 – Hashing & Blooms!

Agenda for CS #9



- 1) Recap
- 2) Open addressing
- 3) Bloom Filters
- 4) Introduction to kd trees
- 5) Exercises
- 6) Q&A



Journey so far ...



CS#1
Introduction
& Analysis
framework

CS#2
Analysis and
Master's
Theorem

CS#3
Linear Data
structures

CS#4
Introduction
to Trees &
traversals

CS#5
Heaps ,
Heap Sort!

CS#6
Priority
Queues,
Introduction
to Graphs

CS#7
Exploring
graph
traversals &
BST

CS#8
Hashing &
Collision.
Webinar 1: Stacks &
Amortized.
Webinar 2: Trees

Mid Semester Examination – All the Best!

Quadratic probing



- Insert an item (k, e)
 - Compute $i = h(k)$
 - If $A[i]$ is not occupied then place element e in $A[i]$
 - If $A[i]$ is already occupied (collision)
 - Iteratively try the buckets $A[(i + f(j)) \bmod N]$,
 - for $j = 0, 1, 2, \dots, N-1$, where $f(j) = j^2$,
 - until finding an empty bucket or have examined the entire array
- Can only guarantee a successful insertion when the hash table is at most half full !
- Can suffer from secondary clustering
 - *If two keys have the same initial position, then their probe sequences are the same*

Quadratic probing



- Insert the keys using quadratic probing in this order into a hash table using the hash function $h(k) = k \bmod 7$. Show your work step by step. Keys: 14, 8, 21, 2, 7
 - $h(k) = 0, 1, 0, 2, 0$



Work Integrated Learning Programmes

Quadratic probing



$$i = h(k) = k \bmod 7$$

$A[(i + j^2) \bmod N]$
 $j = 0 \rightarrow A[0+0 \bmod 7] = A[0]$
 $j = 1 \rightarrow A[0+1 \bmod 7] = A[1]$
 $j = 2 \rightarrow A[0+4 \bmod 7] = A[4]$

insert(14)

$$14 \% 7 = 0$$

0	14
1	
2	
3	
4	
5	
6	

1

insert(8)

$$8 \% 7 = 1$$

0	14
1	8
2	
3	
4	
5	
6	

1

insert(21)

$$21 \% 7 = 0$$

0	14
1	8
2	
3	
4	21
5	
6	

3

insert(2)

$$2 \% 7 = 2$$

0	14
1	8
2	2
3	
4	21
5	
6	

1

Quadratic probing



insert(7)
 $7 \% 7 = 0$

0	14
1	8
2	2
3	
4	21
5	
6	

j	Cells tried $h(k) = (h(k) + j^2) \bmod N$
0	$0 \bmod 7 = 0$
1	$(0+1) \bmod 7 = A[1]$
2	$(0+4) \bmod 7 = A[4]$
3	$(0+9) \bmod 7 = A[2]$
4	$(0+16) \bmod 7 = A[2]$
5	$(0+25) \bmod 7 = A[4]$
6	$(0+36) \bmod 7 = A[1]$

Linear vs Quadratic Probing



- An advantage of linear probing is that it can reach every location in the hash table.
- This property is important since it guarantees the success of the *insertItem* operation when the hash table is not full.
- Quadratic probing can only guarantees a successful *insertItem* operation when the hash table is atmost half full.

Double hashing



- Double hashing uses a secondary hash function $d(k)$
- Handles collisions by placing an item in the first available cell of the series

$$(i + j d(k)) \bmod N$$

where $j = 0, 1, \dots, N-1$ and $i = h(k)$ and $d(k)$ is secondary hash function

- The secondary hash function cannot have zero values.
- The table size N must be a prime to allow probing of all the cells.
- Choose a secondary hash function that will attempt to minimize clustering as much as possible. Usually taken as $q - (k \bmod q)$ where q is prime and $q < N$

Example of Double Hashing



- Consider a hash table storing integer keys that handles collision with double hashing
 - $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - (k \bmod 7)$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order into the hash table

k	$h(k)$	$d(k)$
18	5	3
41	2	1
22	9	6
44	5	5
59	7	4
32	6	3
31	5	4
73	8	4



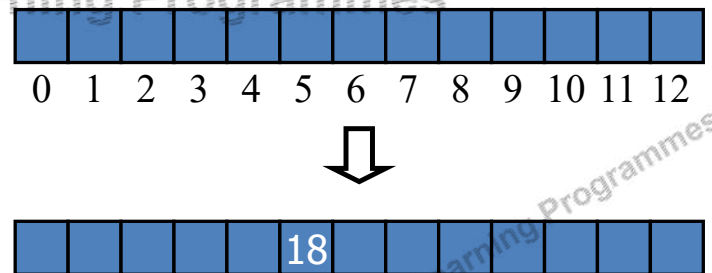
Example of Double Hashing



Insert key 18

- place an item in the first available cell of the series $(i + j d(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = h(k) = 5; j = 0; d(k) = 3; N = 13$
- $5 + 0 \bmod 13 = 5$
- $A[5] = 18$

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	



Example of Double Hashing



Insert key 41

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 2; j = 0; d(k) = 1; N = 13$
- $2 + 0 \bmod 13 = 2$
- $A[2] = 41$



k	$h(k)$	$d(k)$	Probes
18	5	3	5
41	2	1	2
22	9	6	9
44	5	5	5 10
59	7	4	7
32	6	3	6
31	5	4	5 9 0
73	8	4	8

					18							
0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18							
0	1	2	3	4	5	6	7	8	9	10	11	12

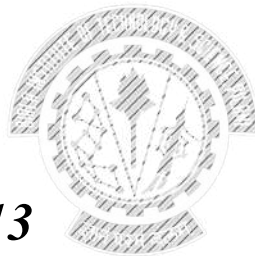
13

Example of Double Hashing



Insert key 22

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 9; j = 0; d(k) = 6; N = 13$
- $9 + 0 \bmod 13 = 9$
- $A[9] = 22$



k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9
73	8	4	8	

		41			18									
0	1	2	3	4	5	6	7	8	9	10	11	12		



		41			18				22					
0	1	2	3	4	5	6	7	8	9	10	11	12		

Example of Double Hashing



Insert key 44

- place an item in the first available cell of the series
 $(i + jd(k)) \bmod N$
 for $j = 0, 1, \dots, N-1$
- $i = 5; j = 0; d(k) = 5; N = 13$
- $5 + 0 \bmod 13 = 5; A[5] - \text{occupied}$
- Try with $j=1$
- $5 + (1 * 5) \bmod 13 = 10$
- $A[10] = 44$

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

		41			18				22				
0	1	2	3	4	5	6	7	8	9	10	11	12	



		41			18				22	44			
0	1	2	3	4	5	6	7	8	9	10	11	12	



Example of Double Hashing

Insert key 59

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 7; j = 0; d(k) = 4; N = 13$
- $7 + 0 \bmod 13 = 7$
- $A[7] = 59$

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

		41			18				22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18		59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing



Insert key 32

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 6; j = 0; d(k) = 3; N = 13$
- $6 + 0 \bmod 13 = 6$
- $A[6] = 32$

k	$h(k)$	$d(k)$	Probes
18	5	3	5
41	2	1	2
22	9	6	9
44	5	5	5 10
59	7	4	7
32	6	3	6
31	5	4	5 9 0
73	8	4	8

		41			18		59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18	32	59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing



Insert key 31

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 5; j = 0; d(k) = 4; N = 13$
- $5 + 0 \bmod 13 = 5$
- $5 + (1 * 4) \bmod 13 = 9$ ($j=1$)
- $5 + (2 * 4) \bmod 13 = 0$ ($j=2$)
- $A[0] = 31$

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

	41			18	32	59		22	44			
0	1	2	3	4	5	6	7	8	9	10	11	12



31		41				18	32	59		22	44	
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing



Insert key 73

- place an item in the first available cell of the series $(i + jd(k)) \bmod N$ for $j = 0, 1, \dots, N-1$
- $i = 8; j = 0; d(k) = 4; N = 13$
- $8 + 0 \bmod 13 = 8$
- $A[8] = 73$

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

31	41	18	32	59	22	44						
----	----	----	----	----	----	----	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12



31	41			18	32	59	73	22	44			
----	----	--	--	----	----	----	----	----	----	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12

Hash Table



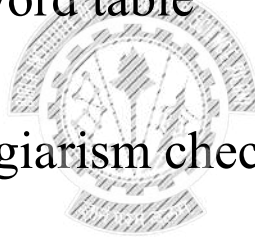
- The worst case running time of operations like insert, delete and search is $O(n)$ where n is the number of items in the hash table.

Method	Worst case performance
Separate Chaining	$O(N)$
Linear Probing	$O(N)$
Quadratic Probing	$O(N)$
Double Hashing	$O(N)$

Applications



- DNS resolution
- Password Verification (local and server-side)
- Compilers: Symbol/keyword table
- Online spell check tools
- Pattern matching and plagiarism check



Work Integrated Learning Programmes

Motivation for Bloom Filters ?

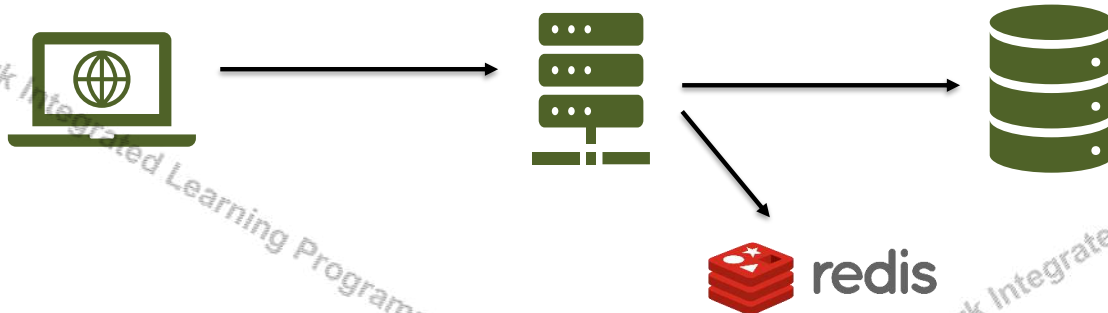


Check if the username "Atul" exists ?

GET call for "Atul" :



GET call for "Atul" :



innovate achieve lead

Motivation for B

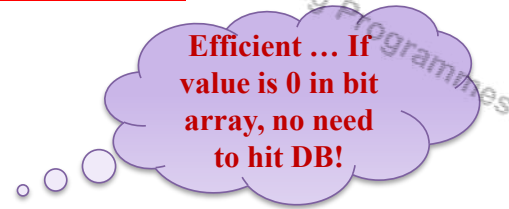
Check if the username "Atul" exists

Work Integrated Learning Programmes

?

innovate achieve lead

Efficient ... If value is 0 in bit array, no need



(Atul) = 3

Work Integrated Learning Programmes

Diagram illustrating a hash table with 128 slots (indices 0 to 127). The slots are arranged in two rows of 64 slots each. The first row contains indices 0 to 63, and the second row contains indices 64 to 127. The values stored in the slots are as follows:

Index	Value
0	1
1	1
2	1
3	3
4	4
...	...
125	0
126	0
127	1

Note : The hash values 3 and 1 are assumption here. In reality this value depends on hash function used.

$$f(Ravi) = 0$$

23

Bloom Filters



- Our first esoteric data structure is called a bloom filter, named for its creator, Burton Howard Bloom, who invented the data structure in 1970.
- A bloom filter is a space efficient, probabilistic data structure that is used *to tell whether a member is in a set*.
- Bloom filters are a bit odd because they can **definitely** tell you whether an element is not in the set, but can only say whether the element is **possibly** in the set.
- In other words: “*false positives*” are possible, but “*false negatives*” are not possible with bloom filters.
- A *false positive* would say that the element is in the set when it isn't, and a *false negative* would say that the element is not in the set when it is.

Bloom Filters



- The idea is that we have a “bit array.” We will model a bit array with a regular array

a bit array :

1	0	1	1	0	1	1	1
---	---	---	---	---	---	---	---

- Bloom Filters: start with an empty bit array (all zeros), and k hash functions.

- $k1 = (13 - (x \% 13)) \% 7,$

- $k2 = (3 + 5x) \% 7, \text{ etc.}$

0	1	2	3	4	5	6
0	0	0	0	0	0	0

Bloom Filters



- The hash functions should be independent, and the optimal amount is calculable based on the number of items you are hashing, and the length of your table (*see Wikipedia for details*).
- Values then get hashed by all k hashes, and the bit in the hashed position is set to 1 in each case.
- Example : Insert **129** $\rightarrow x=129$

$$\begin{aligned} k1 &= (13 - (x \% 13)) \% 7 \\ &= (13 - (129 \% 13)) \% 7 \\ &= (13 - 12) \% 7 \\ &= 1 \% 7 \\ &= 1 \end{aligned}$$

$$\begin{aligned} k2 &= (3 + 5 * x) \% 7 \\ &= (3 + 5 * 129) \% 7 \\ &= 648 \% 7 \\ &= 4 \end{aligned}$$

0	1	2	3	4	5	6
0	1	0	0	1	0	0

$k1 == 1$, so we change bit 1 to a 1

$k2 == 4$, so we change bit 4 to a 1

Bloom Filters



Insert **479** : $x = 479$ and $k1 = (13 - (x \% 13)) \% 7$, $k2 = (3 + 5x) \% 7$

$$\begin{aligned} k1 &= (13 - (x \% 13)) \% 7 \\ &= (13 - (479 \% 13)) \% 7 \\ &= (13 - 11) \% 7 \\ &= 2 \end{aligned}$$

$$\begin{aligned} k2 &= (3 + 5x) \% 7 \\ &= (3 + 5 \cdot 479) \% 7 \\ &= (2398) \% 7 \\ &= 4 \end{aligned}$$

0	1	2	3	4	5	6
0	1	1	0	1	0	0

$k1 == 2$, so we change bit 2 to a 1

$k2 == 4$, so we would change bit 4 to a 1, but it is already a 1

Bloom Filters



- To check if 129 is in the table, just hash again using k_1 and k_2 and check the bits in the bit array.
- In this case, $k_1=1$, $k_2=4$: **probably** in the table!

0	1	2	3	4	5	6
0	1	1	0	1	0	0

- To check if 123 is in the table, hash and check the bits.
 $k_1=0$, $k_2=2$: **definitely cannot be** in table because the 0 bit is still 0

0	1	2	3	4	5	6
0	1	1	0	1	0	0

Bloom Filters



- To check if 402 is in the table, hash and check the bits.
k1=1, k2=4: **Probably** in the table (but isn't! False positive!)

$$\begin{aligned}k1 &= (13 - (x \% 13)) \% 7 \\&= (13 - (402 \% 13)) \% 7 \\&= (13 - 12) \% 7 \\&= 1 \% 7 \\&= 1\end{aligned}$$

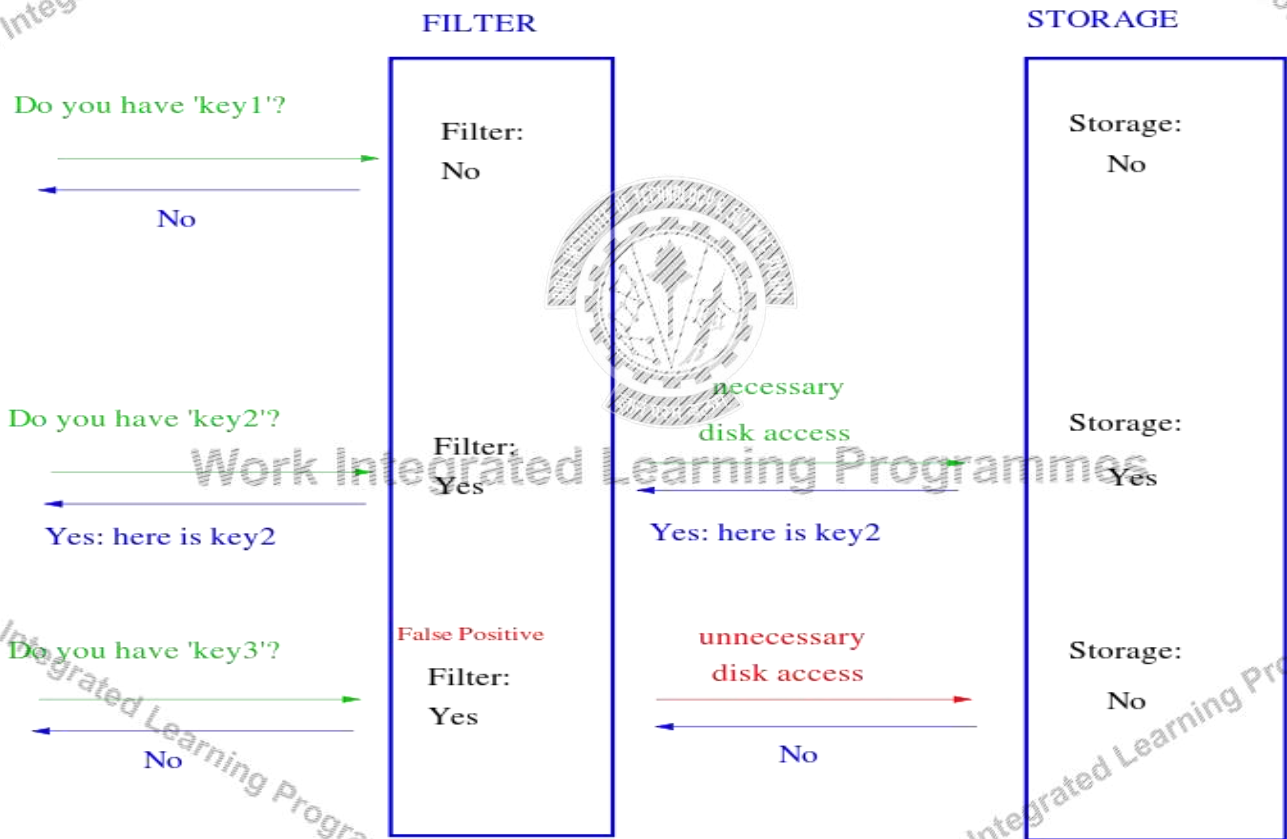
$$\begin{aligned}k2 &= (3 + 5 * x) \% 7 \\&= (3 + 5 * 402) \% 7 \\&= 2013 \% 7 \\&= 4\end{aligned}$$

129 and 402
result in same bits

0	1	2	3	4	5	6
0	1	1	0	1	0	0

Interactive Example: <https://lmlib.github.io/bloomfilter-tutorial/>

Bloom Filters – Pictorial



Bloom Filters Applications



Why would we want a structure that can produce false positives ?

- Example: Google Chrome uses a local Bloom Filter to check for malicious URLs — if there is a hit, a stronger check is performed!
- Weak password detection
- Internet Cache Protocol
- Wallet synchronization in Bitcoin
- Cyber security like virus scanning
- Facebook uses bloom filters for typeahead search, to fetch friends and friends of friends to a user typed query. The bloom filter is just 16 bits per friend connection (an edge in the facebook social graph) and they have called it "world's smallest bloom" 😊

Tip : Just google about these applications to understand better as to why these use bloom filters!

31

Bloom Filters



Positives :

- You have to perform k hashing functions for an element, and then either flip bits, or read bits. Therefore, they perform in $O(k)$ time, which is independent of the number of elements in the structure. Additionally, because the hashes are independent, they can be parallelized, which gives drastically better performance with multiple processors

Negatives :

- There is one more (1st is false positives) negative issue with a Bloom Filter: you can't delete! If you delete, you might delete another inserted value, as well! You could keep a second bloom filter of removals, but then you could get false positives in that filter...

Bloom Filters - Exercise



- Visit following website:
- <http://lmlib.github.io/bloomfilter-tutorial/>
- Add few strings say: “hello”, ‘hi’, ‘bye’
- Then check for possibility of following strings:
 - “hi”
 - “hello”
 - “good”
- Observe the result

What is k-dimensional data?

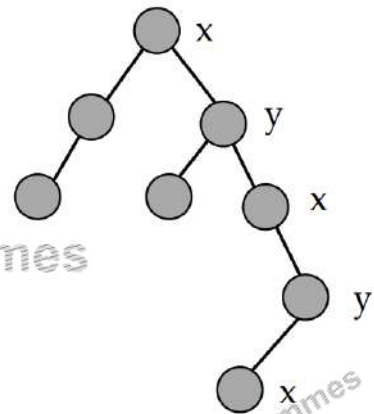


- If we have a set of ages say, {20, 45, 36, 75, 87, 69, 18}, these are one dimensional data.
- Because each data in the array is a single value that represents age.
- What if instead of only age we have to also store the salary for a person?
- The data would look like [{20, 1500}, {45, 5000}, {36, 4000}, {75, 2000}, {87, 0}, {18, 1000}].
- This data is two dimensional as each data set contains two values.
- Similarly, if we add one more attribute to it, say education it would be a 3 dimensional data and so on.

K-d Trees



- K-d trees is a data structure for storing finite set of multi-dimensional [**k-dimensional**] points. It was invented in 1970s by Jon Bentley.
- Name originally meant “3d-trees, 4d-trees” where k was the # of dimensions
- A Binary tree
 - Each level of the tree compares against 1 dimension, also called as “cutting/splitting” dimension
 - Each node contains a point **P = (x,y)**
- Idea: Each level of the tree compares against 1 dimension.



How do we split the points?

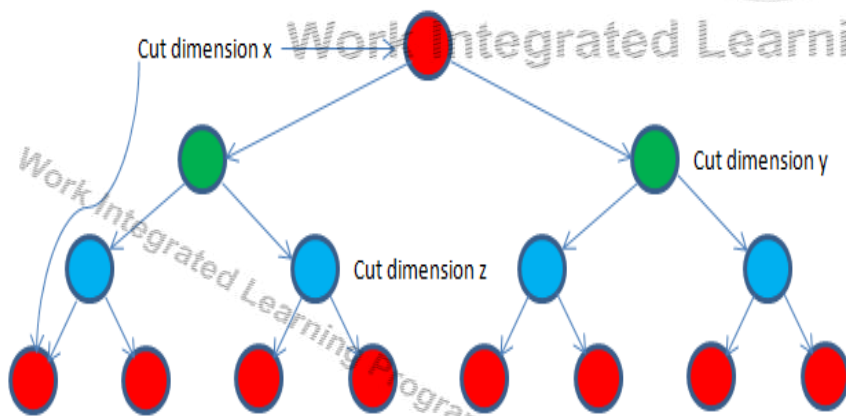


- We can extend the idea of BST to do this!
- This is what Jon Louis Bentley created in 1975.
- K-d tree is called 2-d tree or k-d tree with 2-dimension when $k = 2$ and so on.
- In BST, at each level of the tree we split the data points based on the data value.
- Since, BST deals with just one dimension the question does not arise which dimension.
- In k-d tree since we have more than one dimension.
- At each level we can choose to split the data based on only one dimension.



How do we split the points?

- So if we have 3 dimensions: x , y and z , at first level we split the data sets using x dimension. At 2nd level we do so using y dimension and at 3rd level we use z dimension.
- At 4th level we start again with x dimension and so on.
- If we are splitting the points based on x dimension for a certain level then we call x the *cutting dimension* for this level.

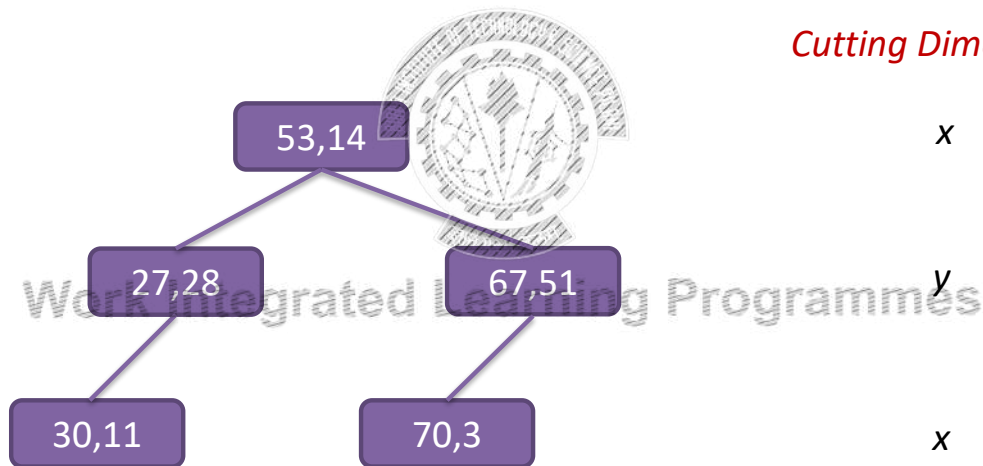


The root would have an x -aligned plane, the root's children would both have y -aligned planes, the root's grandchildren would all have x -aligned planes, and the root's great-grandchildren would all have y -aligned planes and so on.

K-d Tree Construction



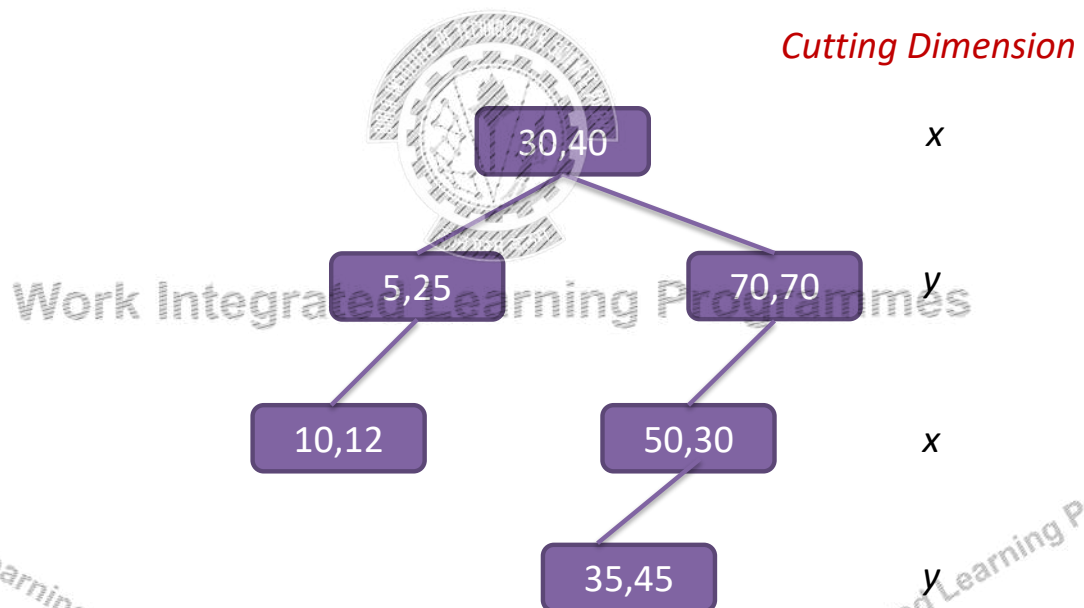
- Insert the following data points into an initially empty 2d tree :
(53,14) , (27,28) , (30,11) , (67,51) , (70,3)



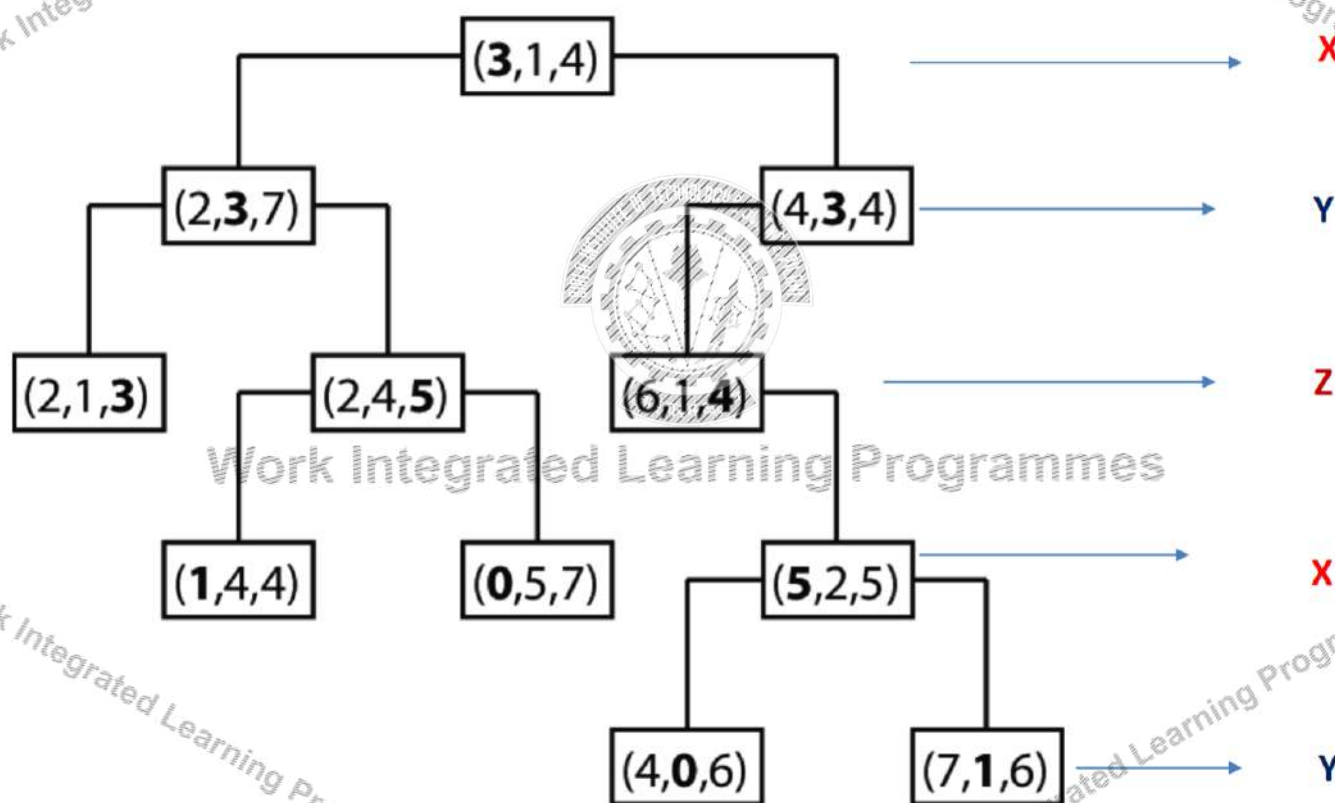
K-d Tree Construction Example 2



- Insert the following data points into an initially empty 2d tree :
(30,40) , (5,25) , (10,12) , (70,70) , (50,30) , (35,45)



K-d Tree with 3 Dimensions



K-d Tree Applications



- Nearest Neighbor [Explore & Self-Study! Will upload some slides as appendix]
- Range Queries
- Search Engines
- Image Search
- Machine learning
- ...
- ...

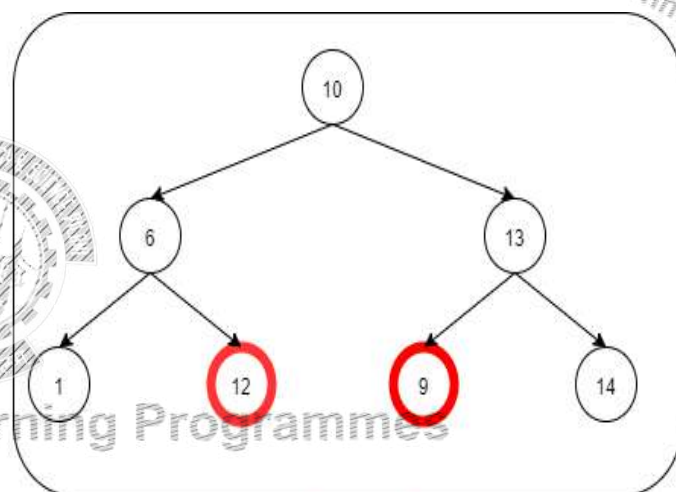
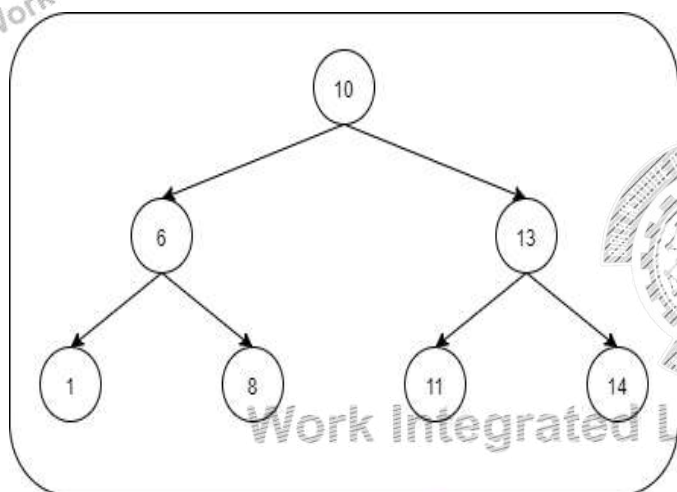


Recap [Binary Search Tree – Property]



- In binary search tree, *all the left subtree elements should be less than root data and all the right subtree elements should be greater than root data.*
 - This is called binary search tree property, note that, this property should be satisfied at **every node** in the tree.
 - The left subtree of a node, contains only nodes with keys **less** than the nodes key.
 - The right subtree of a node, contains only nodes with keys **greater** than the nodes key.
 - Both the left and right subtrees must also be binary search trees.

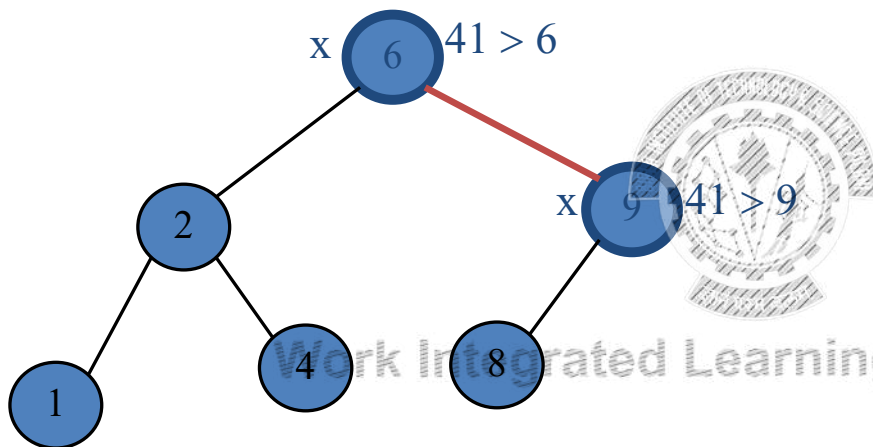
Binary Search Trees



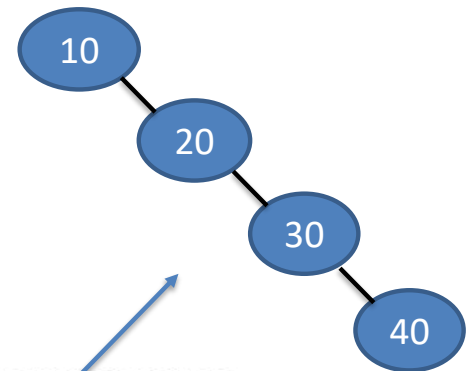
Find in BST



Search element 41



Search element 41



Skewed Tree

What is the time complexity of finding the element in binary search tree ?

$O(n)$ Why?!

Exercise 3



Given the input {4371, 1323, 6173, 4199, 4344, 9679, 1989}, and a fixed table size of 10, and a hash-function $H(X) = X \bmod 10$, show the resulting

1. Linear probing hash-table
2. Quadratic probing hash-table
3. Separate chaining hash-table

Appendix!



Work Integrated Learning Programmes

K-d Trees – Insertion Algorithm



```
insert(Point x, KNode t, int cd) {  
    if t == null  
        t = new KNode(x)  
    else if (x == t.data)  
        // error! duplicate  
    else if (x[cd] < t.data[cd])  
        t.left = insert(x, t.left, (cd+1) % DIM)  
    else  
        t.right = insert(x, t.right, (cd+1) % DIM)  
    return t  
}
```

Time Complexity for insertion into a K-d tree : $O(\log n)$ or in worst case $O(n)$

FindMin in k-d-trees

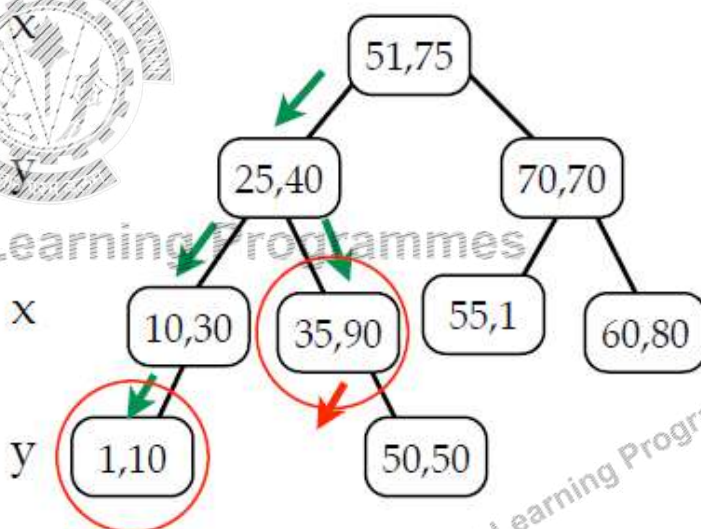
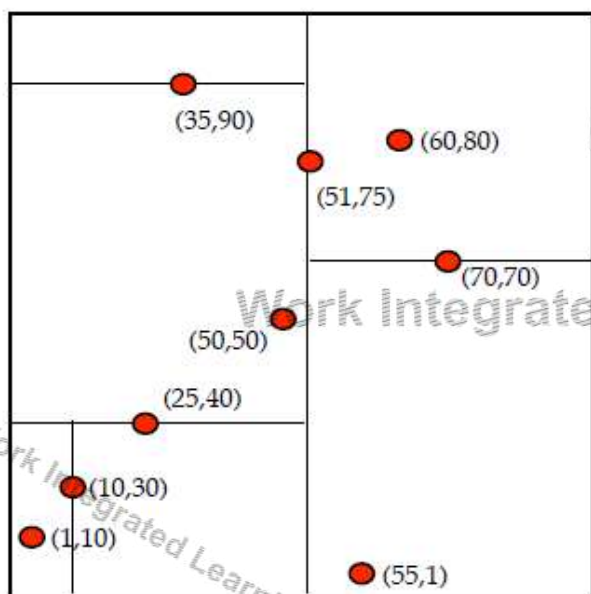


- FindMin(d): find the point with the smallest value in the d^{th} dimension.
- Recursively traverse the tree!
- If $\text{cutdim}(\text{current_node}) = d$, then
 - the minimum can't be in the right subtree, so recurse on just the left subtree.
 - if no left subtree, then current node is the min for tree rooted at this node.
- If $\text{cutdim}(\text{current_node}) \neq d$, then
 - minimum could be in *either* subtree, so recurse on both subtrees,
 - or current node may also be minimum. So we take minimum of three and return.

FindMin in k-d Trees



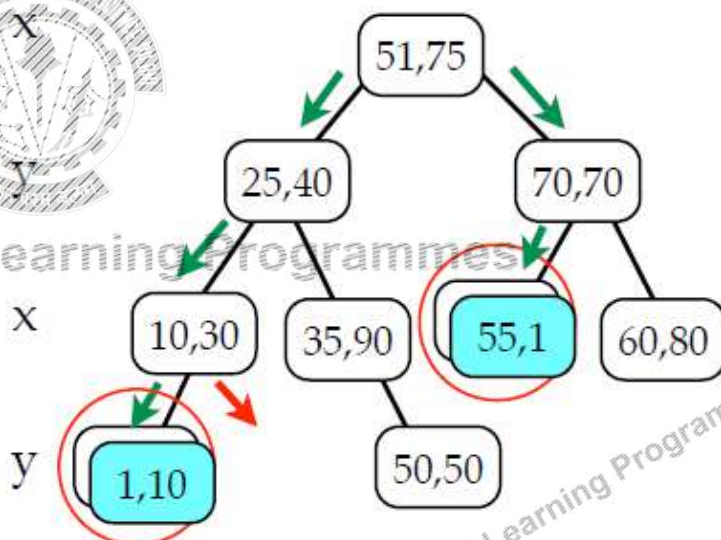
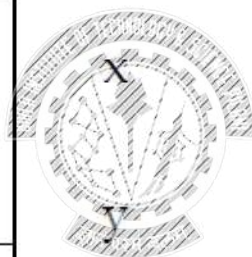
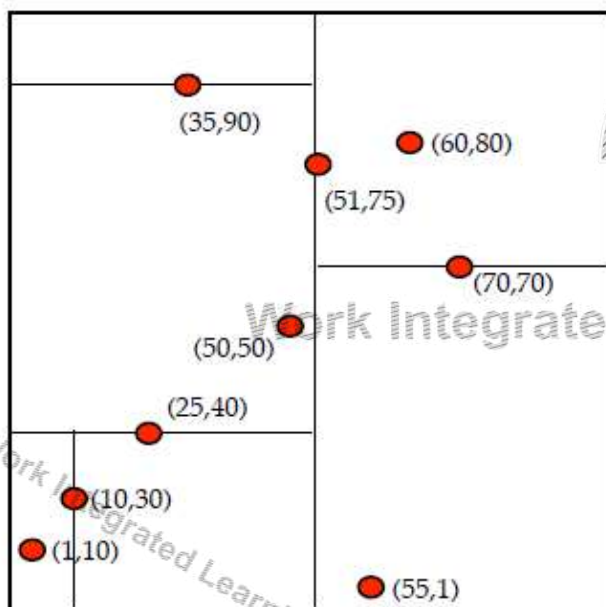
FindMin(x-dimension):



FindMin in k-d Trees



FindMin(y-dimension):



FindMin in k-d Trees Algorithm



```
Point findmin(Node T, int dim, int cd):
    // empty tree
    if T == NULL: return NULL

    // T splits on the dimension we're searching
    // => only visit left subtree
    if cd == dim:
        if t.left == NULL: return t.data
        else return findmin(T.left, dim, (cd+1)%DIM)

    // T splits on a different dimension
    // => have to search both subtrees
    else:
        return minimum(
            findmin(T.left, dim, (cd+1)%DIM),
            findmin(T.right, dim, (cd+1)%DIM)
            T.data
        )
```

51

K-d Trees – Deletion



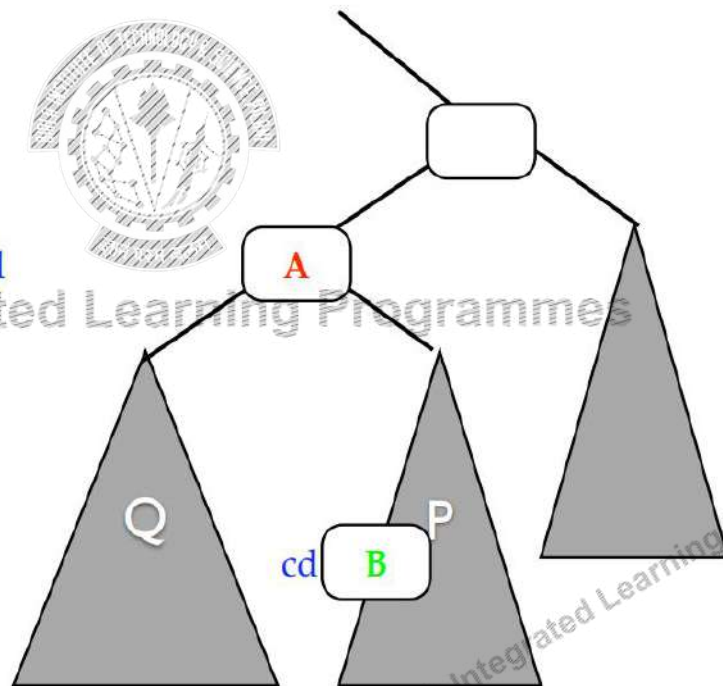
- Deletion from a k-d tree has many cases (just as in BST). Lets look at each of them.

Want to delete node **A**.

Assume cutting dimension of **A** is **cd**

In BST, we'd $\text{findmin}(\text{A.right})$.

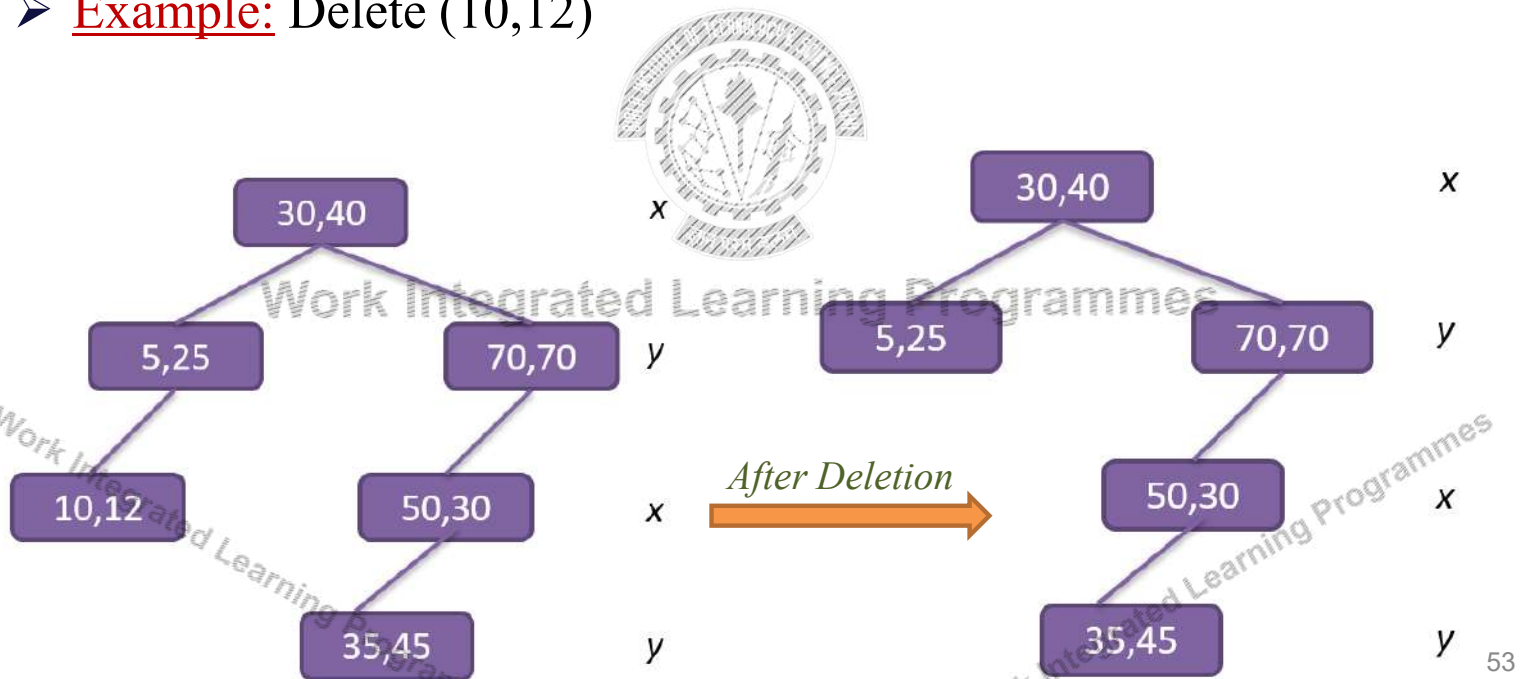
Here, we have to $\text{findmin}(\text{A.right}, \text{cd})$





K-d Tree – Deletion – Case 1

- Case 1: If the current node contains the point to be deleted and the node to be deleted is a leaf node, just delete it.
- Example: Delete (10,12)

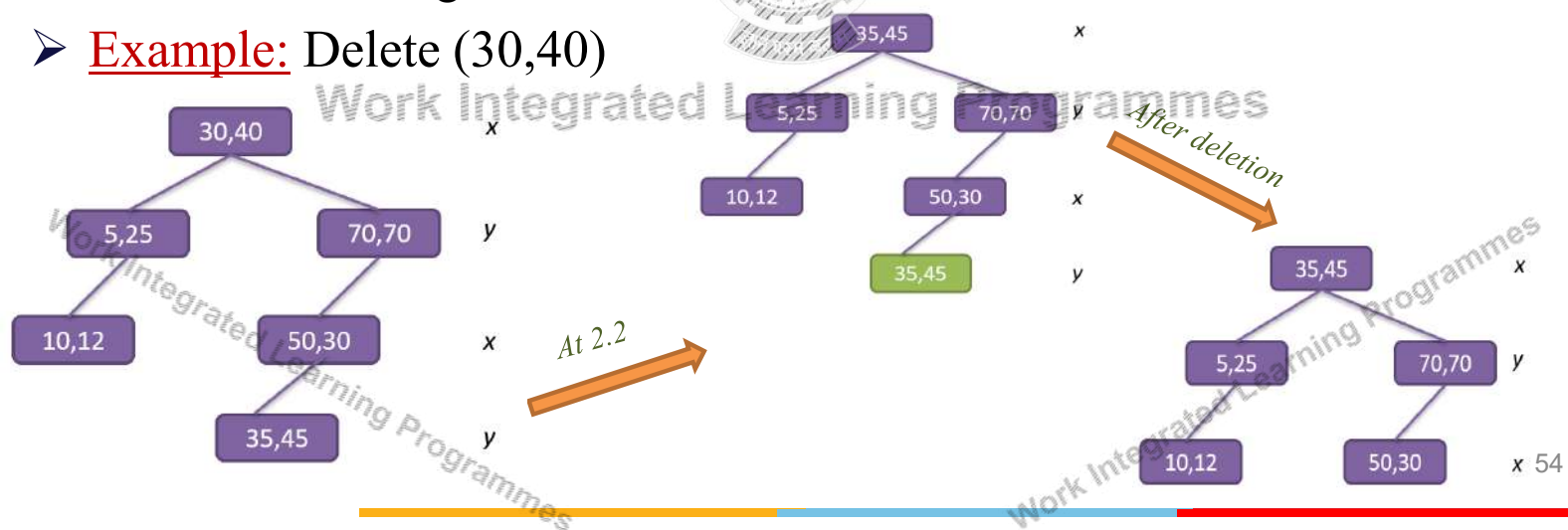




K-d Tree – Deletion – Case 2

- Case 2: If the current node contains the point to be deleted and the node to be deleted has a right subtree, then:
 - 2.1 Find the minimum of current node's dimension in **right** subtree i.e. FindMin(currentNode's dim) in right subtree.
 - 2.2 Replace the node with the node found in 2.1 and recursively delete minimum in right subtree.

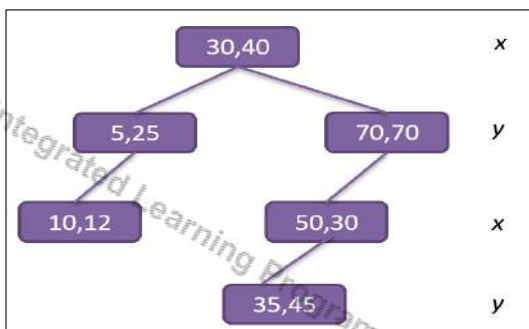
➤ Example: Delete (30,40)



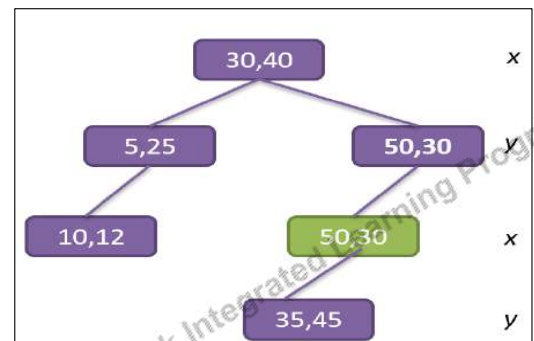


K-d Tree – Deletion – Case 3

- **Case 3:** If the current node contains the point to be deleted and the node to be deleted has a left subtree (no right subtree), then:
 - **3.1** Find the minimum of current node's dimension in **left** subtree i.e. FindMin(currentNode's dim) in left subtree.
 - **3.2** Replace the node with the node found in 3.1 and recursively delete minimum in left subtree.
 - **3.3** Make new left subtree as right child of current node [*Caution!*]
- **Example:** Delete (70,70)



At 3.2,
Min is (50,30)

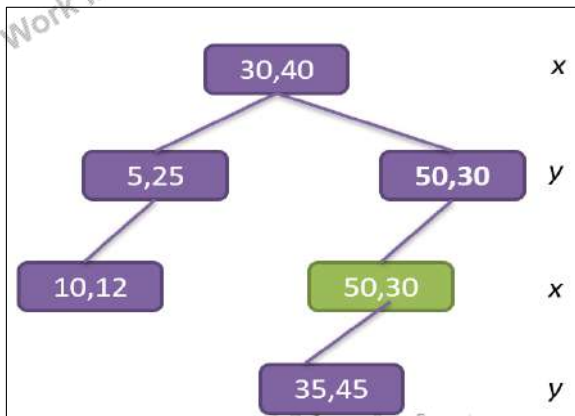


55

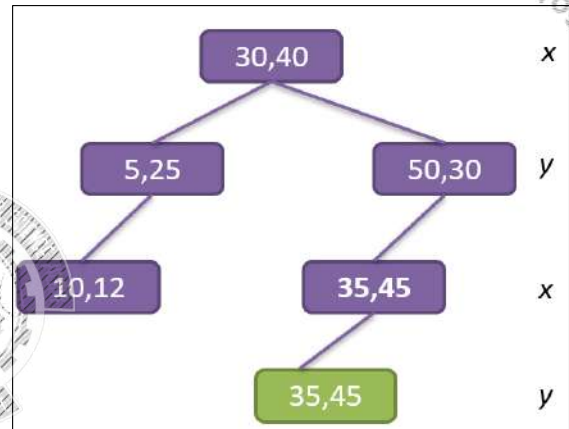
K-d Tree – Deletion – Case 3



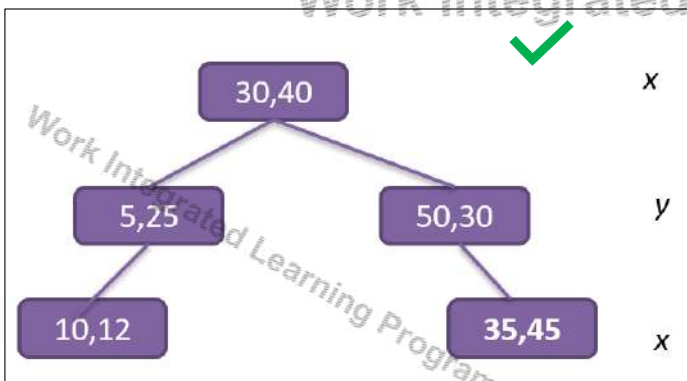
Recursively delete in left-subtree !!



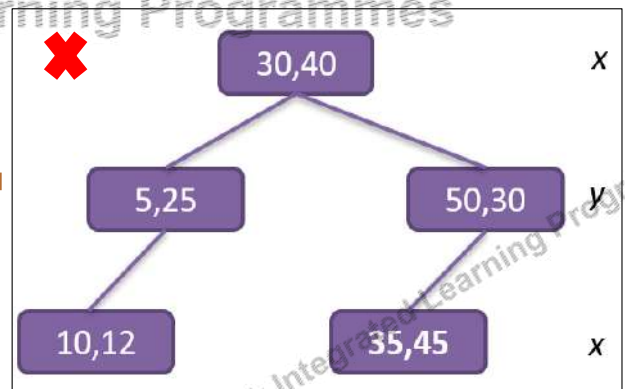
At 3,2,
Min is (35,45)



After deletion



After 3.3

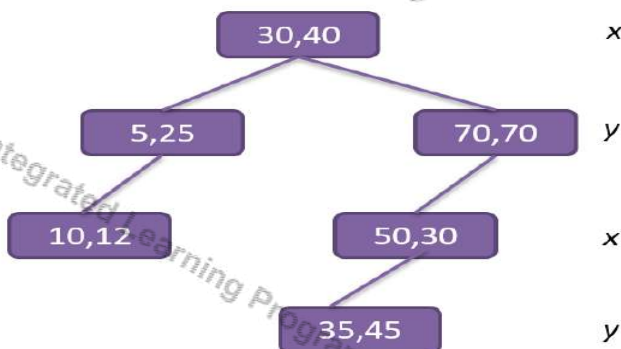




K-d Tree – Deletion – Case 4

Case 4: If the current node does not contain the point to be deleted, then:

- **4.1** If the node to be deleted is smaller than the current node on current dimension, recur for the left subtree. Otherwise, recur for the right subtree.
- **4.2** Eventually after 4.1, we will reach any of the other three cases. Use that case and perform deletion!
- **Example:** Delete (70,70) and current node is (30,40)



Here, $70 > 30$, hence recur on right of (30,40) to find (70,70). Then, perform Case 3.

K-d Tree – Deletion Algorithm



```
Point delete(Point x, Node T, int cd):
if T == NULL: error point not found!
next_cd = (cd+1)%DIM

// This is the point to delete:
if x = T.data:
    // use min(cd) from right subtree:
    if t.right != NULL:
        t.data = findmin(T.right, cd, next_cd)
        t.right = delete(t.data, t.right, next_cd)
    // swap subtrees and use min(cd) from new right:
    else if T.left != NULL:
        t.data = findmin(T.left, cd, next_cd)
        t.right = delete(t.data, t.left, next_cd)
    else
        t = null    // we're a leaf: just remove

// this is not the point, so search for it:
else if x[cd] < t.data[cd]:
    t.left = delete(x, t.left, next_cd)
else
    t.right = delete(x, t.right, next_cd)

return t
```




Work Integrated Learning Programmes

AVL Trees next!



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



60

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #10

DSECLZG519 – AVL Trees

Work Integrated Learning Programmes

Agenda for Session # 10



- Balance Factor ?
- AVL Trees
 - Motivation for AVL ?
 - Types of Rotations
 - LL
 - RR
 - LR
 - RL
 - Operations on AVL
 - Insertion
 - Removal
- Analysis of AVL tree
- Exercises





Balanced Binary Search Trees

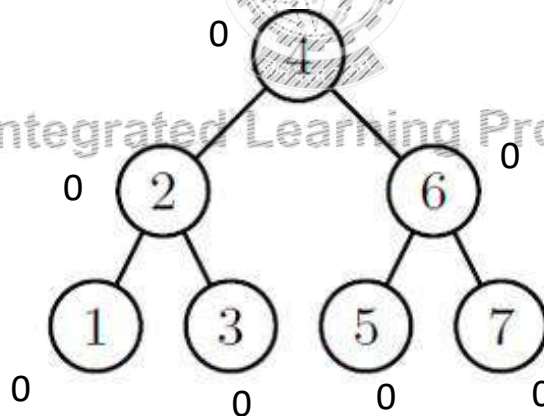
- We have seen different trees whose worst case complexity is $O(n)$ (including BST).
- Where n is the number of nodes in the tree. This happens when the trees are skew trees ☹
- Will try to reduce this worst case complexity to $O(\log n)$ *by imposing restrictions on the heights.*
- In general, the height balanced trees, are represented with **HB(K)**, where K is the difference between left subtree height and right subtree height.
- Sometimes K is called *balance factor*.

$$\text{Balance Factor} = \text{Height}(\text{Left subtree}) - \text{Height}(\text{right subtree})$$

Balanced Binary Search Trees



- If $HB(K)$, if $K = 0$ (if balance factor is zero), then we call such binary search trees as full balanced binary search trees.
- That means, in $HB(0)$ binary search tree, the difference between left subtree and right subtree height should be *at most zero*.



Note: Numbers next to the node indicate the balance factor

AVL (Adelson-Velsky and Landis) Trees

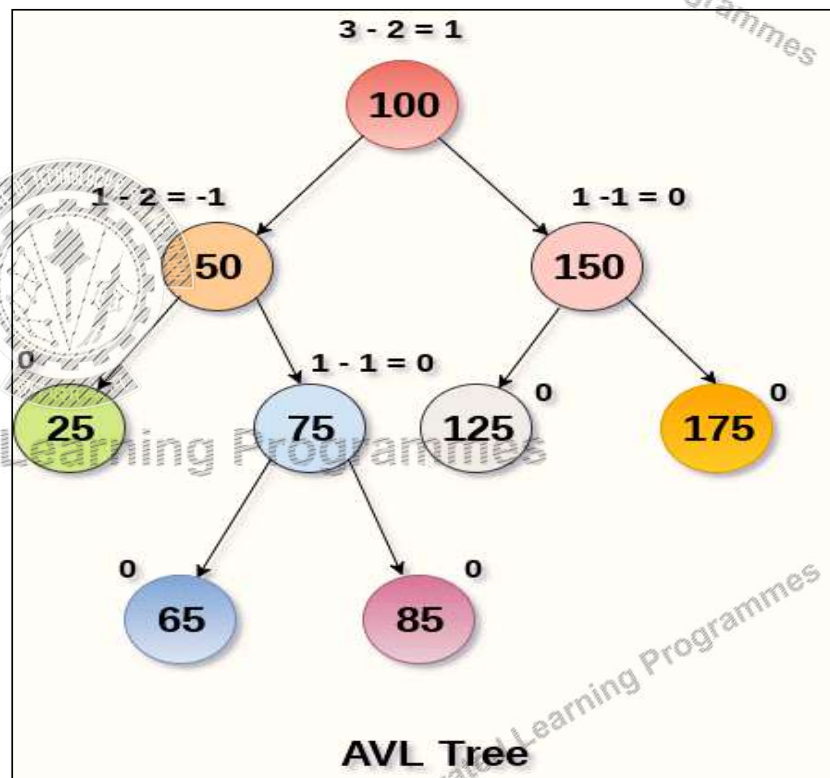


- In a height balanced tree $HB(K)$, if $K = 1$ (if balance factor is one), such binary search tree is called an AVL tree.
- That means an AVL tree is a binary search tree with a balance condition the difference between left subtree height and right subtree height is **at most 1**.
- Hence, in an AVL tree, each node is associated with a balance factor which can be :
 - **Balance factor of 0** : If the height of the left subtree and height of the right subtree are same.
 - **Balance factor of 1** : If the height of the left subtree is 1 more than the height of the right subtree.
 - **Balance factor of -1** : If the height of the left subtree is 1 less than the height of the right subtree.

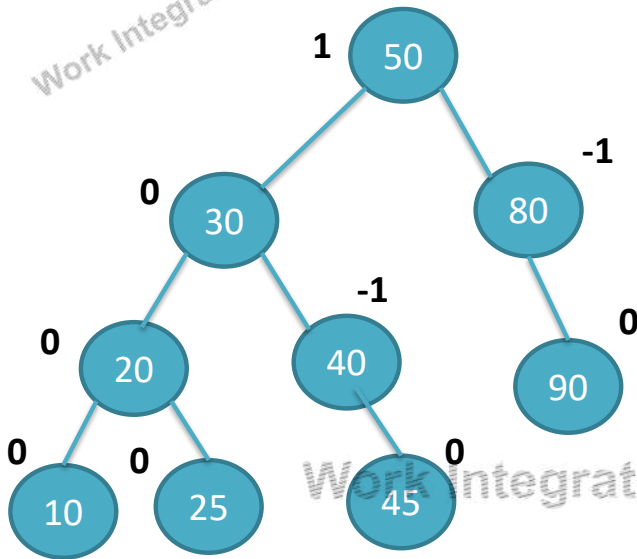
Properties of AVL Trees



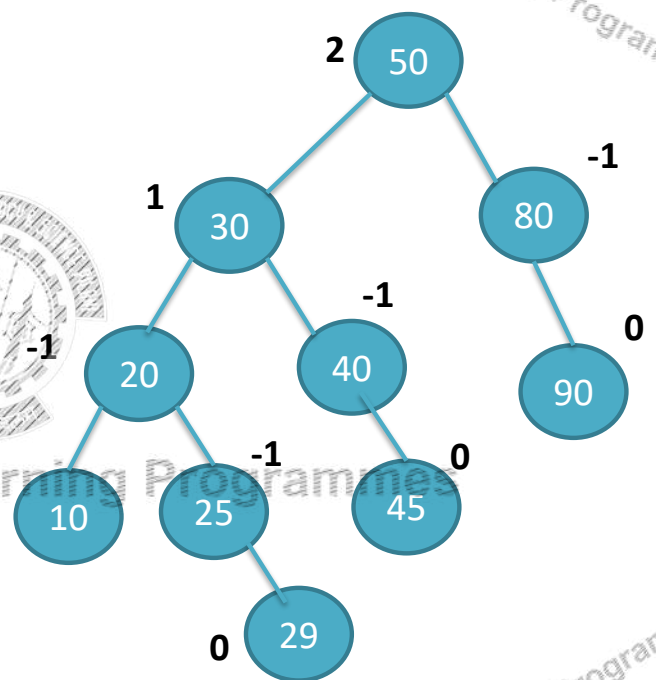
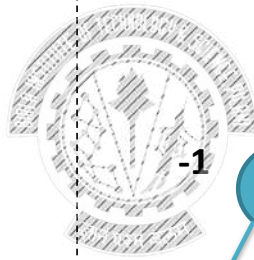
- It is a binary search tree, and
- For any node X, the height of left subtree of X and height of right subtree of X differ by *at most 1*.
- i.e. An AVL tree is a height balanced BST !



Examples



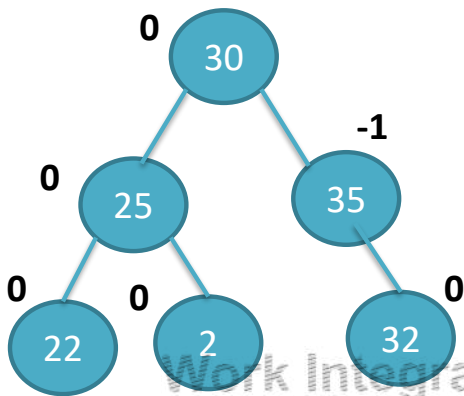
AVL
Tree ✓



AVL
Tree ✗

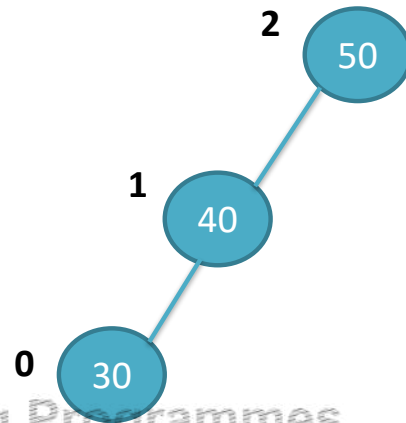
*Reason : Balance
factor of 50 is 2*

Examples



Reason : This is not a BST !

AVL
Tree **×**



Reason : Balance factor of 50 is 2

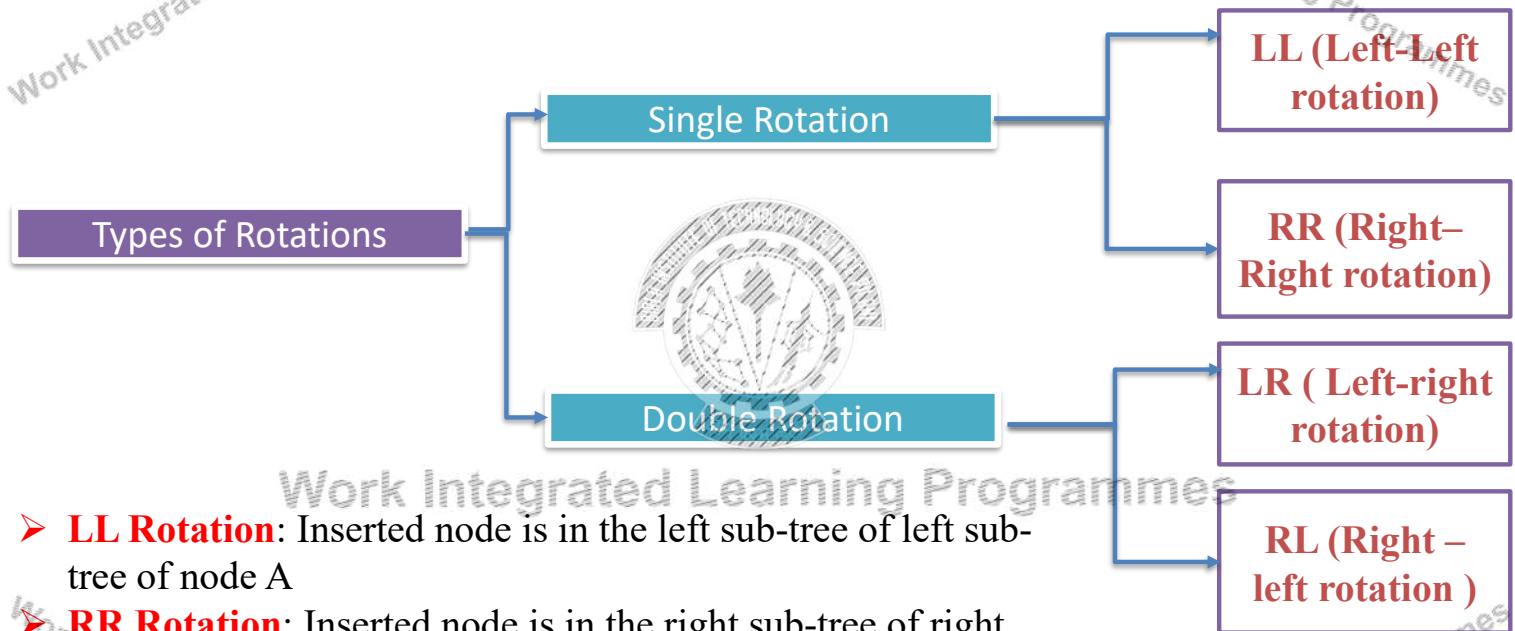
AVL
Tree **×**

Creating an AVL Tree



- Creating an AVL tree is nothing but repeated insertion of elements / items into binary search tree [BST]. But, after adding each element / item into the tree, a check must be made to ensure that the balance factor of **each node** is -1 or 0 or 1.
- If so, then the tree is balanced and the tree is a AVL Tree.
- If the balance factor of **any** node is not either -1 or 0 or 1, it is necessary to rearrange the nodes in the tree so that the balance factor of all the nodes will either be -1 or 0 or 1.
- This rearrangement is achieved using “**rotations**”.
- The different types of rotations are LL , RR , LR and RL.

Rotations in AVL Tree



- **LL Rotation:** Inserted node is in the left sub-tree of left sub-tree of node A
- **RR Rotation:** Inserted node is in the right sub-tree of right sub-tree of node A
- **LR Rotation:** Inserted node is in the right sub-tree of left sub-tree of node A
- **RL Rotation:** Inserted node is in the left sub-tree of right sub-tree of node A



Which rotation is required ?

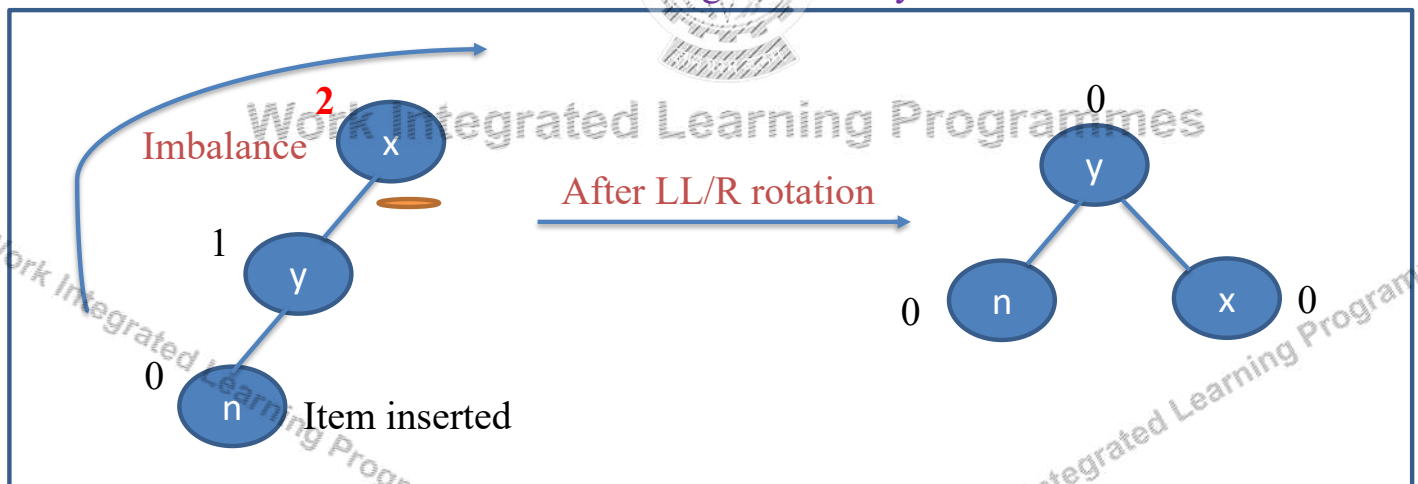
- After inserting a node, trace backward towards the root and compute the balance factors along the path.
- If we find a node whose balance factor is other than -1 or 0 or 1, then rotation is required. But how do we find which of the four rotations is required ? We follow the below process :
 - **Step 1**: Identify the first node where balance factor is **not** -1/0/1 in the path.
 - **Step 2**: Identify two other nodes below the node where imbalance occurs in the path.
- If the three nodes identified after Step (1) and (2) are in a straight line, single rotation is required (Either LL/R-rotation or RR/L rotation).
- If the three nodes identifies are **not** in a straight line, we will need double rotation (LR rotation or RL rotation).

LL imbalance (R rotation) in AVL Tree



- If the balance factor of a node where the imbalance occurs is **2**, then the tree is heavy towards left. So, this is called left-left imbalance and to balance it we rotate right.
- This is called as LL rotation or R-rotation. This is a **single** rotation and is in clock wise.

Case 1: No right child of y

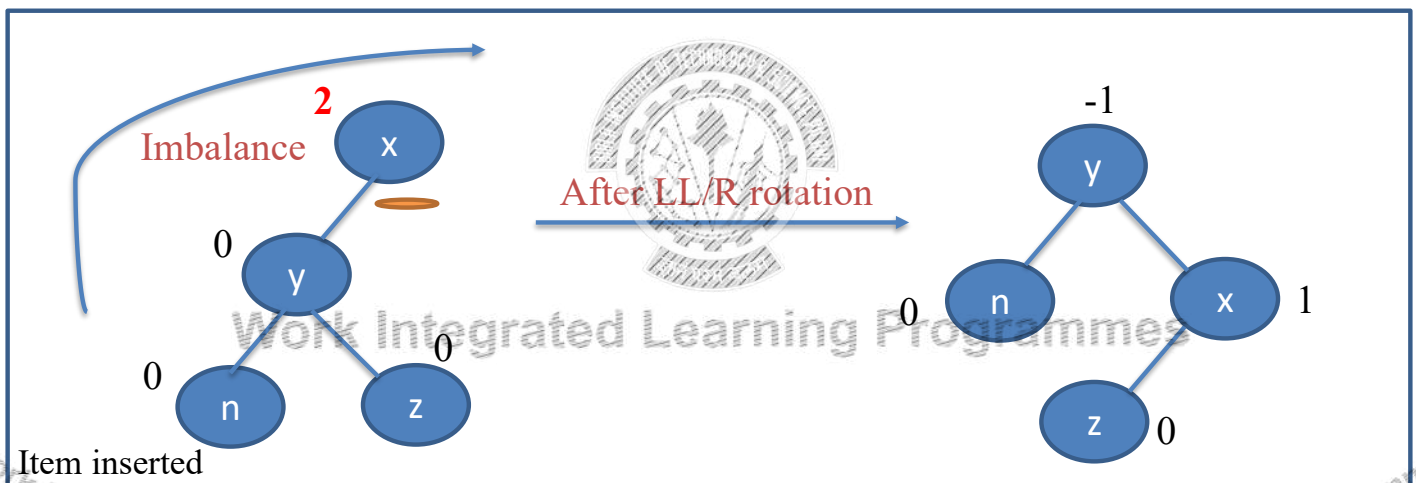


13

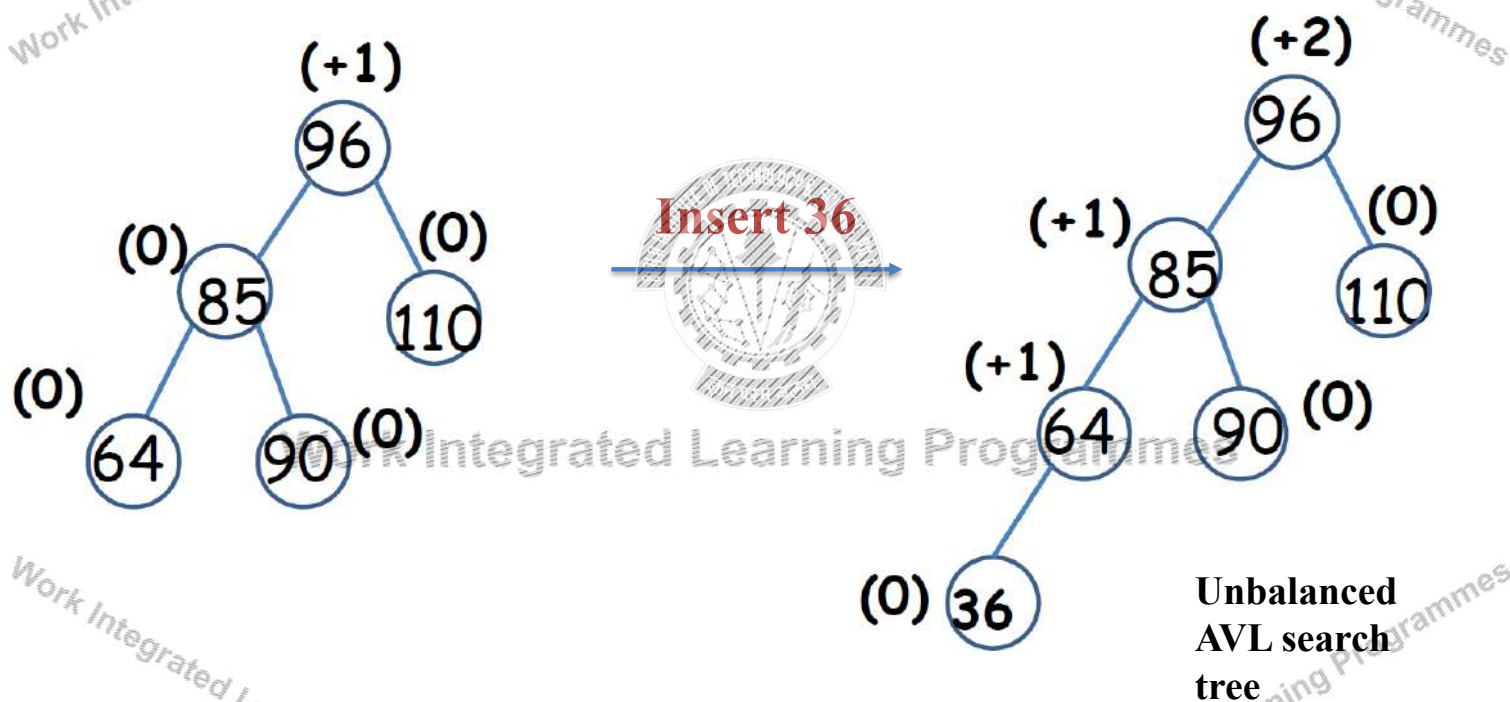
LL imbalance (R rotation) in AVL Tree



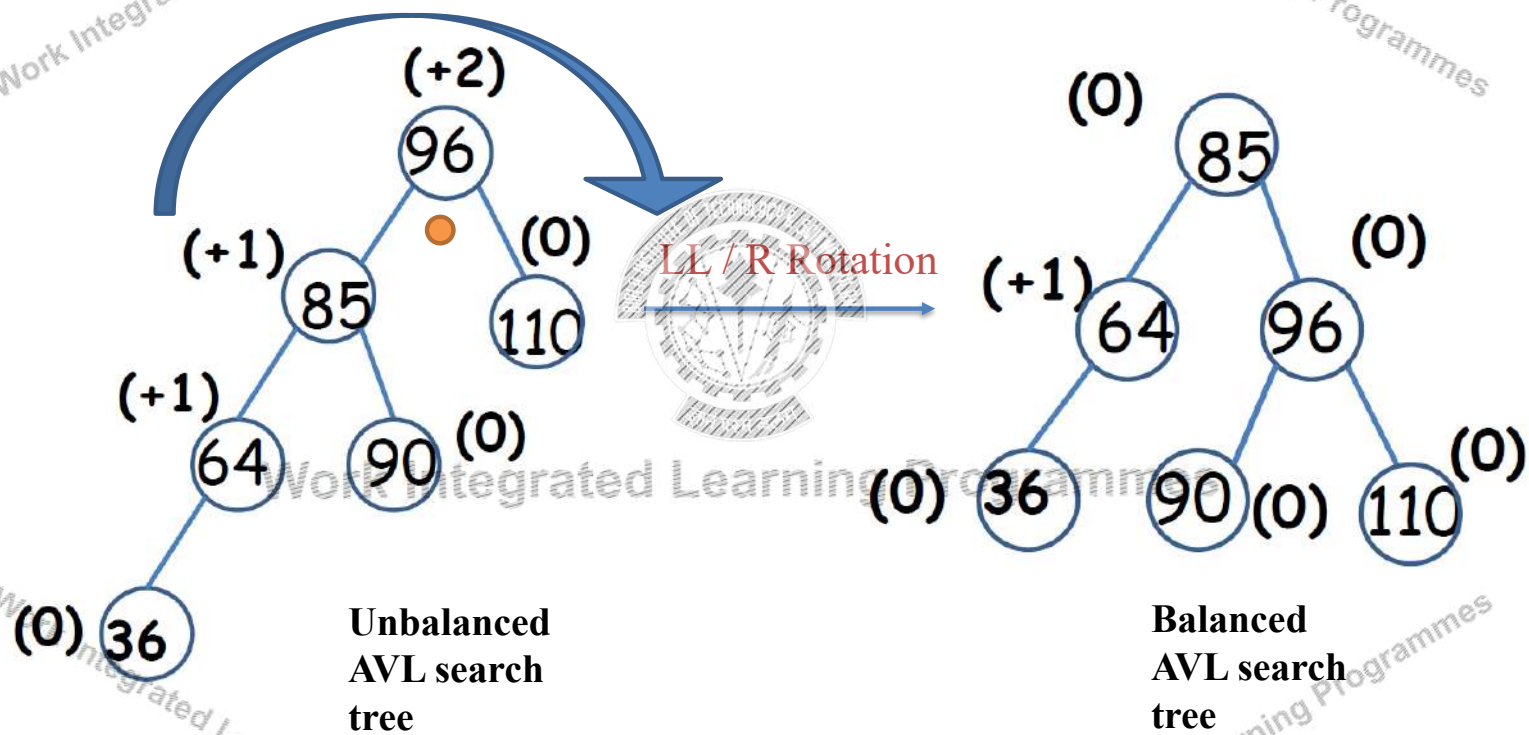
Case 2: There is a right child of y



LL / R Rotation Example



LL / R Rotation Example

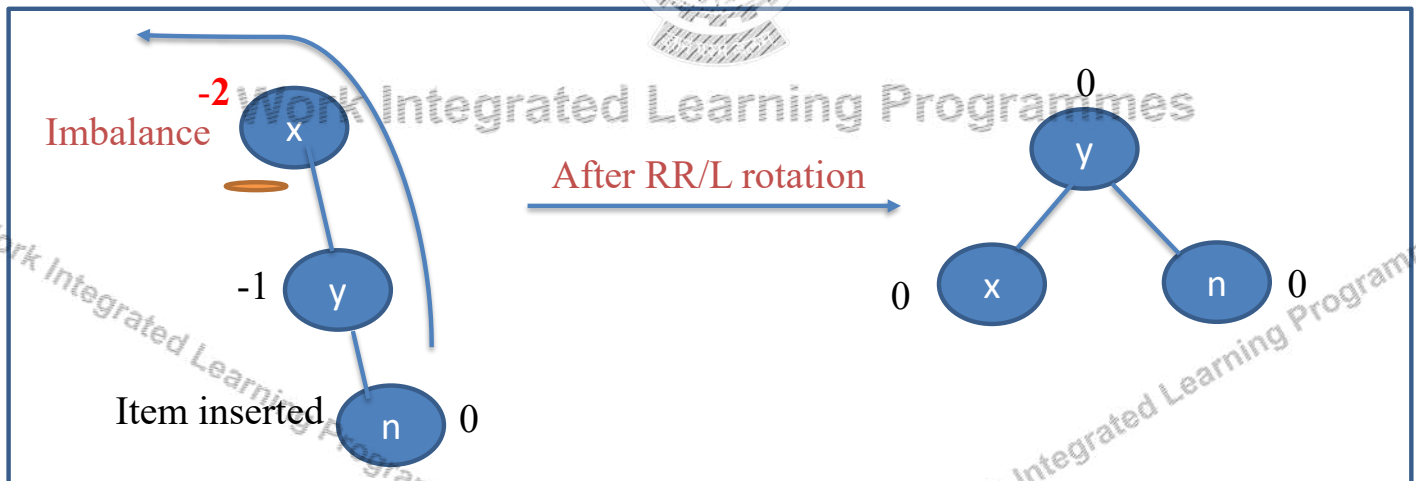


RR Imbalance (L-rotation) in AVL Tree



- If the balance factor of a node where imbalance occurs is **-2**, then the tree is heavy towards right. So, this is called right-right imbalance and to balance it we rotate left.
- This is called RR-rotation or L rotation. This is a **single** rotation and is anti-clock wise.

Case 1: No left child of y

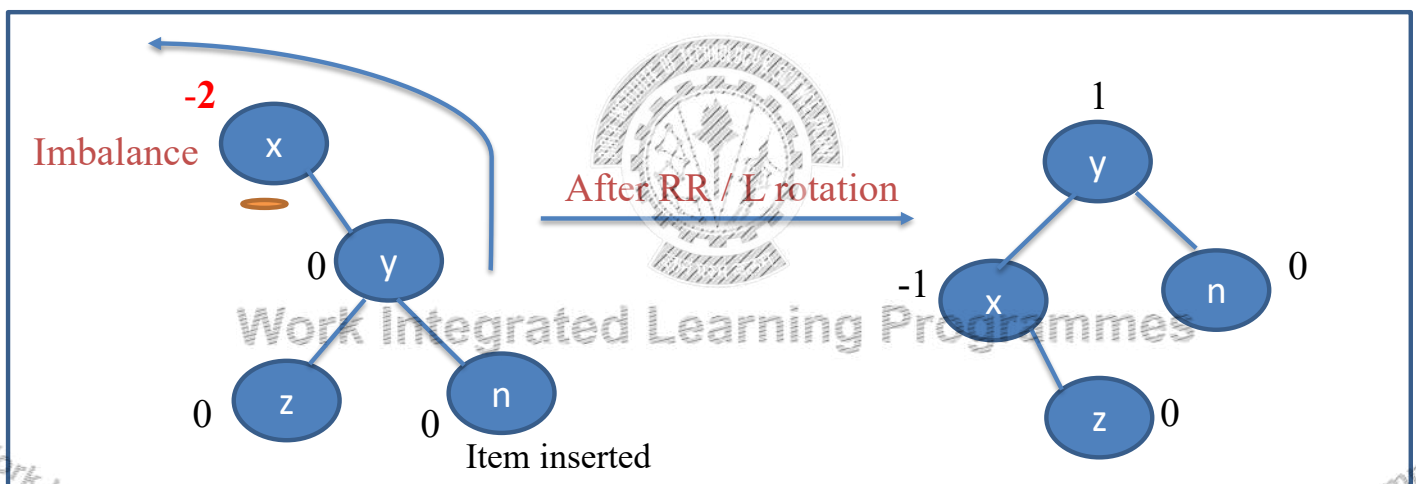


17

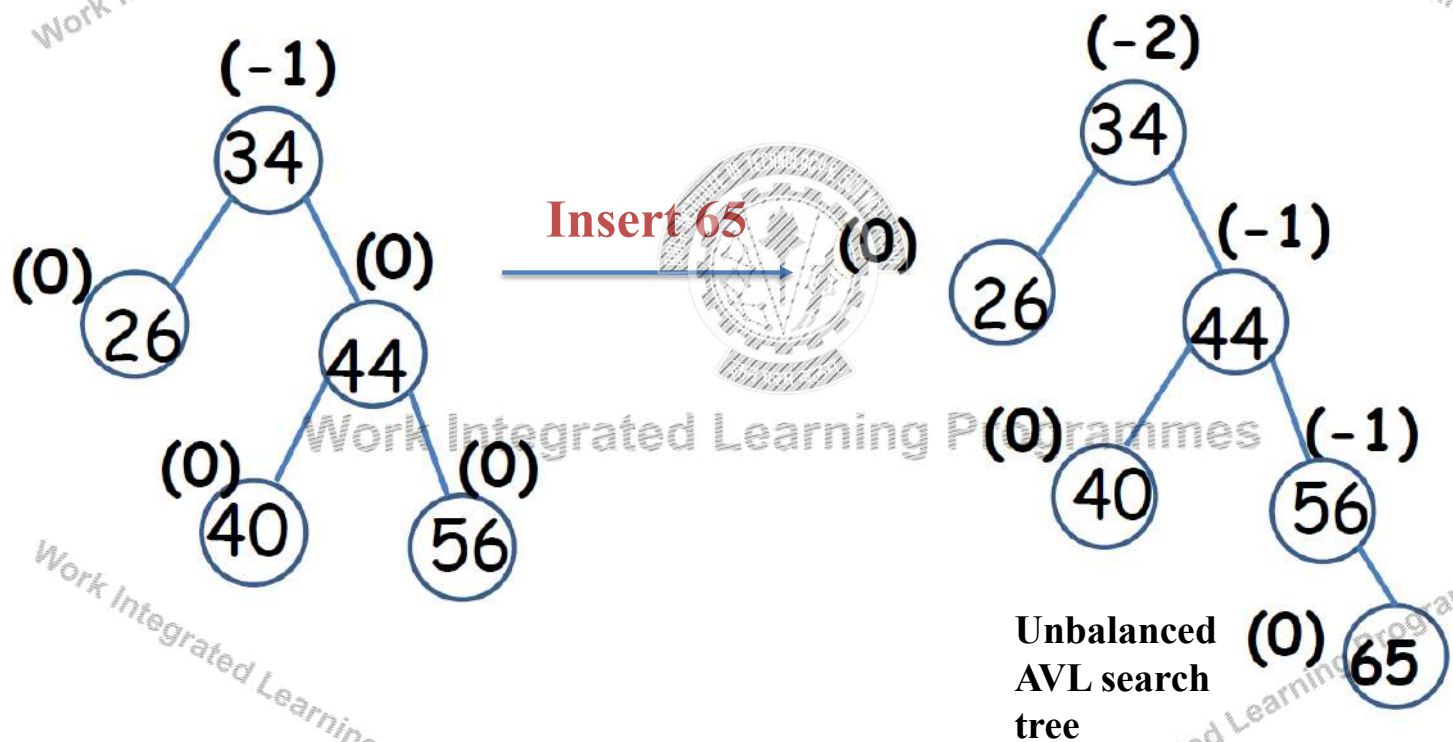
RR Imbalance (L-rotation) in AVL Tree



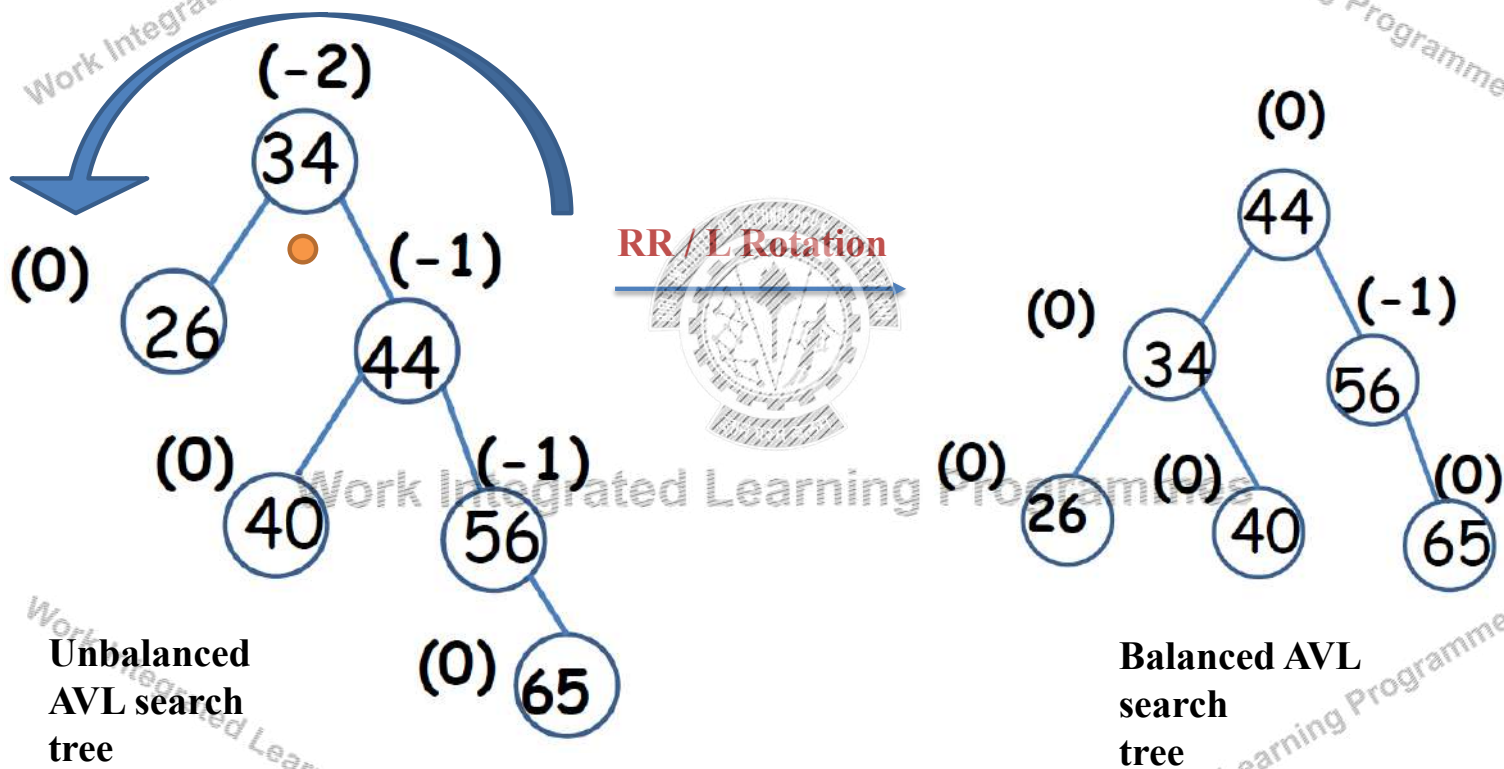
Case 2: There is a left child of y



RR / L Rotation Example



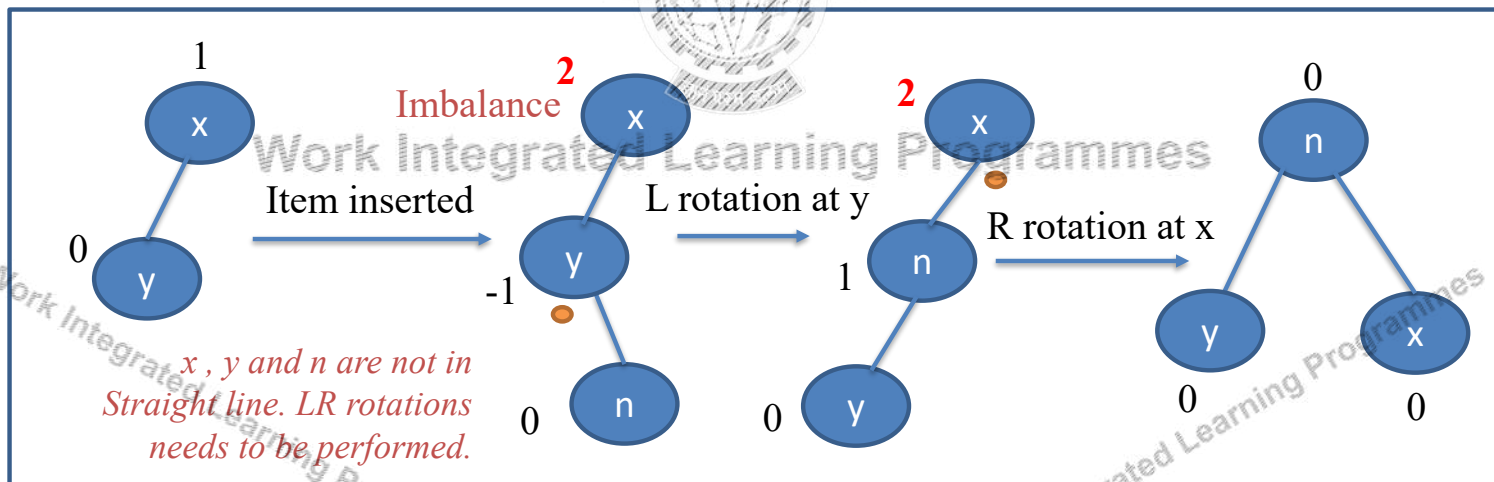
RR / L Rotation Example



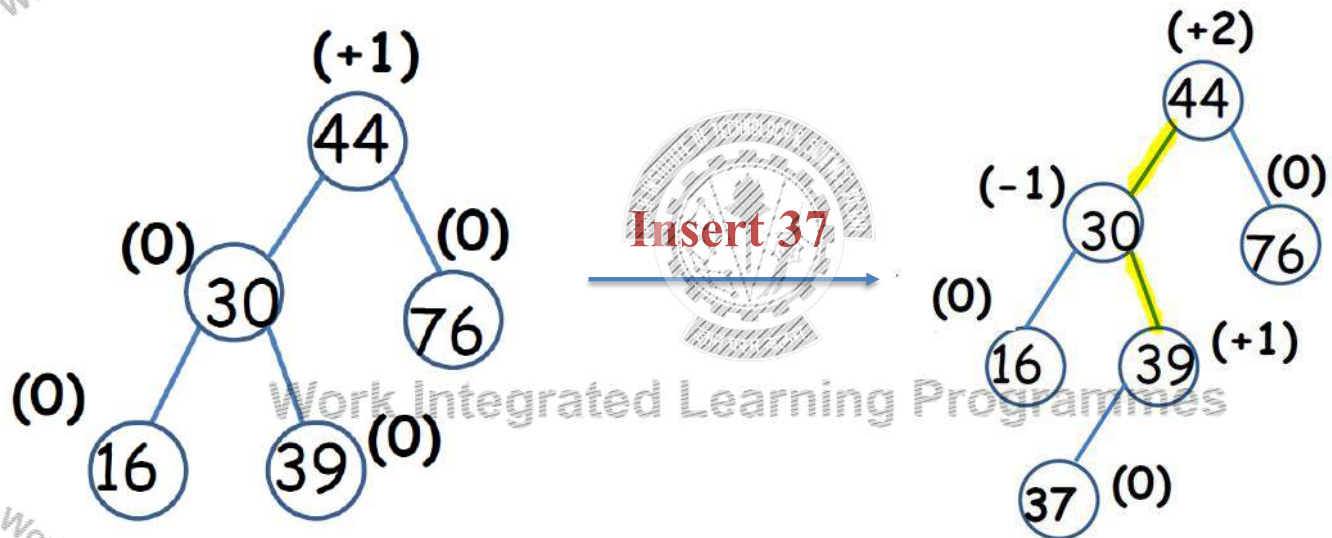
Left Right rotation (LR) in AVL Tree



- If the three nodes identified are **not in a straight line**, then double rotation is required. A double-rotation is a combination of two single-rotations. If the order of imbalance is LR, then LR rotation is needed.
- The first step will be L rotation (anti) and then the R(clock) rotation.

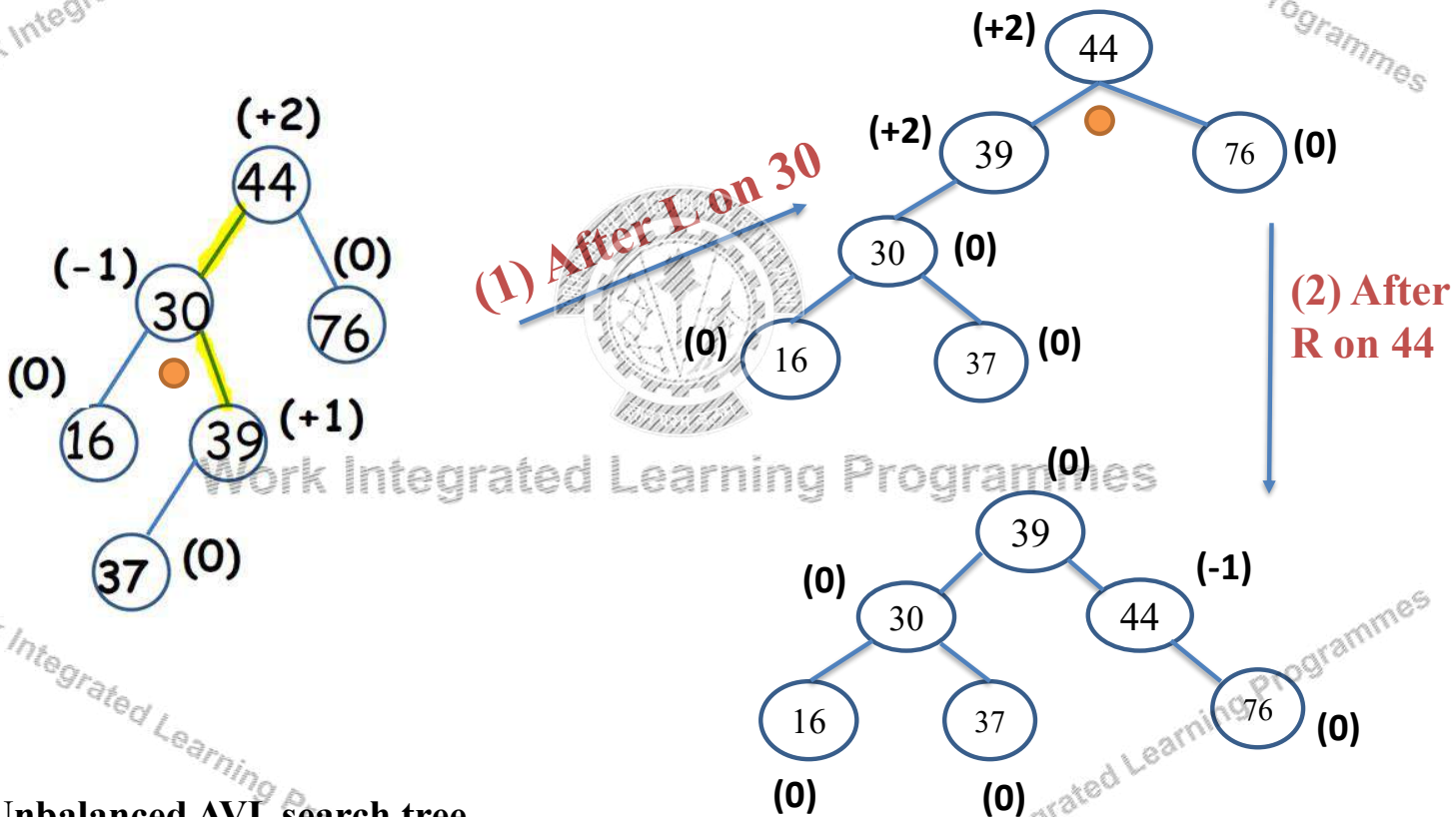


(LR) rotation Example



Unbalanced AVL search tree

(LR) rotation Example



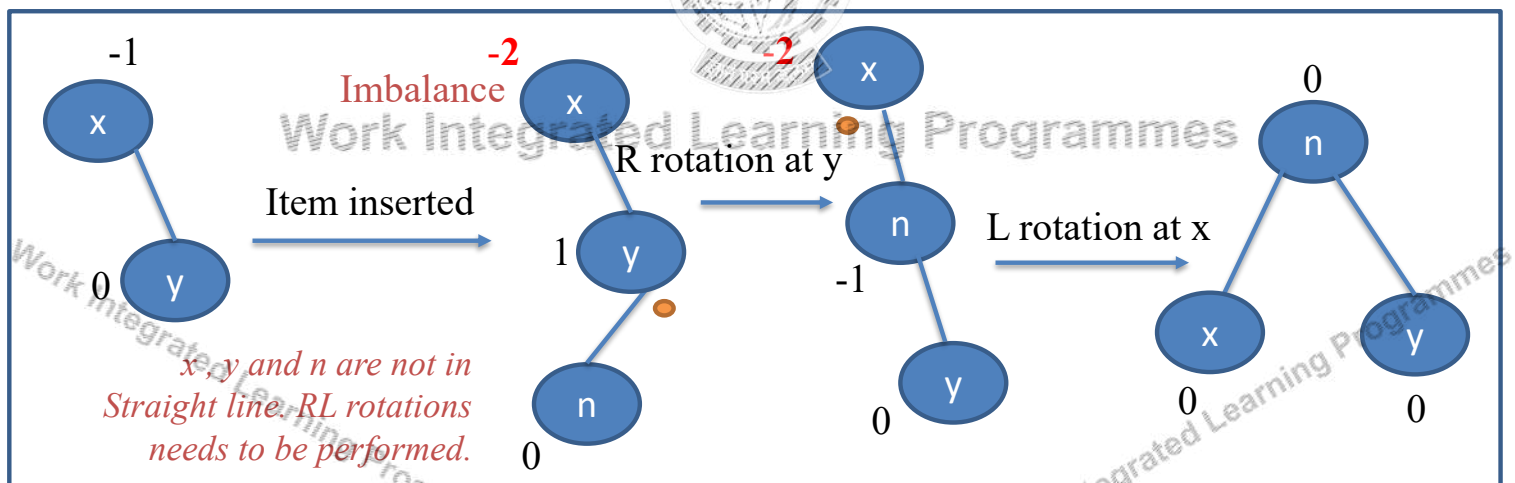
Unbalanced AVL search tree

Balanced AVL search tree

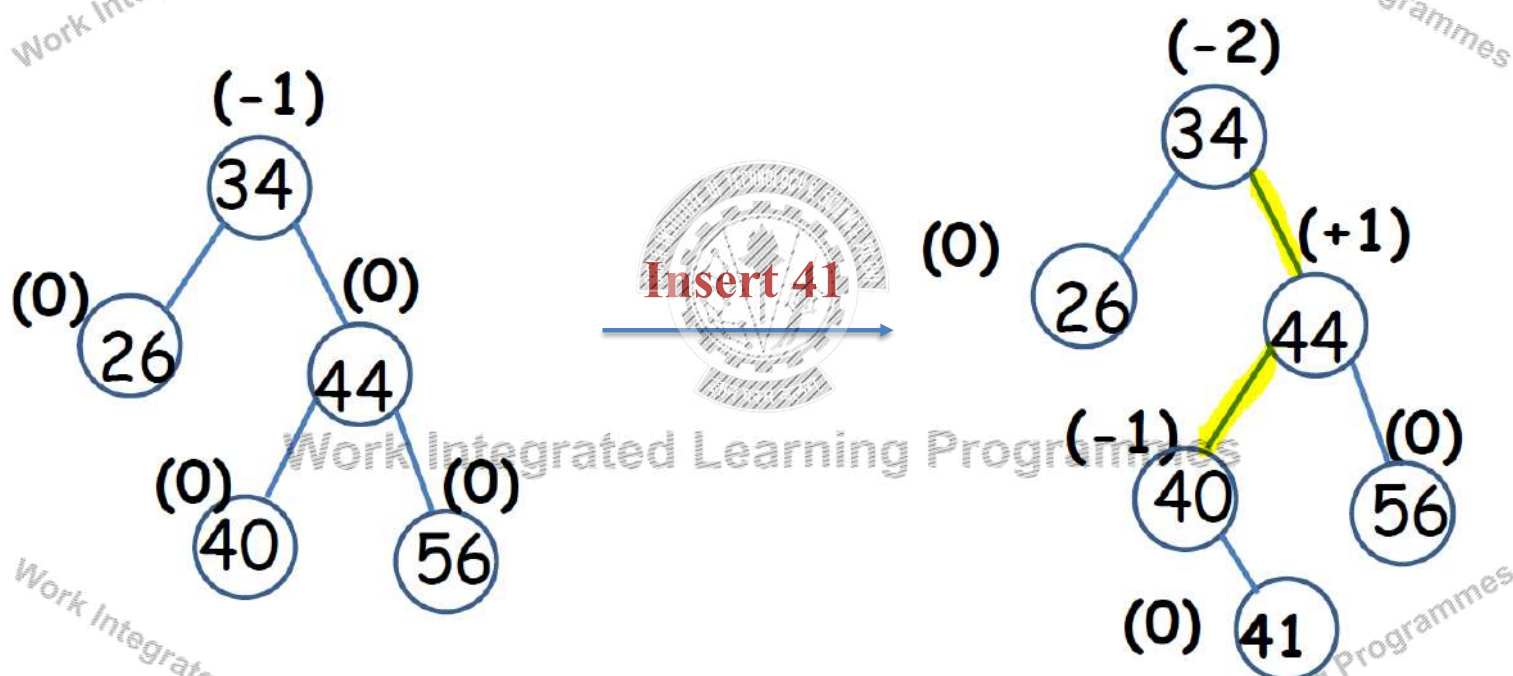
Right Left rotation (RL) in AVL Tree



- If the three nodes identified are not in a straight line, then double rotation is required. A double-rotation is a combination of two single-rotations. If the order of imbalance is RL, then RL rotation is needed.
- The first step will be R rotation (clockwise) and then the L(anti-clock) rotation.



(RL) rotation Example



Unbalanced AVL search tree

innovate achieve lead



Ready Reckoner ☺



Involved Nodes in Straight line ?	Balance Factor	Type of rotation	Rotation Direction	Where?
Yes	+2	R / LL rotation	Clockwise	On the impacted node
Yes	-2	L / RR rotation	Anti-Clockwise	On the impacted node
No	+2	LR rotation	(1) L rotation (2) R rotation	(1) Below the impacted node (2) On the originally impacted node
No	-2	RL rotation	(1) R rotation (2) L rotation	(1) Below the impacted node (2) On the originally impacted node

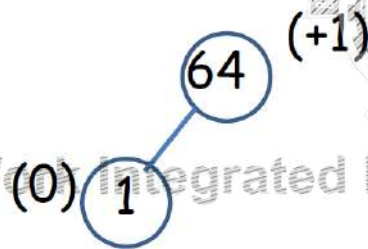
Insertion Example



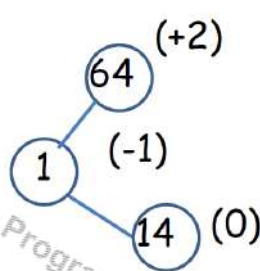
- Construct an AVL search tree by inserting the following elements in the order of their occurrence :

64, 1, 14, 26, 13, 110, 98, 85

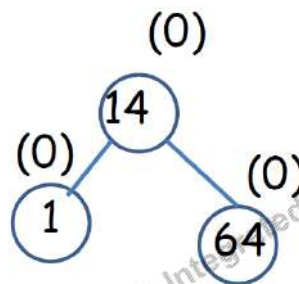
Insert 64, 1



Insert 14



LR
⇒

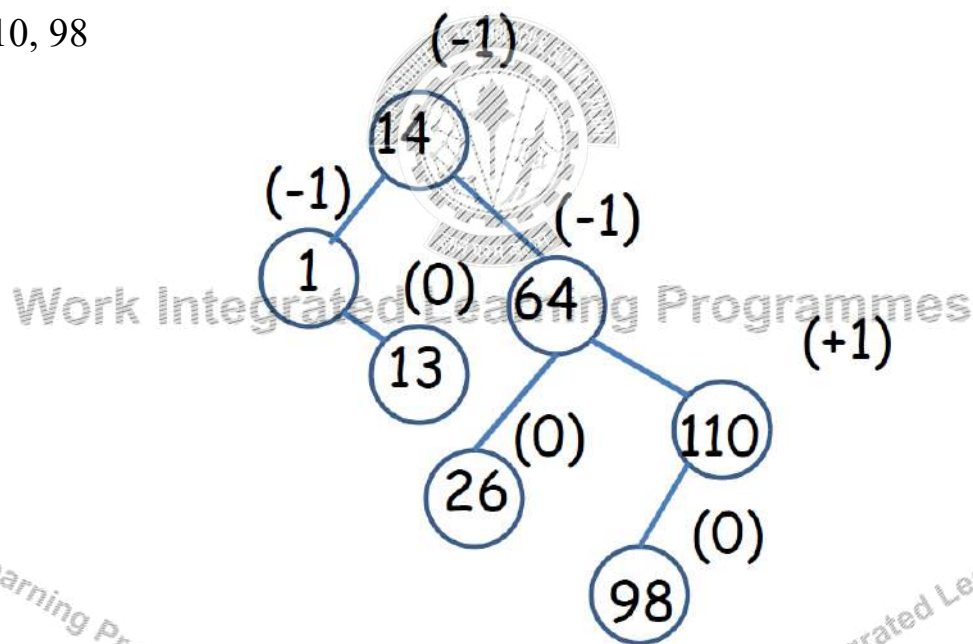


Insertion Example



64, 1, 14, 26, 13, 110, 98, 85

Insert 26, 13, 110, 98

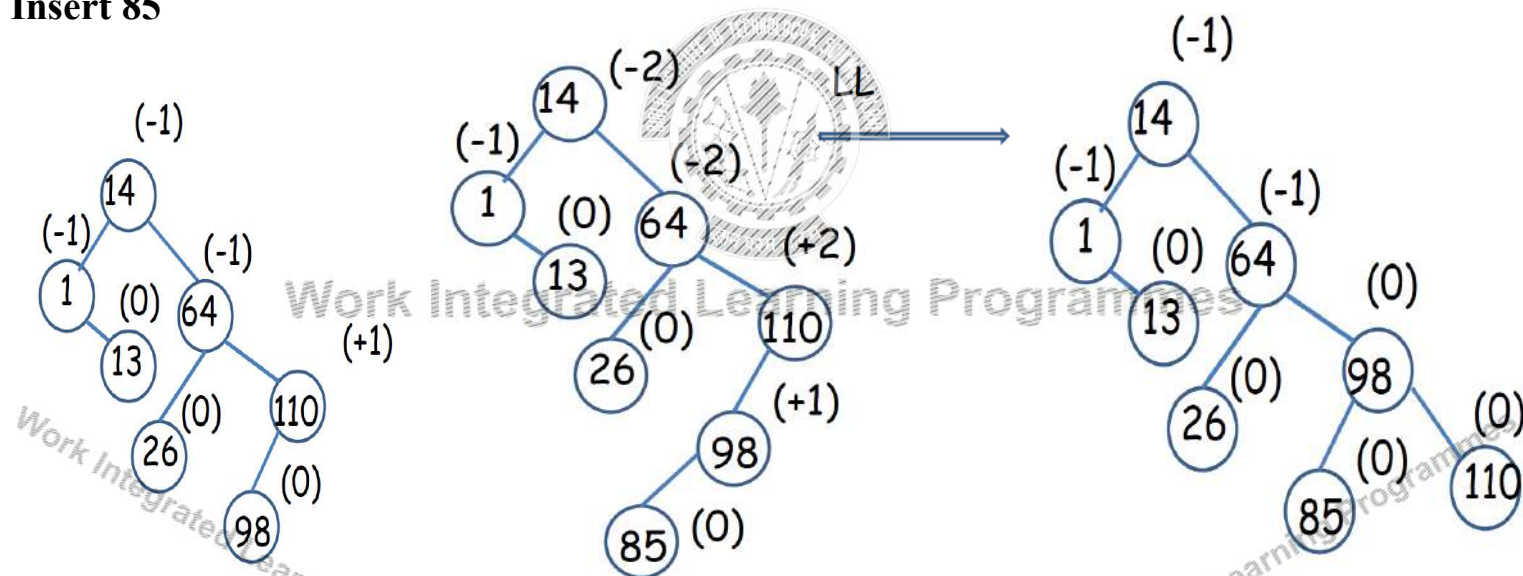


Insertion Example



64, 1, 14, 26, 13, 110, 98, **85**

Insert 85



Balanced AVL search tree

30

Removal from AVL Tree



- *Deletion in AVL search tree proceed the same way as the deletion in binary search tree (So, all the 3 cases of BST is applicable here as well – Node to be deleted has no child, 1 child and 2 child).*
- However, in the event of imbalance due to deletion, one or more rotation need to be applied to balance the AVL tree.

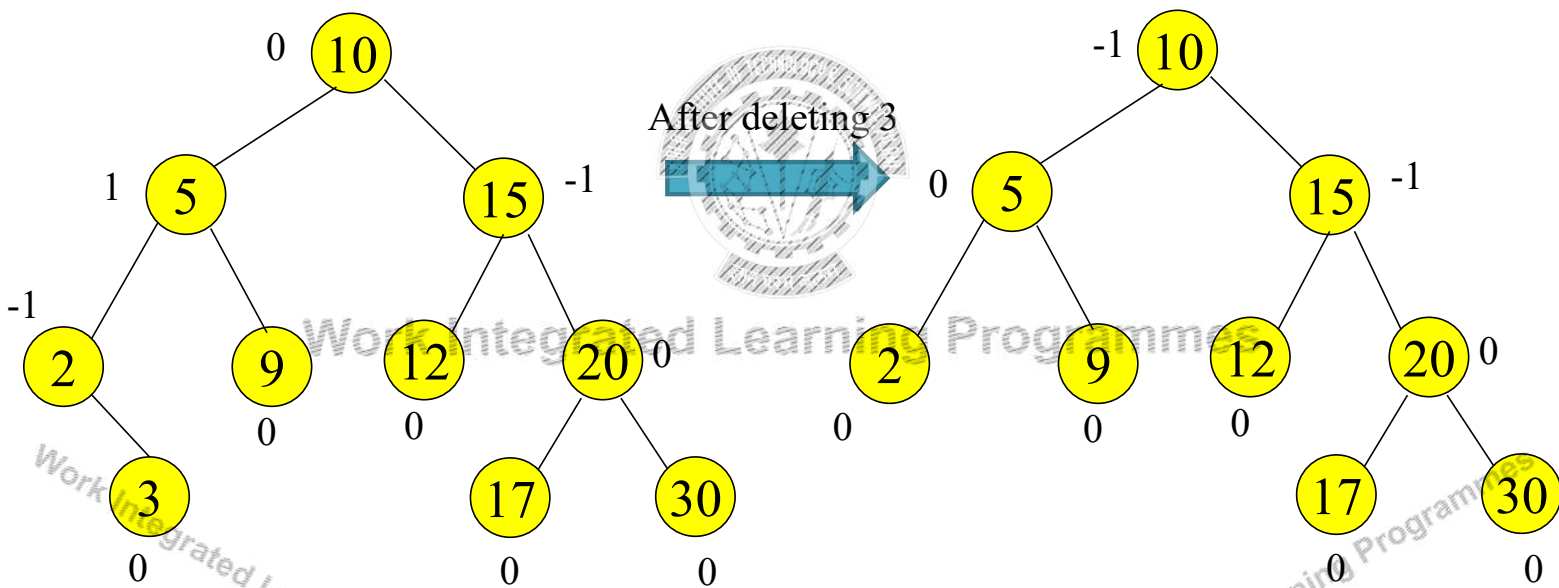
Work Integrated Learning Programmes

AVL Deletion



Delete 3 from :

*3 has no children!! Easy delete & recalculate
Balance factor. No need of rotations ☺*

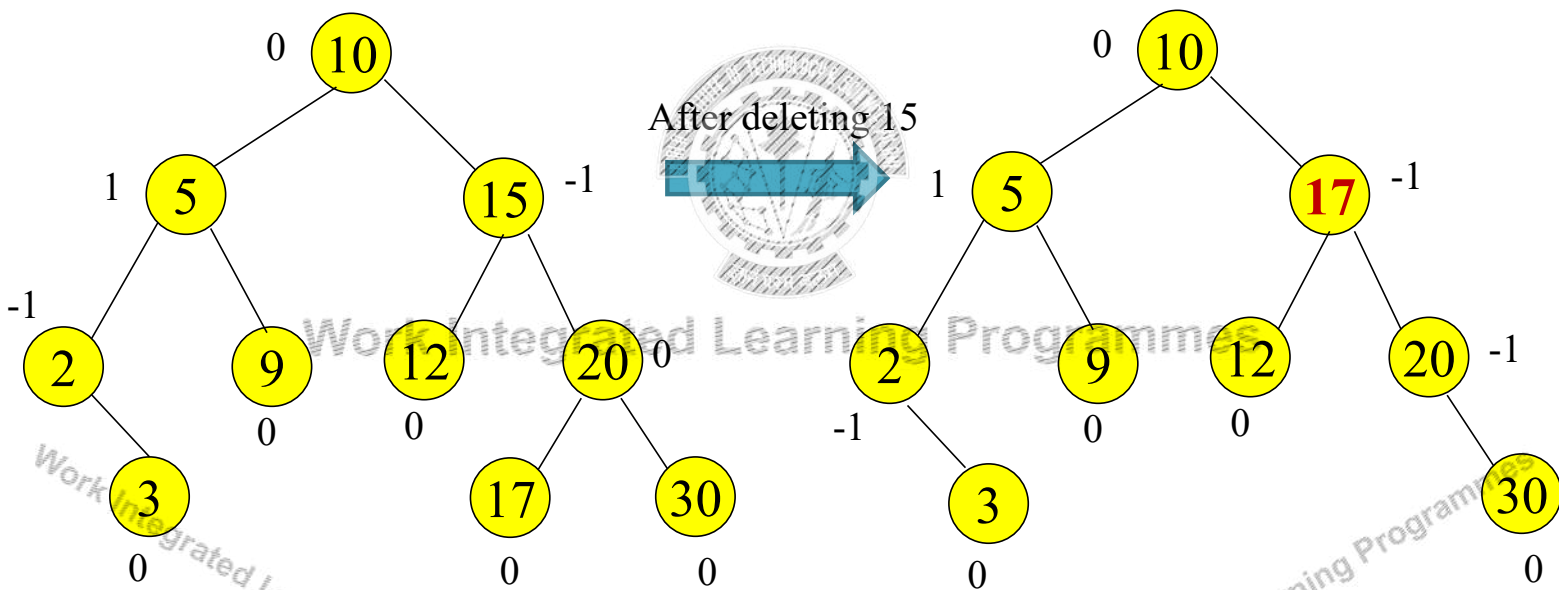


AVL Deletion



Delete 15 from :

15 has 2 children!! Find the inorder successor and place in position of 15

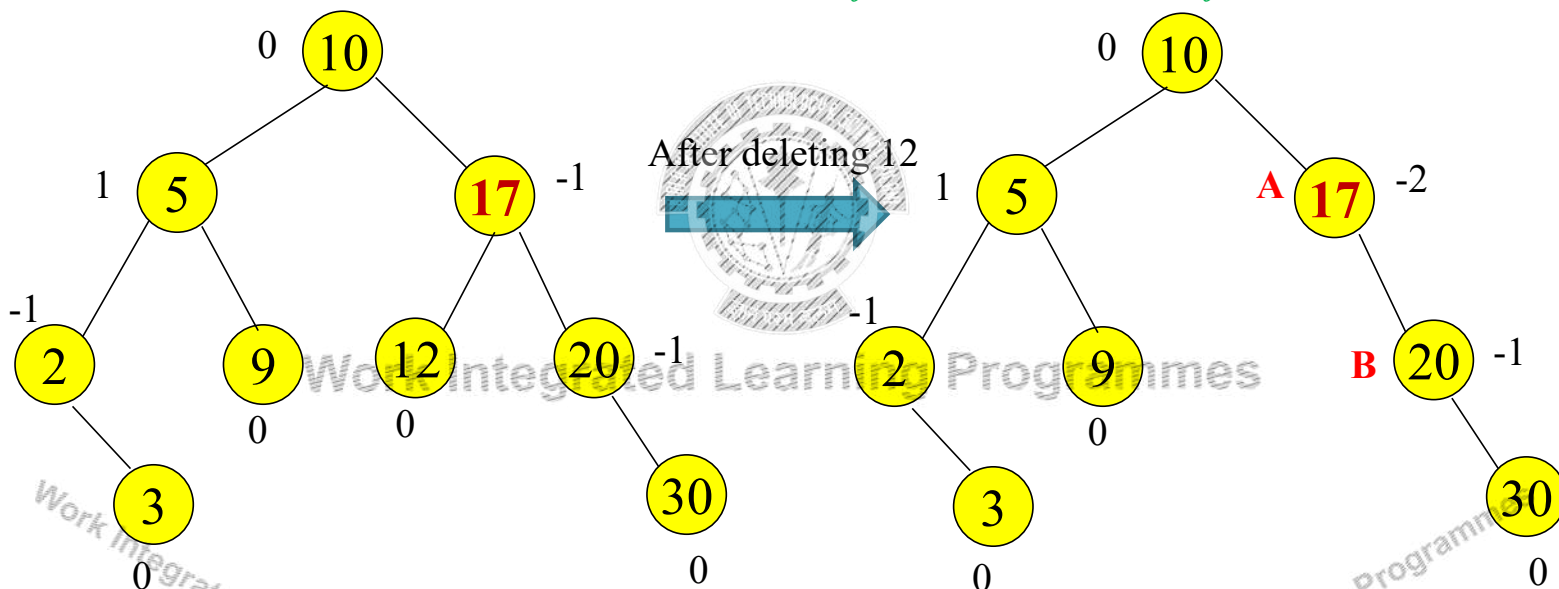


AVL Deletion



Delete 12 from :

12 has no children!! Easy delete & recalculate
Balance factor. There is need of rotations ☹️

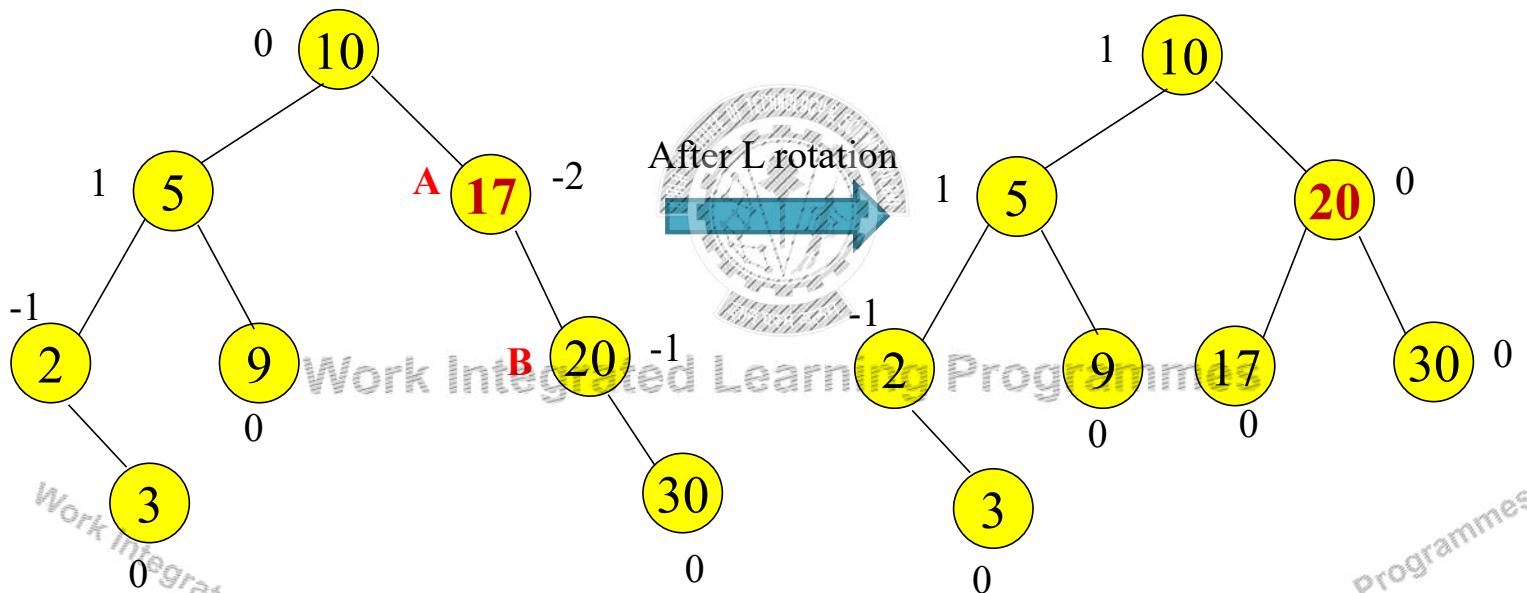


We are in L-1 Case, so
perform L rotation

AVL Deletion



Delete 12 from :



Analysis of AVL Trees



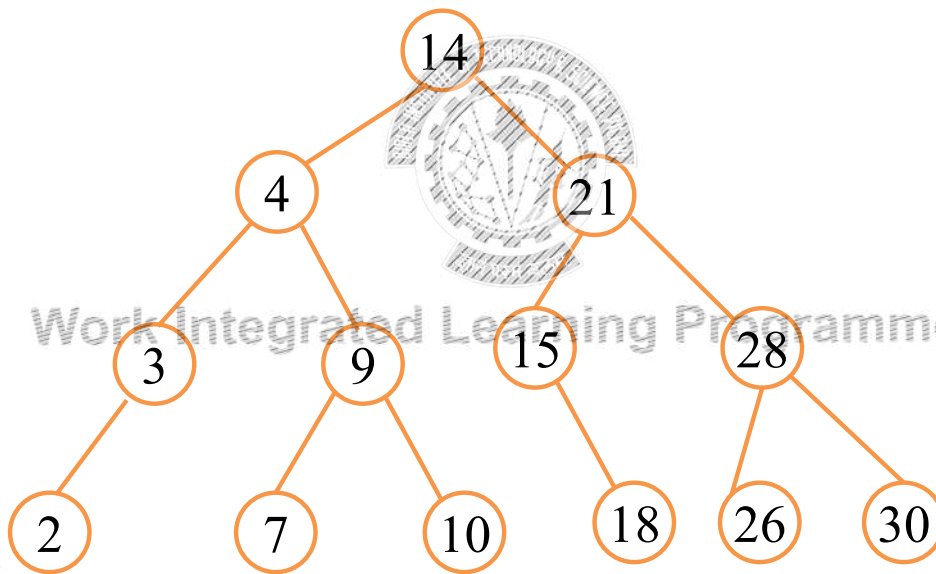
- Due to the balancing property, the insertion, deletion and search operations take $O(\log n)$ in both the average and the worst cases. Therefore, AVL trees give us an edge over Binary Search Trees which have an $O(n)$ time complexity in the worst case scenario.
- The space complexity of an AVL tree is $O(n)$ in both the average and the worst case.

Work Integrated Learning Programmes

AVL Deletion - Exercise



- Delete 2,3,10,18,4,9,14,7 and 15 in the same order from the below AVL and show all the steps !



Exercises



1. Write an algorithm to check whether a given a binary tree is a valid binary search tree (BST) or not.
2. Write an algorithm to find K^{th} smallest element in BST
3. Construct the AVL tree for : 63, 9, 19, 27, 18, 108, 99, 81
4. Construct the AVL tree for : H, I, J, B, A, E, C, F, D, G, K, L
5. Delete F, G, L from the tree constructed in (4)

Interactive Example

1. Try <https://www.cs.usfca.edu/~galles/visualization/AVLtree.html> and add some elements and remove some elements and observe rotations happen!

Exploration

Explore uses of AVL.



We will start with algorithms from the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under: [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



40

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Goa | Hyderabad



Work Integrated Learning Programmes

Contact Session #11

DSECLZG519 - Introduction to Greedy Techniques

2

Agenda for Session # 11



➤ Introduction to Greedy algorithms!

- What is Greedy technique ?
- Fractional Knapsack problem
- Minimum Spanning Tree
 - Kruskal's algorithm
 - Prim's algorithm
- Single Source Shortest Path – Dijkstra's Algorithm

➤ Exercises 😊

Greedy Algorithms



There are a lot of algorithmic types :

- Brute force algorithms
- Simple recursive algorithms
- **Greedy algorithms**
- Divide and conquer algorithms
- Dynamic programming algorithms
- Backtracking algorithms
- Branch and bound algorithms
- Randomized algorithms
- Approximation algorithms

Optimization problems



- An **optimization problem** is one in which you want to find, not just *a* solution, but the *best* solution 😊
- A “greedy algorithm” sometimes works well for optimization problems
- A **greedy algorithm** works in phases. At each phase:
 - You take the best you can get right now, without regard for future consequences
 - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

Counting Money



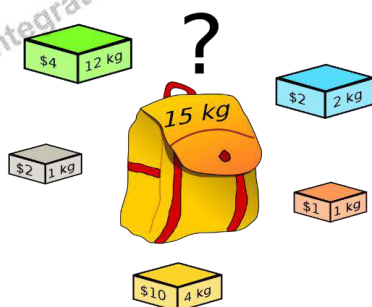
- Suppose you want to count out a certain amount of money, using the fewest possible bills and coins
- A greedy algorithm would do this would be:
At each step, take the largest possible bill or coin that does not overshoot
 - Example: To make \$6.39, you can choose:
 - a \$5 bill
 - a \$1 bill, to make \$6
 - a 25¢ coin, to make \$6.25
 - a 10¢ coin, to make \$6.35
 - four 1¢ coins, to make \$6.39
- For US money, the greedy algorithm always gives the optimum solution.
- Try for other currencies ...

A failure of the Greedy Algorithm



- In some (fictional) monetary system, “krons” come in 1 kron, 7 kron, and 10 kron coins
- Using a greedy algorithm to count out 15 krons, you would get
 - A 10 kron piece
 - Five 1 kron pieces, for a total of 15 krons
 - This requires six coins
- A better solution would be to use two 7 kron pieces and one 1 kron piece
 - This only requires three coins
- In this case, the greedy algorithm results in a feasible solution, but not in an optimal solution !!

Knapsack Problem or Container Problem



Based on the nature of the items, Knapsack problems are categorized as :

- Fractional Knapsack
- 0/1 Knapsack

Objects O	1	2	3	4	5	6	7
Profits P	10	5	15	7	6	18	3
Weights W	2	3	5	7	1	4	1

N= 7
W = 15

Constraint:

The sum of the weights of the objects we place in the knapsack must be less than Or equal to the capacity of the knapsack i.e. $\sum x_i w_i \leq m$

Objective

Maximize the profit : $\sum X_i P_i$

Knapsack Problem Example



Objects O	1	2	3	4	5	6	7
Profits P	10	5	15	7	6	18	3
Weights W	2	3	5	7	1	4	1

N = 7
W = 15

P/W	5	1.3	3	1	6	4.5	3
-----	---	-----	---	---	---	-----	---

x	1				1	1	
---	---	--	--	--	---	---	--

- **Step 1:** Select Object whose P/W is highest. Here, O5 is selected and put it fully in Knapsack. Now the Available capacity in Knapsack is $15 - 1 = 14\text{Kg}$
- **Step 2:** Select the next Object whose P/W is highest. Here, O1 is selected and put it fully in Knapsack. Now the Available capacity in Knapsack is $14 - 2 = 12\text{Kg}$
- **Step 3:** Select the next Object whose P/W is highest. Here, O6 is selected and put it fully in Knapsack. Now the Available capacity in Knapsack is $12 - 4 = 8\text{Kg}$

Knapsack Problem Example



Objects O	1	2	3	4	5	6	7
Profits P	10	5	15	7	6	18	3
Weights W	2	3	5	7	1	4	1
P/W	5	1.3	3	1	6	4.5	3
x	1	2/3	1	0	1	1	1

N= 7
W = 15

- **Step 4:** Select Object whose P/W is highest. Here, O3 and O7 both yield the same. We select O3 put it fully in Knapsack. Now the Available capacity in Knapsack is $8 - 5 = 3\text{Kg}$
- **Step 5:** Select the next Object whose P/W is highest. Here, O7 is selected and put it fully in Knapsack. Now the Available capacity in Knapsack is $3 - 1 = 2\text{Kg}$
- **Step 6:** Select the next Object whose P/W is highest. Here, O2 is selected but we cannot place it fully in the knapsack as the capacity available in knapsack is just 2Kg. Hence, We take 2/3 of O2. Knapsack is now full!

10

Knapsack Problem Example



Objects O	1	2	3	4	5	6	7
Profits P	10	5	15	7	6	18	3
Weights W	2	3	5	7	1	4	1
P/W	5	1.3	3	1	6	4.5	3
x	1	2/3	1	0	1	1	1

N= 7
W = 15

- $\sum X_i W_i = (1 * 2) + \left(\frac{2}{3} * 3\right) + (1 * 5) + (0 * 7) + (1 * 1) + (1 * 4) + (1 * 1)$
 $= 2 + 2 + 5 + 0 + 1 + 4 + 1 = 15$, so the constraint $\sum x_i w_i \leq m$ is satisfied,
 where m is the capacity of the knapsack.
- $\sum X_i P_i = (1 * 10) + \left(\frac{2}{3} * 5\right) + (1 * 15) + (0 * 7) + (1 * 6) + (1 * 18) + (1 * 3)$
 $= 10 + 3.33 + 15 + 0 + 6 + 18 + 3 = 55.33$, so the objective was to maximize
 $\sum X_i P_i$ which is also done. Maximum Profit is **55.33**



Job Sequencing with Deadlines

- Problem: Given an array of jobs/tasks where every job has a deadline associated with it. If the job is finished before/on the deadline, there is a profit associated with it. It is also given that every job takes a single unit of time, so the minimum possible deadline for any job is 1. How to maximize total profit if only one job can be scheduled at a time?

N=5

Jobs ID	1	2	3	4	5
Profit	20	15	10	5	1
Deadline	2	2	1	3	3

8AM — 9AM — 10AM — 11AM

12



Job Sequencing with Deadlines

N=5

Jobs ID	1	2	3	4	5
Profit	20	15	10	5	1
Deadline	2	2	1	3	3

8AM (S0) — 9AM (S1) — 10AM (S2) — 11AM (S3)

Considered	Slot Assigned	Solution	Profit
-	-	Null	0
J1	[1-2]	J1	20
J2	[0-1],[1-2]	J1,J2	20+15 = 35
J3	J3 Rejected (as deadline can't be meet)	J1,J2	35
J4	[0-1],[1-2],[2-3]	J1,J2,J4	35+5 = 40
J5	J5 Rejected (as deadline can't be meet)	J1,J2,J4	40

13

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Exercises



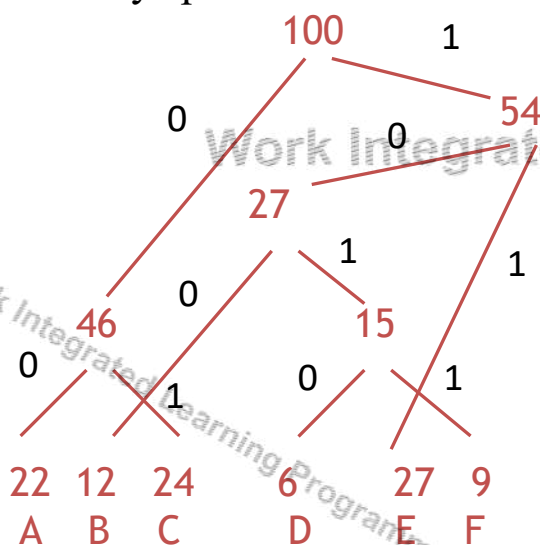
1. For the fractional knapsack problem, $n=3, m=20$, $(p_1, p_2, p_3) = (25, 24, 15)$ and $(w_1, w_2, w_3) = (18, 15, 10)$, find the feasible and optimal solution.
2. Consider a fractional knapsack problem of finding the optimal solution, where $m=15$, $(p_1, p_2, \dots, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and $(w_1, w_2, \dots, w_7) = (2, 3, 5, 7, 1, 4, 1)$.
3. Consider the problem of Job sequencing using Greedy method. The number of jobs are 5. Deadlines are $\{d_1, d_2, d_3, d_4, d_5\} = \{1, 2, 4, 1, 3\}$ and profits are $\{p_1, p_2, p_3, p_4, p_5\} = \{1, 5, 20, 15, 10\}$
4. Solve the job sequencing problem where there are 4 jobs. Profits are $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$ and deadlines are $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Exploration : Applications of Greedy algorithm other than said ones!

Huffman Encoding [Webinar 3]



- A *Huffman code* is a particular type of optimal prefix *code* that is commonly used for lossless data compression. Huffman developed a nice greedy algorithm for solving this problem and producing a minimum-cost(optimum) prefix code. The code that it produces is called a Huffman-code.
- The Huffman encoding algorithm is a greedy algorithm
- You always pick the two smallest numbers to combine



A=00
B=100
C=01
D=1010
E=11
F=1011

- Average bits/char:

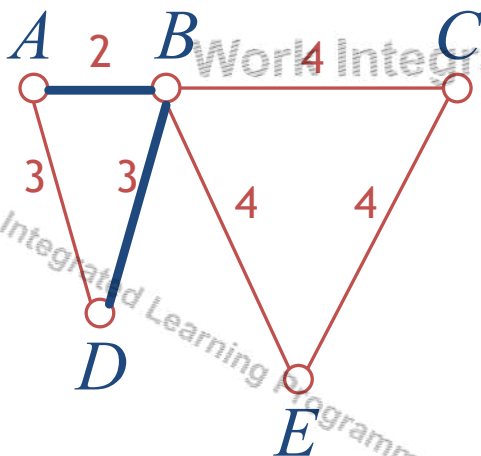
$$0.22*2 + 0.12*3 + 0.24*2 + 0.06*4 + 0.27*2 + 0.09*4 = 2.42$$

- The Huffman algorithm finds an optimal solution

Traveling Salesman



- A salesman must visit every city (starting from city **A**), and wants to cover the least possible distance
 - He can revisit a city (and reuse a road) if necessary
- He does this by using a greedy algorithm: He goes to the next nearest city from wherever he is



- From **A** he goes to **B**
- From **B** he goes to **D**
- This is *not* going to result in a shortest path!
- The best result he can get now will be **ABDBCE**, at a cost of **16**
- An actual least-cost path from **A** is **ADBCE**, at a cost of **14**



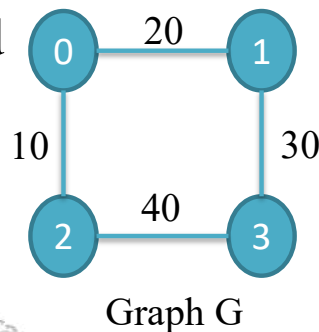
Spanning Tree

A spanning tree is a subgraph of a graph such that the subgraph spans (includes) all nodes, is connected and is acyclic (does not contain cycle/loop/circuit). Formally, a spanning tree of graph G is defined as a sub graph $G' = (V', E')$ with the following properties :

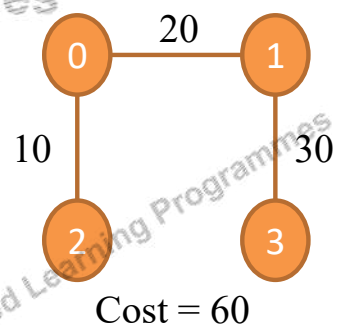
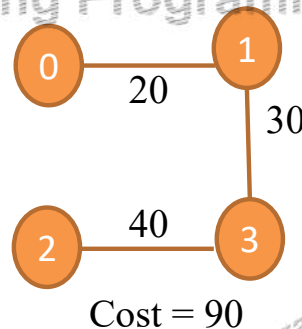
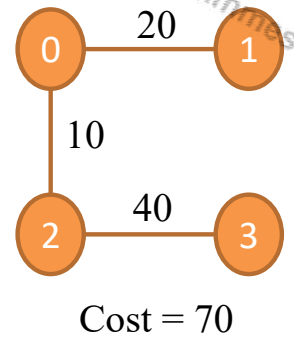
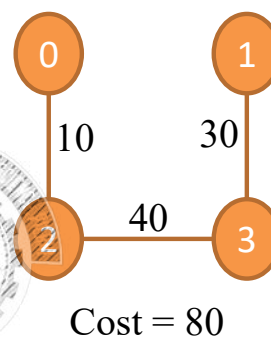
✓ G' is connected

✓ $V' = V$

✓ G' is acyclic



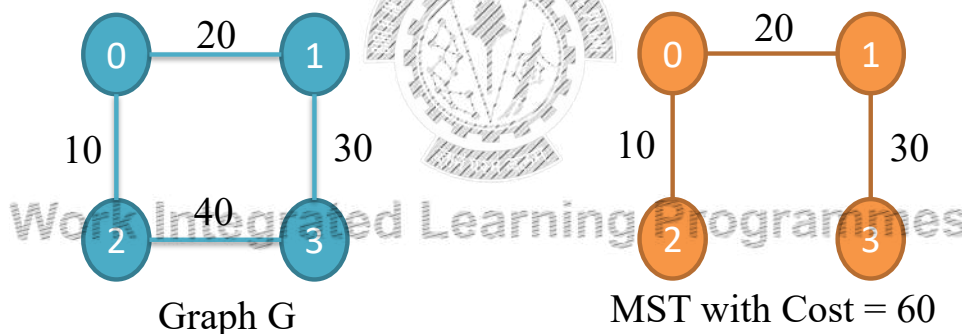
Spanning Trees





Minimum Spanning Tree (MST)

- A minimum spanning tree of a given graph G is a spanning tree whose cost is minimum.
- For the previous Graph G , the minimum spanning tree is

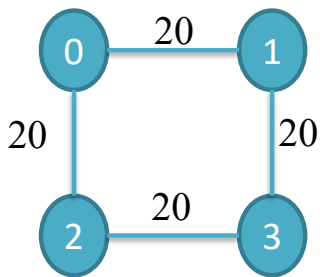


- A graph will have only one MST iff the weights associated with all the edges in the graph are distinct. Example for above Graph G , there is only one MST with cost 60.

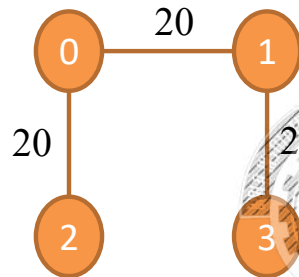


Minimum Spanning Tree (MST)

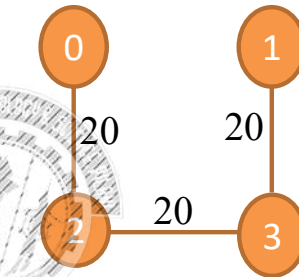
➤ Consider the below Graph G,



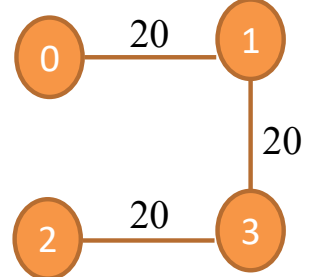
Graph G



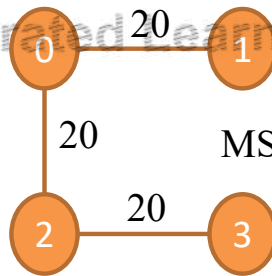
MST with Cost = 60



MST with Cost = 60



MST with Cost = 60



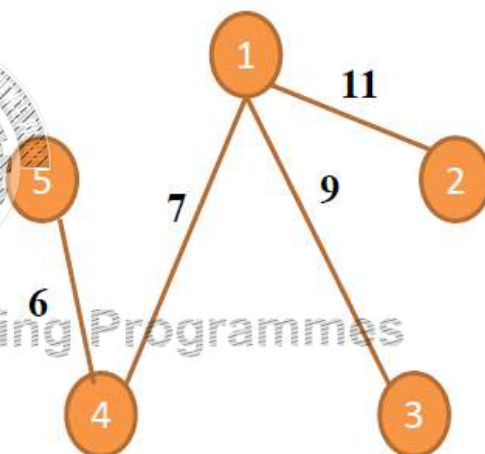
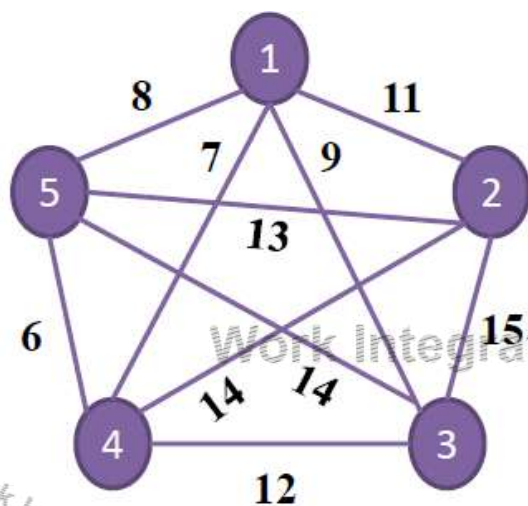
MST with Cost = 60

➤ Here, since the weights are not distinct, we have more than 1 MST.

Classroom Exercise [3 min]



Draw the MST for the below Graph G and also provide its cost :



MST with Cost = 6+7+9+11 = 33 units

Minimum Spanning Tree



There are plenty of algorithms for finding MST like Kruskal, Prim-Jarnik, Borůvka's, Edmonds' and so on.

We will look at two most popular greedy algorithms to find the MST:

➤ **Kruskal's Algorithm:**

Consider edges in ascending order of cost. Add the next edge to T unless doing so would create a cycle.

➤ **Prim's Algorithm:**

Start with any vertex S and greedily grow a tree T from S. At each step, add the cheapest edge to T that has exactly one endpoint in T.

- Both the algorithms produce optimal solutions and are greedy techniques.
- Both the algorithms *work with undirected graphs only !!* (We will see why these algorithms cannot be used for directed graphs).

Kruskal's Algorithm

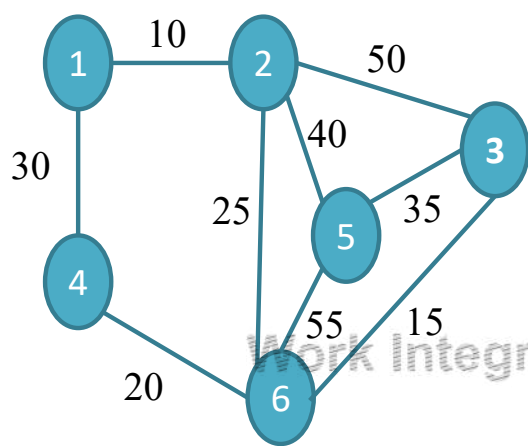


- Kruskal's algorithm is a greedy algorithm used to obtain MST using a direct generic method. Here, all the edges are considered in non-decreasing (increasing) order and the edges to be included are based on the order.
- Procedure for Kruskal's approach to MST:
 - Step 01: Construct an edge list E in the increasing order of their costs.
 - Step 02: Pick one edge at a time from the list starting from the least cost edge.
 - Step 03: Check if the inclusion of the edge causes a cycle or a loop in the output.
 - Step 04: If it causes a cycle, discard it. If it does not generate a cycle, include it into the tree.
 - Step 05: Repeat the steps 2 – 4 until a tree is generated with all nodes of the graph.

Kruskal's Example 1



Edge List E in Increasing Order

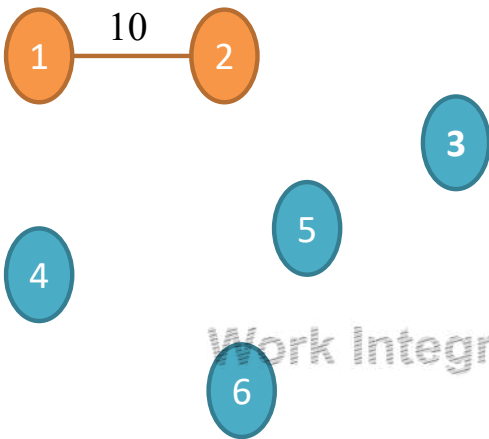


Edge	Cost
1 – 2	10
3 – 6	15
4 – 6	20
2 – 6	25
1 – 4	30
3 – 5	35
2 – 5	40
2 – 3	50
5 – 6	55

Kruskal's Example 1



Kruskal in Action

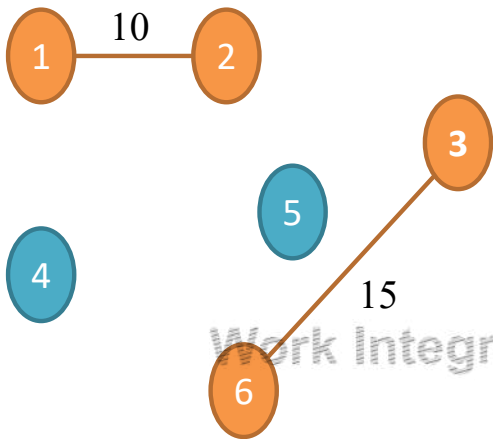


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15		
4 – 6	20		
2 – 6	25		
1 – 4	30		
3 – 5	35		
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Kruskal in Action

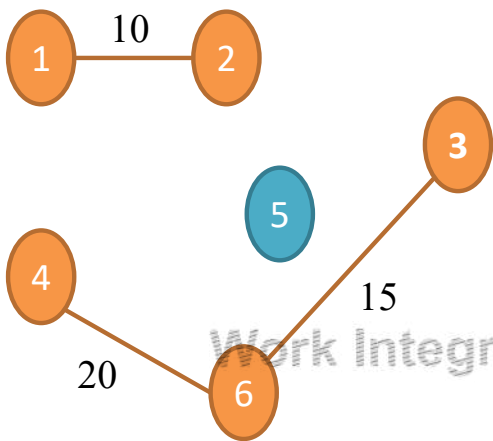


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20		
2 – 6	25		
1 – 4	30		
3 – 5	35		
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Kruskal in Action

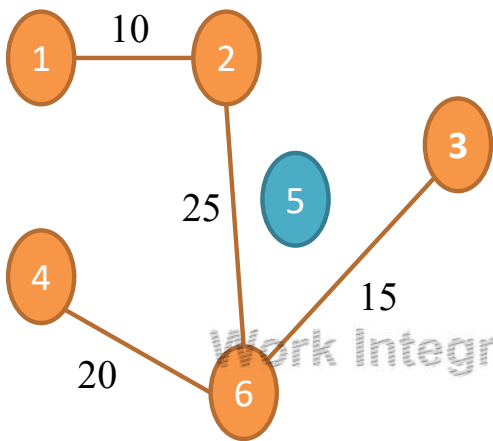


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20	Yes	✓
2 – 6	25		
1 – 4	30		
3 – 5	35		
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Kruskal in Action

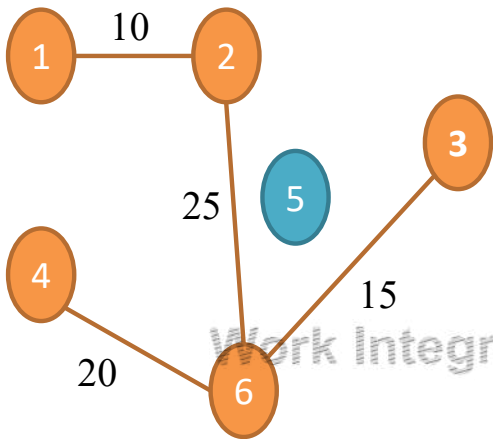


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20	Yes	✓
2 – 6	25	Yes	✓
1 – 4	30		
3 – 5	35		
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Kruskal in Action

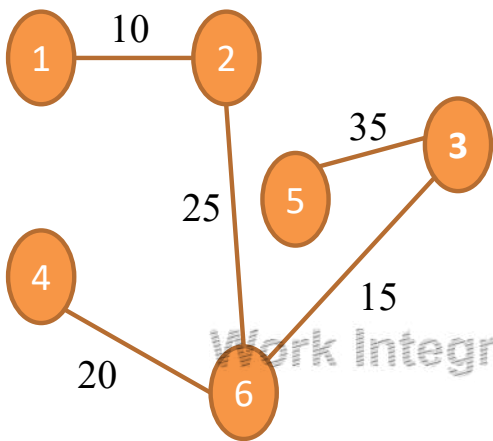


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20	Yes	✓
2 – 6	25	Yes	✓
1 – 4	30	No – Leads to Cycle	✗
3 – 5	35		
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Kruskal in Action

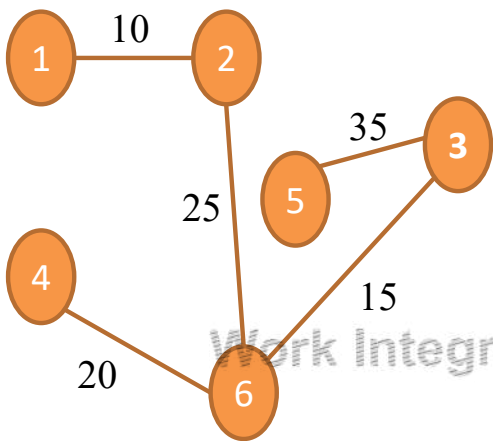


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20	Yes	✓
2 – 6	25	Yes	✓
1 – 4	30	No – Leads to Cycle	✗
3 – 5	35	Yes	✓
2 – 5	40		
2 – 3	50		
5 – 6	55		

Kruskal's Example 1

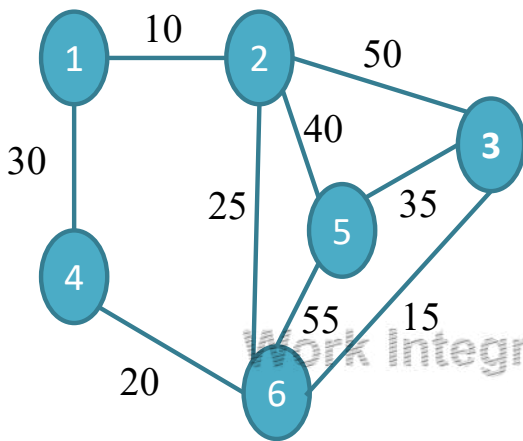


Kruskal in Action

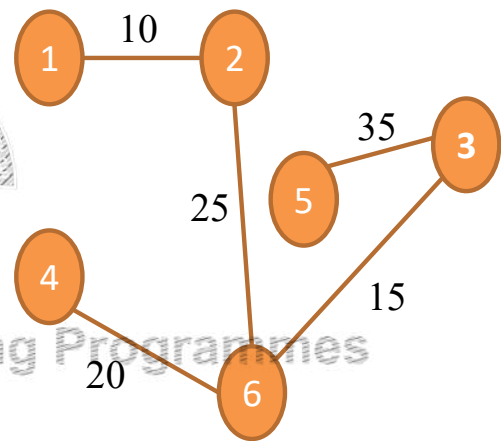


Edge	Cost	Include ?	Status
1 – 2	10	Yes	✓
3 – 6	15	Yes	✓
4 – 6	20	Yes	✓
2 – 6	25	Yes	✓
1 – 4	30	No – Leads to Cycle	✗
3 – 5	35	Yes	✓
2 – 5	40	Not Considered as already all Nodes are covered in MST	
2 – 3	50		
5 – 6	55		

Kruskal's Example 1



Graph G

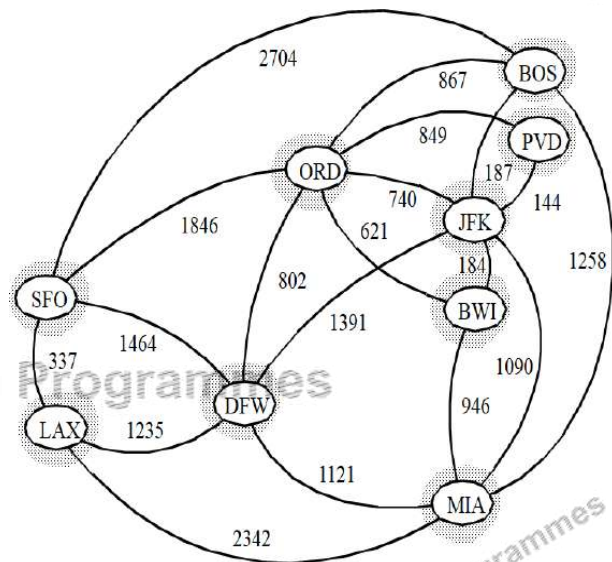


MST with Cost = 10+15+20+25+35 = 105 units

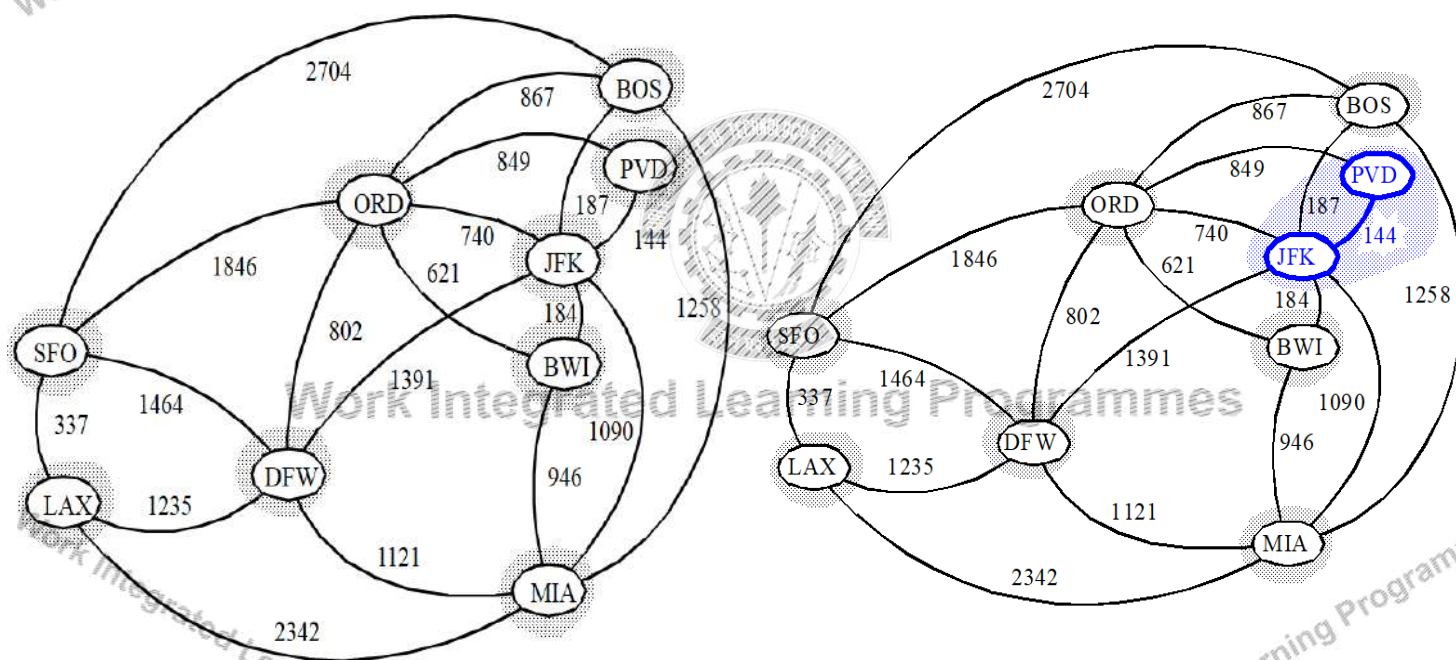
Kruskal's Example 2



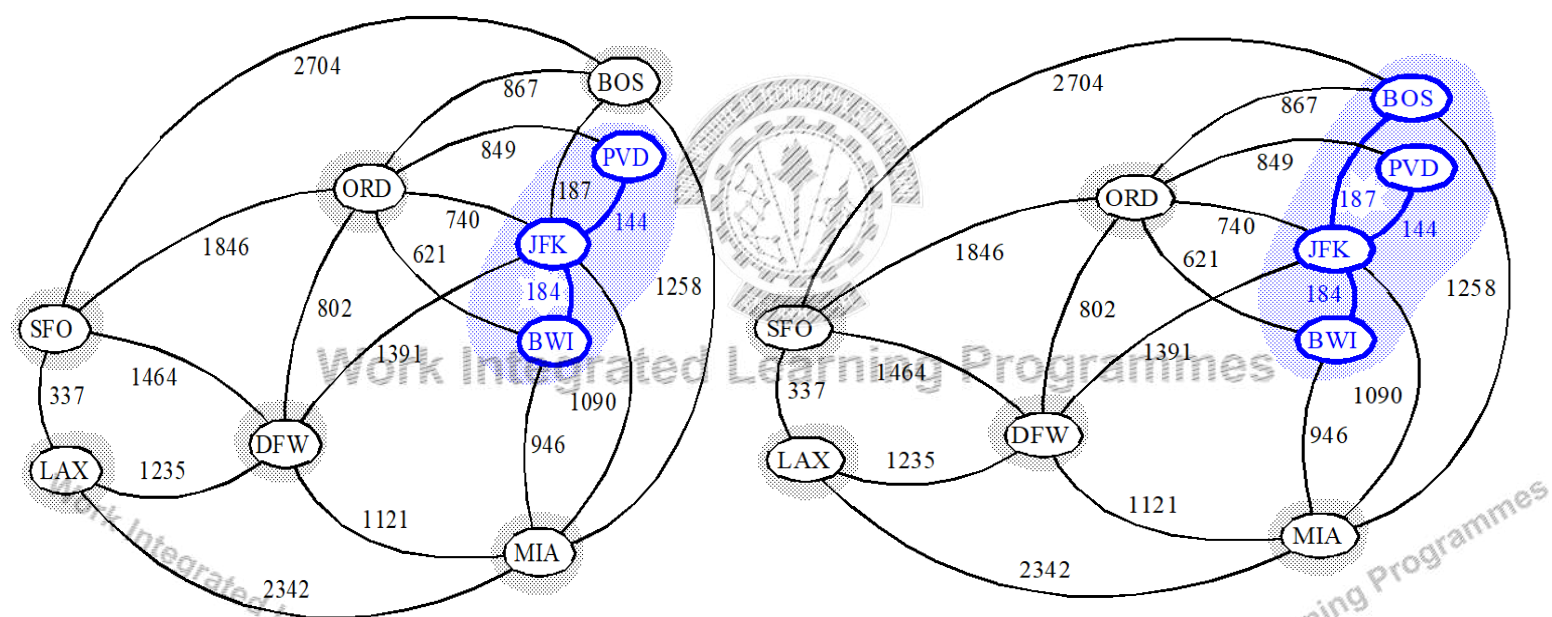
- Kruskal's algorithm is used to build the MST in *clusters*. Initially, each vertex is in its own cluster all by itself.
- The algorithm then considers each edge in turn, ordered by increasing weight.
- If an edge e connects two different clusters, then e is added to the set of edges of the MST, and the two clusters connected by e are merged into a single cluster!
- If, on the other hand, e connects two vertices that are already in same cluster, then e is discarded.
- Once the algorithm has added enough edges to form a spanning tree, it terminates and outputs this tree as the MST.



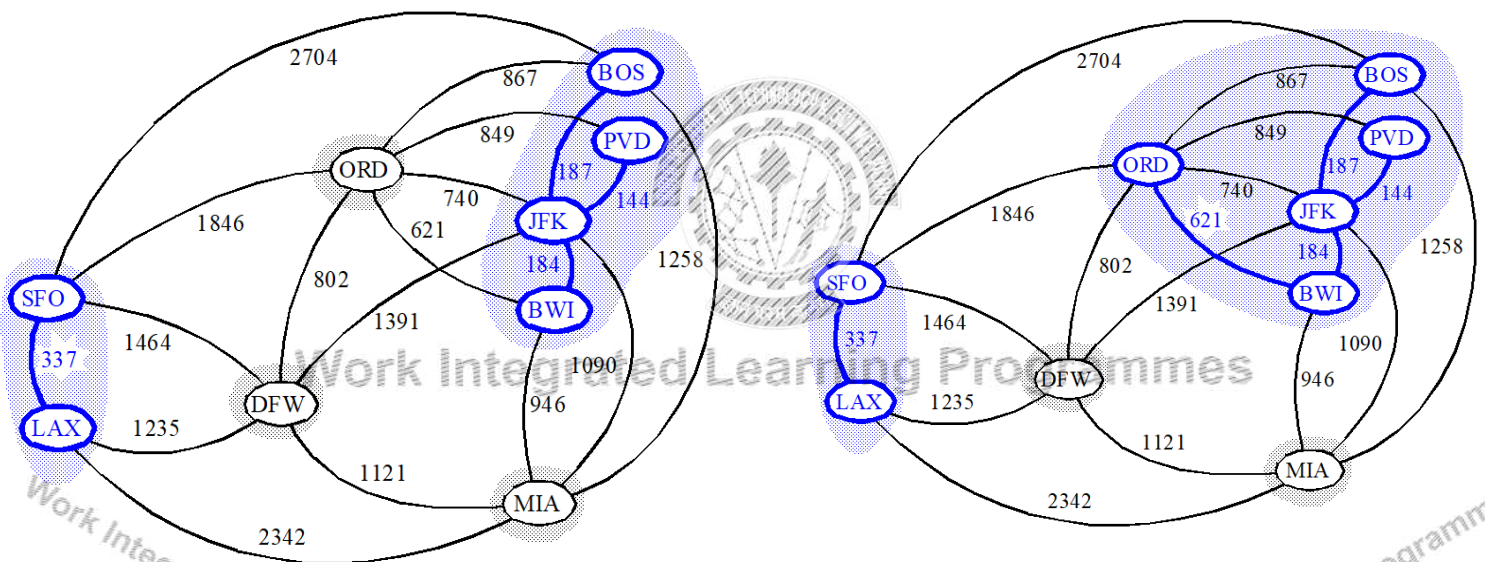
Kruskal's Example 2



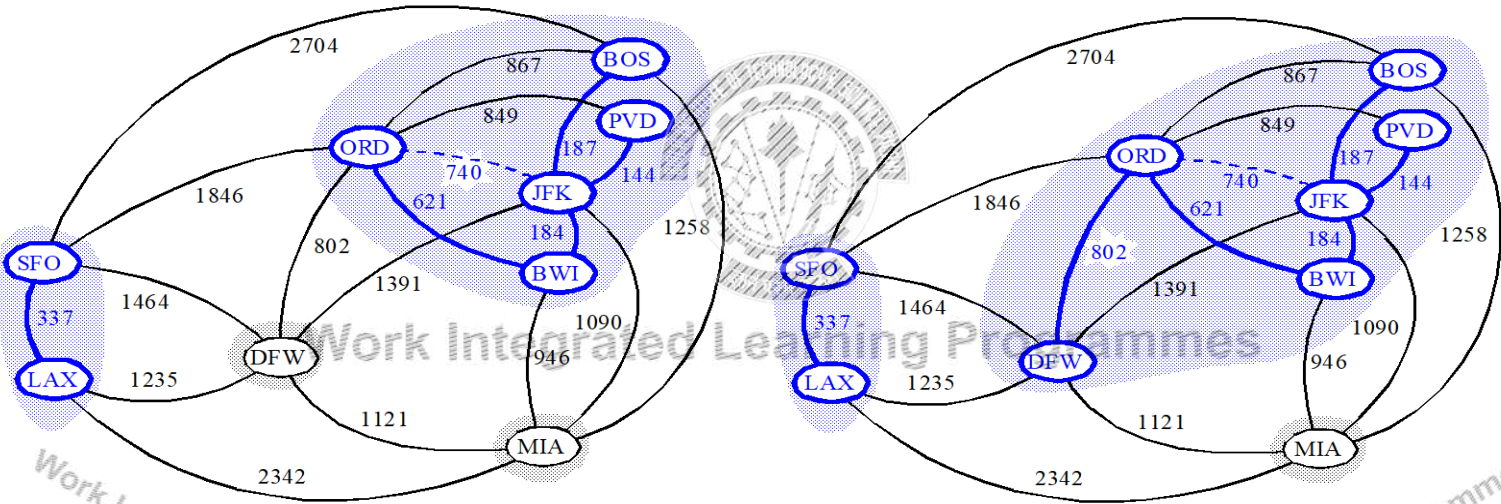
Kruskal's Example 2



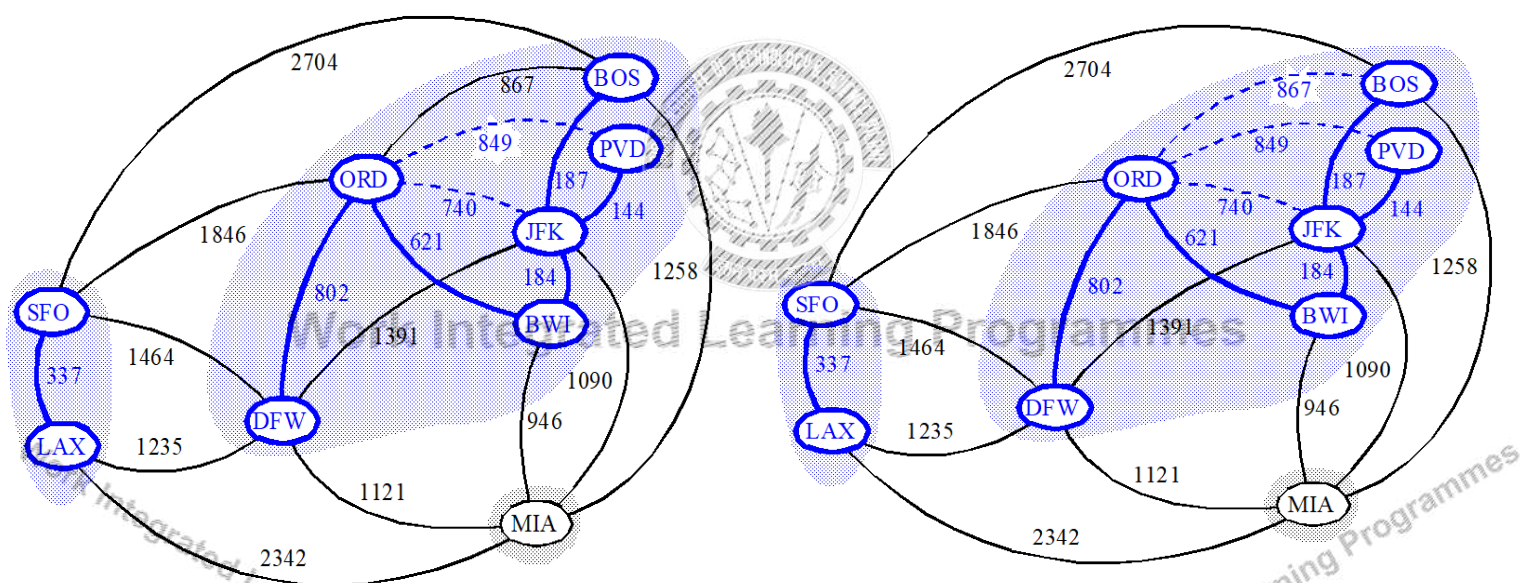
Kruskal's Example 2



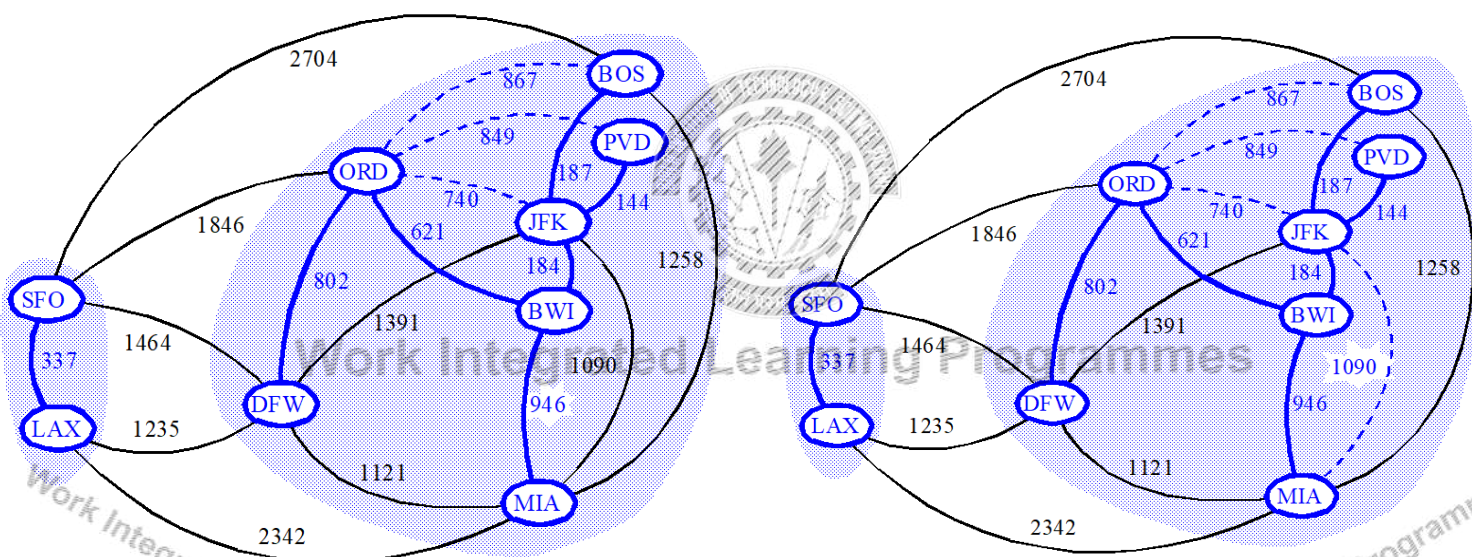
Kruskal's Example 2



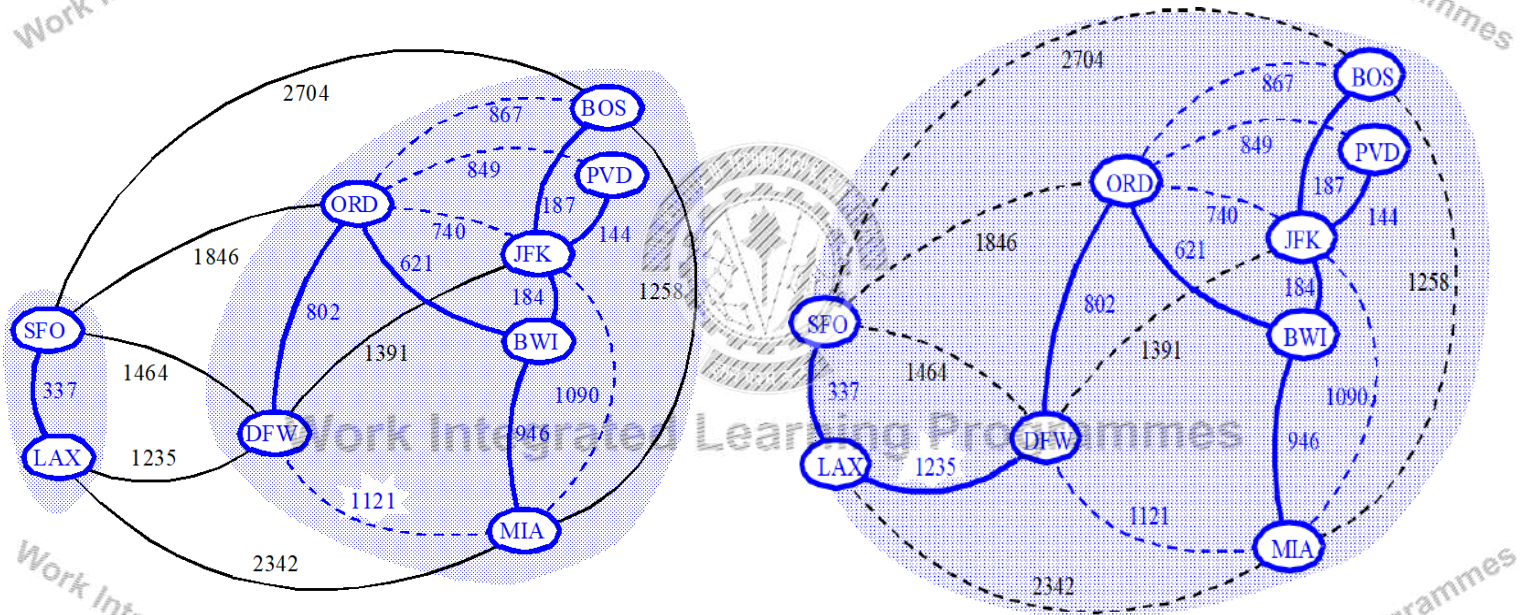
Kruskal's Example 2



Kruskal's Example 2



Kruskal's Example 2



$$\begin{aligned} \text{MST Cost} &= 144 + 184 + 187 + 337 + 621 + 802 + 946 + 1235 \\ &= 4456 \text{ units} \end{aligned}$$

Kruskal's Algorithm



Algorithm KruskalMST(G):

Input: A simple connected weighted graph G with n vertices and m edges

Output: A minimum spanning tree T for G

for each vertex v in G **do**

 Define an elementary cluster $C(v) \leftarrow \{v\}$.

Let Q be a priority queue storing the edges in G , using edge weights as keys

$T \leftarrow \emptyset$ // T will ultimately contain the edges of the MST

while T has fewer than $n - 1$ edges **do**

$(u, v) \leftarrow Q.\text{removeMin}()$

 Let $C(v)$ be the cluster containing v

 Let $C(u)$ be the cluster containing u

if $C(v) \neq C(u)$ **then**

 Add edge (v, u) to T

 Merge $C(v)$ and $C(u)$ into one cluster, that is, union $C(v)$ and $C(u)$

return tree T

$O(m \log n)$

$O(n \log n)$

Explore!!

Total: $O(m+n \log n) = O(m \log n)$ 40

Edge Relaxation



- Generally, we can solve all the shortest path problems by using edge relaxation. The *edge relaxation* is the operation to calculate the reaching cost to the vertex with the aim to lower the cost.

- Consider the below example:

- Here, initially the cost from $1 \rightarrow 2$ is 2 and $1 \rightarrow 3$ is ∞



- Once 2 is visited (added into source), the cost from $1 \rightarrow 3$ is reduced to 5 ($2+3$) from ∞ . This process is called edge relaxation! More formally,

For the edge from the vertex u to the vertex v , if $d[u] + w(u, v) < d[v]$ is satisfied, update $d[v]$ to $d[u] + w(u, v)$

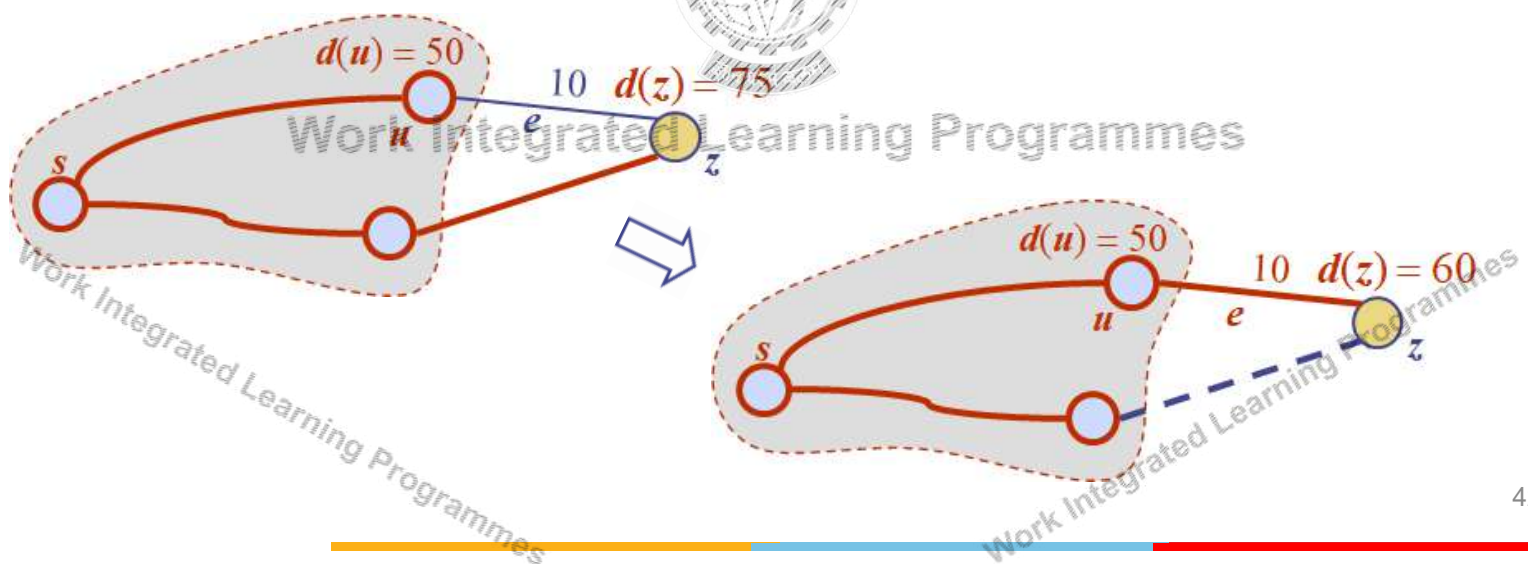
If $(d[u] + c(u, v) < d[v])$
then $d[v] = d[u] + c(u, v)$

Edge Relaxation



- Consider an edge $e = (u, z)$ such that
 - u is the vertex most recently added to the cloud
 - z is not in the cloud
- The relaxation of edge e updates distance $d(z)$ as follows:

$$d(z) \leftarrow \min \{d(z), d(u) + \text{weight}(e)\}$$





Work Integrated Learning Programmes

We will Greedily learn Greedy in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



44

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Contact Session #12

DSECLZG519 – Greedy & Introduction to Divide & Conquer

2

Agenda for Session # 12



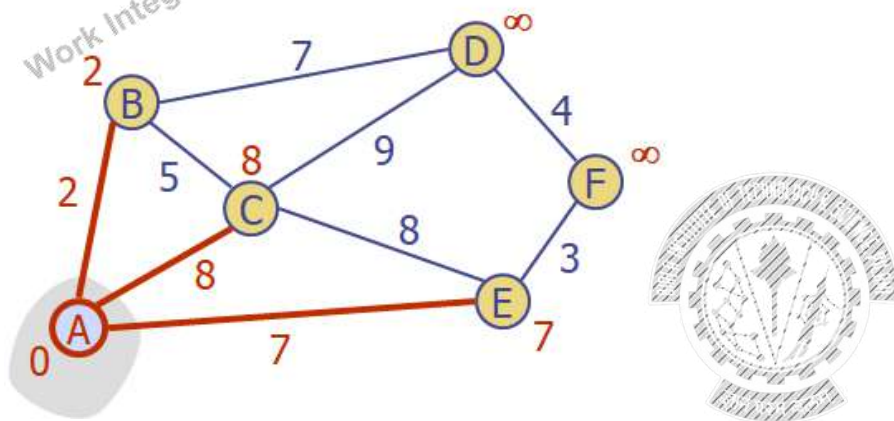
- Recap of CS#11
- Prim's algorithm for MST
- Single source shortest path
- Divide & Conquer
 - Merge Sort
 - Quick Sort
 - Integer Multiplication Problem (Karatsuba)
- Exercises

Prim's Algorithm / Prim-Jarnik Algorithm



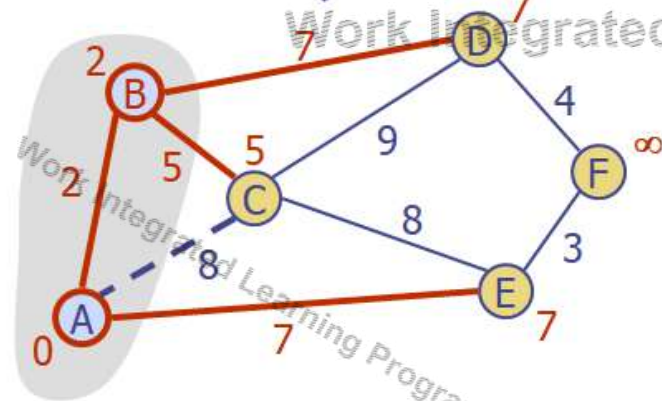
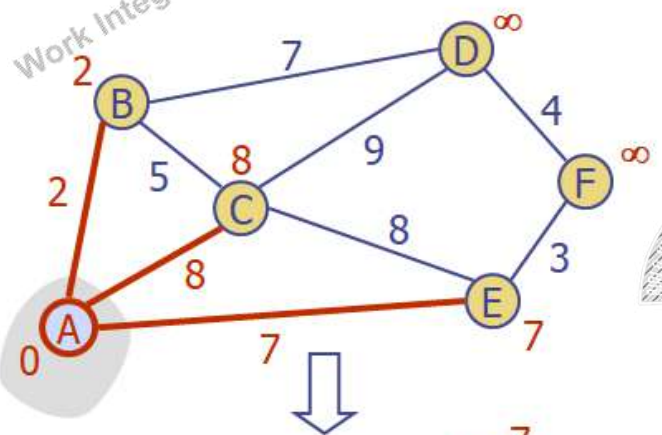
- Prim's algorithm is a greedy strategy used to obtain the MST from a graph edge by edge. The edge to be included is chosen according to the optimization criteria i.e., to choose an edge that results in a minimum increase in the sum of costs of the edges included so far.
- If A is the set of edges selected so far, then A forms a tree, the next edge (u, v) to be included in A is a minimum cost edge not in A , with the property that $A \cup \{(u, v)\}$ is also a tree.
- Steps in Prim's Algorithm:
 - Step 01: Select any vertex (or alphabetically or given vertex) as the source.
 - Step 02: Compute the distances from the source to each other vertex of the graph.
 - Step 03: Choose the vertex which leads to a minimum increase in the sum of costs of the edges included. Add the vertex to the tree iff the addition does not lead to a cycle.
 - Step 04: Repeat the steps 2 and 3 until a tree is generated with all the vertices of the graph.

Prim's Algorithm Example

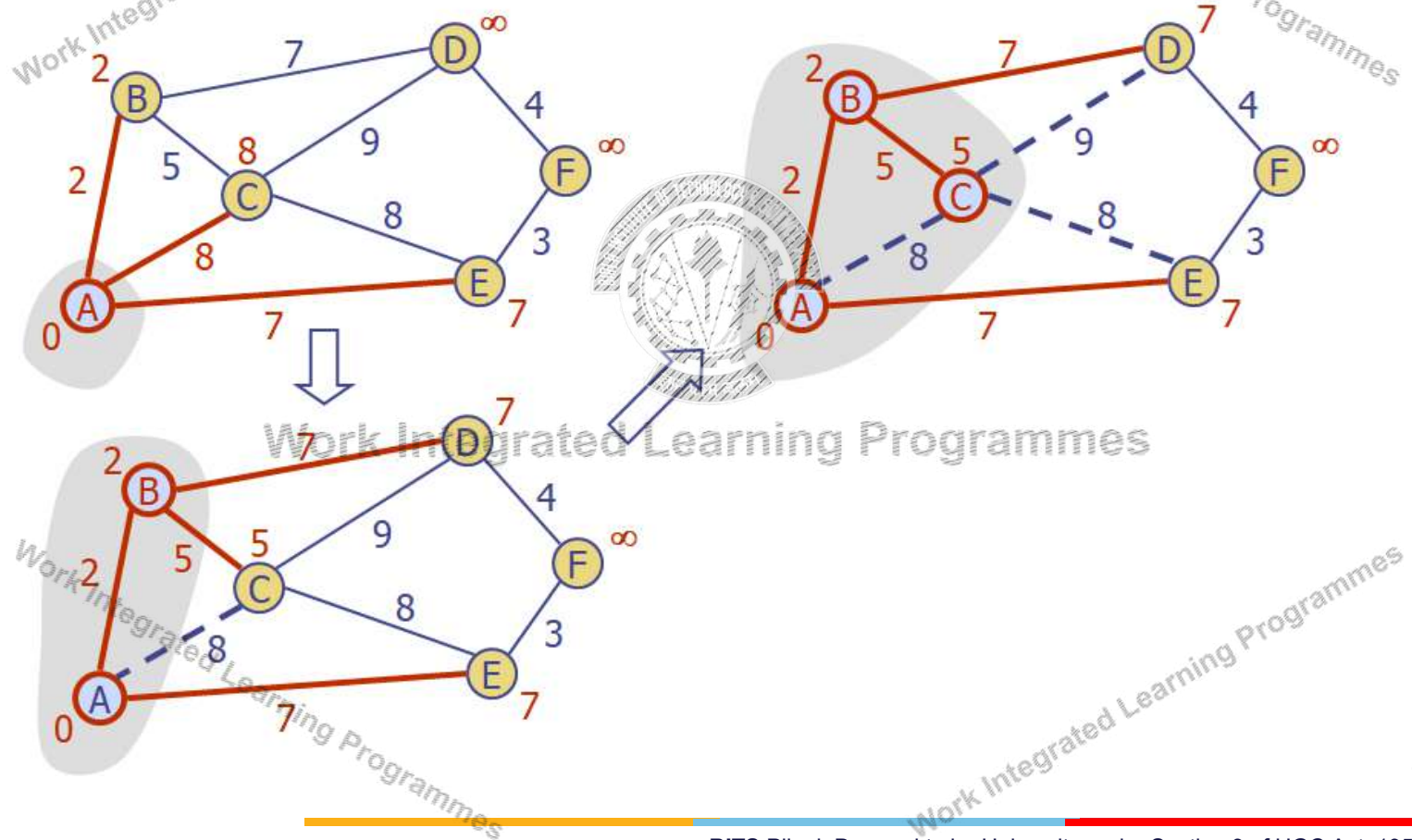


Work Integrated Learning Programmes

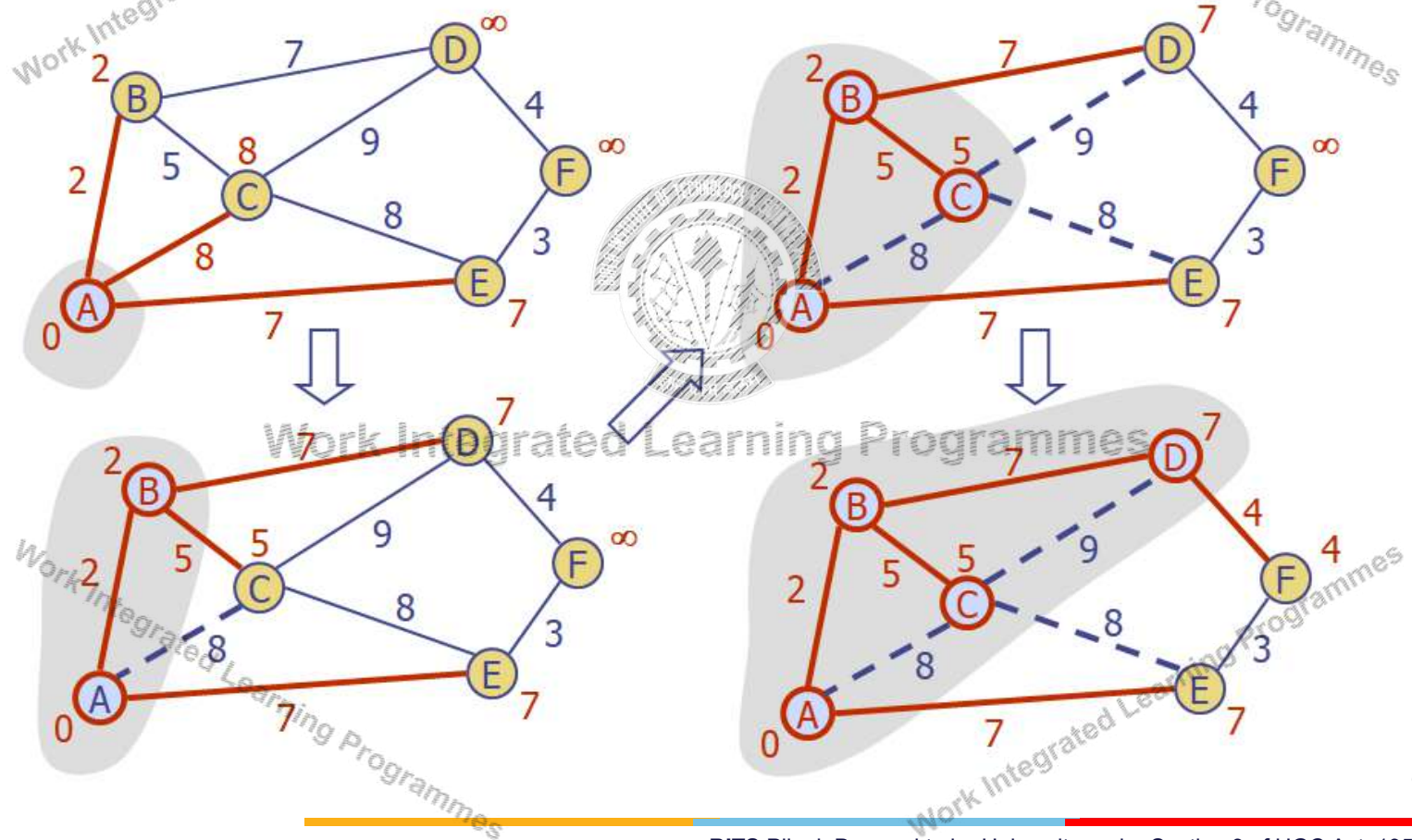
Prim's Algorithm Example



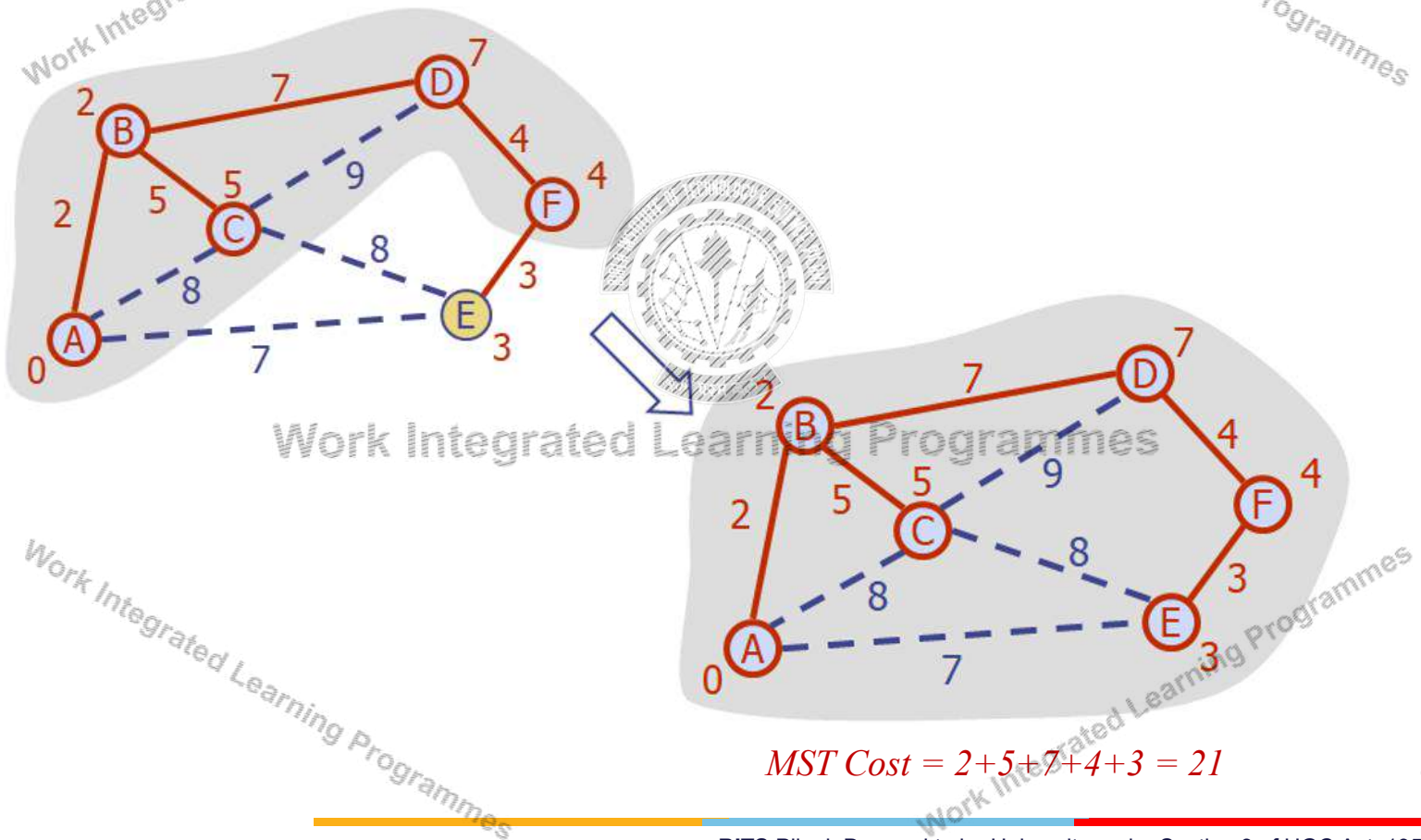
Prim's Algorithm Example



Prim's Algorithm Example



Prim's Algorithm Example 2



$$MST\ Cost = 2 + 5 + 7 + 4 + 3 = 21$$

9

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Prim's / PrimJarnik Algorithm



Algorithm PrimJarnikMST(G):

Input: A weighted connected graph G with n vertices and m edges

Output: A minimum spanning tree T for G

Pick any vertex v of G

$D[v] \leftarrow 0$

for each vertex $u \neq v$ **do**

$D[u] \leftarrow +\infty$

Initialize $T \leftarrow \emptyset$.

Initialize a priority queue Q with an item $((u, \text{null}), D[u])$ for each vertex u , where (u, null) is the element and $D[u]$ is the key.

while Q is not empty **do**

$(u, e) \leftarrow Q.\text{removeMin}()$

 Add vertex u and edge e to T .

for each vertex z adjacent to u such that z is in Q **do**

 // perform the relaxation procedure on edge (u, z)

if $w((u, z)) < D[z]$ **then**

$D[z] \leftarrow w((u, z))$

 Change to $(z, (u, z))$ the element of vertex z in Q .

 Change to $D[z]$ the key of vertex z in Q .

return the tree T

Complexity: $O(m \log n)$

Kruskal vs Prim's Algorithm



Prims Algorithm	Kruskal Algorithm
It start to build the MST from any of the Node.	It start to build the MST from Minimum weighted vertex in the graph.
Adjencary Matrix , Binary Heap or Fibonacci Heap is used in Prims algorithm	Disjoint Set is used in Kruskal Algorithm.
Prims Algorithm run faster in dense graphs	Kruskal Algorithm run faster in sparse graphs
Time Complexity is $O(EV \log V)$ with binay heap and $O(E+V \log V)$ with fibonacci heap.	Time Complexity is $O(E \log V)$
The next Node included must be connected with the node we traverse	The next edge include may or may not be connected but should not form the cycle.
It traverse the one node several time in order to get it minimum distance	It travese the edge only once and based on cycle it will either reject it or accept it,
Greedy Algorithm	Greedy Algorithm

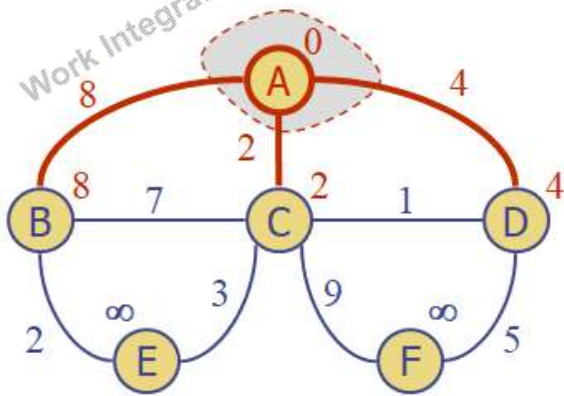
Single Source Shortest Path



- The aim of the single source shortest path problem is to find the shortest path between a selected source and every other vertex in the graph. To solve this problem, Edsger-w-Dijkstra has devised an amazing algorithm in the 1950's which is still popular and widely used.
- The problem can be stated as “For a given weighted graph G with vertices, a shortest path has to be computed from a given vertex v (called source) to all the other vertices”.
- The shortest path computed can either be a direct path or an indirect path.
- To formulate a greedy paradigm solution for this problem, a multistage solution which is optimal in every stage has to be devised.
- First the shortest path from the starting vertex to the nearest vertex has to be computed, then the next shortest and so on.

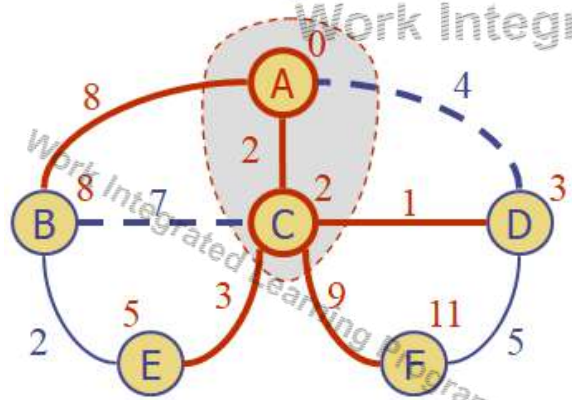
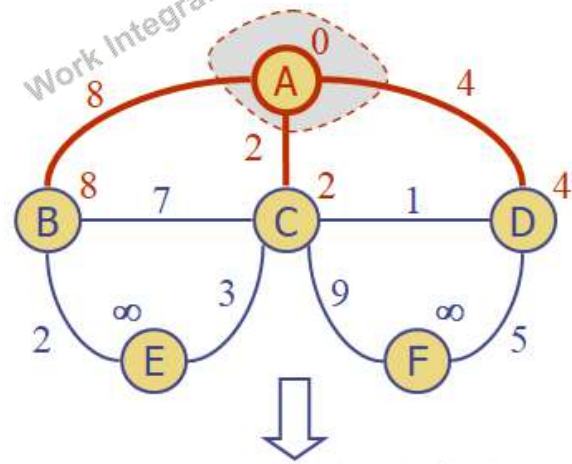
12

Dijkstra's Algorithm Example 2

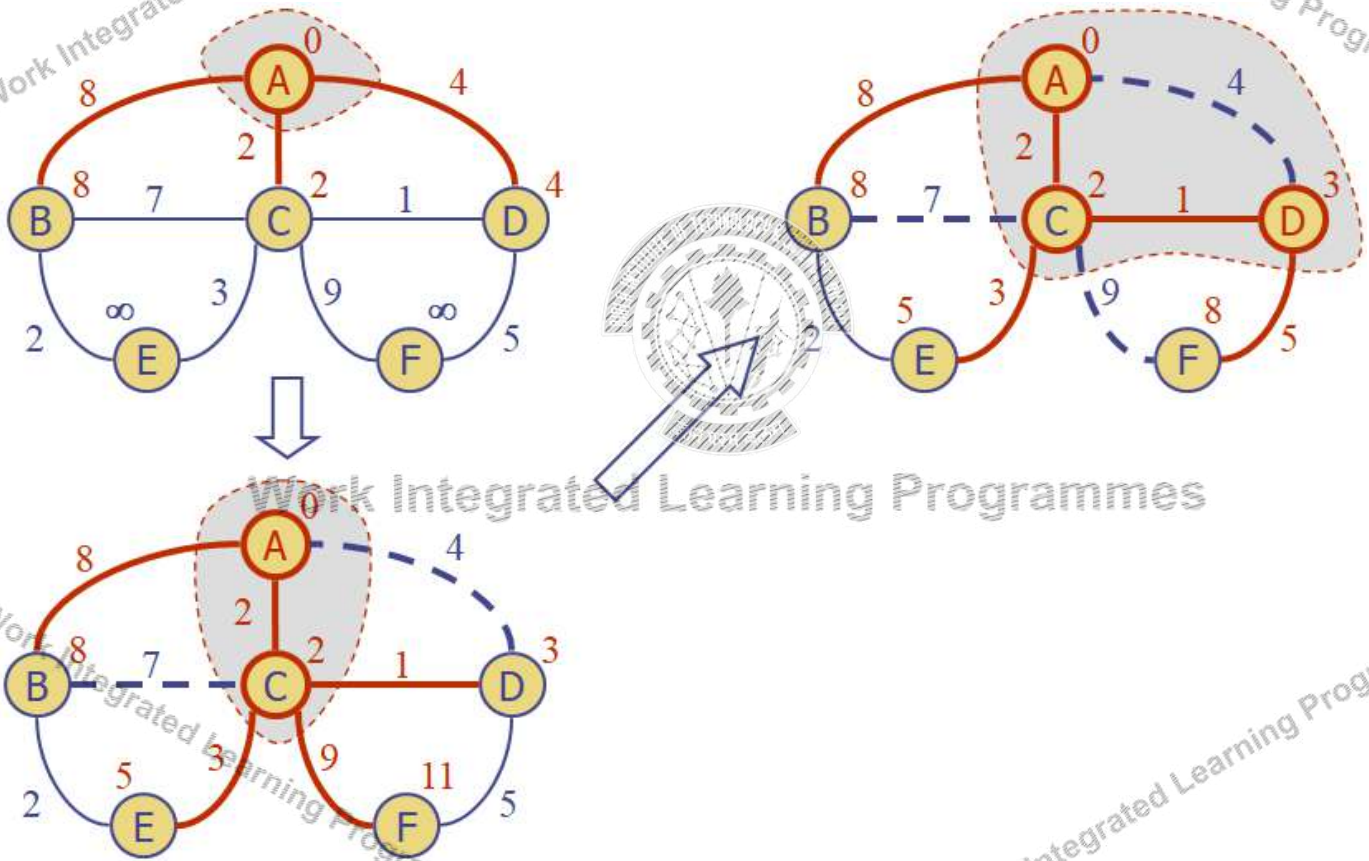


Work Integrated Learning Programmes

Dijkstra's Algorithm Example 2



Dijkstra's Algorithm Example 2

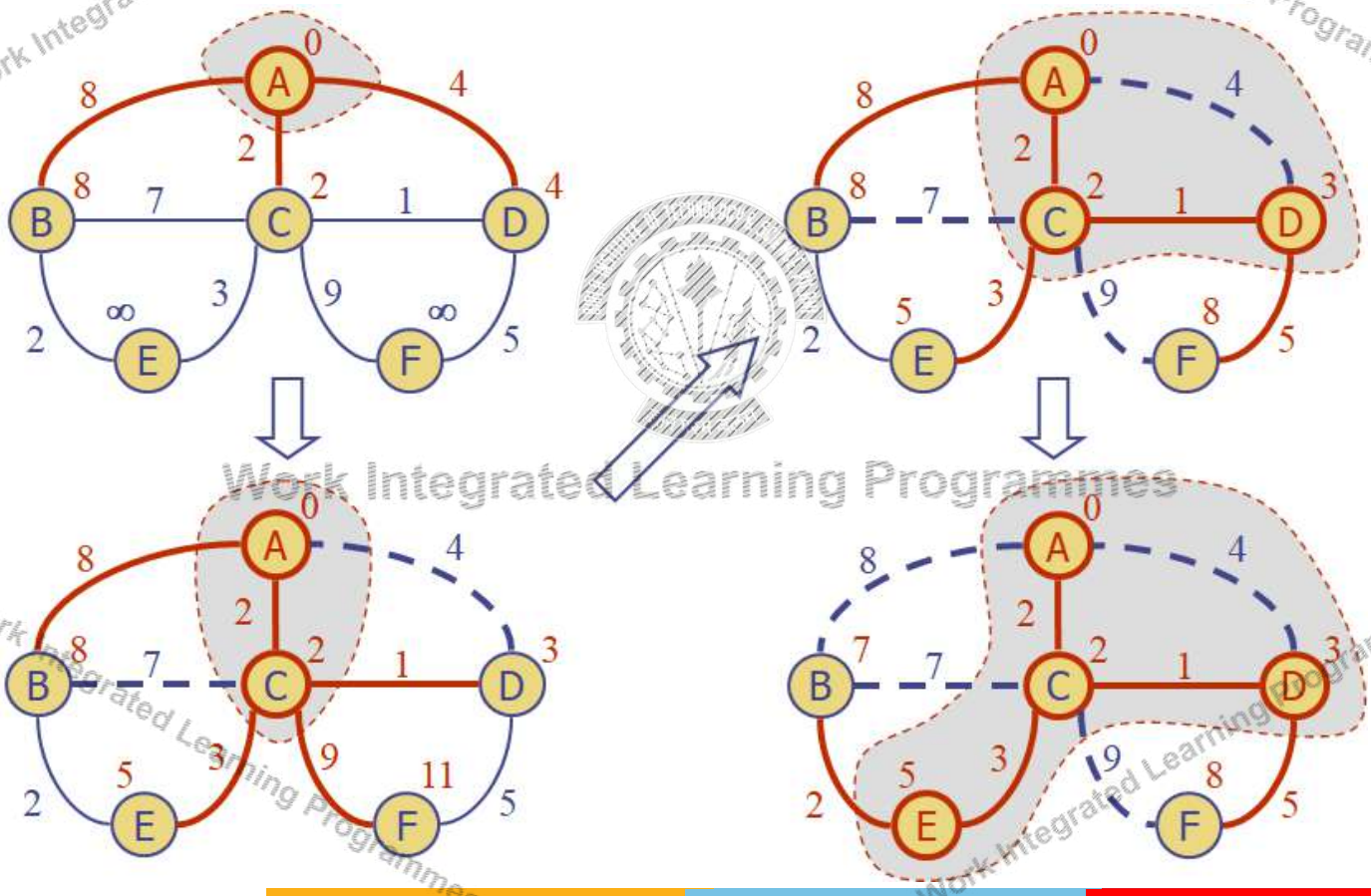


Dijkstra's Algorithm Example 2



Work Integrated Learning Programmes

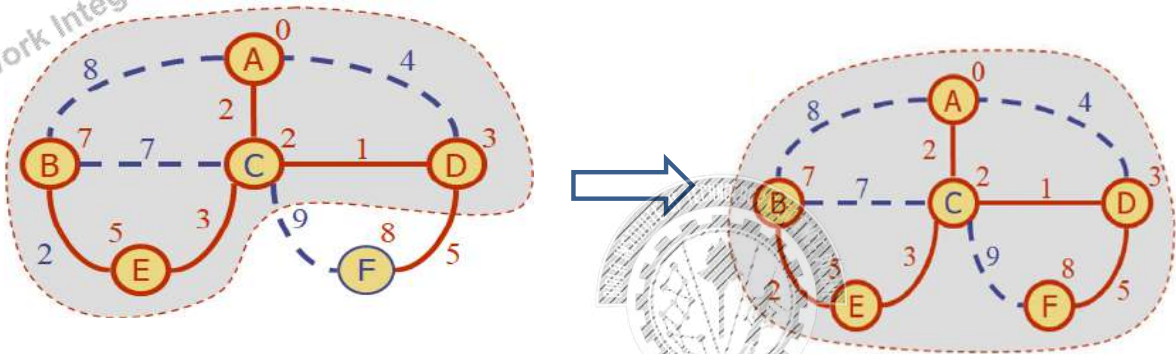
Work Integrated Learning Programmes



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Dijkstra's Algorithm Example 2



The Optimal Solution or Shortest Path from Source Vertex (A) is :

Path	Shortest Path	Shortest Cost/Distance
A → A	-	0
A → B	A → C → E → B	7
A → C	A → C	2
A → D	A → C → D	3
A → E	A → C → E	5
A → F	A → C → D → F	8

Dijkstra's Algorithm

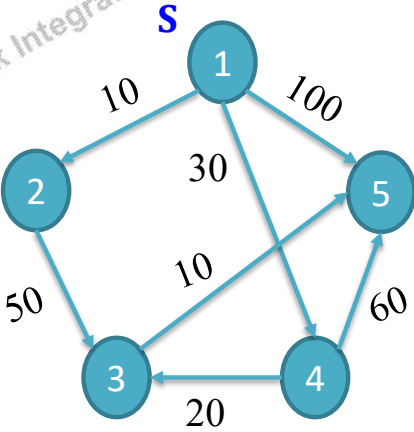


- Dijkstra's Algorithm will work for both Directed and undirected graphs but only when the weights are positive!

Procedure:

- **Step 01:** Construct the cost adjacency matrix for the given graph.
- **Step 02:** Assume a vertex as the source (alphabetically) (if source is not mentioned) and compute the distance from the source to all other vertices as $D[w]=c(s,w)$ or $c(s,u)+c(u,w)$ i.e. direct distance or indirect distance. This is also known as relaxation.
- **Step 03:** Pick the shortest path of the computed distance.
- **Step 04:** The vertex causing the shorted path is also included into the source.
- **Step 05:** Repeat the steps till all shortest paths are evaluated.

Dijkstra's Algorithm Example 1



Cost Adjacency Matrix

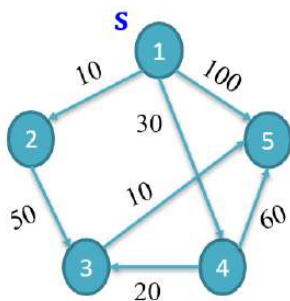
	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

Step 01 :

S (Source)	V = V - S	Distance D[v]	u	D[u]
1	2 3 4 5	D[2] = C(1,2) = 10 ✓ D[3] = C(1,3) = ∞ D[4] = C(1,4) = 30 D[5] = C(1,5) = 100	2	10

Add u (which is 2) to the source and continue ...

Dijkstra's Algorithm Example 1



Cost Adjacency Matrix

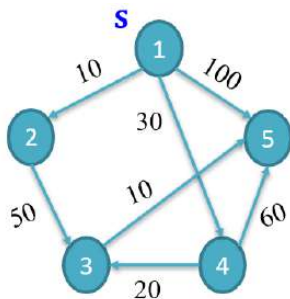
	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

Step 02 :

S (Source)	V = V - S	Distance D[v]	U	D[u]
1,2	3 4 5	D[3] = min{prev value, C(1,2)+C(2,3)}=min(∞ , 10 + 50) = 60 D[4] = min{prev value, C(1,2)+C(2,4)}=min(30,10+ ∞) = 30 ✓ D[5] =min {prev value, C(1,2)+C(2,5)}=min(100,10+ ∞) = 100	4	30

Add u (which is 4) to the source and continue ...

Dijkstra's Algorithm Example 1



Cost Adjacency Matrix

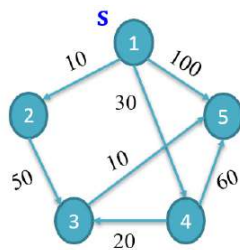
	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

Step 03 :

S (Source)	V = V - S	Distance D[v]	U	D[u]
1,2,4	3 5	$D[3] = \min\{\text{prev value}, C(1,4)+C(4,3)\} = \min(60, 30 + 20) = 50$ ✓ $D[5] = \min\{\text{prev value}, C(1,4)+C(4,5)\} = \min(100, 30+60) = 90$	3	50

Add u (which is 3) to the source and continue ...

Dijkstra's Algorithm Example 1



Cost Adjacency Matrix

	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

Step 04 :

S (Source)	V = V - S	Distance D[v]	U	D[u]
1,2,4,3	5	$D[5] = \min \{ \text{prev value}, C(1,3)+C(3,5) \} = \min(90, 50+10) = 60$	5	60

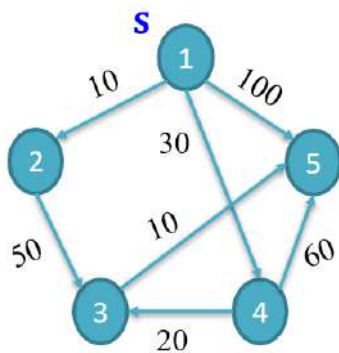
Add u (which is 5) to the source and continue ...

Step 05 :

S (Source)	V = V - S	Distance D[v]	U	D[u]
1,2,4,3,5	ϕ	-	-	-

Since V is null, stop the process.

Dijkstra's Algorithm Example 1



Cost Adjacency Matrix

	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

The Optimal Solution or Shortest Path from Source Vertex (1) is :

Path	Shortest Path	Shortest Cost/Distance
1 $\rightarrow$ 1	-	0
1 $\rightarrow$ 2	1 $\rightarrow$ 2	10
1 $\rightarrow$ 3	1 $\rightarrow$ 4 $\rightarrow$ 3	50
1 $\rightarrow$ 4	1 $\rightarrow$ 4	30
1 $\rightarrow$ 5	1 $\rightarrow$ 4 $\rightarrow$ 3 $\rightarrow$ 5	60

Dijkstra's Algorithm



Algorithm DijkstraShortestPaths(G, v):

Input: A simple undirected weighted graph G with nonnegative edge weights, and a distinguished vertex v of G

Output: A label, $D[u]$, for each vertex u of G , such that $D[u]$ is the distance from v to u in G

$D[v] \leftarrow 0$

for each vertex $u \neq v$ of G **do**

$D[u] \leftarrow +\infty$

Let a priority queue, Q , contain all the vertices of G using the D labels as keys.

while Q is not empty **do**

// pull a new vertex u into the cloud

$u \leftarrow Q.\text{removeMin}()$

for each vertex z adjacent to u such that z is in Q **do**

// perform the **relaxation** procedure on edge (u, z)

if $D[u] + w((u, z)) < D[z]$ **then**

$D[z] \leftarrow D[u] + w((u, z))$

Change the key for vertex z in Q to $D[z]$

return the label $D[u]$ of each vertex u

$O(n \log n)$

$O(m \log n)$

$O((m+n) \log n) \rightarrow O(m \log n)$ 24

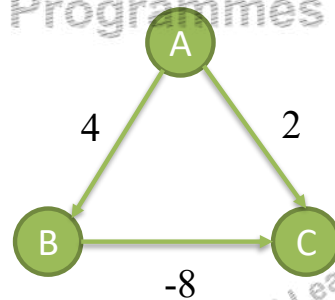
Dijkstra's Algorithm Fails !



- Note that the Dijkstra's Algorithm will fail if the weights of the graph are negative!
- Recall that in Dijkstra's algorithm, *once a vertex is marked as "closed" (and out of the open set) - the algorithm found the shortest path to it*, and will never have to develop this node again - it assumes the path developed to this path is the shortest. But with negative weights - it might not be true.

Dijkstra from A will first develop C (and add C to the source), and will later fail to find $A \rightarrow B \rightarrow C$

Hence, Dijkstra's algorithm will only work when the weights are positive. If they are negative then Bellman Ford Algorithm (Dynamic Programming technique) helps us!



Divide & Conquer Strategy



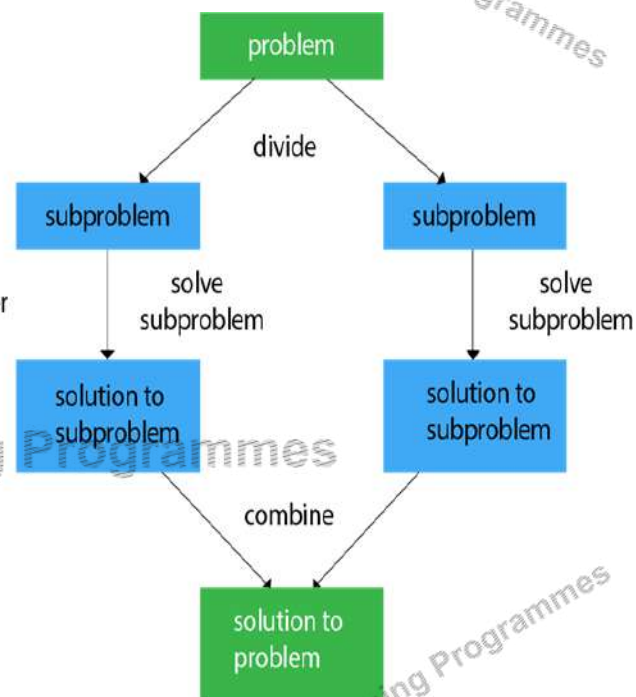
- Divide and Conquer (D&C) is an important algorithm design strategy based on multi-branched recursion. A divide and conquer algorithm works by recursively breaking down a problem into two or more sub-problems of the same (or related) type, until these become simple enough to be solved directly.
- The solutions to the sub-problems are then combined to give a solution to the original problem.
- The divide and conquer approach yields some of the most important and efficient algorithms in Computer Science / Data Science 😊
- This technique is the basis of efficient algorithms for all kinds of problems, such as sorting, multiplying large numbers, syntactic analysis, and in scientific research.

Divide & Conquer Strategy



➤ Divide and Conquer Algorithms work in the following manner :

- 1) A problem's instance is divided into several smaller instances of the same problem, ideally of the same size (**Divide**).
- 2) The smaller instances are solved, typically recursively. (**Conquer**).
- 3) The solutions obtained for the smaller instances are combined to get a solution to the original instance of the problem (**Combine**).



Advantages of Divide & Conquer Strategy



- **Solving Difficult Problems:** D&C is a powerful tool for solving conceptually difficult problems, such as the classic towers of Hanoi problem. All it requires is a way of breaking the problem into sub-problems, of solving the trivial cases and the combining sub-problems to the original form.
- **Algorithm Efficiency:** D&C technique often helps in the discovery of efficient algorithms. It was the key to Karatsuba's multiplication, quick sort, merge sort algorithms and others. Often D&C led to an improvement in the asymptotic cost of the solution.
- **Parallelism:** D&C algorithms are naturally adapted for execution in multi-processor machines.
- **Efficient Memory access**

Some common Divide & Conquer Algorithms



- Binary Search
- Merge Sort
- Quick Sort
- Integer Multiplication – Karatsuba’s Algorithm
- Finding maximum and minimum from a list [Exercise Question]
- Strassen’s Matrix Multiplication [Self – Study]
- ...
- ...

Search Techniques



- We already know linear search in which we start from the leftmost element and compare it with the search term (Key);
- If the search term matches the number on the index we are currently on, then the search operation is successful and the index is returned!
- But, if the numbers don't match, then we go to the number on the next index and follow the same procedure till the number is found or we have reached the end of the list.
- If the number isn't present, we return that the search was unsuccessful. The time complexity of linear sort is $O(n)$. This may hence take enormous time when there are many inputs.
- Which Algorithmic Strategy is linear Search ? *Brute Force !!*

Binary Search



- Binary search is one such divide and conquer algorithm to assist with the given problem; note that a sorted array should be used in this case.
- This search algorithm recursively divides the array into two sub-arrays that may contain the search term.
- It discards one of the sub-array by utilizing the fact that items are sorted.
- It continues halving the sub-arrays until it finds the search term or it narrows down the list to a single item.
- Since binary search discards the sub-array it's pseudo Divide & Conquer algorithm. What makes binary search efficient is the fact that if it doesn't find the search term in each iteration, it just reduces the array/list to it's half for the next iteration.
- The time complexity of binary search is $O(\log n)$, where n is the number of elements in an array. If the search term is at the center of the array, it's considered to be the best case since the element is found instantly in a go. Hence the best case complexity will be $O(1)$

31

Merge Sort



➤ Merge-sort on an input sequence S with n elements consists of three steps:

- **Divide**: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
- **Recur**: recursively sort S_1 and S_2
- **Conquer**: merge S_1 and S_2 into a unique sorted sequence

Algorithm *mergeSort*(S, C)

Input sequence S with n elements, comparator C

Output sequence S sorted according to C

if $S.size() > 1$

$(S_1, S_2) \leftarrow partition(S, n/2)$

mergeSort(S_1, C)

mergeSort(S_2, C)

$S \leftarrow merge(S_1, S_2)$

Merge Sort



- The *conquer step* of merge-sort consists of merging two sorted sequences A and B into a sorted sequence S containing the union of the elements of A and B
- Merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes $O(n)$ time

Algorithm *merge*(A, B)

Input sequences A and B with $n/2$ elements each

Output sorted sequence of $A \cup B$

$S \leftarrow$ empty sequence

while $\neg A.isEmpty() \wedge \neg B.isEmpty()$

if $A.first().element() < B.first().element()$

$S.insertLast(A.remove(A.first()))$

else

$S.insertLast(B.remove(B.first()))$

while $\neg A.isEmpty()$

$S.insertLast(A.remove(A.first()))$

while $\neg B.isEmpty()$

$S.insertLast(B.remove(B.first()))$

return S

Merge Sort



Merging two sorted lists, `merge(A,B)`



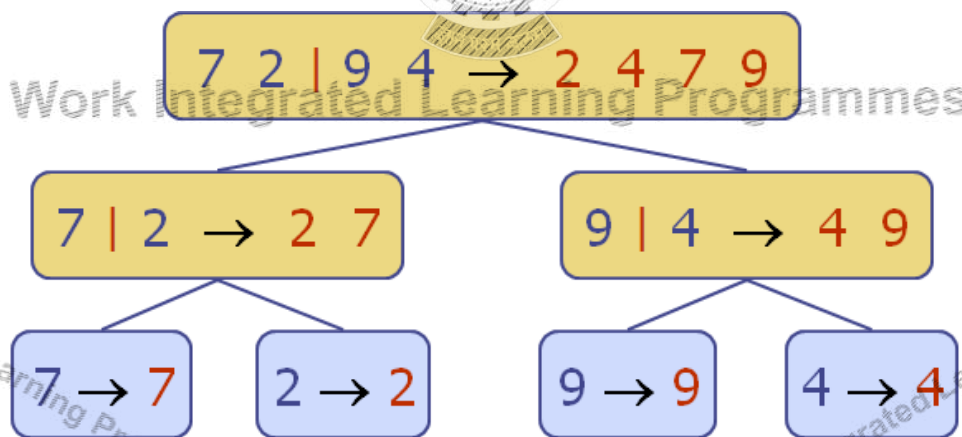
Result:

--	--	--	--	--	--	--	--

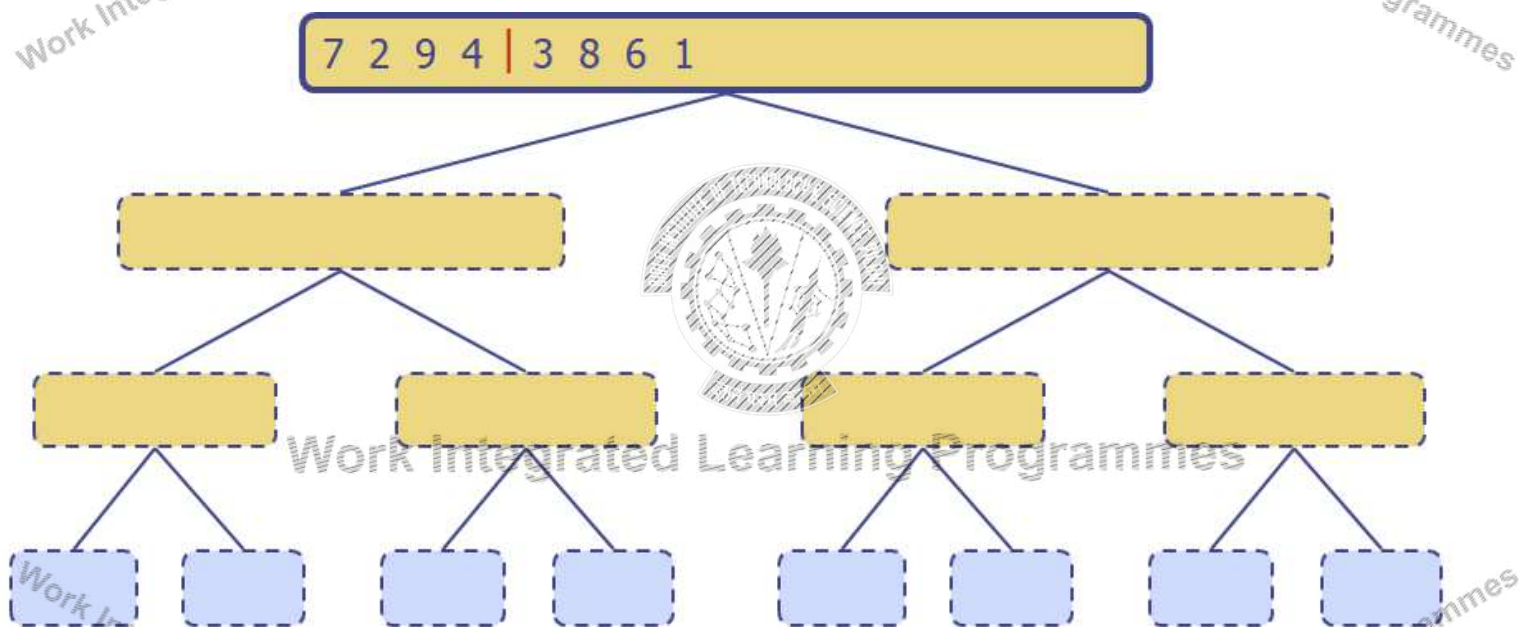
Merge Sort Example 1



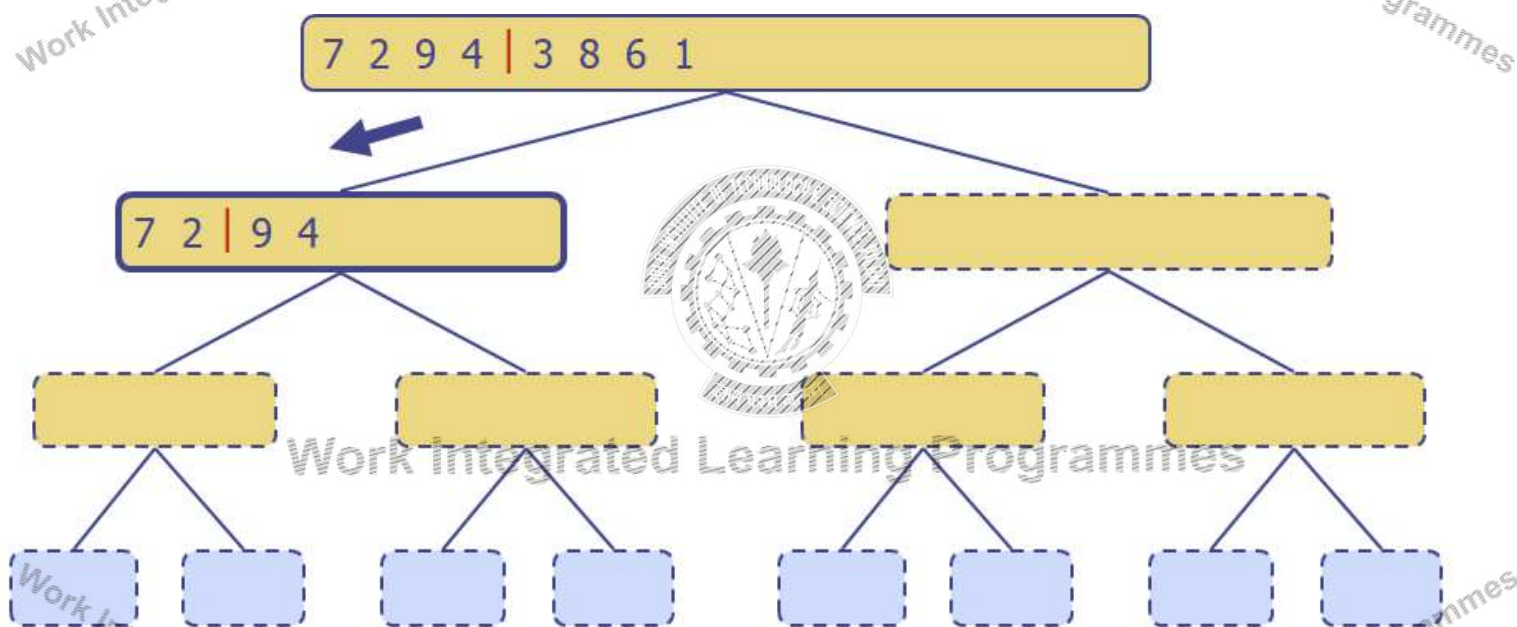
- An execution of merge-sort is depicted by a binary tree
 - each node represents a recursive call of merge-sort and stores
 - unsorted sequence before the execution and its partition
 - sorted sequence at the end of the execution
 - the root is the initial call
 - the leaves are calls on subsequences of size 0 or 1



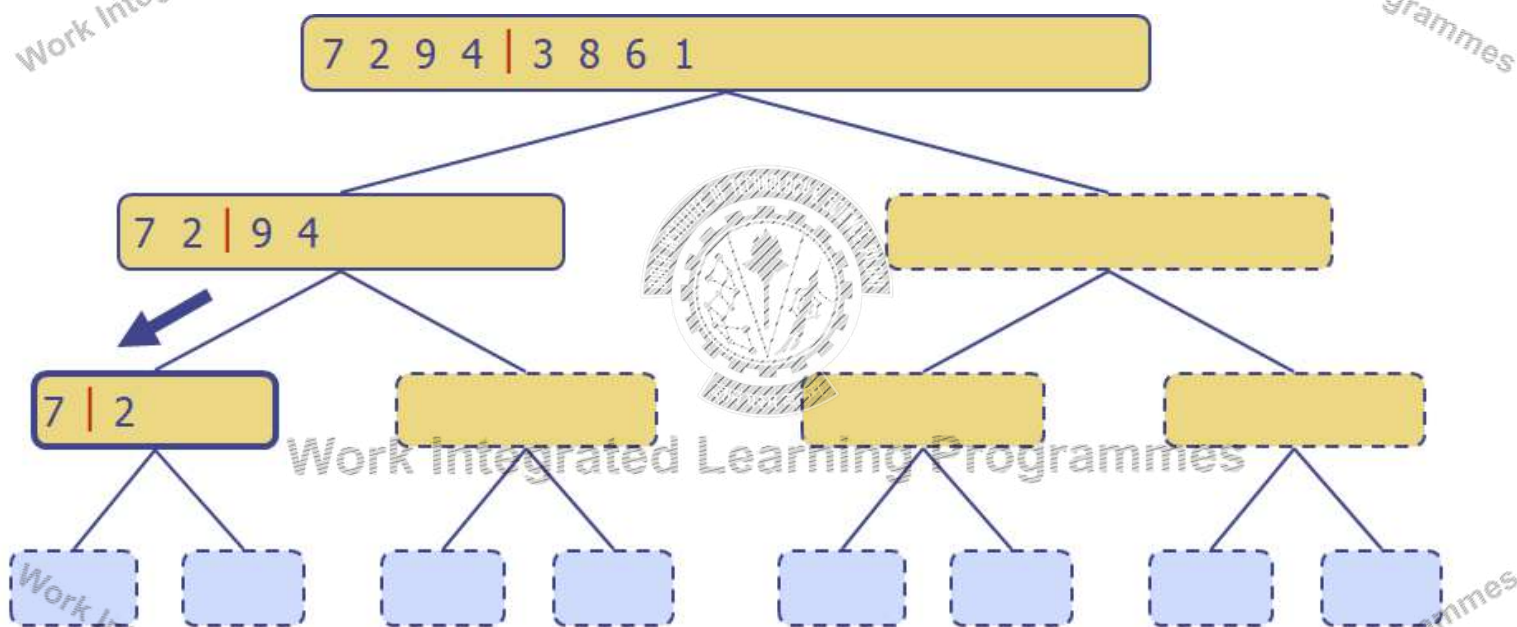
Merge Sort Example 1



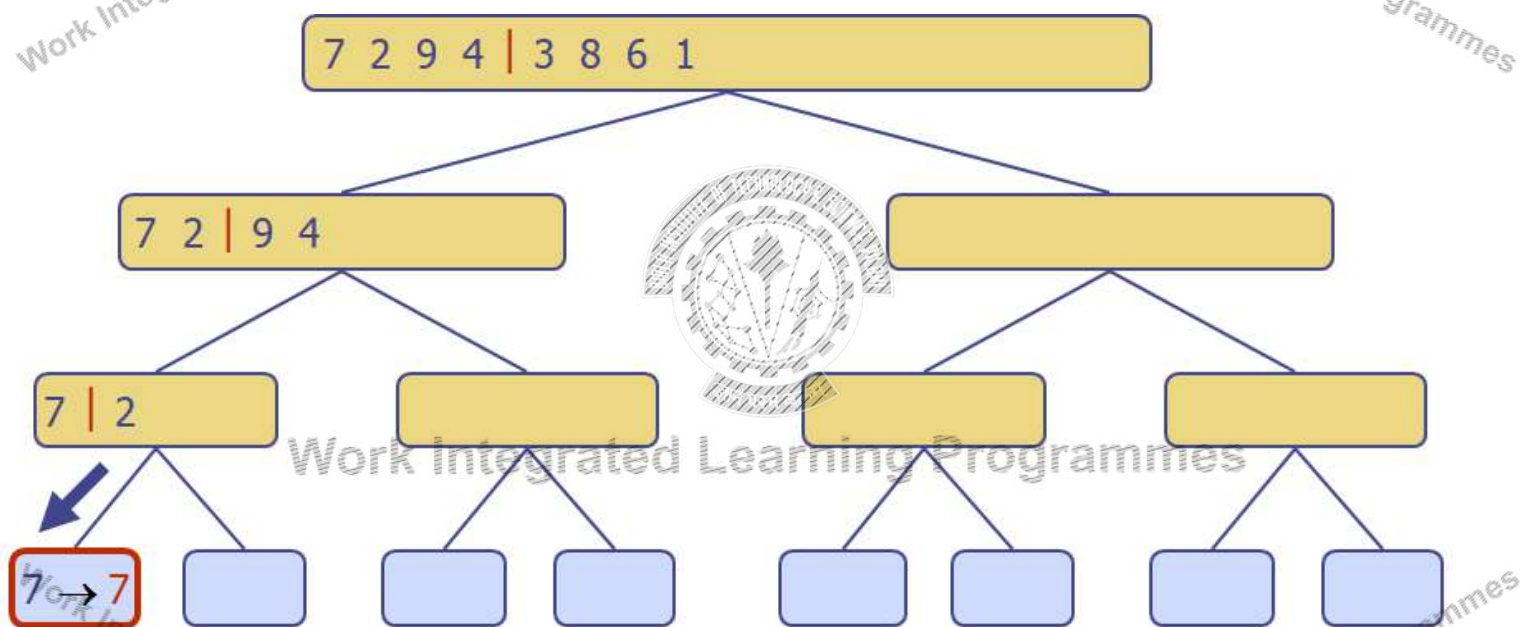
Merge Sort Example 1



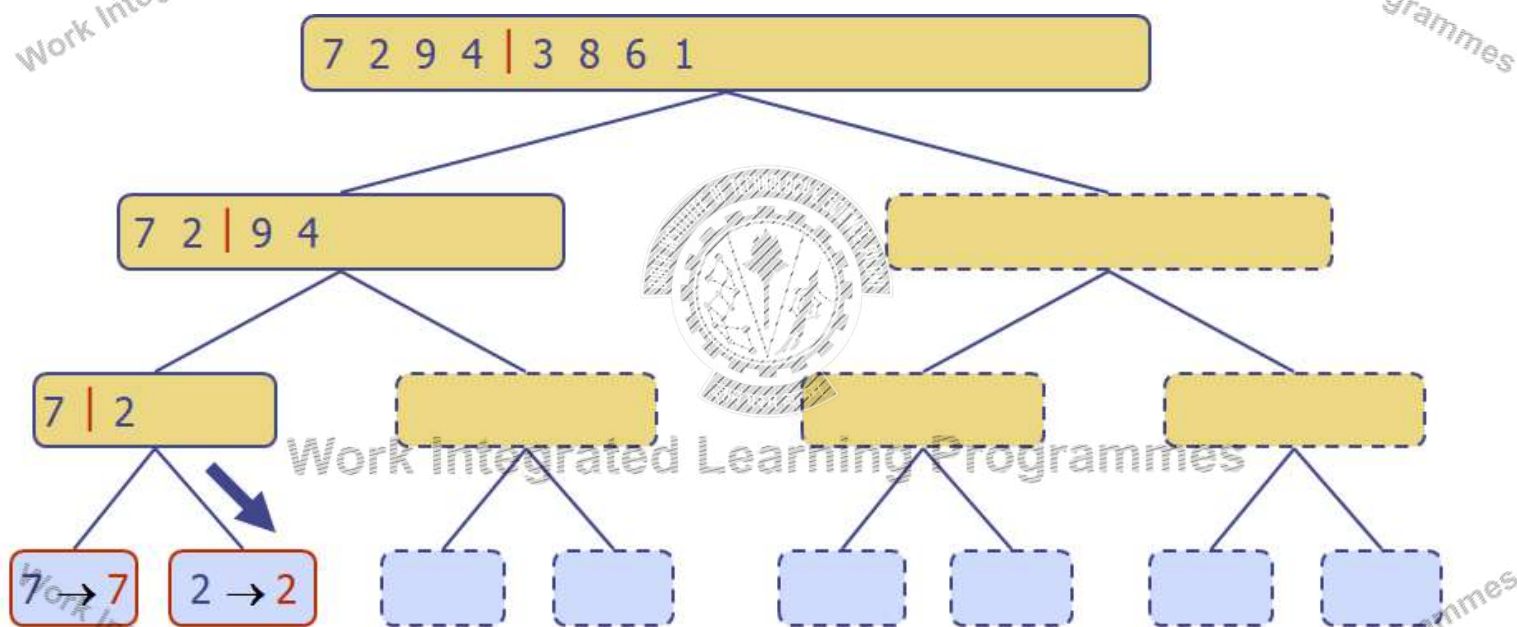
Merge Sort Example 1



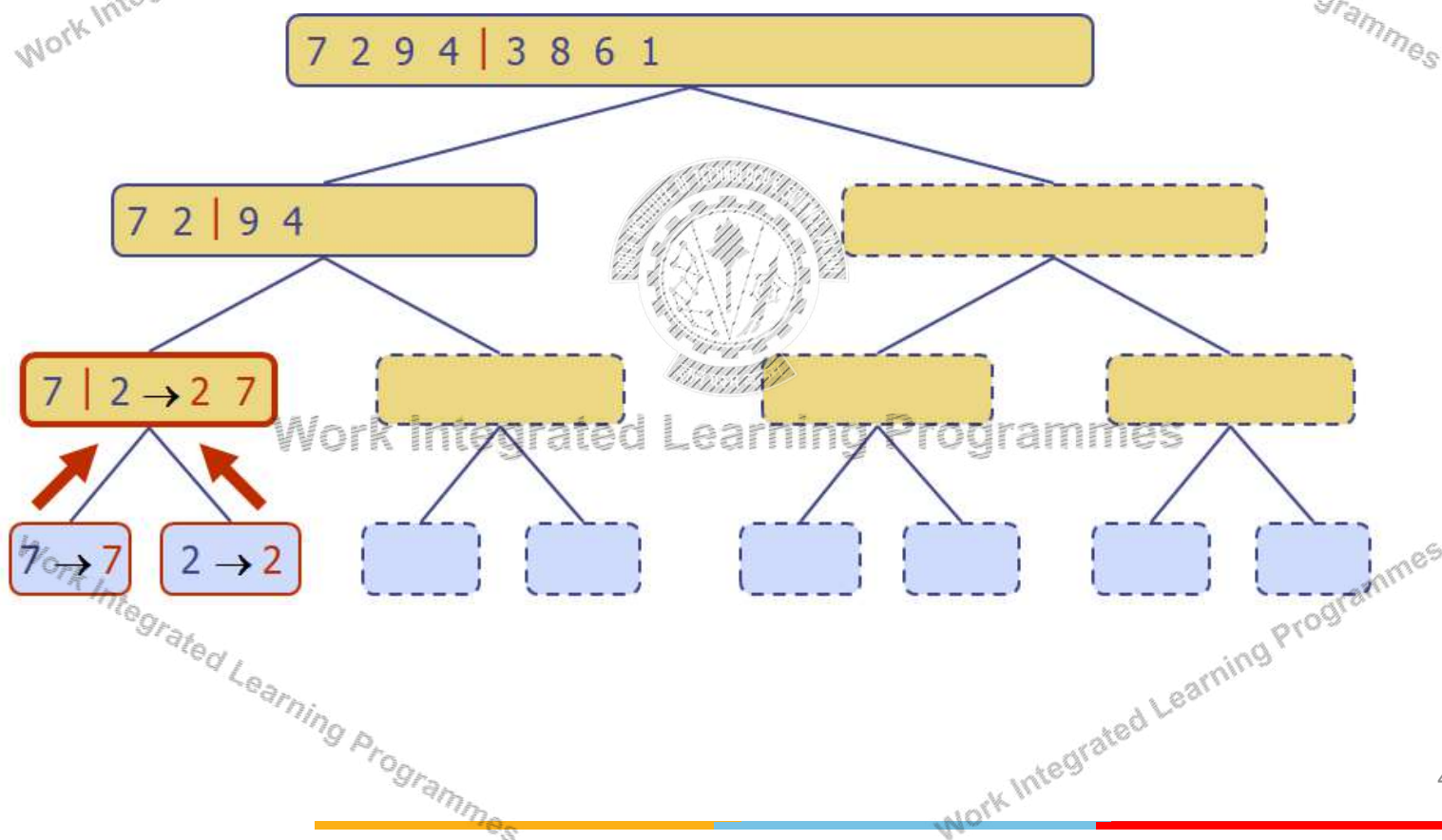
Merge Sort Example 1



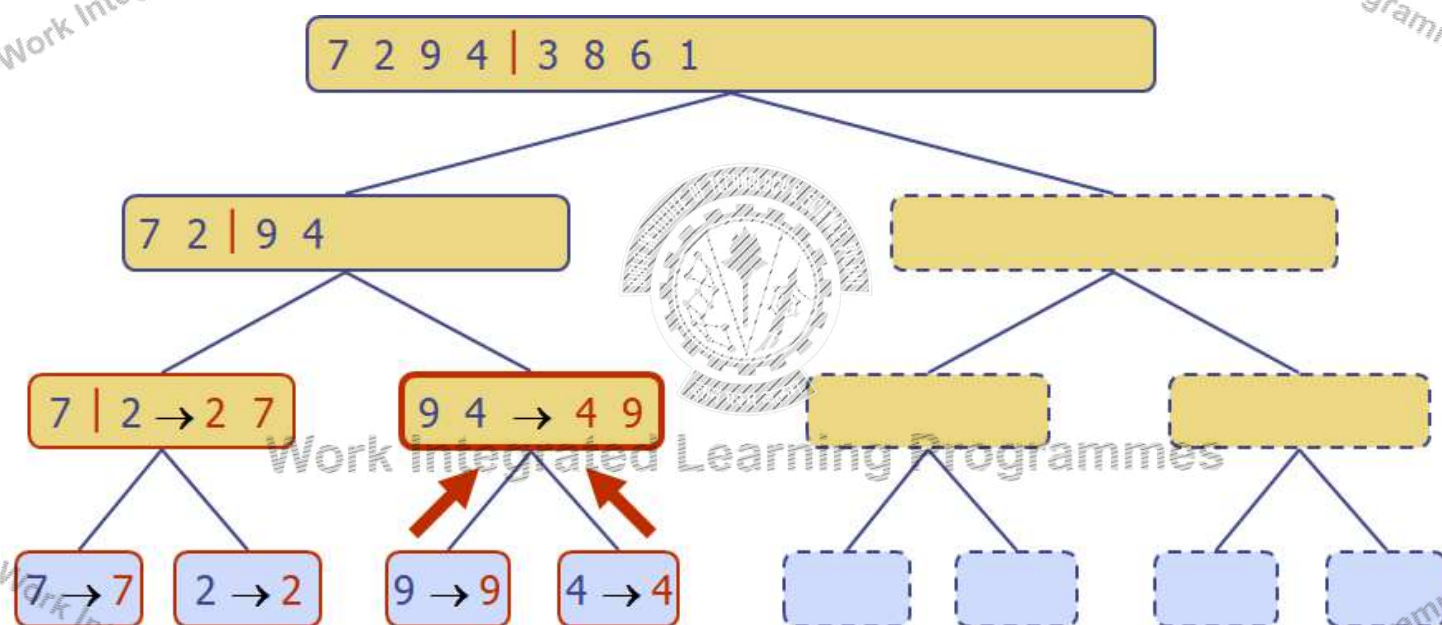
Merge Sort Example 1



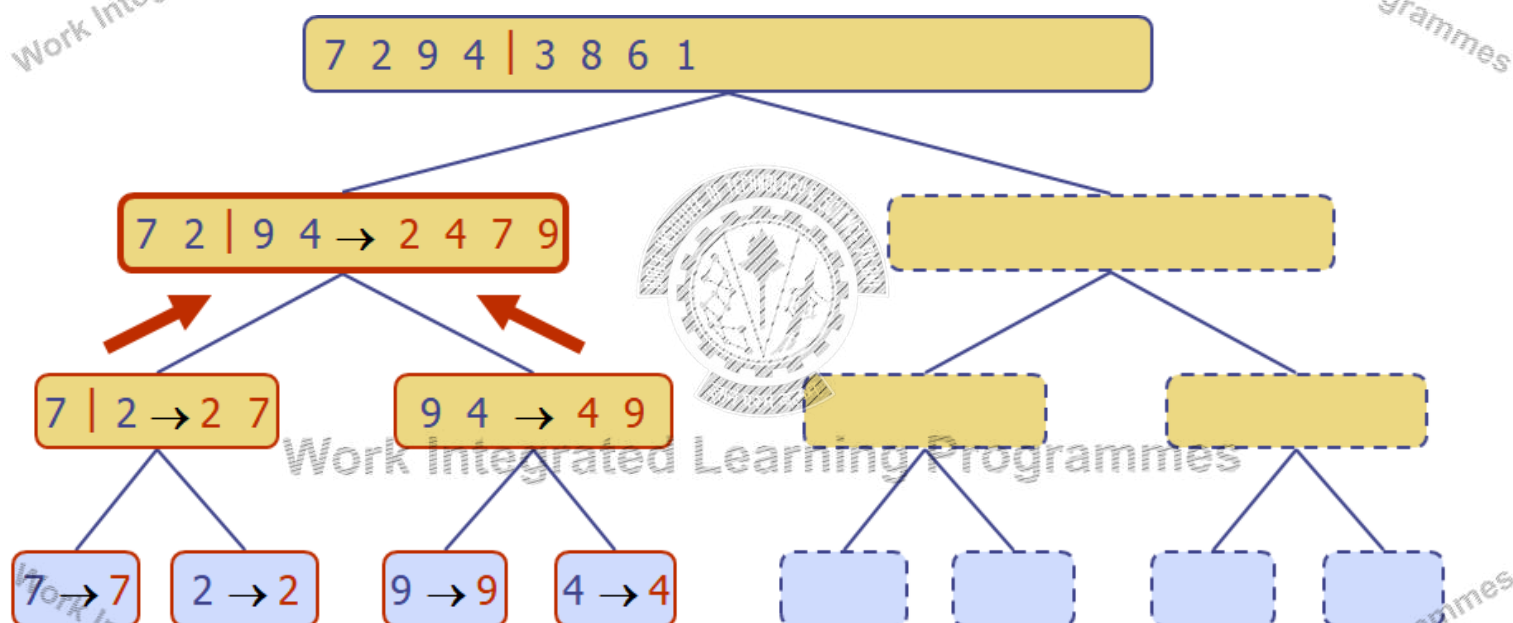
Merge Sort Example 1



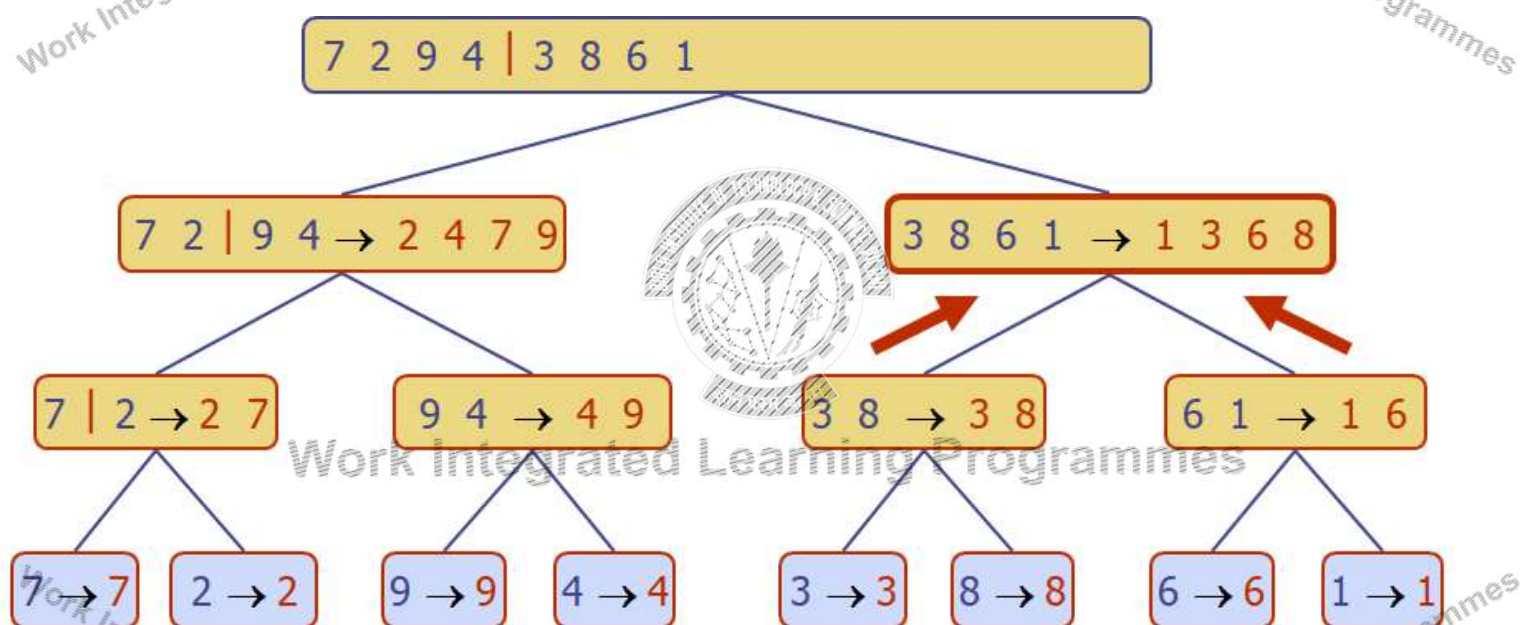
Merge Sort Example 1



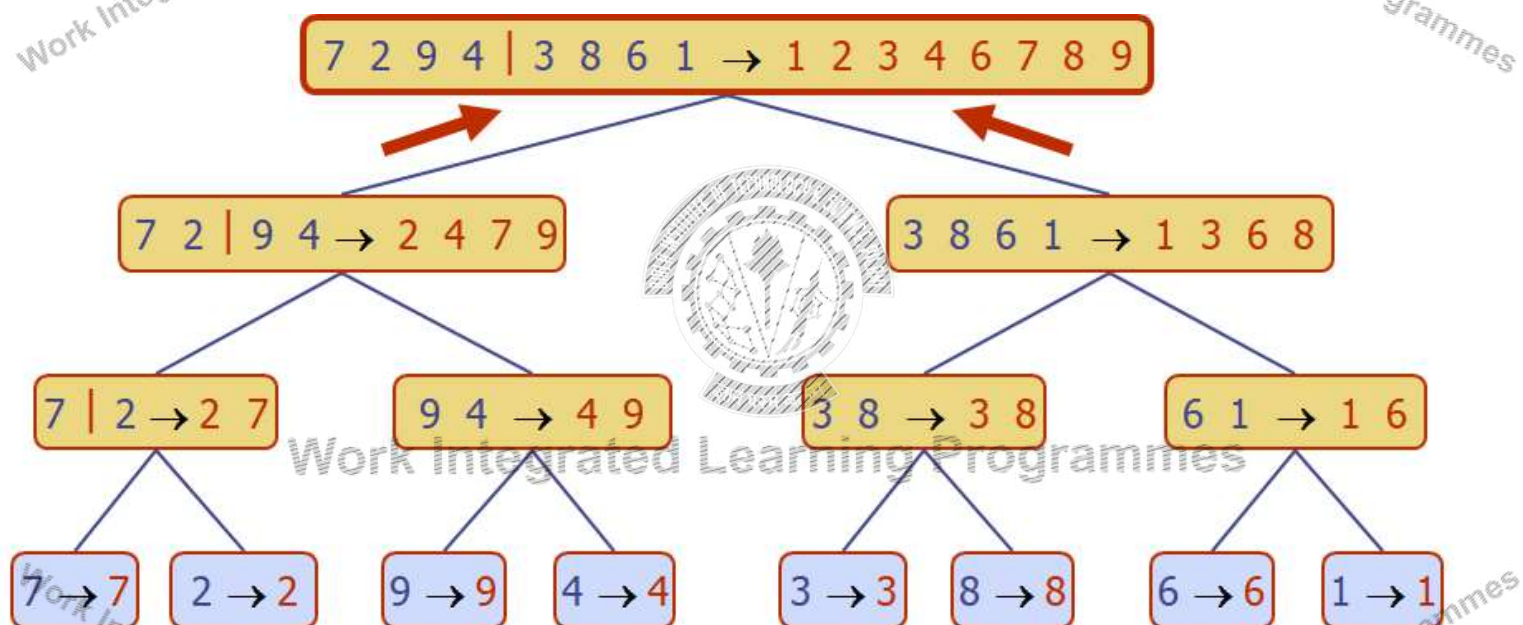
Merge Sort Example 1



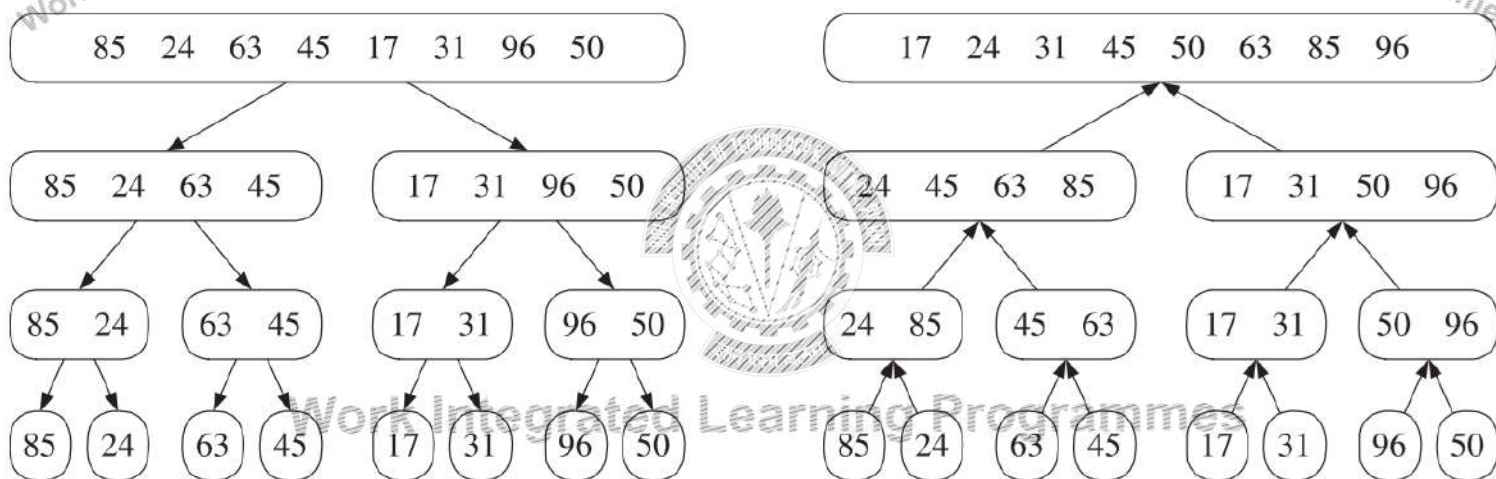
Merge Sort Example 1



Merge Sort Example 1



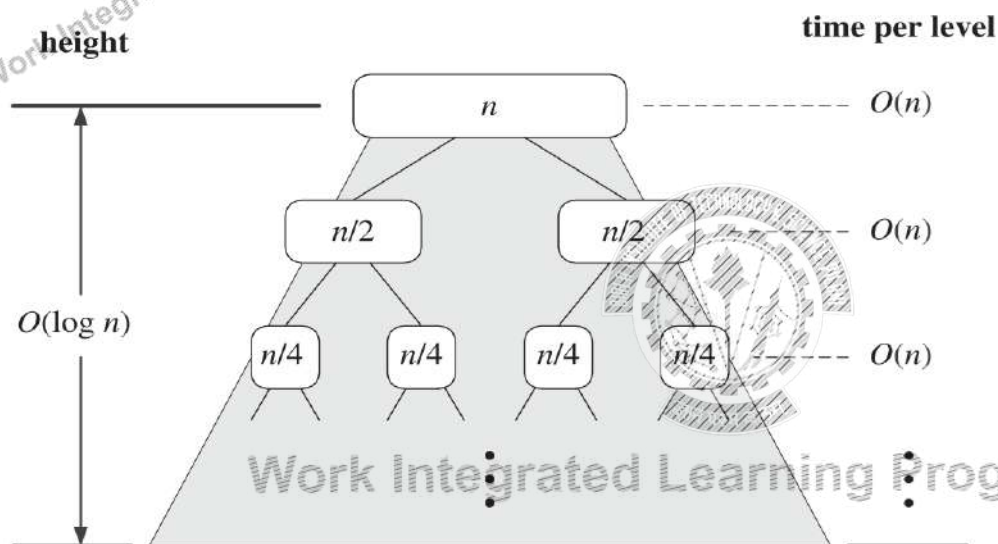
Merge Sort Example 2



Input sequence processed at each node - Divide

Output sequence generated at each node - Conquer

Merge Sort



Total time: $O(n \log n)$

$$t(n) = \begin{cases} b & \text{if } n = 1 \text{ or } n = 0 \\ t(\lceil n/2 \rceil) + t(\lfloor n/2 \rfloor) + cn & \text{otherwise} \end{cases}$$

Assume, n is a power of 2 :

$$t(n) = \begin{cases} b & \text{if } n = 1 \\ 2t(n/2) + cn & \text{otherwise} \end{cases}$$

47

Merge Sort Analysis



$$t(n) = \begin{cases} b & \text{if } n = 1 \\ 2t(n/2) + cn & \text{otherwise} \end{cases}$$

By repeated substitution,

$$\begin{aligned} t(n) &= 2(2t(n/2^2) + (cn/2)) + cn \\ &= 2^2t(n/2^2) + 2cn. \end{aligned}$$

Substituting Again...

$$t(n) = 2^3t(n/2^3) + 3cn$$

Substituting Again...

$$t(n) = 2^4t(n/2^4) + 4cn$$

Generalizing it ...

$$t(n) = 2^i t(n/2^i) + icn$$

when $i = \log n$, $2^i = n$

$$\begin{aligned} t(n) &= 2^{\log n} t(n/2^{\log n}) + (\log n)cn \\ &= nt(1) + cn \log n \\ &= nb + cn \log n. \end{aligned}$$

This implies,

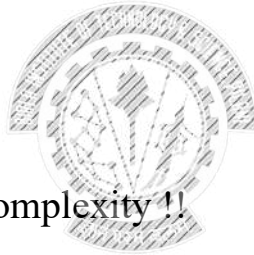
$$t(n) \text{ is } O(n \log n)$$

Merge Sort Analysis



$$t(n) = \begin{cases} b & \text{if } n = 1 \text{ or } n = 0 \\ t(\lceil n/2 \rceil) + t(\lfloor n/2 \rfloor) + cn & \text{otherwise} \end{cases}$$

$$t(n) = \begin{cases} b & \text{if } n = 1 \\ 2t(n/2) + cn & \text{otherwise} \end{cases}$$



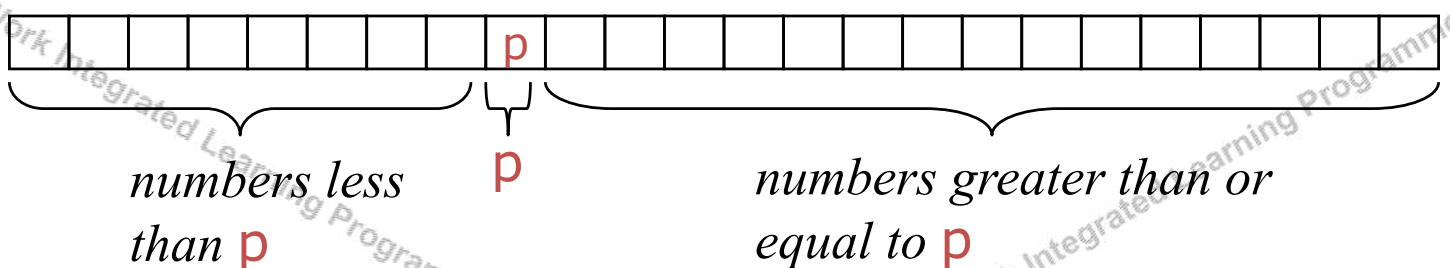
Apply Master's theorem to get the complexity !!

Work Integrated Learning Programmes

Quick Sort

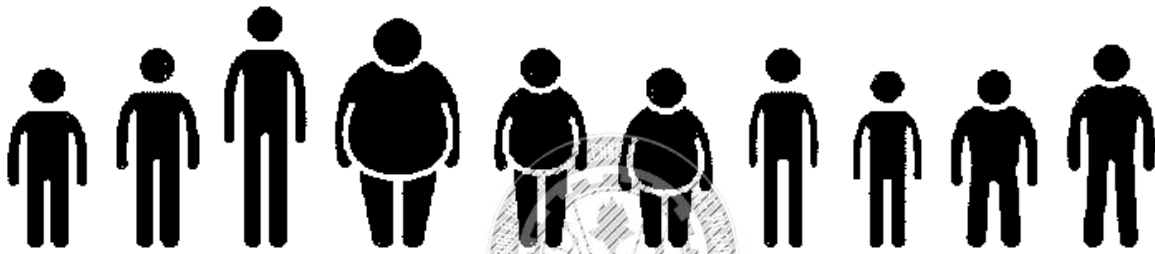


- Quick is a very popular sorting algorithm which is based on the divide and conquer strategy. The steps involved here are :
 - Pick some number p from the array
 - Move all numbers less than p to the beginning of the array
 - Move all numbers greater than (or equal to) p to the end of the array
 - Quicksort the numbers less than p
 - Quicksort the numbers greater than or equal to p

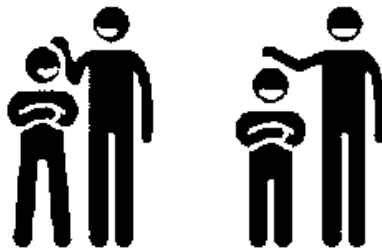




Quick Sort Analogy – Assembly Line



*Compare and get into **right** position !!*
Right position means anyone in front / left of you is smaller than you
and anyone after you / behind you are greater than you!



Exercises Set 1



Exercise-1:

Assume that the sequence to be sorted is represented as a doubly linked list.

Given this,

- (a) What is the complexity of divide step?
- (b) What is the complexity of merge step?
- (c) Characterize the performance of merge sort on doubly linked list as a recurrence & solve it.
- (d) Characterize the result obtained in (c) using Big Oh notation

Exercise-2:

Assume you are given a sequence of keys with duplicates. How will you adapt merge sort algorithm to produce sorted array as output which has no duplicate entries? Write the modified algorithm. What is its complexity?

Exercise Set 2



➤ Apply Merge sort and quick sort on the following numbers and show the tracing :

a) 10,76,83,2,1,68,94,1003,368,53

b) 45,68,21,6,-6,2,78,3,6,457,1

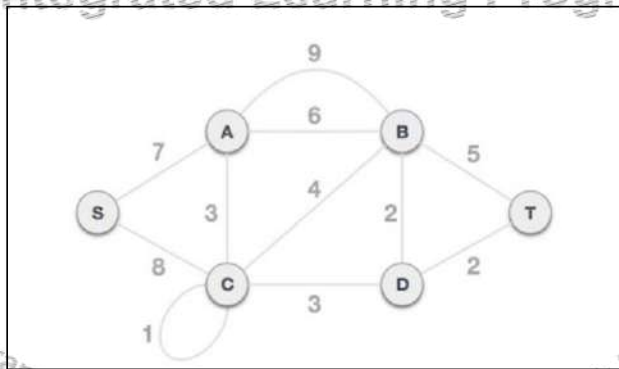
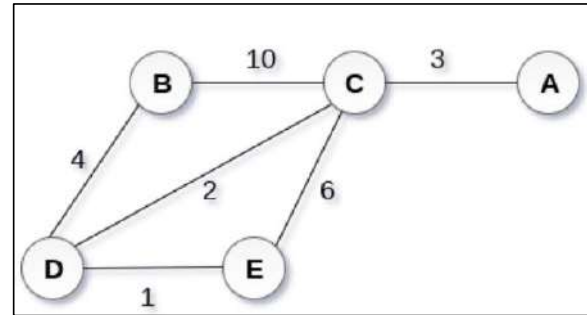
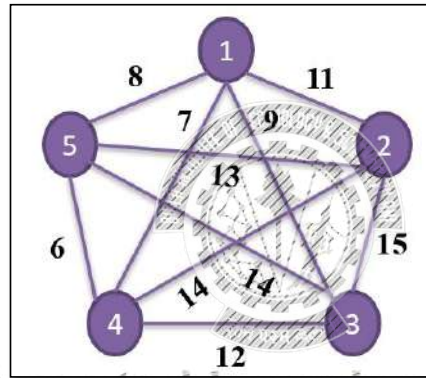
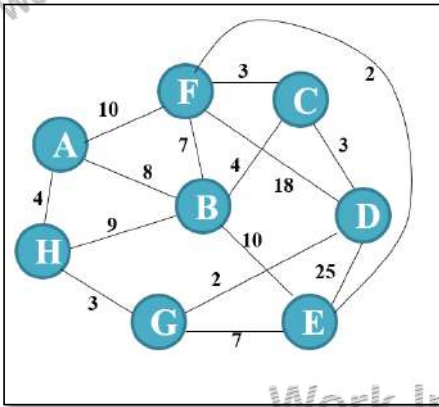


Work Integrated Learning Programmes

Exercises Set



1) Apply Dijkstra's algorithm on the given graphs.





Work Integrated Learning Programmes

We will explore D&C further in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



56

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Contact Session #13

DSECLZG519 – Divide & Conquer

Agenda for Session # 13



➤ Recap of CS#12

➤ Divide & Conquer

➤ Quick Sort

➤ Integer Multiplication Problem (Karatsuba)

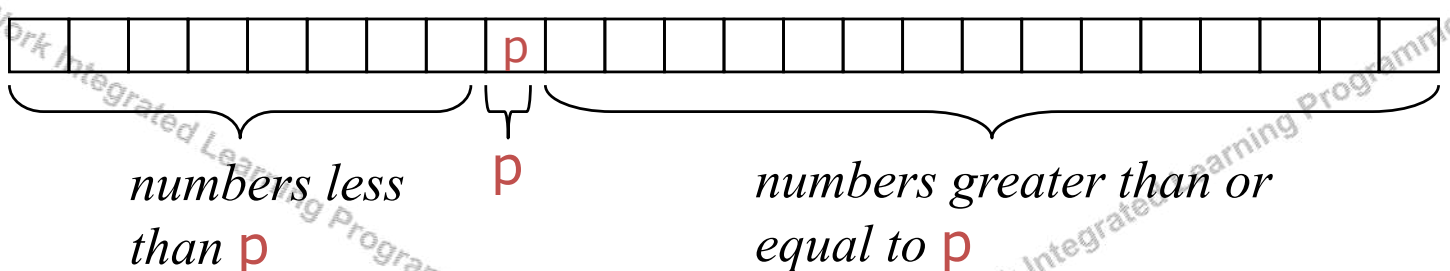
➤ Introduction to DP

➤ Exercises

Quick Sort

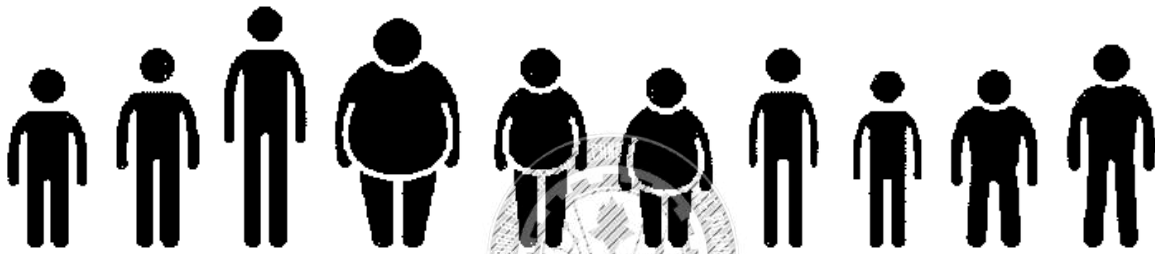


- Quick is a very popular sorting algorithm which is based on the divide and conquer strategy. The steps involved here are :
 - Pick some number p from the array
 - Move all numbers less than p to the beginning of the array
 - Move all numbers greater than (or equal to) p to the end of the array
 - Quicksort the numbers less than p
 - Quicksort the numbers greater than or equal to p

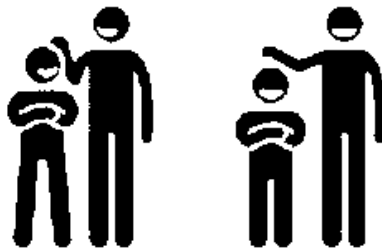




Quick Sort Analogy – Assembly Line



*Compare and get into **right** position !!*
Right position means anyone in front / left of you is smaller than you
and anyone after you / behind you are greater than you!



Quick Sort Algorithm



- Given an array of n elements (e.g., integers):
- If array only contains one element, return
- Else
 - pick one element to use as *pivot*.
 - **Partition** elements into two sub-arrays:
 - Elements less than or equal to pivot
 - Elements greater than pivot
 - Quicksort two sub-arrays
 - Return results

Quick Sort Example



We are given array of n integers to sort:

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

- **Pick Pivot Element:** There are a number of ways to pick the pivot element. In this example, we will use the first element in the array:

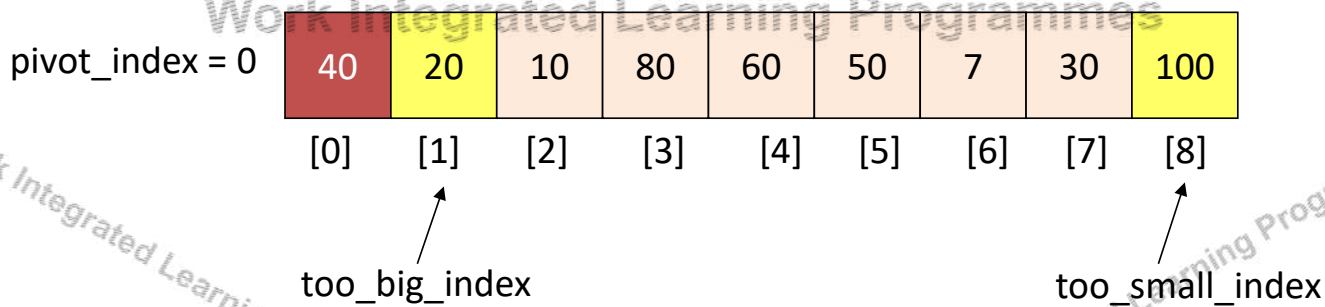
40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Partitioning Array in Quick Sort



- Given a pivot, partition the elements of the array such that the resulting array consists of:
 - One sub-array that contains elements $\geq$ pivot
 - Another sub-array that contains elements $<$ pivot
- The sub-arrays are stored in the original data array.
- Partitioning loops through, swapping elements below/above pivot.

Quick Sort



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$



pivot_index = 0

40	20	10	80	60	50	7	30	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$



pivot_index = 0

40	20	10	80	60	50	7	30	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$



pivot_index = 0

40	20	10	80	60	50	7	30	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$

pivot_index = 0

40	20	10	80	60	50	7	30	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

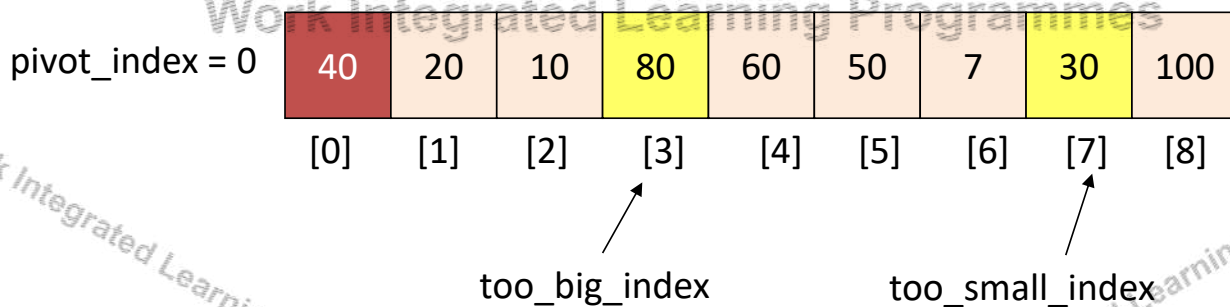
too_big_index

too_small_index

Quick Sort



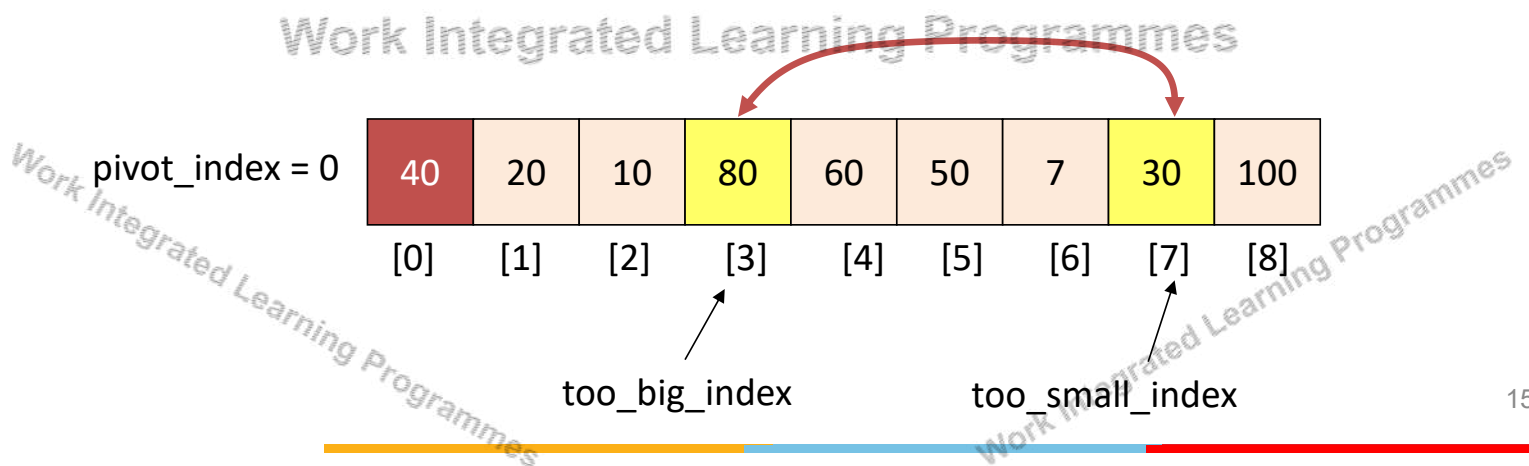
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$



Quick Sort



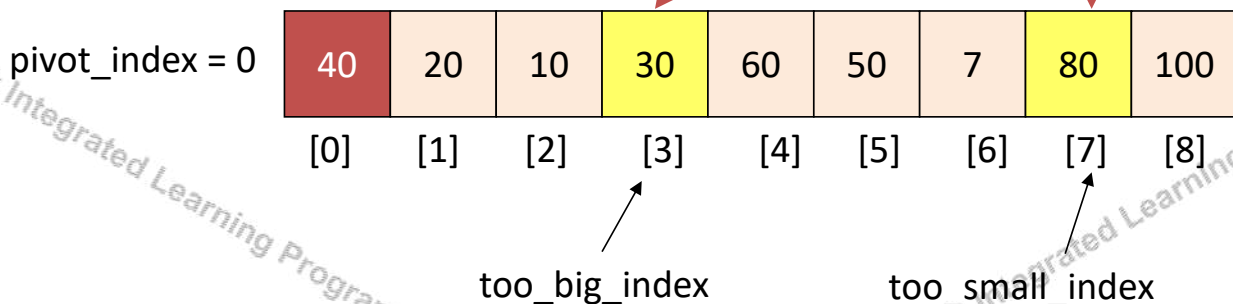
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$



Quick Sort



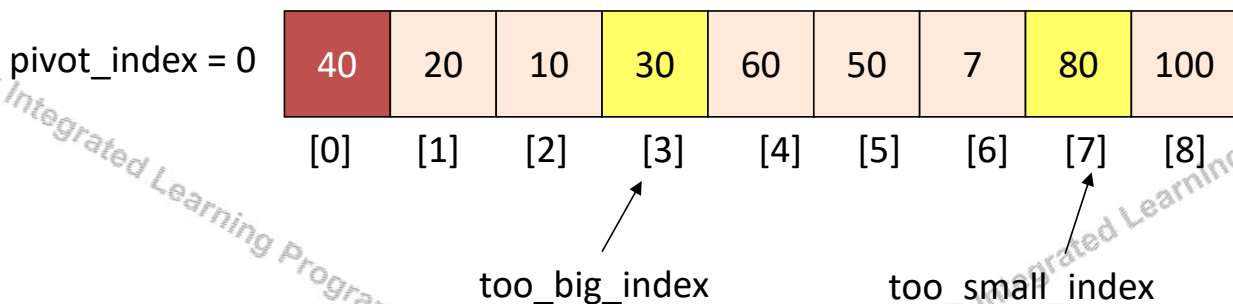
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$



Quick Sort



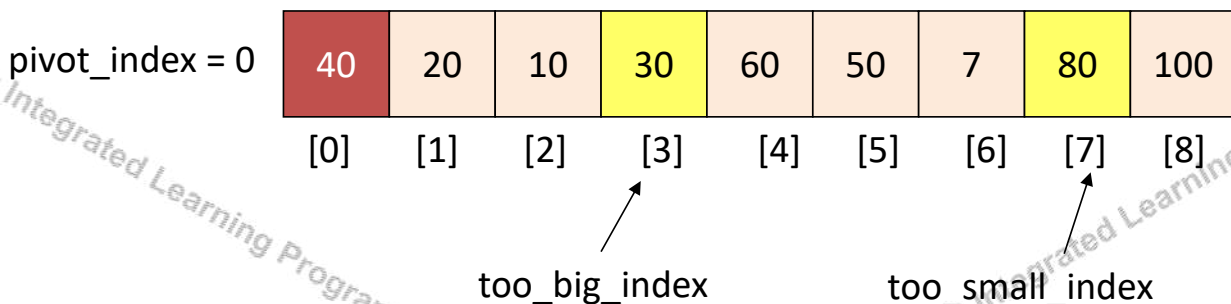
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

pivot_index = 0

40	20	10	30	60	50	7	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

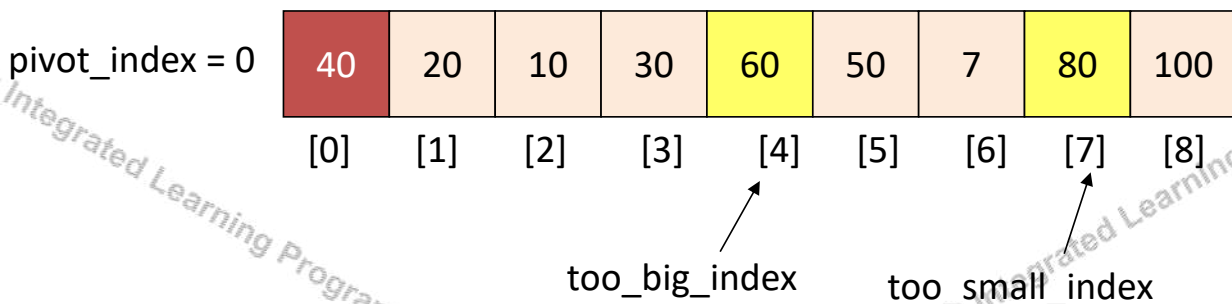
too_big_index

too_small_index

Quick Sort



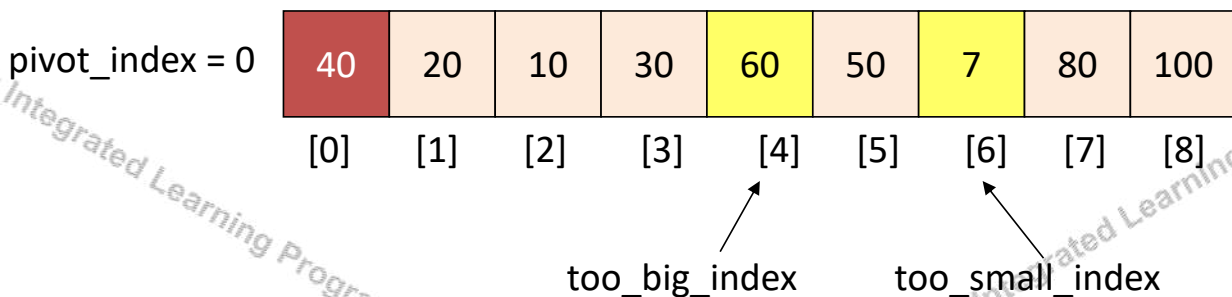
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
- 2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



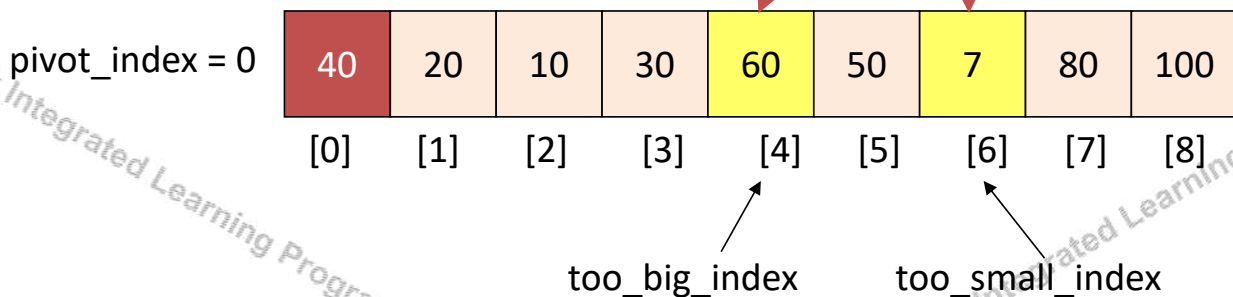
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
- 2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



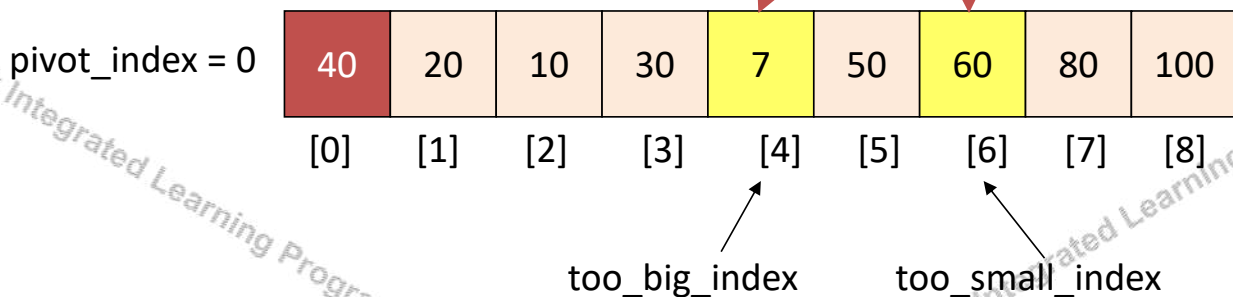
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
- 3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
- 3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
- 4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

pivot_index = 0

40	20	10	30	7	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

pivot_index = 0

40	20	10	30	7	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

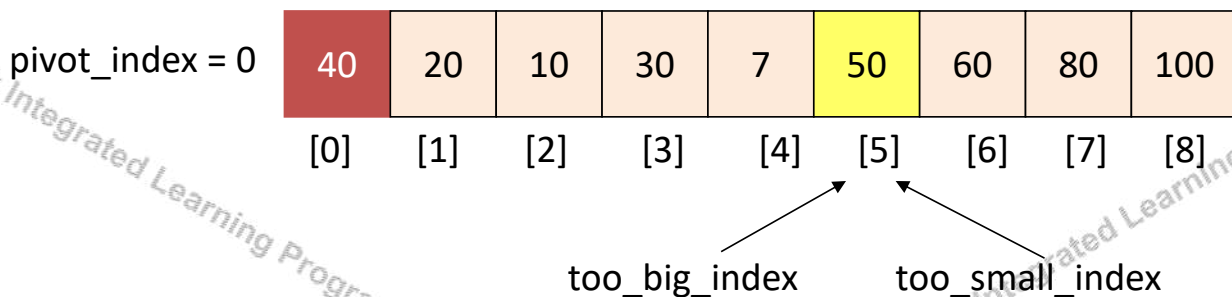
too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
- 2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
- 2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

pivot_index = 0

40	20	10	30	7	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
- 3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

No Swap

pivot_index = 0

40	20	10	30	7	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 $++\text{too_big_index}$
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 $--\text{too_small_index}$
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
- 4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.

pivot_index = 0

40	20	10	30	7	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

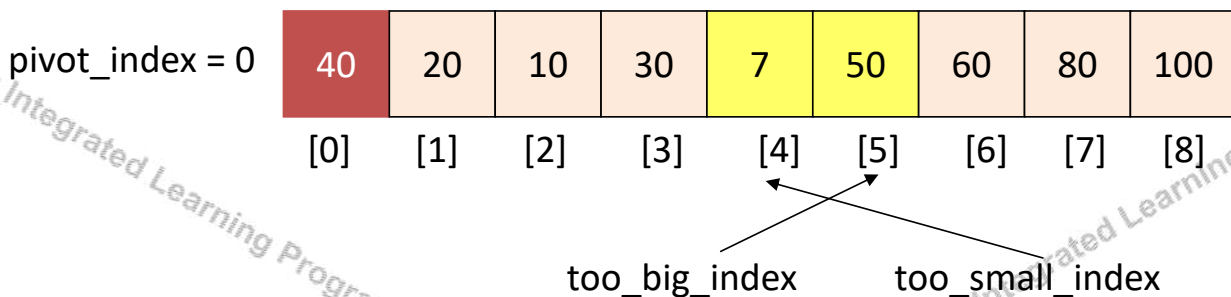
too_big_index

too_small_index

Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
- 5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$



Quick Sort



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
- 5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

pivot_index = 4

7	20	10	30	40	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

31

Quick Sort Partition Result



7	20	10	30	40	50	60	80	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

← $\leq \text{data}[\text{pivot}]$ $> \text{data}[\text{pivot}]$ →

Quick Sort Algorithm



```
procedure quickSort(left, right)

  if right-left <= 0
    return
  else
    pivot = A[right]
    partition = partitionFunc(left, right, pivot)
    quickSort(left, partition-1)
    quickSort(partition+1, right)
  end if
end procedure
```

```
function partitionFunc(left, right, pivot)
  leftPointer = left
  rightPointer = right - 1

  while True do
    while A[++leftPointer] < pivot do
      do-nothing
    end while

    while rightPointer > 0 && A[--rightPointer] > pivot do
      do-nothing
    end while

    if leftPointer > rightPointer
      break
    else
      swap leftPointer, rightPointer
    end if
  end while

  swap leftPointer, right
  return leftPointer
end function
```

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Quick Sort Analysis



➤ Assume that keys are random, uniformly distributed.

➤ What is best case / average running time?

– Recursion:

1. Partition splits array in two sub-arrays of size $n/2$
2. Quicksort each sub-array

– Depth of recursion tree? $\Theta(\log_2 n)$

– Number of accesses in partition? $\Theta(n)$

➤ Best case running time: $\Theta(n \log_2 n)$

➤ Unable to get it intuitively, then solve :

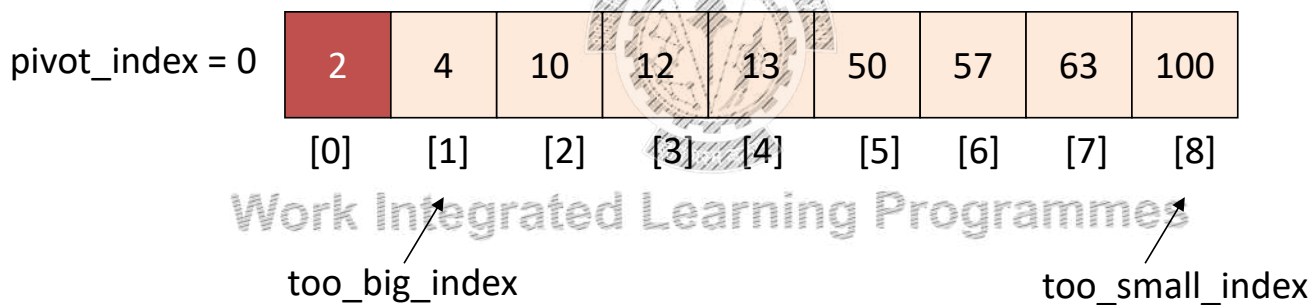
$$T(n) = \begin{cases} 0 & \text{if } n = 1 \text{ (sorting not required)} \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n & \text{otherwise} \end{cases} \quad \rightarrow \quad \Theta(n \log_2 n)$$

34

Quick Sort Worst Case Analysis



- Assume first element is chosen as pivot.
- Assume we get array that is already in order:



Quick Sort Worst Case Analysis



- 1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

$\text{pivot_index} = 0$

2	4	10	12	13	50	57	63	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

Quick Sort Worst Case Analysis



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

$\text{pivot_index} = 0$

2	4	10	12	13	50	57	63	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

37

Quick Sort Worst Case Analysis



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
- 2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

$\text{pivot_index} = 0$

2	4	10	12	13	50	57	63	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

too_big_index

too_small_index

38

Quick Sort Worst Case Analysis



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
- 3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

$\text{pivot_index} = 0$

2	4	10	12	13	50	57	63	100
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

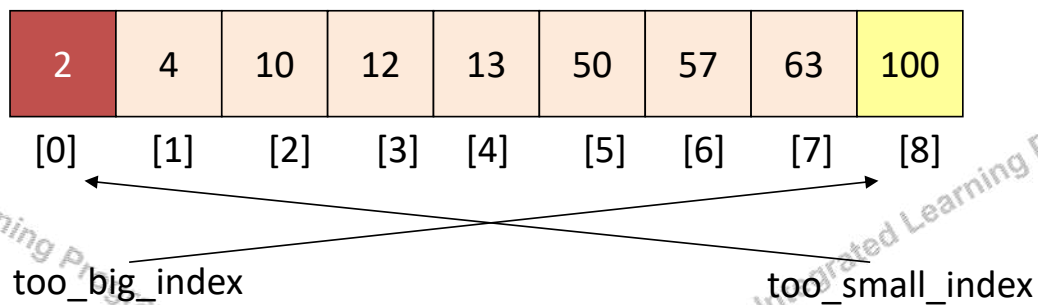
too_big_index

too_small_index

Quick Sort Worst Case Analysis



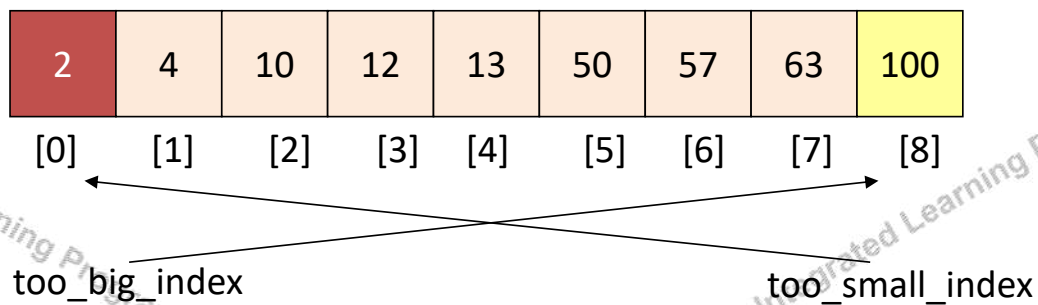
1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
- 4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$



Quick Sort Worst Case Analysis



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
- 5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$



Quick Sort Worst Case Analysis



1. While $\text{data}[\text{too_big_index}] \leq \text{data}[\text{pivot}]$
 - ++too_big_index
2. While $\text{data}[\text{too_small_index}] > \text{data}[\text{pivot}]$
 - --too_small_index
3. If $\text{too_big_index} < \text{too_small_index}$
 - swap $\text{data}[\text{too_big_index}]$ and $\text{data}[\text{too_small_index}]$
4. While $\text{too_small_index} > \text{too_big_index}$, go to 1.
- 5. Swap $\text{data}[\text{too_small_index}]$ and $\text{data}[\text{pivot_index}]$

Work Integrated Learning Programmes

pivot_index = 0	2	4	10	12	13	50	57	63	100
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

← $\leq \text{data}[\text{pivot}]$ $> \text{data}[\text{pivot}]$ →

Quick Sort Worst Case Analysis



- Assume that keys are random, uniformly distributed.
- Best case running time: $O(n \log_2 n)$
- Worst case running time?

- Recursion:

1. Partition splits array in two sub-arrays:

- one sub-array of size 0
- the other sub-array of size $n-1$

2. Quicksort each sub-array

- Depth of recursion tree? **$O(n)$**
- Number of accesses per partition? **$O(n)$**
- Overall : **$O(n^2)$**

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(0) + T(n-1) + n & \text{otherwise} \end{cases} \quad \text{➤ } \mathbf{O(n^2)}$$

Quick Sort Worst Case Analysis



- Assume that keys are random, uniformly distributed.
- Best case running time: $\Theta(n \log_2 n)$
- Worst case running time? $O(n^2)$
- How to avoid the worst case ?
 - One approach is to improve the pivot selection :
 - Pick median value of three elements from data array data[0], data[n/2], and data[n-1]. Use this median value as pivot.
- Use Randomization!

Randomized Quick Sort



- The goal of randomized quick sort is to decrease the chances of getting the worst case complexity.
- In the below example, quick sort will perform poorly and complexity would be $O(n^2)$.

Example :

1	2	3	4	5	6
---	---	---	---	---	---

- In randomized Quick sort, either make the input randomized by shuffling the input or by choosing the pivot randomly.
- Even with Randomized Quick sort, there can be a situation that the complexity is $O(n^2)$ but experientially it is proven that that case is rare! Probabilistically also its rare !! (Try to think ...)
- *A randomized algorithm is an algorithm that employs a degree of randomness as part of its logic. More on it in your next semester!*

45

innovate achieve lead



innovate achieve lead

compute $a + b$.

$O(n)$ bit operations.

Work Integrated Learning Program

$\Theta(n^2)$ bit operations.

Integer Multiplication

To multiply two 2-digit integers:

- Multiply *four* 1-digit integers.
- Add, shift some 2-digit integers to obtain result.

$$\begin{aligned}
 x &= 10 \cdot x_1 + x_0 \\
 y &= 10 \cdot y_1 + y_0 \\
 xy &= (10 \cdot x_1 + x_0)(10 \cdot y_1 + y_0) \\
 &= 100 \cdot x_1 y_1 + 10 \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0
 \end{aligned}$$



4	5	y_1	y_0
3	2	x_1	x_0
1	0	$x_0 \cdot y_0$	
0	8	$x_0 \cdot y_1$	
1	5	$x_1 \cdot y_0$	
1	2	$x_1 \cdot y_1$	
1	4	4	0

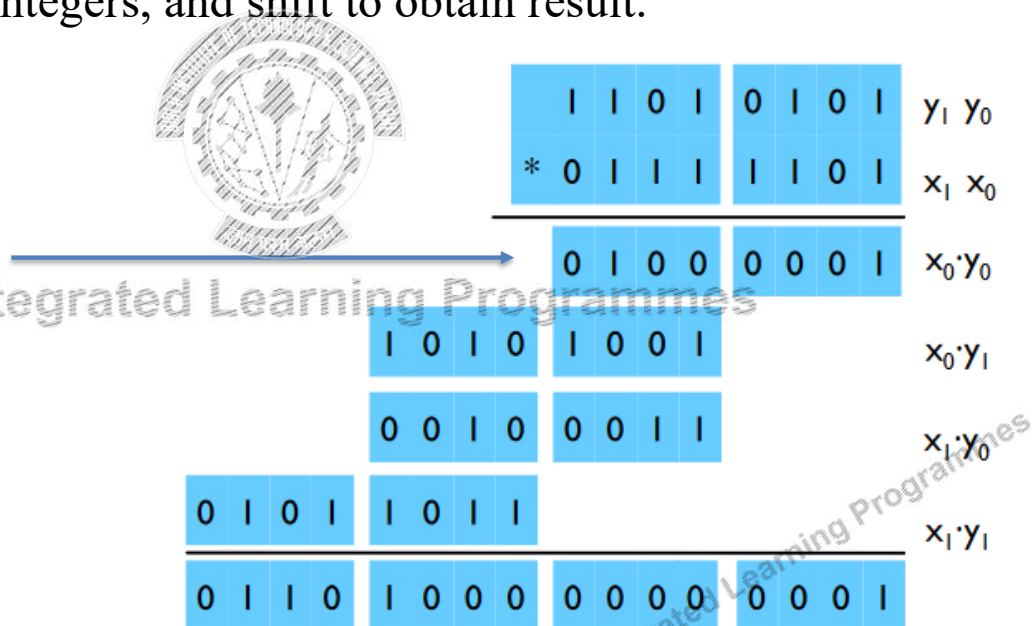
Same idea works for long integers – Can split them into 4 half-sized integers !!

Integer Multiplication



To multiply two n -bit integers:

- Multiply *four* $\frac{1}{2}n$ -bit integers.
- Add two $\frac{1}{2}n$ -bit integers, and shift to obtain result.



Integer Multiplication



To multiply two n -bit integers:

- Multiply *four* $\frac{1}{2}n$ -bit integers.
- Add two $\frac{1}{2}n$ -bit integers, and shift to obtain result.

$$\begin{aligned}
 x &= 2^{n/2} \cdot x_1 + x_0 \\
 y &= 2^{n/2} \cdot y_1 + y_0 \\
 xy &= (2^{n/2} \cdot x_1 + x_0)(2^{n/2} \cdot y_1 + y_0) \\
 &= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0
 \end{aligned}$$

	1	1	0	1	0	1	0	1	$y_1 y_0$
*	0	1	1	1	1	1	0	1	$x_1 x_0$
	0	1	0	0	0	0	0	1	$x_0 y_0$
	1	0	1	0	1	0	0	1	$x_0 y_1$
	0	0	1	0	0	0	1	1	$x_1 y_0$
	0	1	0	1	1	0	1	1	$x_1 y_1$
	0	1	1	0	1	0	0	0	1

$$T(n) = \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, shift}} \Rightarrow T(n) = \Theta(n^2)$$

Integer Multiplication Key Trick



Key trick: 2 multiplies for the price of 1

$$\begin{aligned}x &= 2^{n/2} \cdot x_1 + x_0 \\y &= 2^{n/2} \cdot y_1 + y_0 \\xy &= (2^{n/2} \cdot x_1 + x_0)(2^{n/2} \cdot y_1 + y_0) \\&= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0\end{aligned}$$

Instead of 4 subproblems, we only need 3 (with the help of clever insight).

Work Integrated Learning Programmes

$$\begin{aligned}\alpha &= x_1 + x_0 \\ \beta &= y_1 + y_0 \\ \alpha\beta &= (x_1 + x_0)(y_1 + y_0) \\ &= x_1 y_1 + (x_1 y_0 + x_0 y_1) + x_0 y_0 \\ (x_1 y_0 + x_0 y_1) &= \alpha\beta - x_1 y_1 - x_0 y_0\end{aligned}$$

50

Karatsuba Multiplication



To multiply two n -bit integers:

- Add two $\frac{1}{2}n$ bit integers.
- Multiply three $\frac{1}{2}n$ -bit integers.
- Add, subtract, and shift $\frac{1}{2}n$ -bit integers to obtain result.

$$\begin{aligned}x &= 2^{n/2} \cdot x_1 + x_0 \\y &= 2^{n/2} \cdot y_1 + y_0 \\xy &= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0 \\&= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot ((x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0) + x_0 y_0\end{aligned}$$

ABACC

$$\begin{aligned}T(n) &= 3 T(n/2) + O(n) \\&= O(n^{\log 3}) \\&= \mathbf{O(n^{1.584...})}\end{aligned}$$

Karatsuba Multiplication Simplified



Three subproblems:

$$- a = x_H y_H$$

$$- d = x_L y_L$$

$$- e = (x_H + x_L) (y_H + y_L) - a - d$$

$$\text{Then } xy = a r^n + e r^{n/2} + d$$

Where, r is the radix/base of the number system.

H/1 stands for higher order terms

L/0 stands for lower order terms

Worked Example



Compute $1234 * 4321$ using Karatsuba's Insight!

Solution:

Firstly, $X = 1234$ and $Y = 4321$

$X_H = 12$, $X_L = 34$ and $Y_H = 43$, $Y_L = 21$

○ $a_1 = 12 * 43$

○ $d_1 = 34 * 21$

○ $e_1 = (12+34) * (43+21) - a_1 - d_1$
 $= 46 * 64 - a_1 - d_1$

➤ Need to recursively solve these !! We have 3 subproblems, a_1 , d_1 and e_1

$$- a = x_H y_H$$

$$- d = x_L y_L$$

$$- e = (x_H + x_L) (y_H + y_L) - a - d$$

$$\text{Then } xy = a r^n + e r^{n/2} + d$$

Worked Example



Solving for 1st subproblem a_1 :

Firstly, $X = 12$ and $Y = 43$

$X_H = 1$, $X_L = 2$ and $Y_H = 4$, $Y_L = 3$

$$a_1 = 12 * 43$$

$$\begin{aligned} - a &= x_H y_H \\ - d &= x_L y_L \\ - e &= (x_H + x_L) (y_H + y_L) - a - d \end{aligned}$$

Then $xy = a r^n + e r^{n/2} + d$

Subproblems:

○ $a_2 = 1 * 4 = 4$

○ $d_2 = 2 * 3 = 6$

○ $e_2 = (1+2)(4+3) - a_2 - d_2$
 $= 21 - 4 - 6 = 11$

Solution to $a_1 = 4 * 10^2 + 11 * 10 + 6$ [*10 because base is decimal, $n=2$ as its two digit multiplication*]

$$= 516$$

Worked Example



Solving for 2nd subproblem d_1 :

Firstly, $X = 34$ and $Y = 21$

$X_H = 3$, $X_L = 4$ and $Y_H = 2$, $Y_L = 1$

$$d_1 = 34 * 21$$

$$- a = x_H y_H$$

$$- d = x_L y_L$$

$$- e = (x_H + x_L) (y_H + y_L) - a - d$$

$$\text{Then } xy = a r^n + e r^{n/2} + d$$

Subproblems:

○ $a_2 = 3 * 2 = 6$

○ $d_2 = 4 * 1 = 4$

○ $e_2 = (3+4)(2+1) - a_2 - d_2$
 $= 21 - 6 - 4 = 11$

Solution to $d_1 = 6 * 10^2 + 11 * 10 + 4$ [10 because base is decimal,
 $n=2$ as its two digit multiplication]

$$= 714$$

Worked Example



Solving for 3rd subproblem e_1 :

Firstly, $X = 46$ and $Y = 64$

$X_H = 4$, $X_L = 6$ and $Y_H = 6$, $Y_L = 4$

$e_1 = 46 * 64 - a_1 - d_1$ (where a_1, d_1 are solutions of previous subproblems)

Subproblems:

○ $a_2 = 4 * 6 = 24$

○ $d_2 = 6 * 4 = 24$

○ $e_2 = (4+6)(6+4) - a_2 - d_2$
 $= 100 - 24 - 24$
 $= 52$

Solution to $e_1 = [24 * 10^2 + 52 * 10 + 24] - 516 - 714$
 $= 1714$

– $a = x_H y_H$

– $d = x_L y_L$

– $e = (x_H + x_L)(y_H + y_L) - a - d$

Then $xy = a r^n + e r^{n/2} + d$

Worked Example



Solving for final answer:

$$\begin{aligned} - a &= x_H y_H \\ - d &= x_L y_L \\ - e &= (x_H + x_L) (y_H + y_L) - a - d \\ \text{Then } xy &= a r^n + e r^{n/2} + d \end{aligned}$$

Solution of 1st Subproblem

Solution of 3rd Subproblem

$$1234 * 4321 = 516 * 10^4 + 1714 * 10^2 + 714$$
$$= 5,332,114$$

Solution of 2nd Subproblem

Integer Multiplication Algorithm



function multiply(x, y)

Input: Positive integers x and y , in binary

Output: Their product

$n = \max(\text{size of } x, \text{size of } y)$

if $n = 1$: return xy

$x_L, x_R = \text{leftmost } \lfloor n/2 \rfloor, \text{ rightmost } \lfloor n/2 \rfloor \text{ bits of } x$

$y_L, y_R = \text{leftmost } \lfloor n/2 \rfloor, \text{ rightmost } \lfloor n/2 \rfloor \text{ bits of } y$

$P_1 = \text{multiply}(x_L, y_L)$

$P_2 = \text{multiply}(x_R, y_R)$

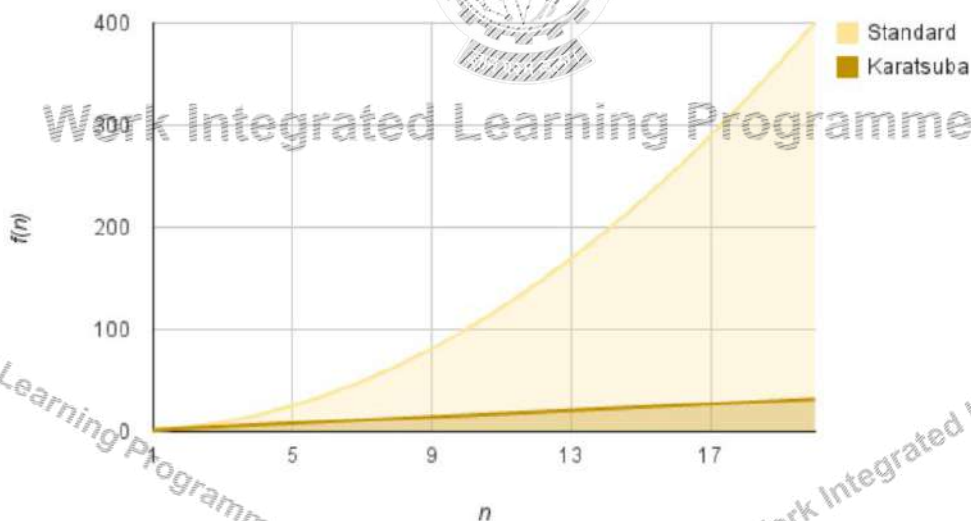
$P_3 = \text{multiply}(x_L + x_R, y_L + y_R)$

return $P_1 \times 2^n + (P_3 - P_1 - P_2) \times 2^{n/2} + P_2$

Power of Karatsuba's Algo



- Assuming that we replace two of the multiplications with only one makes the program faster. The question is how fast. Karatsuba improves the multiplication process by replacing the initial complexity of $O(n^2)$ by $O(n^{1.59})$, which as you can see on the diagram below is much faster for big n .



Exercises Set 1



Exercise-1:

Assume that the sequence to be sorted is represented as a doubly linked list.

Given this,

- (a) What is the complexity of divide step?
- (b) What is the complexity of merge step?
- (c) Characterize the performance of merge sort on doubly linked list as a recurrence & solve it.
- (d) Characterize the result obtained in (c) using Big Oh notation

Exercise-2:

Assume you are given a sequence of keys with duplicates. How will you adapt merge sort algorithm to produce sorted array as output which has no duplicate entries? Write the modified algorithm. What is its complexity?

Exercise Set 2



➤ Apply Merge sort and quick sort on the following numbers and show the tracing :

a) 10,76,83,2,1,68,94,1003,368,53

b) 45,68,21,6,-6,2,78,3,6,457,1



Work Integrated Learning Programmes

Motivation for DP



- What is “ $1+1+1+1+1=$ ” ?
- Counting “*Five !*”
- I add another “ $1+$ ” on the left.
- “What about that answer now?”
- “*Six!*”
- “How did you answer that it’s six so fast ?”
- “You just added one more!” “So you didn’t need to recount because you remembered there were Five!”
- Dynamic Programming is just a fancy way to say: remembering things to save time later!”

Those who cannot remember the past are condemned to repeat it.

Introduction to Dynamic Programming



- Dynamic programming is a method of solving the problem with overlapping subproblems. In this approach, we precompute and **store** similar subproblem solutions to build the solution of a complex problem.
- This method works by dividing the problem into sub-problem and getting the solution for the sub-problems using which the solution for the given problem can be obtained.
- Once a sub-problem is solved, the result is stored in a table and never recalculated. When a sub-problem of earlier instance is encountered, instead of recalculating the values, it is simply retrieved from the table thus saving time.
- *Note: The word programming in dynamic programming has no particular connection to computer programming. Here, programming is the process of obtaining the plan to get the optimal solution.*

Where do we need Dynamic Programming?



- If we are given a problem that can be broken down into smaller sub-problems and these smaller sub-problems can still be broken into smaller ones, and if we manage to find out that there are some **over-lapping** sub-problems, then we've encountered a DP problem.
- Dynamic Programming combines solutions to sub-problems. It is mainly used when solutions of the **same** subproblems are needed again and again. In this, computed solutions to subproblems are stored in a table so that these don't have to be recomputed.
- So Dynamic Programming is not useful when there are no common (overlapping) subproblems because there is no point in storing the solutions if they are not needed again. For example, Binary Search does not have an overlapping sub-problem. Whereas the recursive program of Fibonacci numbers has many overlapping sub-problems!
- DP can be used when the sub-problems are dependent. When they are independent subproblems, we can use D&C 😊
- The core idea of dynamic programming is to avoid repeated work by remembering partial results.

64

Applications of DP Strategy



➤ Areas :

- Bioinformatics.
- Control theory.
- Information theory.
- Operations research.
- Computer science: theory, graphics, AI, systems,



Some Common Problems solved using DP



- Fibonacci Series
- Transitive Closure – Warshall's Algorithm
- All pair Shortest path – Floyd Warshall's Algorithm
- 0/1 Knapsack Problem
- Matrix Chain Multiplication
- Sum of subset problem
- Binomial co-efficient
- Longest common Sequence
- Bellman Ford Algorithm
- Viterbi for hidden Markov models.
- Unix diff for comparing two files.
- And many more

Self-Study & Next Semesters!

Fibonacci Series



$$\text{Fib}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

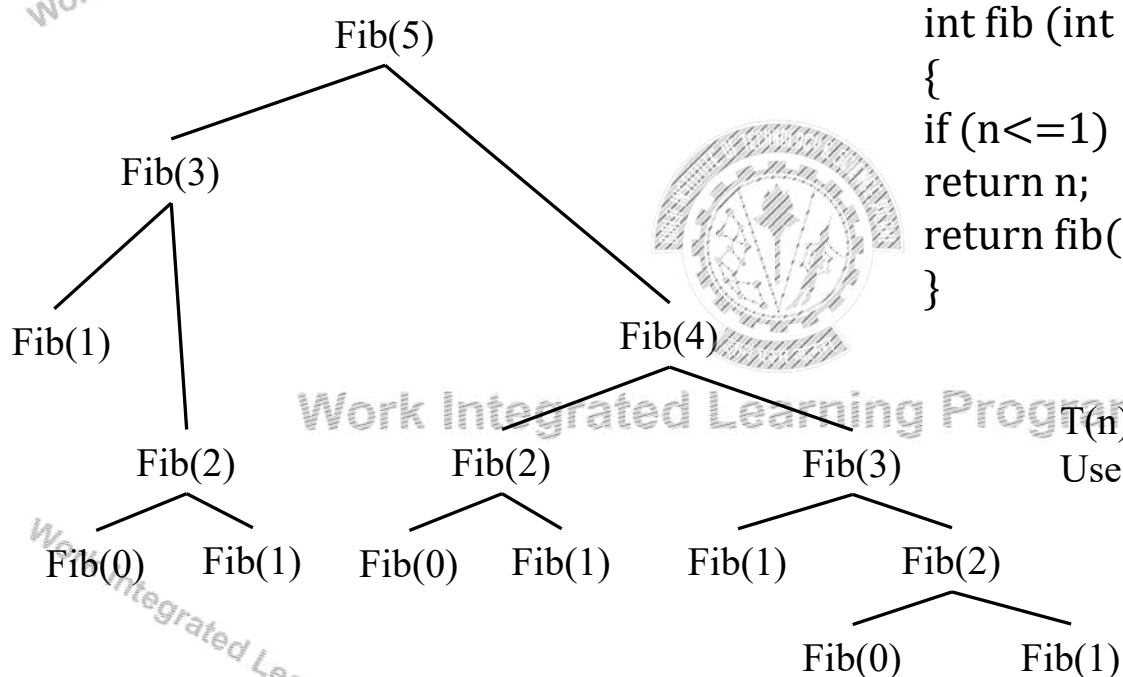
```
int fib (int n)
{
if (n<=1)
return n;
return fib(n-2)+fib(n-1);
}
```

The Fibonacci Sequence

1,1,2,3,5,8,13,21,34,55,89,144,233,377...

1+1=2	13+21=34
1+2=3	21+34=55
2+3=5	34+55=89
3+5=8	55+89=144
5+8=13	89+144=233
8+13=21	144+233=377

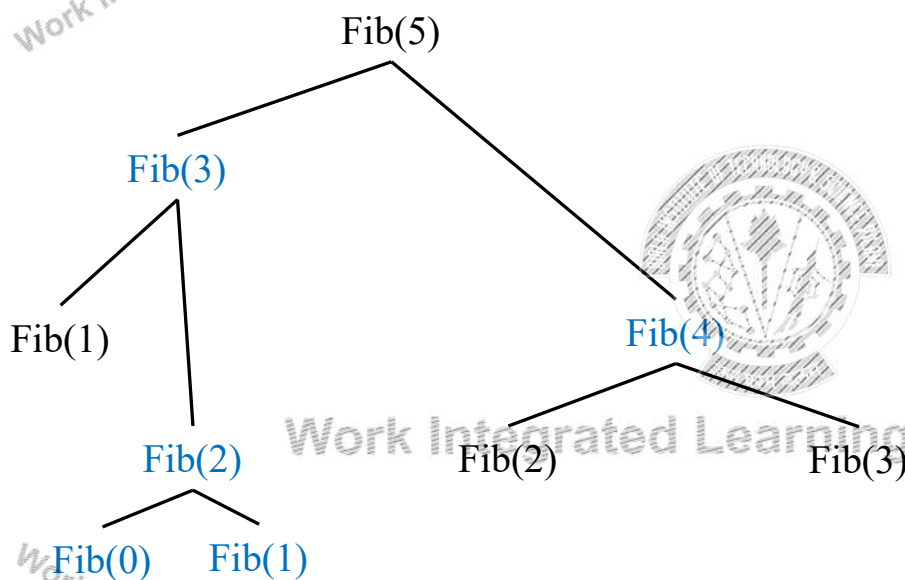
Fibonacci Series



```
int fib (int n)
{
    if (n<=1)
        return n;
    return fib(n-2)+fib(n-1);
}
```

$T(n) = 2T(n-1) + 1$
Use master theorem = $O(2^n)$

Recursion with Memoization



```
int fib (int n)
{
    if (n<=1)
        return n;
    If  $f_n$  is in memory return  $f_n$ 
    else
         $f_n \leftarrow \text{fib}(n-2) + \text{fib}(n-1)$ ;
        Save  $f_n$  in memory
    return  $f_n$ 
}
```

From the above figure it can be seen, subtrees that correspond to subproblems that have already been solved are pruned from this recursive call tree. The time complexity is thus **$O(n)$** . In short, **DP = recursion + re-use**

Steps of Dynamic Programming



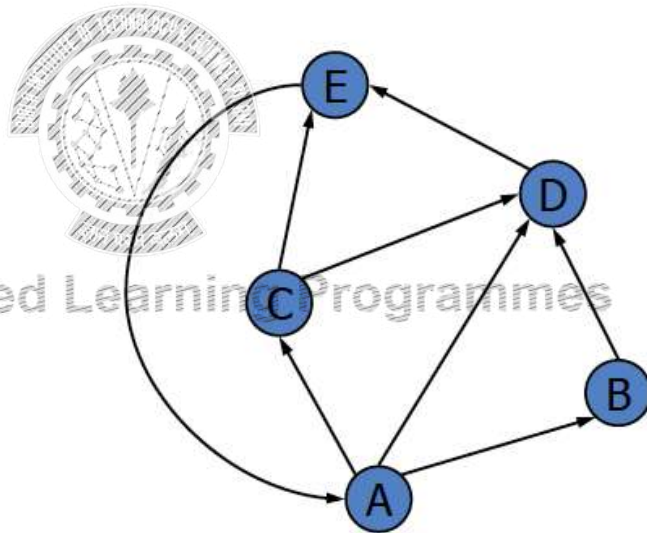
- Characterize the structure of an optimal solution
 - Recursively define the value of an optimal solution
 - Compute the optimal solution for all smaller sub-problems in a bottom-up approach (preferably by using a table).
 - Construct an optimal solution from computed information
-
- The common theme of any problem solved using dynamic programming is to computing solutions for smaller sub-problems and filling it into the table for future reference (while solving other subproblems).

Directed Graph



- A **digraph** is a graph whose edges are all directed ..
- Short for “directed graph”
- Applications

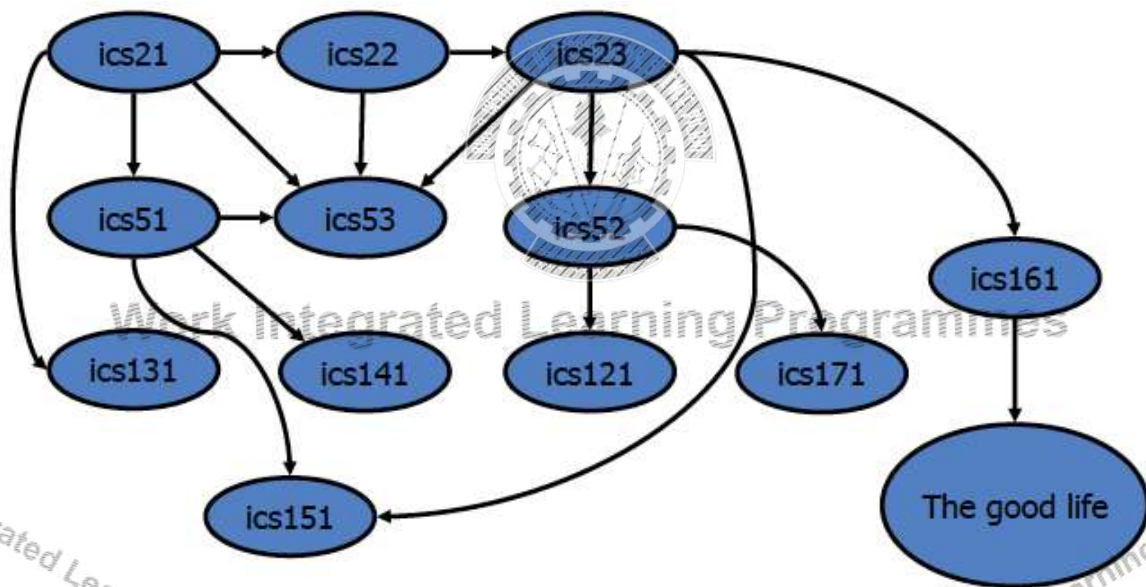
- One-way streets
- Flights
- Task scheduling



Digraph Application



Task / Job : Scheduling: Edge(a, b) means task ***a*** must be completed before ***b*** can be started.

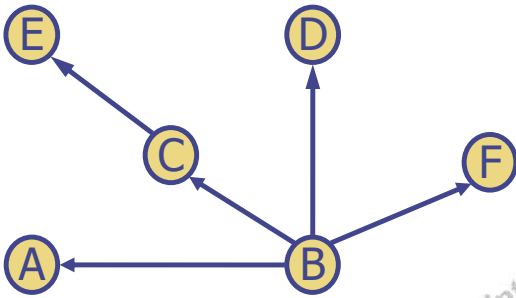
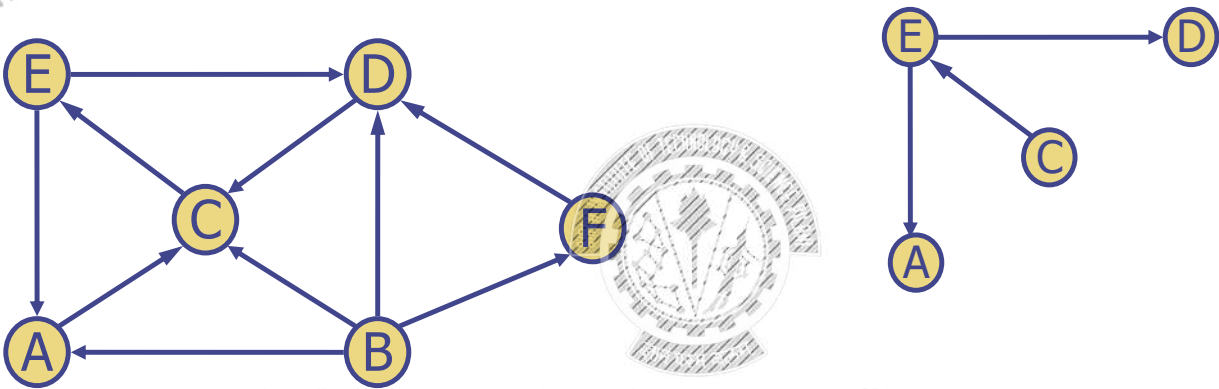


Reachability



- A fundamental issue with directed graphs is the notion of reachability
- Reachability, which deals with determining where we can get to in a directed graph.
- Given vertices u and v of a digraph G , we say that u **reaches** v (and v is **reachable** from u) if G has a directed path from u to v .
- A vertex v reaches an edges (w,z) if v reaches the origin vertex w of the edge .

Reachability Examples



Reachability



- Interesting problems that deal with reachability in a digraph G including the following
 - *Given vertices u and v , determine whether u reaches v ?*
- Find all the vertices of G that are reachable from a given vertex S .
- Determine whether G is strongly connected.
- Compute the transitive closure G^* of G .

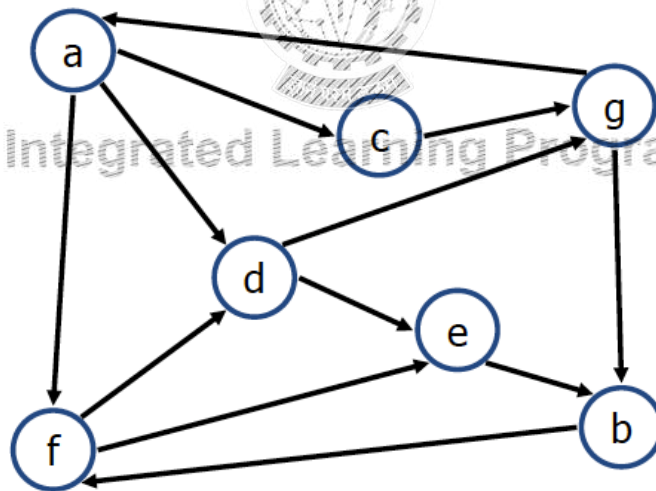
Strongly connected



- A digraph G is strongly connected, if for any two vertices u and v of G :

u reaches v **and** v reaches u .

- Each vertex can reach all other vertices:



Transitive closure



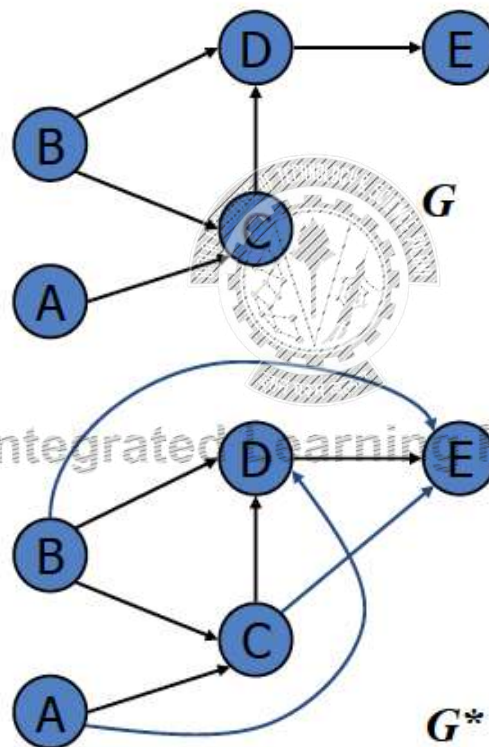
➤ The transitive closure provides reachability information about a digraph.

➤ Given a digraph G , the transitive closure of G is the digraph G^* such that

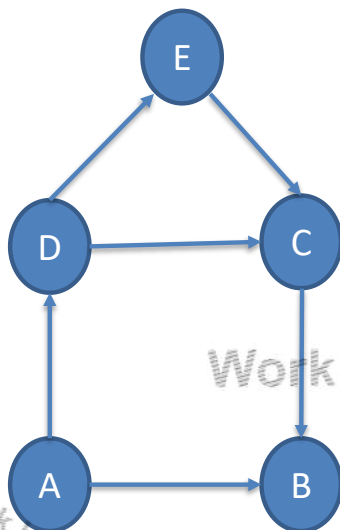
✓ G^* has the same vertices as G

✓ If G has a directed path from u to v ($u \neq v$), G^* has a directed edge from u to v .

Transitive closure (Example)



Transitive Closure & Reachability Matrix



Adjacency Matrix A :



	A	B	C	D	E
A	0	1	0	1	0
B	0	0	0	0	0
C	0	1	0	0	0
D	0	0	1	0	1
E	0	0	1	0	0

Reachability Matrix/
Transitive Closure T :

	A	B	C	D	E
A	1	1	1	1	1
B	0	1	0	0	0
C	0	1	1	0	0
D	0	1	1	1	1
E	0	1	1	0	1

Computing the transitive Closure



- We can perform DFS starting at each vertex to see which vertices w are reachable from v , adding an edge (v, w) to the transitive closure for each such W .
 - $O(n(n+m))$
- Use Warshall's algorithm.

If there's a way to get from **A** to **B** and from **B** to **C**, then there's a way to get from **A** to **C**.





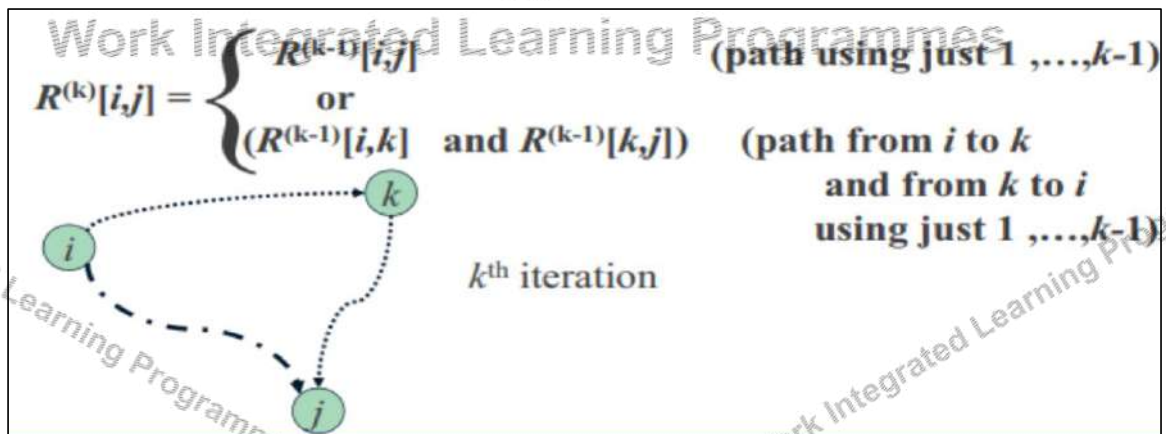
Floyd-Warshall Transitive Closure

- Constructs the transitive closure of a given directed graph by a sequence of matrices :

$$R^0, R^1, R^2, R^3, \dots, R^{(n-1)}, R^n$$

- ✓ each of them are of order $n \times n$ where
- ✓ 'n' - number of vertices in digraph.

- R^n – Transitive closure.
- Vertices are numbered from 1 to 'n'.



Floyd-Warshall Transitive Closure Formula for Matrix Generation



$$R^k[i,j] = R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$$

So,

k=0, Adjacency Matrix

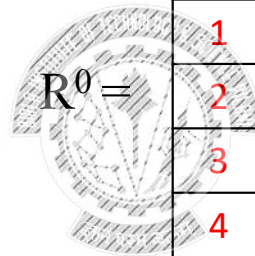
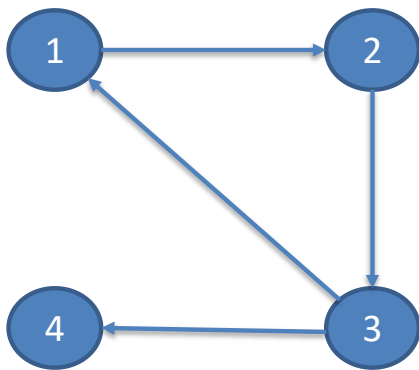
$$K=1, R^1[i,j] = R^{(0)}[i,j] \text{ or } (R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$$

$$K=2, R^2[i,j] = R^{(1)}[i,j] \text{ or } (R^{(1)}[i,2] \text{ and } R^{(1)}[2,j])$$

$$K=3, R^3[i,j] = R^{(2)}[i,j] \text{ or } (R^{(2)}[i,3] \text{ and } R^{(2)}[3,j])$$

-
-
-

Example



$R^0 =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	0

R^0 is the adjacency matrix of the digraph.



Example – Generate R^1 (Row1)

Now find R^1 Matrix !

$k=1$, $R^1[i,j] = R^{(0)}[i,j]$ or $(R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$

$$R^0 =$$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	0

$$R^{(1)} =$$

	1	2	3	4
1	0	1	0	0
2				
3				
4				

- ✓ $R^1[1,1] = R^{(0)}[1,1]$ (false). So, Check condition 2, $(R^{(0)}[1,1] \text{ and } R^{(0)}[1,1])$ are not 1 in R^0 Hence, $R^1[1,1] = 0$
- ✓ $R^1[1,2] = R^{(0)}[1,2]$ (true) . Hence, $R^1[1,2] = 1$
- ✓ $R^1[1,3] = R^{(0)}[1,3]$ (false). So, Check condition 2, $(R^{(0)}[1,1] \text{ and } R^{(0)}[1,3])$ are not 1 in R^0 Hence, $R^1[1,3] = 0$
- ✓ $R^1[1,4] = R^{(0)}[1,4]$ (false). So, Check condition 2, $(R^{(0)}[1,1] \text{ and } R^{(0)}[1,4])$ are not 1 in R^0 Hence, $R^1[1,4] = 0$



Example – Generate R^1 (Row 2)

Now find R^1 Matrix !

$k=1$. $R^1[i,i] = R^{(0)}[i,i]$ or $(R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$

$$R^0 =$$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	0

$$R^{(1)} =$$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3				
4				

- ✓ $R^1[2,1] = R^{(0)}[2,1]$ (false). So, Check condition 2, $(R^{(0)}[2,1] \text{ and } R^{(0)}[1,1])$ are not 1 in R^0 Hence, $R^1[2,1] = 0$
- ✓ $R^1[2,2] = R^{(0)}[2,2]$ (false). So, Check condition 2, $(R^{(0)}[2,1] \text{ and } R^{(0)}[1,2])$ are not 1 in R^0 Hence, $R^1[2,2] = 0$
- ✓ $R^1[2,3] = R^{(0)}[2,3]$ (true). Hence, $R^1[2,3] = 1$
- ✓ $R^1[2,4] = R^{(0)}[2,4]$ (false). So, Check condition 2, $(R^{(0)}[2,1] \text{ and } R^{(0)}[1,4])$ are not 1 in R^0 Hence, $R^1[2,4] = 0$

86



Example – Generate R^1 (Row 3)

Now find R^1 Matrix !

$k=1, R^1[i,j] = R^{(0)}[i,j]$ or $(R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$

$R^0 =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	0

$R^{(1)} =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	1	0	1
4				

- ✓ $R^1[3,1] = R^{(0)}[3,1]$ (true). Hence, $R^1[3,1] = 1$
- ✓ $R^1[3,2] = R^{(0)}[3,2]$ (false). So, Check condition 2, $(R^{(0)}[3,1] \text{ and } R^{(0)}[1,2])$ are 1 in R^0 Hence, $R^1[3,2] = 1$
- ✓ $R^1[3,3] = R^{(0)}[3,3]$ (false). So, Check condition 2, $(R^{(0)}[3,1] \text{ and } R^{(0)}[1,3])$ are not 1 in R^0 Hence, $R^1[3,3] = 0$
- ✓ $R^1[3,4] = R^{(0)}[3,4]$ (true). Hence, $R^1[3,4] = 1$



Example – Generate R^1 (Row 4)

Now find R^1 Matrix !

$k=1, R^1[i,j] = R^{(0)}[i,j]$ or $(R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$

$R^0 =$					$R^{(1)} =$				
	1	2	3	4		1	2	3	4
1	0	1	0	0	1	0	1	0	0
2	0	0	1	0	2	0	0	1	0
3	1	0	0	1	3	1	1	0	1
4	0	0	0	0	4	0	0	0	0

- ✓ $R^1[4,1] = R^{(0)}[4,1]$ (false). So, Check condition 2, $(R^{(0)}[4,1] \text{ and } R^{(0)}[1,1])$ are not 1 in R^0 Hence, $R^1[4,1] = 0$
- ✓ $R^1[4,2] = R^{(0)}[4,2]$ (false). So, Check condition 2, $(R^{(0)}[4,1] \text{ and } R^{(0)}[1,2])$ are not 1 in R^0 Hence, $R^1[4,2] = 0$
- ✓ $R^1[4,3] = R^{(0)}[4,3]$ (false). So, Check condition 2, $(R^{(0)}[4,1] \text{ and } R^{(0)}[1,3])$ are not 1 in R^0 Hence, $R^1[4,3] = 0$
- ✓ $R^1[4,4] = R^{(0)}[4,4]$ (false). So, Check condition 2, $(R^{(0)}[4,1] \text{ and } R^{(0)}[1,4])$ are not 1 in R^0 Hence, $R^1[4,4] = 0$

Example – Generate R^2 (Row 1)



Now find R^2 Matrix

$k=2$, $R^2[i,j] = R^{(1)}[i,j]$ or $(R^{(1)}[i,2] \text{ and } R^{(1)}[2,j])$

$R^{(1)} =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	1	0	1
4	0	0	0	0

$R^{(2)} =$

	1	2	3	4
1	0	1	1	0
2				
3				
4				

- ✓ $R^2[1,1] = R^{(1)}[1,1]$ (false). So, Check condition 2, $(R^{(1)}[1,2] \text{ and } R^{(1)}[2,1])$ are not 1 in R^1) Hence, $R^2[1,1] = 0$
- ✓ $R^2[1,2] = R^{(1)}[1,2]$ (true) . Hence, $R^2[1,2] = 1$
- ✓ $R^2[1,3] = R^{(1)}[1,3]$ (false). So, Check condition 2, $(R^{(1)}[1,2] \text{ and } R^{(1)}[2,3])$ are 1 in R^1) Hence, $R^2[1,3] = 1$
- ✓ $R^2[1,4] = R^{(1)}[1,4]$ (false). So, Check condition 2, $(R^{(1)}[1,2] \text{ and } R^{(1)}[2,4])$ are not 1 in R^1) Hence, $R^2[1,4] = 0$

89

Example – Generate R^2

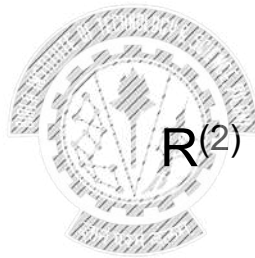


Now find R^2 Matrix

$k=2$, $R^2[i,j] = R^{(1)}[i,j]$ or $(R^{(1)}[i,2] \text{ and } R^{(1)}[2,j])$

$R^{(1)} =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	1	0	1
4	0	0	0	0



$R^{(2)} =$

	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	1	1	1	1
4	0	0	0	0

Similarly, we calculate and fill all the cells of $R^{(2)}$

Tip: Try them offline and verify the $R^{(2)}$ matrix that we have got here 😊



Example – Generate R^3 (Row1)

Now find R^3 Matrix

$k=3$, $R^3[i,j] = R^{(2)}[i,j]$ or $(R^{(2)}[i,3]$ and $R^{(2)}[3,j])$

	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	1	1	1	1
4	0	0	0	0

 $R^{(2)} =$

	1	2	3	4
1	1	1	1	1
2				
3				
4				

 $R^{(3)} =$

- ✓ $R^3[1,1] = R^{(2)}[1,1]$ (false). So, Check condition 2, $(R^{(2)}[1,3]$ and $R^{(2)}[3,1]$ are 1 in $R^{(2)}$) Hence, $R^3[1,1] = 1$
- ✓ $R^3[1,2] = R^{(2)}[1,2]$ (true). Hence, $R^3[1,2] = 1$
- ✓ $R^3[1,3] = R^{(2)}[1,3]$ (true). Hence, $R^3[1,3] = 1$
- ✓ $R^3[1,4] = R^{(2)}[1,4]$ (false). So, Check condition 2, $(R^{(2)}[1,3]$ and $R^{(2)}[3,4]$ are 1 in $R^{(2)}$) Hence, $R^3[1,4] = 1$

91

Example – Generate R^3



Now find R^3 Matrix

$$k=3, R^3[i,j] = R^{(2)}[i,j] \text{ or } (R^{(2)}[i,3] \text{ and } R^{(2)}[3,j])$$

$$R^{(2)} =$$

	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	1	1	1	1
4	0	0	0	0

$$R^{(3)} =$$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

Similarly, we calculate and fill all the cells of $R^{(3)}$

Tip : Try them offline and verify the $R^{(3)}$ matrix that we have got here 😊

Example – Generate R^4



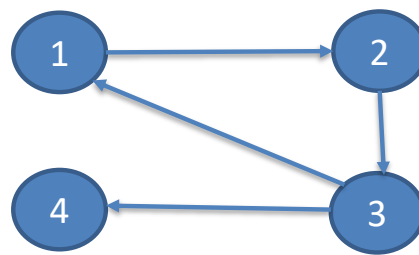
Now find R^4 Matrix

$$k=4, R^4[i,j] = R^{(3)}[i,j] \text{ or } (R^{(3)}[i,4] \text{ and } R^{(3)}[4,j])$$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

 $R^{(3)} =$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

 $R^{(4)} =$ 

Warshall's Algorithm



Algorithm: warshall (A[1..n,1..n])

//implements warshall's algorithm for computing the transitive closure.

//input: the adjacency matrix A of a digraph with n-vertices

//output: the transitive closure of the digraph.

$R^{(0)} \leftarrow A$

for K $\leftarrow$ 1 to n do

 for i $\leftarrow$ 1 to n do

 for j $\leftarrow$ 1 to n do

$R^{(k)}[i,j] \leftarrow R^{(k-1)}[i,j]$ or $(R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$

return $R^{(n)}$

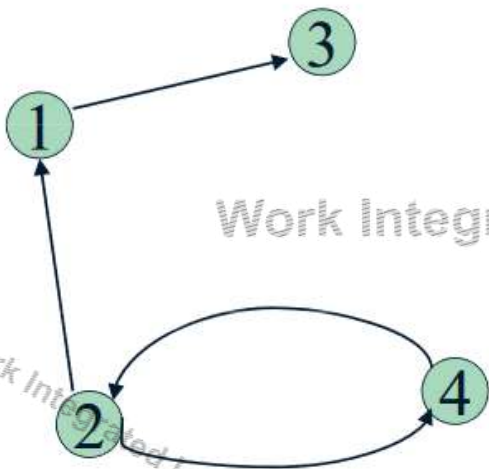
Now its your turn to find the time complexity: ☺

Exercises

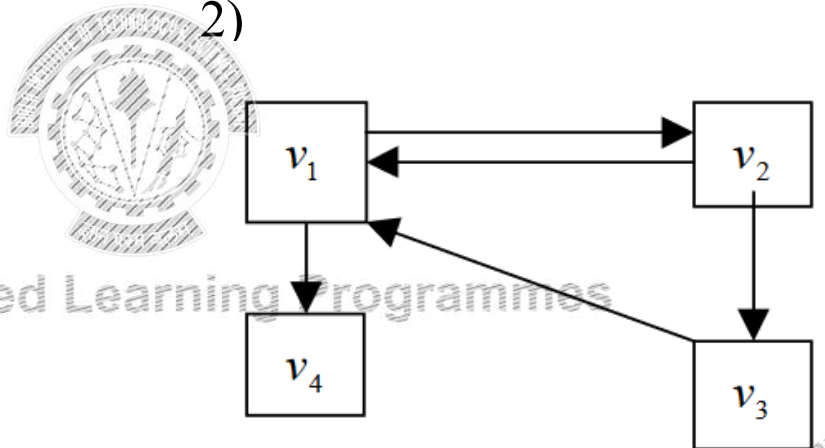


Apply the Warshall algorithm to find the transitive Closure matrix T for the following digraph's:

1)



2)





Work Integrated Learning Programmes

We will explore DP in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



97

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Contact Session #14

DSECLZG519 –Dynamic Programming

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Agenda for Session # 14



- Recap of CS#13
- Warshall's algorithm for Transitive Closure
- Floyd's algorithm for All Pair Shortest Path
- Matrix chain multiplication using DP
- Exercises

Computing the transitive Closure



- We can perform DFS starting at each vertex to see which vertices w are reachable from v , adding an edge (v, w) to the transitive closure for each such w .
 - $O(n(n+m))$
- Use Warshall's algorithm.

If there's a way to get from **A** to **B** and from **B** to **C**, then there's a way to get from **A** to **C**.





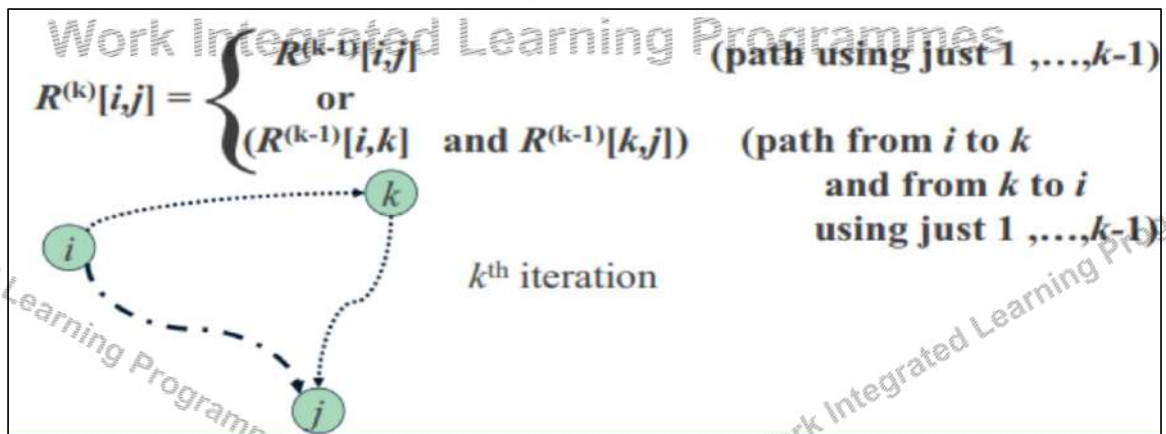
Floyd-Warshall Transitive Closure

- Constructs the transitive closure of a given directed graph by a sequence of matrices :

$$R^0, R^1, R^2, R^3, \dots, R^{(n-1)}, R^n$$

- ✓ each of them are of order $n \times n$ where
- ✓ 'n' - number of vertices in digraph.

- R^n – Transitive closure.
- Vertices are numbered from 1 to 'n'.



5

Floyd-Warshall Transitive Closure Formula for Matrix Generation



$$R^k[i,j] = R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$$

So,

k=0, Adjacency Matrix

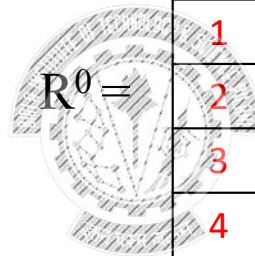
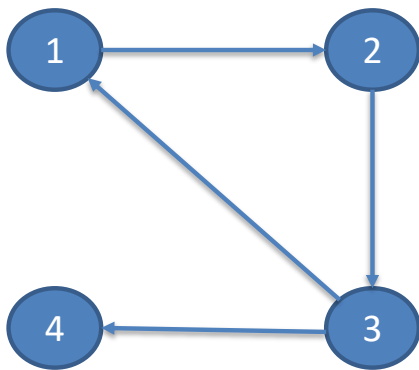
$$K=1, R^1[i,j] = R^{(0)}[i,j] \text{ or } (R^{(0)}[i,1] \text{ and } R^{(0)}[1,j])$$

$$K=2, R^2[i,j] = R^{(1)}[i,j] \text{ or } (R^{(1)}[i,2] \text{ and } R^{(1)}[2,j])$$

$$K=3, R^3[i,j] = R^{(2)}[i,j] \text{ or } (R^{(2)}[i,3] \text{ and } R^{(2)}[3,j])$$

-
-
-

Example



$R^0 =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	0	0	1
4	0	0	0	0

R^0 is the adjacency matrix of the digraph.



Example – Generate R^1

Now find R^1 Matrix !

$k=1, R^1[i,j] = R^{(0)}[i,j]$ or $(R^{(0)}[i,1]$ and $R^{(0)}[1,j])$

$R^0 =$					$R^{(1)} =$				
	1	2	3	4		1	2	3	4
1	0	1	0	0	1	0	1	0	0
2	0	0	1	0	2	0	0	1	0
3	1	0	0	1	3	1	1	0	1
4	0	0	0	0	4	0	0	0	0

- ✓ $R^1[4,1] = R^{(0)}[4,1]$ (false). So, Check condition 2, $(R^{(0)}[4,1]$ and $R^{(0)}[1,1]$ are not 1 in R^0) Hence, $R^1[4,1] = 0$
- ✓ $R^1[4,2] = R^{(0)}[4,2]$ (false) . So, Check condition 2, $(R^{(0)}[4,1]$ and $R^{(0)}[1,2]$ are not 1 in R^0) Hence, $R^1[4,2] = 0$
- ✓ $R^1[4,3] = R^{(0)}[4,3]$ (false). So, Check condition 2, $(R^{(0)}[4,1]$ and $R^{(0)}[1,3]$ are not 1 in R^0) Hence, $R^1[4,3] = 0$
- ✓ $R^1[4,4] = R^{(0)}[4,4]$ (false). So, Check condition 2, $(R^{(0)}[4,1]$ and $R^{(0)}[1,4]$ are not 1 in R^0) Hence, $R^1[4,4] = 0$

Example – Generate R^2

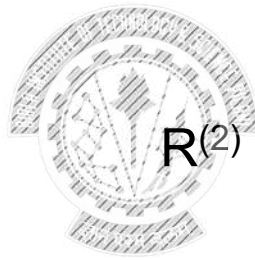


Now find R^2 Matrix

$k=2$, $R^2[i,j] = R^{(1)}[i,j]$ or $(R^{(1)}[i,2] \text{ and } R^{(1)}[2,j])$

$R^{(1)} =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	1	1	0	1
4	0	0	0	0



$R^{(2)} =$

	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	1	1	1	1
4	0	0	0	0

Similarly, we calculate and fill all the cells of $R^{(2)}$

Tip: Try them offline and verify the $R^{(2)}$ matrix that we have got here 😊

Example – Generate R^3



Now find R^3 Matrix

$$k=3, R^3[i,j] = R^{(2)}[i,j] \text{ or } (R^{(2)}[i,3] \text{ and } R^{(2)}[3,j])$$

$$R^{(2)} =$$

	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	1	1	1	1
4	0	0	0	0

$$R^{(3)} =$$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

Similarly, we calculate and fill all the cells of $R^{(3)}$

Tip : Try them offline and verify the $R^{(3)}$ matrix that we have got here 😊

Example – Generate R^4



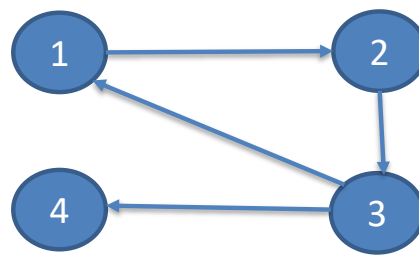
Now find R^4 Matrix

$k=4$, $R^4[i,j] = R^{(3)}[i,j]$ or $(R^{(3)}[i,4] \text{ and } R^{(3)}[4,j])$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

 $R^{(3)} =$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

 $R^{(4)} =$ 

Warshall's Algorithm



Algorithm: warshall (A[1..n,1..n])

//implements warshall's algorithm for computing the transitive closure.

//input: the adjacency matrix A of a digraph with n-vertices

//output: the transitive closure of the digraph.

$R^{(0)} \leftarrow A$

for K $\leftarrow$ 1 to n do

 for i $\leftarrow$ 1 to n do

 for j $\leftarrow$ 1 to n do

$R^{(k)}[i,j] \leftarrow R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$

return $R^{(n)}$

Now its your turn to find the time complexity: ☺



All Pair Shortest Path Problem

- Let $G = (V, E)$ be a digraph with n vertices. Let “C” be the cost adjacency matrix for G , such that $C[i, i]$ i.e. cost of self loops is 0. The $C[i, j]$ is the cost of the edge $(i, j) \in E$.
- The $C[i, j] = \infty$ if there does not exist a path between vertex i and vertex j .
- All pairs shortest path problem is to determine a matrix A such that $A[i, j]$ is the length of a shortest path from i to j .
- In other words, all pair shortest path problem is one where we want to find the shortest path from every node to every other node.
- This can be obtained greedily also! If we run the Dijkstra's algorithm by selecting every vertex as source then we can achieve this task with a complexity of $O(n^2) * n = O(n^3)$

All Pair Shortest Path Problem



- An alternate solution to the same problem can be obtained by dynamic programming strategy and the algorithm is called Floyd's algorithm or Floyd – Warshall's algorithm.
- Floyd's algorithm works on the same principles and is also referred at times as Floyd – Warshall's algorithm.

Work Integrated Learning Programmes

Warshall's Algorithm for Transitive Closure



ALGORITHM *Warshall*($A[1..n, 1..n]$)

//Implements Warshall's algorithm for computing the transitive closure

//Input: The adjacency matrix A of a digraph with n vertices

//Output: The transitive closure of the digraph

$R^{(0)} \leftarrow A$

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j] \text{ or } (R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j])$

return $R^{(n)}$

$$R^k[i,j] = R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$$

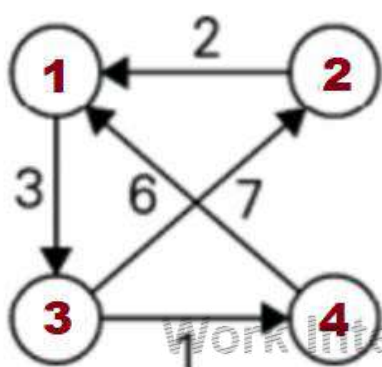
Floyd's Algorithm: All Pairs Shortest Paths



- Problem: In a weighted (di)graph, find the shortest paths between every pair of vertices.
- *Same idea:* construct solution through a series of matrices $D^{(0)}$, $D^{(1)}$, ... $D^{(n)}$ using increasing subsets of the vertices allowed as intermediate. [Note: Few books might name these matrices as A or Cost instead of D].
- On the k^{th} iteration, the algorithm determines shortest paths between every pair of vertices i, j that use only vertices among $1 \dots k$ as intermediate.

$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$

Floyd's Algorithm Example



D^0	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

D^0 is the cost -adjacency matrix of the digraph.

Floyd's Example – Matrix Generation $D^{(1)}$



Now, find D^1 Matrix !

$$D^{(k)}[i,j] = \min \{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$$

$$k=1, D^1[i,j] = \min \{ D^{(0)}[i,j], D^{(0)}[i,1] + D^{(0)}[1,j] \}$$

D^0	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

	1	2	3	4
1	0	∞	3	∞
2	2	0		
3	∞		0	
4	6			0

- ✓ $D^1[2, 3] = \min \{ D^{(0)}[2,3], D^{(0)}[2,1] + D^{(0)}[1,3] \} = \min \{ \infty, 2 + 3 \} = 5$
- ✓ $D^1[2, 4] = \min \{ D^{(0)}[2,4], D^{(0)}[2,1] + D^{(0)}[1,4] \} = \min \{ \infty, 2 + \infty \} = \infty$
- ✓ $D^1[3, 2] = \min \{ D^{(0)}[3,2], D^{(0)}[3,1] + D^{(0)}[1,2] \} = \min \{ 7, \infty + \infty \} = 7$
- ✓ $D^1[3, 4] = \min \{ D^{(0)}[3,4], D^{(0)}[3,1] + D^{(0)}[1,4] \} = \min \{ 1, \infty + \infty \} = 1$
- ✓ $D^1[4, 2] = \min \{ D^{(0)}[4,2], D^{(0)}[4,1] + D^{(0)}[1,2] \} = \min \{ \infty, 6 + \infty \} = \infty$
- ✓ $D^1[4, 3] = \min \{ D^{(0)}[4,3], D^{(0)}[4,1] + D^{(0)}[1,3] \} = \min \{ \infty, 6 + 3 \} = 9$

Floyd's Example – Matrix Generation $D^{(2)}$



Now, find D^2 Matrix !

$$D^{(k)}[i,j] = \min \{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$$

$$k=2, D^2[i,j] = \min \{ D^{(1)}[i,j], D^{(1)}[i,2] + D^{(1)}[2,j] \}$$

D^1	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	∞	7	0	1
4	6	∞	9	0

$D^{(2)} =$

D^2	1	2	3	4
1	0	∞		
2	2	0	5	∞
3		7	0	
4		∞		0

- ✓ $D^2[1, 3] = \min \{ D^{(1)}[1,3], D^{(1)}[1,2] + D^{(1)}[2,3] \} = \min \{ 3, \infty + 5 \} = 3$
- ✓ $D^2[1, 4] = \min \{ D^{(1)}[1,4], D^{(1)}[1,2] + D^{(1)}[2,4] \} = \min \{ \infty, \infty + \infty \} = \infty$
- ✓ $D^2[3, 1] = \min \{ D^{(1)}[3,1], D^{(1)}[3,2] + D^{(1)}[2,1] \} = \min \{ \infty, 7 + 2 \} = 9$
- ✓ $D^2[3, 4] = \min \{ D^{(1)}[3,4], D^{(1)}[3,2] + D^{(1)}[2,4] \} = \min \{ 1, 7 + \infty \} = 1$
- ✓ $D^2[4, 1] = \min \{ D^{(1)}[4,1], D^{(1)}[4,2] + D^{(1)}[2,1] \} = \min \{ 6, \infty + 2 \} = 6$
- ✓ $D^2[4, 3] = \min \{ D^{(1)}[4,3], D^{(1)}[4,2] + D^{(1)}[2,3] \} = \min \{ 9, \infty + 5 \} = 9$

Floyd's Example – Matrix Generation $D^{(3)}$



Now, find D^3 Matrix !

$$D^{(k)}[i,j] = \min \{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$$

$$k=3, D^3[i,j] = \min \{ D^{(2)}[i,j], D^{(2)}[i,3] + D^{(2)}[3,j] \}$$

D^2	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	9	7	0	1
4	6	∞	9	0

D^3	1	2	3	4
1	0		3	
2		0	5	
3	9	7	0	1
4			9	0

- ✓ $D^3[1, 2] = \min \{ D^{(2)}[1,2], D^{(2)}[1,3] + D^{(2)}[3,2] \} = \min \{ \infty, 3 + 7 \} = 10$
- ✓ $D^3[1, 4] = \min \{ D^{(2)}[1,4], D^{(2)}[1,3] + D^{(2)}[3,4] \} = \min \{ \infty, 3 + 1 \} = 4$
- ✓ $D^3[2, 1] = \min \{ D^{(2)}[2,1], D^{(2)}[2,3] + D^{(2)}[3,1] \} = \min \{ 2, 5 + 9 \} = 2$
- ✓ $D^3[2, 4] = \min \{ D^{(2)}[2,4], D^{(2)}[2,3] + D^{(2)}[3,4] \} = \min \{ \infty, 5 + 1 \} = 6$
- ✓ $D^3[4, 1] = \min \{ D^{(2)}[4,1], D^{(2)}[4,3] + D^{(2)}[3,1] \} = \min \{ 6, 9 + 9 \} = 6$
- ✓ $D^3[4, 2] = \min \{ D^{(2)}[4,2], D^{(2)}[4,3] + D^{(2)}[3,2] \} = \min \{ \infty, 9 + 7 \} = 16$

Floyd's Example – Matrix Generation $D^{(4)}$



Now, find D^4 Matrix !

$$D^{(k)}[i,j] = \min \{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$$

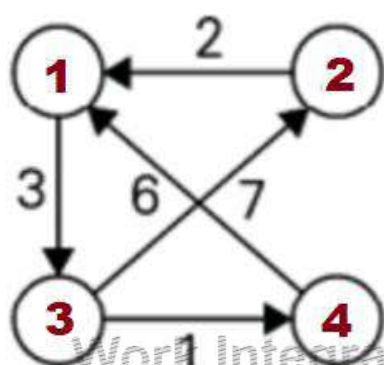
$$k=4, D^4[i,j] = \min \{ D^{(3)}[i,j], D^{(3)}[i,4] + D^{(3)}[4,j] \}$$

D^3	1	2	3	4
1	0	10	3	4
2	2	0	5	6
3	9	7	0	1
4	6	16	9	0

D^4	1	2	3	4
1	0			4
2		0		6
3			0	1
4	6	16	9	0

- ✓ $D^4[1, 2] = \min \{ D^{(3)}[1,2], D^{(3)}[1,4] + D^{(3)}[4,2] \} = \min \{ 10, 4 + 16 \} = 10$
- ✓ $D^4[1, 3] = \min \{ D^{(3)}[1,3], D^{(3)}[1,4] + D^{(3)}[4,3] \} = \min \{ 3, 4 + 9 \} = 3$
- ✓ $D^4[2, 1] = \min \{ D^{(3)}[2,1], D^{(3)}[2,4] + D^{(3)}[4,1] \} = \min \{ 2, 6 + 6 \} = 2$
- ✓ $D^4[2, 3] = \min \{ D^{(3)}[2,3], D^{(3)}[2,4] + D^{(3)}[4,3] \} = \min \{ 5, 6 + 9 \} = 5$
- ✓ $D^4[3, 1] = \min \{ D^{(3)}[3,1], D^{(3)}[3,4] + D^{(3)}[4,1] \} = \min \{ 9, 1 + 6 \} = 7$
- ✓ $D^4[3, 2] = \min \{ D^{(3)}[3,2], D^{(3)}[3,4] + D^{(3)}[4,2] \} = \min \{ 7, 1 + 16 \} = 7$

Floyd's Example



D^4	1	2	3	4
1	0	10	3	4
2	2	0	5	6
3	7	7	0	1
4	6	16	9	0

D^4 matrix presents the shortest path cost between every pair

Floyd's Algorithm



ALGORITHM *Floyd*($W[1..n, 1..n]$)

//Implements Floyd's algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

Time complexity: $O(n^3)$

Floyd's vs Dijkstra's Algorithm



- Floyd's algorithm is used for all pair shortest path problem.
 - This is a DP strategy.
- Dijkstra's algorithm is used for single source shortest path problem.
 - This is a greedy strategy.
- The Dijkstra's algorithm can also be used for all pair shortest path problem if we run the algorithm on every vertex. Doing so we get a complexity of $O(n^3)$.
- Floyd's algorithm also produces the same complexity $O(n^3)$. Then why do we prefer Floyd's algorithm ?
- We prefer Floyd's algorithm over Dijkstra's for APSP problem because :
 - ✓ Floyd's works with negative weights as well. (But not negative weight cycles) but Dijkstra's may not work if weights are negative.
 - ✓ Floyd's algorithm can be implemented in a distributed system unlike Dijkstra's.

Floyd's vs Dijkstra's Algorithm



- Floyd's algorithm is used for all pair shortest path problem.
 - This is a DP strategy.
- Dijkstra's algorithm is used for single source shortest path problem.
 - This is a greedy strategy.
- The Dijkstra's algorithm can also be used for all pair shortest path problem if we run the algorithm on every vertex. Doing so we get a complexity of $O(n^3)$.
- Floyd's algorithm also produces the same complexity $O(n^3)$. Then why do we prefer Floyd's algorithm ?
- We prefer Floyd's algorithm over Dijkstra's for APSP problem because :
 - ✓ Floyd's works with negative weights as well. (But not negative weight cycles) but Dijkstra's may not work if weights are negative.
 - ✓ Floyd's algorithm can be implemented in a distributed system unlike Dijkstra's.



Matrix Multiplication

- Let A be an $p \times q$ matrix, and B be a $q \times r$ matrix. Their product is defined to be the $p \times r$ matrix C where

$$C(i,j) = \sum_{k=1}^q A(i,k) \cdot B(k,j)$$

- $A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}_{2 \times 3}$ $B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix}_{3 \times 2}$

Must be same!

- $C = A \times B = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} \end{bmatrix}_{2 \times 2}$

- Number of Multiplications = 12 ($p \times q \times r$)

Matrix Chain Product / Matrix Chain Multiplication



Matrices : $[A] \times [B] \times [C]$

Dimensions: 2×3 3×4 4×2



Is Matrix Multiplication Associative ?

Yes!

So, we can perform either
 $(A \times B) \times C$ or $A \times (B \times C)$

Number of individual multiplications needed ?

27

Matrix Chain Product / Matrix Chain Multiplication



Matrices : $[A] \times [B] \times [C]$

Dimensions: $2 \times 3 \quad 3 \times 4 \quad 4 \times 2$

$(A \times B) \times C$ or $A \times (B \times C)$

Number of Multiplications = ?

$(A \times B) = 2 * 3 * 4 = 24$ & resultant

Dimension will be 2×4

$(A \times B) \times C = 2 * 4 * 2 = 16$

So, a total of $24 + 16 = 40$ multiplications

Number of Multiplications = ?

$(B \times C) = 3 * 4 * 2 = 24$ & resultant

Dimension will be 3×2

$A \times (B \times C) = 2 * 3 * 2 = 12$

So, a total of $24 + 12 = 36$ multiplications

Number of multiplications : 40 and 36

Matrix Chain Multiplication



- Now, let us consider we need to evaluate a long product of matrices :

$$A_1 \cdot A_2 \cdot A_3 \dots A_{n-1} \cdot A_n$$

- We can place the parentheses wherever we like 😊 [Since, matrix multiplication is associative].
- But, our motive is to place the parentheses in such a way that the total number of individual/scalar multiplications needed is minimum.
- *So, the matrix chain multiplication problem is to determine the parenthesization of the expression defining the product that minimizes the total number of scalar multiplications performed.*
- Fact Check : Can we rearrange the terms ?
 - No, matrix multiplication is not commutative
- How many ways are there to fully parenthesize this product ?

Matrix Chain Multiplication



- How many ways are there to fully parenthesize an expression ?
- *Equivalently: How many full/proper binary trees on n leaves are there?*

Let us say, we have to multiply 4 matrices :

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4$$

- ✓ $(A_1 \cdot A_2) (A_3 \cdot A_4)$
- ✓ $A_1 (A_2 (A_3 \cdot A_4))$
- ✓ $A_1 ((A_2 \cdot A_3) A_4)$
- ✓ $((A_1 \cdot A_2) A_3) A_4$
- ✓ $(A_1 (A_2 \cdot A_3)) A_4$

When $n=4$, we can have 5 different ways to parenthesize it!

Matrix Chain Multiplication



- In Simple, if we want to know the number of different ways the parenthesization can be done is the modified *Catalan* number formula which is :

$$\frac{2(n-1)C_{(n-1)}}{n}$$

So, When n=4,

$$\frac{{}^6C_3}{4} = \frac{20}{4} = 5$$

So, When n=5,

$$\frac{{}^8C_4}{5} = \frac{70}{5} = 14$$

Matrix Chain Multiplication



Suppose we have matrices $A_1, A_2, \dots, A_{n-1}, A_n$ of size $p_0, p_1, \dots, p_n$, so that A_i has dimension $p_{i-1} \times p_i$.

Choosing a particular parenthesization affects the total number of multiplication.

Example:

$$A_1 : 10 \times 100$$

$$A_2 : 100 \times 5$$

$$A_3 : 5 \times 50$$

$$(A_1 A_2) A_3 : 10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 7500$$

$$A_1 (A_2 A_3) : 100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 75000$$

This is vaguely reminiscent of Huffman coding, or merging a collection of already sorted lists.

But there are important differences: we must keep the matrices in order. We can choose the shape of the tree, but not the leaf-order.

Also, the cost function is more complicated:

If A is $p \times q$ matrix, B $q \times r$ then the cost of multiplying them is $p \cdot q \cdot r$ and the result has dimension $p \times r$.

As a consequence, the simple Greedy Algorithm in Huffman coding won't work here.

Matrix Chain Multiplication



- The brute force way is very costly and runs in exponential time. We can improve the performance by making few observations about the nature of the matrix chain-product problem.
- The first observation is that the problem can be split into subproblems which are related.
- Hence, we can use the Dynamic Programming Strategy here!
- Let us derive the formula and algorithm on how to apply the DP strategy to the matrix chain multiplication problem.
- Let us consider the previous example and derive the formula ...

Matrix Chain Multiplication



Matrices : $[A] \times [B] \times [C]$

Dimensions: 2×3 3×4 4×2

$(A \times B) \times C$ or $A \times (B \times C)$

Number of Multiplications = ?

$(A \times B) = 2 * 3 * 4 = 24$ & resultant

Dimension will be 2×4

$(A \times B) \times C = 2 * 4 * 2 = 16$

So, a total of $24 + 16 = 40$ multiplications

Number of Multiplications = ?

$(B \times C) = 3 * 4 * 2 = 24$ & resultant

Dimension will be 3×2

$A \times (B \times C) = 2 * 3 * 2 = 12$

So, a total of $24 + 12 = 36$ multiplications

Number of multiplications : 40 and 36

Matrix Chain Multiplication



Cost[1,3] = ?

C[1,3] = ?

Matrices : [A] X [B] X [C]

Dimensions: 2×3 3×4 4×2
 $d_0 \ d_1$ $d_1 \ d_2$ $d_2 \ d_3$

$(A \times B) \times C$

$A \times (B \times C)$

➤ $(A \times B) = 2 * 3 * 4 = 24$, which is $C[1,2]$

➤ $C = 0$, as its single matrix (no multiplication involved) $\rightarrow C[3,3]$

➤ Resultant of $A \times B$ is dimension 2×4

➤ Now, $(A \times B) \times C$:

➤ Dimensions : 2×4 4×2
 $d_0 \ d_2$ $d_2 \ d_3$

➤ $(A \times B) \times C = 2 * 4 * 2 = 16$

$C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 = 40$

➤ $A = 0$, as its single matrix (no multiplication involved) $\rightarrow C[1,1]$

➤ $(B \times C) = 3 * 4 * 2 = 24$, which is $C[2,3]$

➤ Resultant of $B \times C$ is dimension 3×2

➤ Now, $A \times (B \times C)$:

➤ Dimensions : 2×3 3×2
 $d_0 \ d_1$ $d_1 \ d_3$

➤ $A \times (B \times C) = 2 * 3 * 2 = 12$

$C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 = 36$

Matrix Chain Multiplication



Cost[1,3] = ?

C[i,j] = ?

Matrices : [A] X [B] X [C]

Dimensions: 2×3 3×4 4×2
 $d_0 \ d_1$ $d_1 \ d_2$ $d_2 \ d_3$

$(A \times B) \times C$

$A \times (B \times C)$

$$C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 = 40$$

$$C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 = 36$$

i

j

i

j

$$C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 = 40$$

$$C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 = 36$$

i

j

d_{i-1}

d_j

i

j

d_{i-1}

d_j

$$C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 = 40$$

$$C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 = 36$$

i

k

k+1

j

d_{i-1}

d_k

d_j

i

k

k+1

j

d_{i-1}

d_k

d_j

Matrix Chain Multiplication



Cost[1,3] = ?

$C[i,j] = ?$

Matrices : $[A] \times [B] \times [C]$

Dimensions: 2×3 3×4 4×2
 $d_0 \ d_1$ $d_1 \ d_2$ $d_2 \ d_3$

$(A \times B) \times C$

$A \times (B \times C)$

$$C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 = 40$$

$$C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 = 36$$

$i \quad k$

$k+1 \quad j$

d_{i-1}

d_k

d_j

$i \quad k$

$k+1 \quad j$

d_{i-1}

d_k

d_j

$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

This is the DP formula for Matrix Chain Multiplication ☺



Matrix Chain Multiplication

- Let us consider the same example where we want to multiply 4 matrices.

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4$$

- Let us use the formula that we derived.

$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

$$C[1,4] = \min_{1 \leq k < 4} \begin{cases} k=1 & C[1,1] + C[2,4] + d_0 \times d_1 \times d_4, \\ k=2 & C[1,2] + C[3,4] + d_0 \times d_2 \times d_4, \\ k=3 & C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \end{cases}$$

$$A_1 (A_2 A_3 A_4)$$

$$(A_1 A_2) (A_3 A_4)$$

$$(A_1 A_2 A_3) A_4$$

Matrix Chain Multiplication



$$C[i,j] = \min_{i \leq k < j} \{C[i, k] + C[k + 1, j] + d_{i-1} \times d_k \times d_j\}$$

$$C[1,4] = \min_{1 \leq k < 4} \begin{cases} k=1 & C[1,1] + C[2,4] + d_0 \times d_1 \times d_4, \\ k=2 & C[1,2] + C[3,4] + d_0 \times d_2 \times d_4, \\ k=3 & C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \end{cases}$$

Here, $C[1,1]$ and $C[4,4]$ are zero. Others are unknown! We need to apply the formula again.

$$C[2,4] = \min_{2 \leq k < 4} \begin{cases} k=2 & C[2,2] + C[3,4] + d_1 \times d_2 \times d_4, \\ k=3 & C[2,3] + C[4,4] + d_1 \times d_3 \times d_4, \end{cases}$$

Matrix Chain Multiplication



$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

$$C[2,4] = \min_{2 \leq k < 4} \begin{cases} k=2 \left\{ C[2,2] + C[3,4] + d_1 \times d_2 \times d_4, \right. \\ k=3 \left\{ C[2,3] + C[4,4] + d_1 \times d_3 \times d_4, \right. \end{cases}$$

➤ Here again, $C[2,2]$ and $C[4,4]$ are zero. $C[3,4]$ is unknown !

Matrix Chain Multiplication



- But, this is a lengthy process 😞
- DP Strategy states us to start with smaller sub-problems and then go stage by stage. In this case, instead of calculating $C[1,4]$ in the beginning, we shall calculate smaller subproblems such as $C[3,4]$, $C[1,2]$ etc.

$$C[1,4] = \min_{1 \leq k < 4} \begin{cases} k=1 \left\{ C[1,1] + C[2,4] + d_0 \times d_1 \times d_4, \right. \\ k=2 \left\{ C[1,2] + C[3,4] + d_0 \times d_2 \times d_4, \right. \\ k=3 \left\{ C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \end{cases}$$

- In this approach, the can also reuse the solutions of smaller subproblems later when needed.
- A table is preferred to solve such dynamic strategy problems! So, that its easier to reuse the solutions.

Matrix Chain Multiplication



- For the matrix chain multiplication problem, we will use two tables as below:

		$\xrightarrow{j}$			
Cost		1	2	3	4
$\downarrow i$	1	0			
	2	-	0		
	3	-	-	0	
	4	-	-	-	0

We will start filling the table from smaller subproblem to bigger. Ex: (1,2) (2,3) (3,4) then (1,3) (2,4) and finally (1,4)

This table would be used to denote which yielded the minimum solution.

k	1	2	3	4
1	0			
2	-	0		
3	-	-	0	
4	-	-	-	0

Matrix Chain Multiplication Example



- We are given the matrices orders in a sequence $\{ 3, 2, 4, 2, 5 \}$.
- This means that the matrices have sizes
 - $3 \times 2, 2 \times 4, 4 \times 2, 2 \times 5$
- Are these dimensions valid and matrix multiplication can be performed on this ?
 - Yes!
- Aim: Find the minimum number of scalar multiplications needed and the parenthesization.
- We know that cost of $C[1,1]$, $C[2,2]$, $C[3,3]$ and $C[4,4]$ is 0. With this in mind, let's start using the formula and fill the Cost/Matrix table.
- Also, as per DP, we will start with smaller subproblems (where difference is small) like $C[1,2]$, $C[2,3]$ and then proceed to $C[1,3]$ etc.

Matrix Chain Multiplication Example



$$C[i,j] = \min_{i \leq k < j} \{ C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j \}$$

$$C[1,2] = \min_{k=1} \{ C[1,1] + C[2,2] + d_0 \times d_1 \times d_2 \} \\ = \min \{ 0+0+ 3 * 2 * 4 \} = 24$$

$$C[2,3] = \min_{k=2} \{ C[2,2] + C[3,3] + d_1 \times d_2 \times d_3 \} \\ = \min \{ 0+0+ 2 * 4 * 2 \} = 16$$

$$C[3,4] = \min_{k=3} \{ C[3,3] + C[4,4] + d_2 \times d_3 \times d_4 \} \\ = \min \{ 0+0+ 4 * 2 * 5 \} = 40$$

- So, we have solved three sub-problems and stored their solutions. Now we will move to slightly bigger sub-problem. *Can you guess which is that?*

A	X	B	X	C	X	D
3	2	2	4	4	2	2
d_0	d_1	d_1	d_2	d_2	d_3	d_3

Cost	1	2	3	4
1	0			
2	-	0		
3	-	-	0	
4	-	-	-	0

k	1	2	3	4
1	0			
2	-	0		
3	-	-	0	
4	-	-	-	0

Matrix Chain Multiplication Example



$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

$k=1,2$

$$\begin{aligned} C[1,3] &= \min_{1 \leq k < 3} \{ C[1,1] + C[2,3] + d_0 \times d_1 \times d_3, \\ &\quad C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 \} \\ &= \min \{ 0+16+ 3 * 2 * 2, \\ &\quad 24+0+ 3 * 4 * 2 \} = \min\{28,48\} \\ &= 28 \end{aligned}$$

$k=2,3$

$$\begin{aligned} C[2,4] &= \min_{2 \leq k < 4} \{ C[2,2] + C[3,4] + d_1 \times d_2 \times d_4, \\ &\quad C[2,3] + C[4,4] + d_1 \times d_3 \times d_4 \} \\ &= \min \{ 0+40+ 2 * 4 * 5, \\ &\quad 16+0+ 2 * 2 * 5 \} = \min\{80,36\} \\ &= 36 \end{aligned}$$

What next ?

A	X	B	X	C	X	D
3	2	2	4	4	2	2
d_0	d_1	d_1	d_2	d_2	d_3	d_3

Cost	1	2	3	4
1	0	24		
2	-	0	16	
3	-	-	0	40
4	-	-	-	0

k	1	2	3	4
1	0	1		
2	-	0	2	
3	-	-	0	3
4	-	-	-	0

45

Matrix Chain Multiplication Example



$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

A	X	B	X	C	X	D
3	2	2	4	4	2	2
d_0	d_1	d_1	d_2	d_2	d_3	d_3

$k=1,2,3$

$$\begin{aligned}
 C[1,4] &= \min_{1 \leq k < 4} \{ C[1,1] + C[2,4] + d_0 \times d_1 \times d_4, \\
 &\quad C[1,2] + C[3,4] + d_0 \times d_2 \times d_4, \\
 &\quad C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \} \\
 &= \min \{ 0+36 + 3 * 2 * 5, \\
 &\quad 24+40 + 3 * 4 * 5, \\
 &\quad 28+00 + 3 * 2 * 5 \} \\
 &= \min\{66,124,58\} = 58
 \end{aligned}$$

Cost	1	2	3	4
1	0	24	28	
2	-	0	16	36
3	-	-	0	40
4	-	-	-	0

k	1	2	3	4
1	0	1	1	
2	-	0	2	3
3	-	-	0	3
4	-	-	-	0

46

Matrix Chain Multiplication Example



$$C[i,j] = \min_{i \leq k < j} \{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

A	X	B	X	C	X	D
3	2	2	4	4	2	2
d ₀	d ₁	d ₁	d ₂	d ₂	d ₃	d ₃

- From the solved table, we can see that C[1,4] is 58.
- Hence, the minimum number of scalar multiplications needed to multiply four matrices of the given dimension is **58**.
- Parenthesization:

➤ The k[1,4] = 3 & hence, parenthesize at 3 😊

➤ (A X B X C) X (D)

➤ How to further parenthesize A X B X C ?

➤ K[1,3] will give us where to parenthesize which is 1 here.

➤ So, **((A) X (B X C)) X (D)**

Cost	1	2	3	4
i ↓				
1	0	24	28	58
2	-	0	16	36
3	-	-	0	40
4	-	-	-	0

k	1	2	3	4
1	0	1	1	3
2	-	0	2	3
3	-	-	0	3
4	-	-	-	0

47

Matrix Chain Multiplication Algorithm



Bottom-up approach: compute an array $C[i, j]$.

For $i = j$ can set $C(i, i) = 0$.

Then compute all $C(i, j)$ for $d = j - i, d = 1, 2, \dots, n - 1$.

for $d = 1, \dots, n - 1$ do

for $i = 1, \dots, n - d$ do

$j = i + d$;

for $k = i, \dots, j - 1$ do

{

$c = C[i, k] + C[k + 1, j] + p[i - 1] p[k] p[j]$;

if ($c < C[i, j]$)

$C[i, j] = c$;

}

Running time is clearly $O(n^3)$ and in fact $\Theta(n^3)$.

0/1 Knapsack Problem



- Given a knapsack with maximum capacity W , and a set S consisting of n items
- Each item i has some weight w_i and benefit value b_i (**all w_i and W are integer values**)
- Problem: How to pack the knapsack to achieve maximum total value of packed items?
- Problem, in other words, is to find

$$\max \sum_{i \in T} b_i \text{ subject to } \sum_{i \in T} w_i \leq W$$

- The problem is called a “0-1” problem, because each item must be entirely accepted or rejected.

0/1 Knapsack Problem : Brute force Approach



- Let's first solve this problem with a straightforward algorithm
- Since there are n items, there are 2^n possible combinations of items.
- We go through all combinations and find the one with maximum value and with total weight less or equal to W
- Running time will be $O(2^n)$

0/1 Knapsack Problem : Dynamic Programming Approach



If items are labeled 1..n, then a subproblem would be to find an optimal solution for $S_k = \{\text{items labeled } 1, 2, \dots, k\}$

- Let's add another parameter: w , which will represent the maximum weight for each subset of items
- The subproblem then will be to compute $V[k, w]$, i.e., to find an optimal solution for $S_k = \{\text{items labeled } 1, 2, \dots, k\}$ in a knapsack of size w
- Assuming knowing $V[i, j]$, where $i=0, 1, 2, \dots, k-1$, $j=0, 1, 2, \dots, w$, how to derive $V[k, w]$?
- Recursive formula for subproblems:

$$V[k, w] = \begin{cases} V[k-1, w] & \text{if } w_k > w \\ \max\{V[k-1, w], V[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

51

0/1 Knapsack : DP Approach Example



Let's run our algorithm on the following data:

- $n = 4$ (# of elements)
- $W = 5$ (max weight of the knapsack)
- Elements (weight, benefit):
(2,3), (3,4), (4,5), (5,6)

Work Integrated Learning Programmes

0/1 Knapsack : DP Approach

Example



i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1						
2						
3						
4						

for $w = 0$ to W
 $V[0,w] = 0$

0/1 Knapsack : DP Approach Example



i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

for $i = 1$ to n
 $V[i,0] = 0$

0/1 Knapsack : DP Approach Example



Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0				
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=1$

$w-w_i=-1$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3			
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=2$

$w-w_i=0$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

56

0/1 Knapsack : DP Approach Example



Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3		
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=3$

$w-w_i=1$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

57

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	
2	0					
3	0					
4	0					

Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

$i=1$

$b_i=3$

$w_i=2$

$w=4$

$w-w_i=2$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



Items:

- | |
|----------|
| 1: (2,3) |
| 2: (3,4) |
| 3: (4,5) |
| 4: (5,6) |

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=5$

$w-w_i=3$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0				
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=1$

$w-w_i=-2$

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3			
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=2$

$w-w_i=-1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4		
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=3$

$w-w_i=0$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=4$

$w-w_i=1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=5$

$w-w_i=2$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4		
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w=1..3$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

65

0/1 Knapsack : DP Approach Example



i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w=4$

$w - w_i = 0$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w=5$

$w - w_i = 1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach Example



$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	

$i=4$

$b_i=6$

$w_i=5$

$w=1..4$

Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach

Example



i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=4$

$b_i=6$

$w_i=5$

$w=5$

$w-w_i=0$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

0/1 Knapsack : DP Approach



$$V[k, w] = \begin{cases} V[k-1, w] & \text{if } w_k > w \\ \max \{V[k-1, w], V[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- The best subset of S_k that has the total weight $\leq w$, either contains item k or not.
- First case: $w_k > w$. Item k can't be part of the solution, since if it was, the total weight would be $> w$, which is unacceptable.
- Second case: $w_k \leq w$. Then the item k can be in the solution, and we choose *the case with greater value*.

0/1 Knapsack : DP Approach Algorithm



for $w = 0$ to W

$V[0, w] = 0$

for $i = 1$ to n

$V[i, 0] = 0$

for $i = 1$ to n

for $w = 0$ to W

if $w_i \leq w$ // item i can be part of the solution

if $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else $V[i, w] = V[i-1, w]$ // $w_i > w$

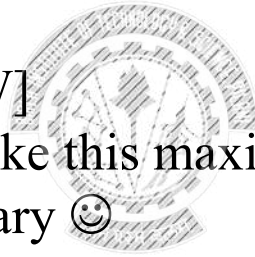
$O(n * W)$



0/1 Knapsack : DP Approach



- How to find out which items are in the optimal subset ?
- This algorithm only finds the max possible value that can be carried in the knapsack
 - i.e., the value in $V[n, W]$
- To know the items that make this maximum value, an addition to this algorithm is necessary 😊



0/1 Knapsack : DP Approach

How to find actual Knapsack Items



- All of the information we need is in the table.
- $V[n, W]$ is the maximal value of items that can be placed in the Knapsack.
- Let $i=n$ and $k=W$

if $V[i, k] \neq V[i-1, k]$ then
mark the i^{th} item as in the knapsack
 $k = k - w_i, i = i - 1$

else

$i = i - 1$ // Assume the i^{th} item is not
in the knapsack

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=4$

$k=5$

$b_i=6$

$w_i=5$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while $i, k > 0$

if $V[i,k] \neq V[i-1,k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=4$

$k=5$

$b_i=6$

$w_i=5$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while $i,k > 0$

if $V[i,k] \neq V[i-1,k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=3$

$k=5$

$b_i=5$

$w_i=4$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while $i,k > 0$

if $V[i,k] \neq V[i-1,k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=2$

$k=5$

$b_i=4$

$w_i=3$

$V[i,k] = 7$

$V[i-1,k] = 3$

$k - w_i = 2$

$i=n, k=W$

while $i,k > 0$

if $V[i,k] \neq V[i-1,k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=1$

$k=2$

$b_i=3$

$w_i=2$

$V[i,k] = 3$

$V[i-1,k] = 0$

$k - w_i = 0$

$i=n, k=W$

while $i, k > 0$

if $V[i,k] \neq V[i-1,k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

0/1 Knapsack : DP Approach

Finding the Items - Example



Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

$i \backslash W$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=0$

$k=0$

$i=n, k=W$

while $i, k > 0$

if $V[i, k] \neq V[i-1, k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

The optimal knapsack should contain {1, 2}

0/1 Knapsack : DP Approach

Finding the Items - Example



i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Items:

- 1: (2,3)
- 2: (3,4)
- 3: (4,5)
- 4: (5,6)

$i=n, k=W$

while $i, k > 0$

if $V[i, k] \neq V[i-1, k]$ then

mark the i^{th} item as in the knapsack

$k = k - w_i, i = i - 1$

else

$i = i - 1$

The optimal knapsack should contain {1, 2}

0/1 Knapsack Example 2



Apply the DP algorithm to the following 0/1 Knapsack instance :

Let $W = 10$ and

i	1	2	3	4
v_i	10	40	30	50
w_i	5	4	6	3

Work Integrated Learning Programmes



Work Integrated Learning Programmes
We will explore Complexity Classes in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



83

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

No Data Structures and Algorithms Design DSECLZG519

Parthasarathy

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



BITS Pilani
Pilani | Dubai | Coimbatore | Hyderabad



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

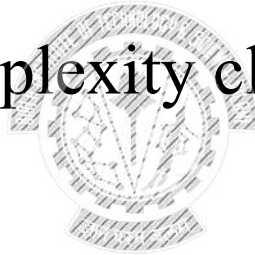
Contact Session #15

DSECLZG519 –Introduction to Complexity Classes

Agenda for Session # 15

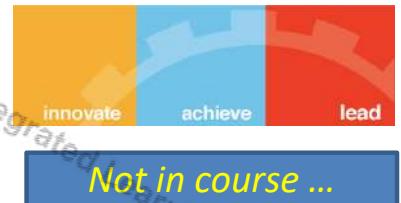


- Recap of CS#14
- Review of 0/1 Knapsack
- Introduction to complexity classes
- Exercises



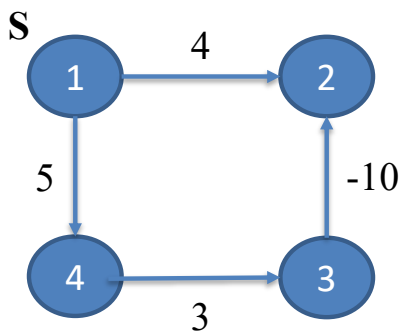
Work Integrated Learning Programmes

Bellman Ford Algorithm



- Solves single shortest path problem in which edge weight may be negative but no negative cycle exists.
- This algorithm works correctly when some of the edges of the directed graph G may have negative weight. When there are no cycles of negative weight, then we can find out the shortest path between source and destination.
- It is slower than Dijkstra's Algorithm but more versatile, as it is capable of handling some of the negative weight edges.
- This algorithm detects the negative cycle in a graph and reports their existence.
- Based on the "**Principle of Relaxation**" in which more accurate values gradually recovered an approximation to the proper distance by until eventually reaching the optimum solution.

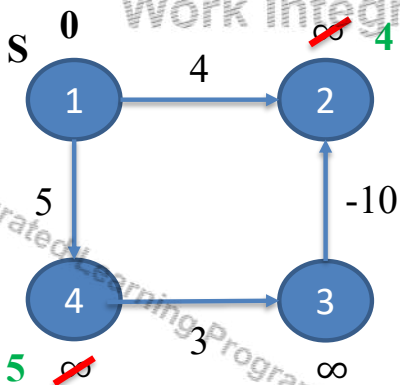
Bellman Ford Algorithm Example



Here, we have 4 nodes, so we will relax $n - 1$, Which is 3 times.

Write down all the edges: (In any order, doesn't matter) : (3,2) (4,3) (1,4) (1,2)

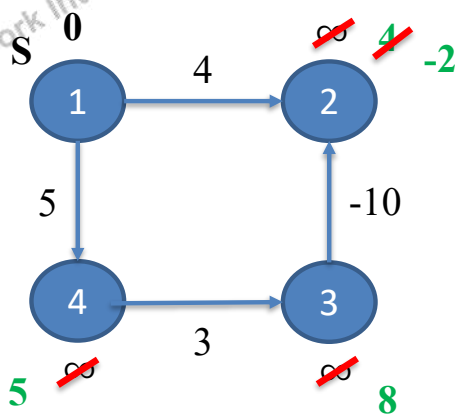
Mark path cost as 0 for Source and ∞ for other nodes. Then start relaxing ☺ for 3 times.



Relaxation – 1st Iteration

- $(3,2) = \infty - 10 < \infty$ (false) , No change
- $(4,3) = \infty + 3 < \infty$ (false), No change
- $(1,4) = 0 + 5 < \infty$ (true). So, change cost at 4 from ∞ to 5
- $(1,2) = 0 + 4 < \infty$ (true). So, change cost at 2 from ∞ to 4

Bellman Ford Algorithm Example



Relaxation – 2nd Iteration

- $(3,2) = \infty - 10 < 4$ (false) , No change
- $(4,3) = 5 + 3 < \infty$ (true), So, change cost at 3 from ∞ to 8
- $(1,4) = 0 + 5 < 5$ (false). No change.
- $(1,2) = 0 + 4 < 4$ (false). No change.

Relaxation – 3rd Iteration

- $(3,2) = 8 - 10 < 4$ (true) , So, change cost at 2 from 4 to -2
- $(4,3) = 5 + 3 < 8$ (false). No change.
- $(1,4) = 0 + 5 < 5$ (false). No change.
- $(1,2) = 0 + 4 < -2$ (false). No change.

Path	Cost
$1 \rightarrow 1$	0
$1 \rightarrow 2$	-2
$1 \rightarrow 3$	8
$1 \rightarrow 4$	5

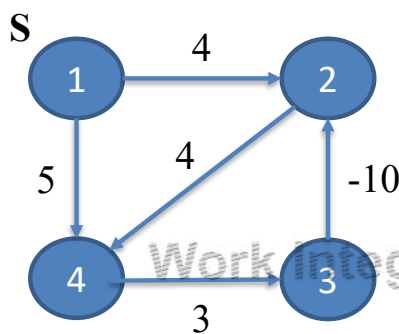
Try to relax one more time ! Are answers changing ? **No!**

6

Negative Edge Cycle ?



- A negative edge cycle is a cycle whose total weight cost is negative.



Observations:

- There exists a cycle $2 - 4 - 3 - 2$
- Its total cost is $4 + 3 + (-10) = -3$
- Such a cycle is called as a negative cycle / negative edge cycle.

- If there is a graph G with negative edge cycle, then even after the $(n-1)$ iteration, the values would still get relaxed. Bellman Ford can detect it cannot be used to find the shortest path! (Since, the edges involved in the negative edge cycle will keep on relaxing).

Quick Game ...



Algorithm	Purpose	Strategy
<i>Kruskal's</i>		
<i>Prim's</i>		
<i>Dijkstra's</i>		
<i>Fractional Knapsack</i>		
<i>Huffman</i>		
<i>Karatsuba</i>		
<i>Merge & Quick Sort</i>		
<i>Bellman Ford</i>		
<i>Floyd's</i>		
<i>Warshall's</i>		
<i>Matrix Chain Product</i>		
<i>0/1 Knapsack</i>		

Answers ...



Algorithm	Purpose	Strategy	Complexity
<i>Kruskal's</i>	Minimum Spanning tree	Greedy	Fill it as an exercise (It also depends on the Data structure used) !!
<i>Prim's</i>	Minimum Spanning tree	Greedy	
<i>Dijkstra's</i>	Single Source shortest path for G which has no negative weights	Greedy	
<i>Fractional Knapsack</i>	Maximize the profit, given the size of knapsack and objects with w and p	Greedy	
<i>Huffman</i>	Encode with variable number of bits per character	Greedy	
<i>Karatsuba</i>	Integer Multiplication	D&C	
<i>Merge & Quick Sort</i>	Sorting	D&C	
<i>Bellman Ford</i>	Single Source shortest path for G which has positive & negative weights	DP	
<i>Floyd's</i>	All pair shortest path with negative & positive weights.	DP	
<i>Warshall's</i>	Transitive Closure	DP	
<i>Matrix Chain Product</i>	To optimally parenthesize such that total scalar multiplications is minimized	DP	
<i>0/1 Knapsack</i>	Maximize the profit, objects cannot be partitioned	DP	

Some other Algorithms to Explore



- Kruskal and Prim's do not work for directed graph, explore about Edmond's algorithm.
- There is another algorithm to find out all pair shortest path problem – Johnson's algorithm.
- Travelling Sales man problem (DP)
- Strassen's Matrix multiplication (D&C)
- String pattern matching Algorithms like:
 - Horspool's algorithm
 - Boyer-more algorithm (Covered in Webinar 4)
 - Knuth-morris-pratt algorithm
- You will learn a lot more algorithm which are '*Data science*' specific in future semesters. Do ask yourself its strategy & complexity and complexity varies if you change the underlying Data structure 😊

10

Complexity Classes Motivation



- **Input:** A set of Jobs $J_1, J_2, \dots, J_n$ and each of them have a size associated with them say $S_1, S_2, \dots, S_n$.
- There are two processors P1 and P2 and your task is to automate the process of scheduling i.e. by writing a program which takes the sizes of the jobs as input (say from an array or file or screen) and schedule them on the processors P1 and P2.
- Assume Pre-emption is not allowed, meaning if we start a job – we cannot stop until it completes.
- The goal is that the last job finishes fastest.
- What are the different ways to do this?
- **Attempt 1:** Say you don't use P2, and schedule every job on P1. Then the time taken will be $S_1 + S_2 + \dots + S_n$

Complexity Classes Motivation



- **Example:** $S_1=10$, $S_2=4$, $S_3=2$, $S_4=14$, $S_5=20$
- You schedule everything in P1 and hence at time unit 50 ($10+4+2+14+20$), all jobs will be completed.
- **Attempt 2:** You want to optimize this approach by using both the processors. You decide to schedule all the odd jobs in P1 and even jobs in P2.

P1	P2
S_1	S_2
S_3	S_4
S_5	

- Now, at time unit 32, all the jobs are completed. You are happy and inform your success to your friends.

Complexity Classes Motivation



- Your friends give an input as follows:
- $S_1=1, S_2=85, S_3=2, S_4=90, S_5=3, S_6=100$
- Your algorithm schedules S_1, S_3 and S_5 on P1 and S_2, S_4 and S_6 on P2

P1	P2
S_1	S_2
S_3	S_4
S_5	S_6
6	275

- The time taken is 275 units to complete the jobs! This is clearly not best!
- *For Example:* If S_1, S_3, S_4, S_6 was scheduled in P1 and S_2, S_5 was scheduled in P2 then by 193 units, all jobs would have got completed!

13

Complexity Classes Motivation



- Based on the previous failure, you realize that you must evenly distribute the jobs (as much as possible) among the processors.
- You come up with another approach!
- **Attempt 3:** If there are P_i processors, you schedule i jobs in the P_i processors. After this for the other jobs, you schedule them on the least lightly loaded processor!
- **Example:** $S_1, S_2, S_3 \dots$. There are 2 processors, so you schedule S_1 in P_1 and S_2 in P_2 .
- From S_3 onwards, we see the least lightly loaded processor and schedule it there. i.e. say if $S_1 > S_2$ then we put S_3 in P_2 and continue this way ... For scheduling S_4 Again check if $S_1 > S_2 + S_3$ if so schedule S_4 on P_2 else on P_1 . Break ties as you want and continue the process ...

P1	P2
S_1	S_2

P1	P2
S_1	S_2
	S_3 (if $S_1 > S_2$)

Complexity Classes Motivation



- You show this algorithm to the friends and this seems to work!
You show this to the boss! All good 😊
- You again face an issue the next day where your algorithm takes T units of time and the boss is able to produce another schedule which is less than T .
- *Reason?*
- Maybe because the job sizes was a mixture and your algorithm did not work.
- **Attempt 4:** You now sort the jobs according to size such that $S_1 < S_2 < S_3 \dots$ and follow the same approach.
- As usual, you show it to your friends and they give you jobs with sizes 2,3,2,3,2.

Complexity Classes Motivation



- You sort them and make it 2,2,2,3,3 and schedule them as follows:

P1	P2
2	2
2	3
3	

At time unit 7, all the jobs got completed.

- Again mockery from friends because optimal for these jobs is to assign 2,2,2 in P1 and 3,3 in P2. In which case, at time unit 6, all the jobs will get completed. So, attempt 4 fails as well.
- By the way, the approach that you are attempting is which strategy?
- Greedy! You are greedily assigning the job to the processor (putting in least lightly loaded processor).

Complexity Classes Motivation



- You try to come up with some other approaches ... Any other algorithmic strategy that can come handy ?
- Probably not ☹ You can surely come up with a smart solution for a given case but an algorithm which works optimally to schedule the jobs always is possibly impossible !!
- *Brute Force the saviour.*
- **Final Attempt (after you are frustrated):**
 - $S_1, S_2, \dots, S_n$
 - Take all the possible subsets of S
 - Choose the minimum and schedule them on the processor.
- Will this always give the correct answer ? **Yes**
- All good and submitted the solution to the boss!

17

Complexity Classes Motivation



- The boss tests your solution by giving some inputs and everything works like a charm!
- Until ...
- The boss gives an input with 1000 Jobs !
- The algorithm runs for
 - 10 min ...
 - 1 hour ...
 - 10 hours
 - 2 days ...
 - 10 days ... and still the algorithm has not stopped
- Now, he calls you for a 1 – 1 meeting 😊
- He asks you the golden question : *“When will the algorithm stop ?”*

Complexity Classes Motivation



- $N = 1000$
- We are create all possible subsets!
- If there are N numbers in a set, how many subsets can be formed ?

$$2^N$$

In this case, 2^{1000}

- Lets assume that in 1 instruction, the computer can process 1 of these subsets.
- Lets say the fastest computer can perform 10^{20} instructions/sec.
- Lets say we use ALL the computers in this world to solve this (currently we have around 2+ billion computers) $\rightarrow 10^{20}$ computers

Complexity Classes Motivation



- So per second we can do 10^{40} instructions if we use all the computers. ($10^{20} * 10^{20}$)
- What about a year ? How many instructions can we perform in a year ?
- $10^{40} * 60 * 60 * 24 * 365$ instructions / year
- Lets generalize this and let's be very generous here :

$$\begin{array}{ccccccc} 10^{40} & * & 60 & * & 60 & * & 24 & * & 365 \\ \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\ 10^{40} & * & 10^2 & * & 10^2 & * & 10^2 & * & 10^3 \end{array} \rightarrow \sim 10^{50} \text{ instructions / year}$$

We are very generous here!

- So maximum we can perform 10^{50} instructions per year and we have 2^{1000} instructions to perform.
- How many years will this take ?

Complexity Classes Motivation



- Required to execute : 2^{1000} instructions
- Capable of executing : 10^{50} instructions/year $\rightarrow \sim 2^{4.50} \rightarrow \sim 2^{200}$ instructions/year
- Number of years required ?
- Number of years required =
$$\frac{\text{Number of Instructions}}{\text{Number of Instructions per year}}$$
- Number of years required =
$$\frac{2^{1000}}{2^{200}} = 2^{1000-200}$$
- So, the number of years needed is 2^{800}
- *Fun Fact:* 2^{800} is $\sim$ equal to number of atoms in this universe ☺).
Realize the blunder of such algorithms?

21

Complexity Classes Motivation



- If you tell this to your boss, what's the consequence ?
- So, what do we tell the boss ?



"I can't find an efficient algorithm,
I guess I'm just too dumb."

What you would ideally like to Say?



Work Integrated Learning Programmes

"I can't find an efficient algorithm,
because no such algorithm is possible!"

This is where Complexity Classes come into picture ☺

Complexity Theory



- We would tell the boss that this is a “*hard*” problem! We will tell that this has not been solved efficiently by anyone yet. If this gets solved, then a lot of similar “*hard*” problems would also get solved.
- We tell him and prove this and save the job!



“I can't find an efficient algorithm,
but neither can all these famous people.”

Complexity Theory



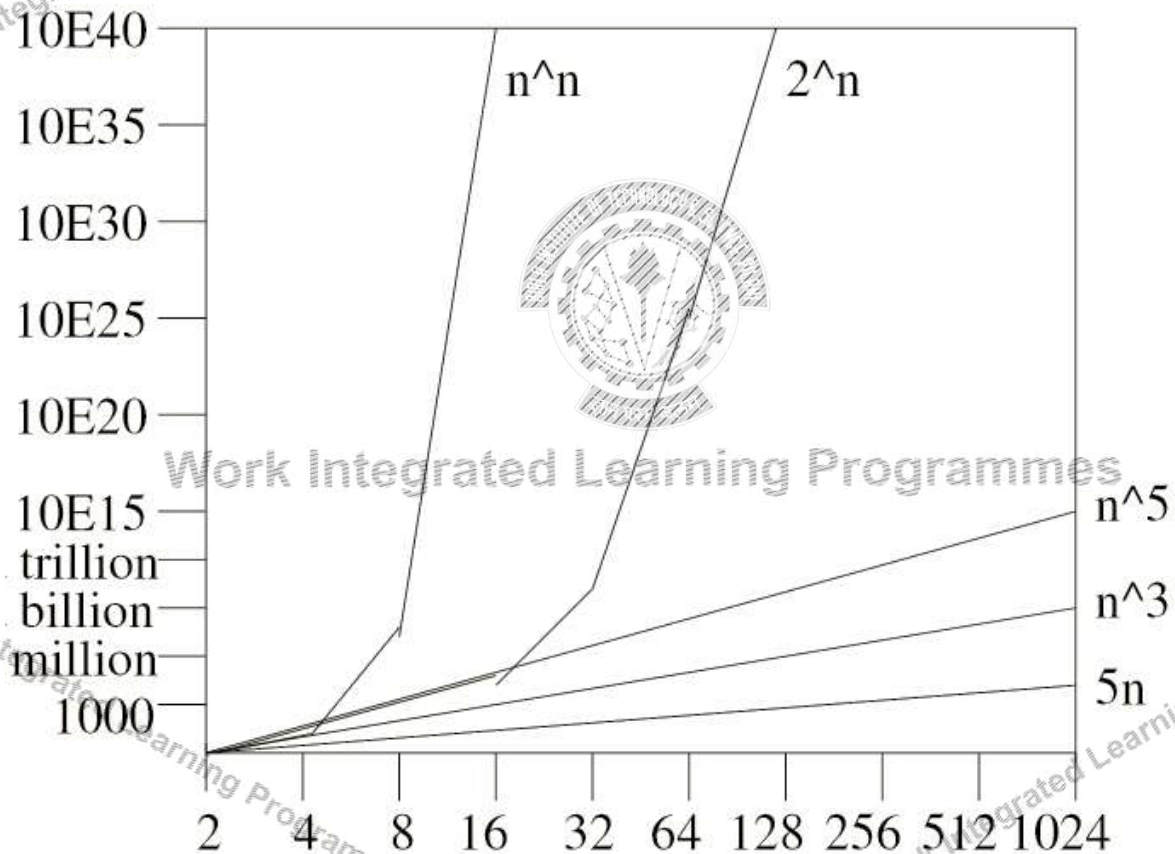
Polynomial Time Taking

- Linear Search – n
- Binary Search – $\log n$
- Insertion Sort – n^2
- Bubble Sort – n^2
- Merge Sort – $n \log n$
- Matrix Multiplication – n^3
-
-
-

Exponential Time Taking

- Job Scheduling – 2^n
- Sum of Subsets – 2^n
- Travelling Salesman – 2^n
- Hamiltonian Cycle – 2^n
- Graph coloring – 2^n
- ...
- ...
- ...

innovate achieve lead



Complexity Theory



- On the basis of this classification of functions into polynomial and exponential, we can classify algorithms:
- **Polynomial-time Algorithms:**

An algorithm whose order of magnitude time performance is bounded from above by a polynomial function n , where n is the size of the inputs.
- **Exponential Algorithms:**

An algorithm whose order of magnitude time performance is not bounded from above by a polynomial function of n .

Complexity Theory



- **Tractable Problem:** a problem that is solvable by a polynomial-time algorithm. The upper bound is polynomial. Example : Searching, sorting, multiplication of integers, finding a MST etc.
- **Intractable Problem:** a problem that cannot be solved by a polynomial-time algorithm. The lower bound is exponential. Example: Towers of Hanoi, SAT, TSP etc.
- **Tractable:** controllable, manageable, malleable, governable, yielding, amenable, complaisant, compliant, adjustable, docile, submissive, obedient, tame, meek, easily handled.
- **Intractable:** unmanageable, uncontrollable, ungovernable, out of control, out of hand, impossible to cope with; difficult, awkward, complex, troublesome, demanding, burdensome, “their problems have become more severe and intractable.

P- Class



- The class P consists of those problems that are solvable in polynomial time, i.e. these problems can be solved in time $O(n^k)$ in worst-case, where k is constant.
- For those problem belonging in this class, the below properties holds good:
 - We can solve them in polynomial time
 - We already know a deterministic algorithm for the same.
- Examples of problems which belong to this class are Binary Search, Sorting, Matrix multiplication, All pair shortest path problem etc.

NP-Class

Not “Non-Polynomial”



- Now there are a lot of programs that don't (necessarily) run in polynomial time on a regular computer, but do run in polynomial time on a non-deterministic Turing machine. These programs solve problems in **NP**, which stands for **non-deterministic polynomial time**. A non-deterministic Turing machine can do everything a regular computer can and more. This means all problems in P are also in NP.
- An equivalent way to define NP is by pointing to the problems that can be **verified in polynomial time**. This means there is not necessarily a polynomial-time way to find a solution, but once you have a solution it only takes polynomial time to verify that it is correct.
- *Verifiable?*
 - If we are somehow given a ‘certificate’ of a solution we can verify the legitimacy in polynomial time.
- *A Turing machine is a hypothetical machine thought of by the mathematician Alan Turing in 1936. Despite its simplicity, the machine can simulate ANY computer algorithm, no matter how complicated it is!*

30

NP-Class

Not “Non-Polynomial”



- NP class consists of those problems which can be solved by a *non-deterministic algorithm* in polynomial time.
- NP stands for **N**on-deterministic **P**olynomial
- The class NP consists of those problems that are *verifiable* in polynomial time.
- Hence, we aren't asking for a way to find a solution, but only to verify that an alleged solution really is correct.
- Every problem in this class can be solved in exponential time using exhaustive search.
- Lets consider what is non-deterministic algorithm with an example.

NP - Class



The kinds of problems we deal with in complexity theory come in pairs: a "search" version and a "verification" version. For example:

- **Problem:** Sorting.
Search version: Input a list of numbers X and output the same list in sorted order (call it Y).
Verification version: input a list of numbers X and another list Y , and output "YES" if Y is the sorted version of X and "NO" otherwise.
- **Problem:** graph coloring.
Search version: Input a network of nodes and edges X , and output colors Y for each node such that no adjacent nodes have the same color.
Verification version: input a network X and a set of colors Y and output "YES" if all adjacent nodes have different colors and "NO" otherwise.

Non-Deterministic



DETERMINISTIC ALGORITHM

For a particular input the computer will give always same output.

NON-DETERMINISTIC ALGORITHM

For a particular input the computer may give different output on different execution.

Can determine the next step of execution.

Cannot determine the next step of execution due to more than one path the algorithm can take.

Non-Deterministic



- In **deterministic algorithm**, for a given particular input, the computer will always produce the same output going through the same states but in case of **non-deterministic algorithm**, for the same input, the compiler may produce different output in different runs.
- *Meaning, non-deterministic algorithms have some degree of randomness.*
- Some of the terms related to the non-deterministic algorithm are defined below:
 - **choice(X)** : chooses any value randomly from the set X.
 - **failure()** : denotes the unsuccessful solution.
 - **success()** : Solution is successful and current thread terminates.

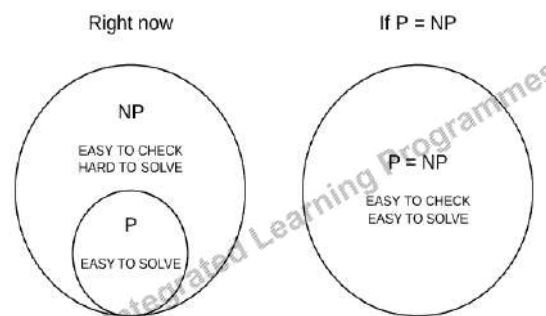
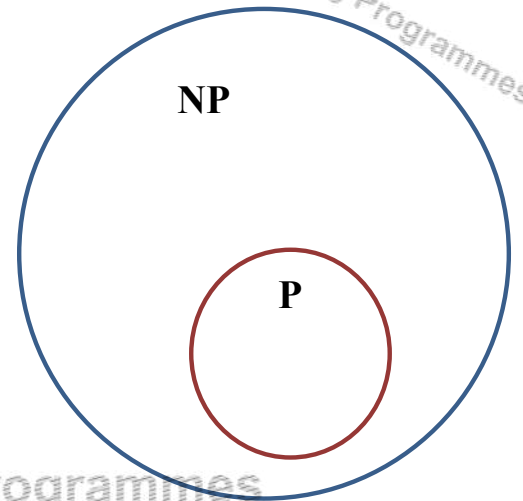
Let's say we have an array 'a' with 'n' elements and a key 'x' is to be searched in the array 'a'. A non-deterministic algorithm for this is as follows:

```
1.j= choice(a, n)
2.if(A[j]==x) then
{
    write(j);
    success();
}
3.write(0); failure();
```

P and NP



- The problem belongs to class **P** if it's easy to find a solution for the problem. The problem belongs to **NP**, if it's easy to check a solution that may have been very tedious to find.
- Clearly, $P \subseteq NP$
- The P vs NP is a famous unsolved Computer Science problem which asks whether every problem whose solution can be quickly (in polynomial time) verified (NP problem) can also be solved quickly (P class).
- In other words, will NP become equal to P ?
- Time will answer this! 😊



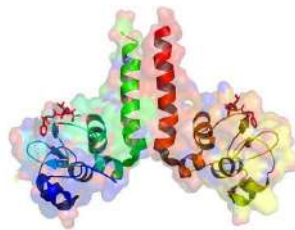
35

P and NP



If $P = NP$, then the world will be different!

- Transport routing problems will be efficient
- Job scheduling's will be best
- Massive advances in ML and AI
- Protein folding problem's could be solved too!
- Many more ...



Downsides of $P = NP$:

- Cryptography will completely go for a toss ☹
- If $P = NP$, then we could break most cryptography algorithms!



<http://www.claymath.org/millennium-problems>

36



Work Integrated Learning Programmes

We will explore further in the next class ...



Thank You for your
time & attention !

Contact : parthasarathypd@wilp.bits-pilani.ac.in

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



38

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956